

2. Transaction Layer Specification §

2.1 Transaction Layer Overview §

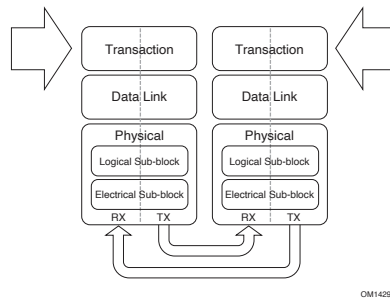


Figure 2-1 Layering Diagram Highlighting the Transaction Layer §

At a high level, the key aspects of the Transaction Layer are:

- A pipelined full Split-Transaction protocol
- Mechanisms for differentiating the ordering and processing requirements of Transaction Layer Packets (TLPs)
- Credit-based flow control
- Optional support for data poisoning and end-to-end data integrity detection.

The Transaction Layer comprehends the following:

- TLP construction and processing
- Association of transaction-level mechanisms with device resources including:
 - Flow Control
 - Virtual Channel management
- Rules for ordering and management of TLPs
 - PCI/PCI-X compatible ordering
 - Traffic Class differentiation
 - UIO Ordering

This chapter specifies the behaviors associated with the Transaction Layer.

2.1.1 Address Spaces, Transaction Types, and Usage §

Transactions form the basis for information transfer between a Requester and Completer. Four address spaces are defined, and different Transaction types are defined, each with its own unique intended usage, as shown in § Table 2-1.

Table 2-1 Transaction Types for Different Address Spaces §

Address Space	Transaction Types	Basic Usage
Memory	Read Write	Transfer data to/from a memory-mapped location
I/O	Read Write	Transfer data to/from an I/O-mapped location
Configuration	Read Write	Device Function configuration/setup
Message	Baseline (including Vendor-Defined)	From event signaling mechanism to general purpose messaging

Details about the rules associated with usage of these address formats and the associated TLP formats are described later in this chapter.

2.1.1.1 Memory Transactions §

Memory Transactions include the following types:

- Read Request/Completion
- Write Request (and Completions for UIO)
- Deferrable Memory Write Request/Completion
- AtomicOp Request/Completion

Memory Transactions use two different address formats:

- Short Address Format 32-bit address
- Long Address Format 64-bit address

Certain Memory Transactions can optionally include a PASID TLP Prefix (NFM) or OHC-A1/OHC-A4 (FM) containing the Process Address Space ID (PASID). See § Section 6.20 for details.

Certain Memory Transactions are required to use only 64-bit address formats.

2.1.1.2 I/O Transactions §

PCI Express supports I/O Space for compatibility with legacy devices that require their use. Future revisions of this specification may deprecate the use of I/O Space. I/O Transactions include the following types:

- Read Request/Completion
- Write Request/Completion

I/O Transactions use a single address format:

- Short Address Format 32-bit address

2.1.1.3 Configuration Transactions §

Configuration Transactions are used to access configuration registers of Functions within devices.

Configuration Transactions include the following types:

- Read Request/Completion
- Write Request/Completion

2.1.1.4 Message Transactions §

The Message Transactions, or simply Messages, are used to support in-band communication of events between devices.

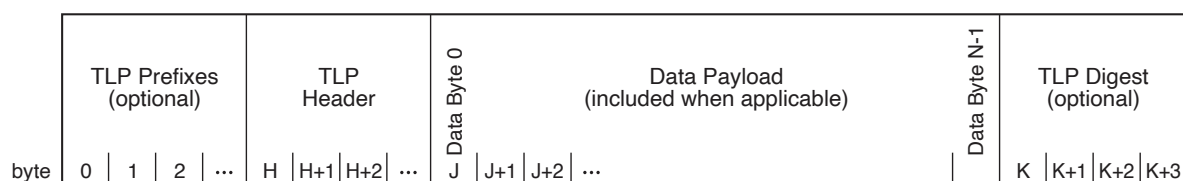
In addition to specific Messages defined in this document, PCI Express provides support for Vendor-Defined Messages using specified Message codes. Except for Vendor-Defined Messages that use the PCI-SIG® Vendor ID (0001h), the definition of specific Vendor-Defined Messages is outside the scope of this document.

This specification establishes a standard framework within which vendors can specify their own Vendor-Defined Messages tailored to fit the specific requirements of their platforms (see § Section 2.2.8.6).

Note that these Vendor-Defined Messages are not guaranteed to be interoperable with components from different vendors.

2.1.2 Packet Format Overview §

Transactions consist of Requests and Completions, which are communicated using packets. § Figure 2-2 shows a high level serialized view of a Non-Flit Mode TLP, consisting of one or more optional TLP Prefixes, a TLP header, a data payload (for some types of packets), and an optional TLP Digest. § Figure 2-3 shows a more detailed view of the TLP. The following sections of this chapter define the detailed structure of the packet headers and digest.



OM14547A

Figure 2-2 Serial View of a Non-Flit Mode TLP §

PCI Express conceptually transfers information as a serialized stream of bytes as shown in § Figure 2-2. Note that at the byte level, information is transmitted/received over the interconnect with the left-most byte of the TLP as shown in § Figure 2-2 being transmitted/received first (byte 0 if one or more optional TLP Prefixes are present else byte H). Refer to § Section 4.2 for details on how individual bytes of the packet are encoded and transmitted over the physical media.

Detailed layouts of the TLP Prefix, TLP Header and TLP Digest (presented in generic form in § Figure 2-3) are drawn with the lower numbered bytes on the left rather than on the right as has traditionally been depicted in other PCI specifications. The header layout is optimized for performance on a serialized interconnect, driven by the requirement that the most time critical information be transferred first. For example, within the TLP header, the most significant byte of the address field is transferred first so that it may be used for early address decode.

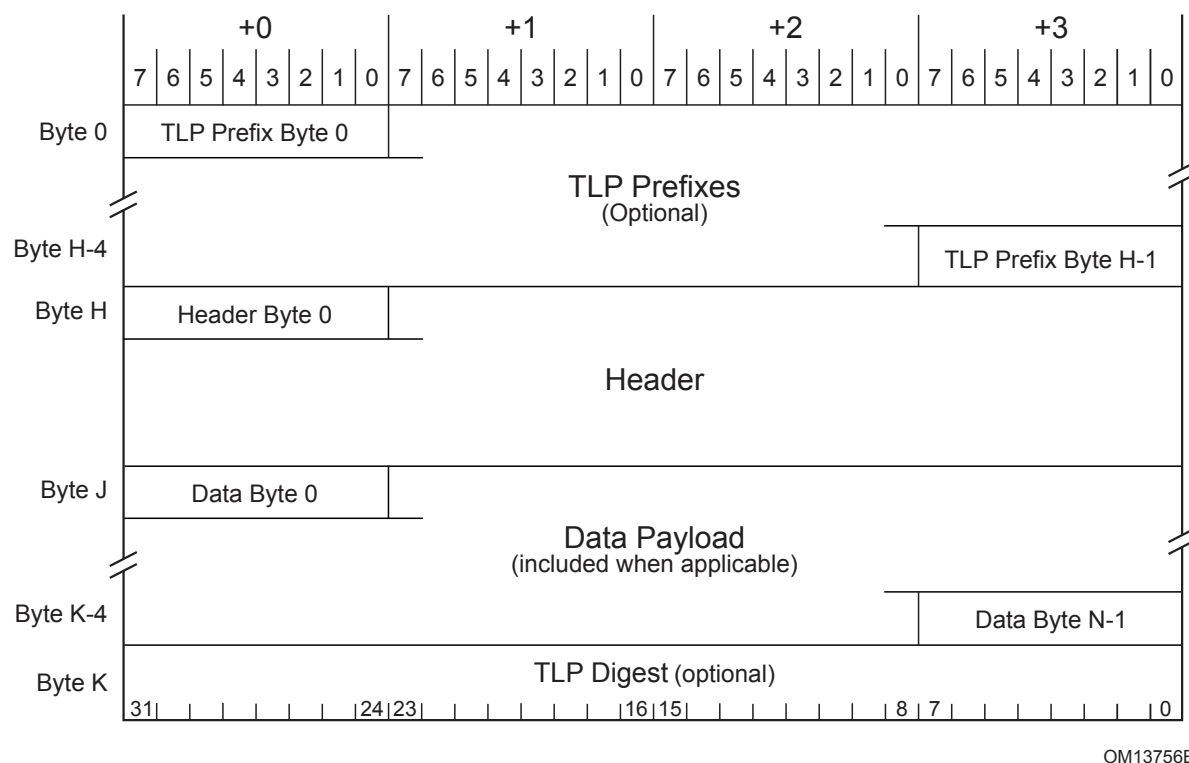


Figure 2-3 Generic TLP Format - Non-Flit Mode §

The data payload within a TLP is depicted with the lowest addressed byte (byte J in § Figure 2-3) shown to the upper left. Detailed layouts depicting data structure organization (such as the Configuration Space depictions in § Chapter 7.) retain the traditional PCI byte layout with the lowest addressed byte shown on the right. Regardless of depiction, all bytes are conceptually transmitted over the Link in increasing byte number order.

Depending on the type of a packet, the header for that packet will include some of the following types of fields:

- Format of the packet
- Type of the packet
- Length for any associated data
- Transaction Descriptor, including:
 - Transaction ID
 - Attributes
 - Traffic Class
- Address/routing information
- Byte Enables
- Message encoding
- Completion status

2.2 Transaction Layer Protocol - Packet Definition §

PCI Express uses a packet based protocol to exchange information between the Transaction Layers of the two components communicating with each other over the Link. PCI Express supports the following basic transaction types: Memory, I/O, Configuration, and Messages. Two addressing formats for Memory Requests are supported: 32 bit and 64 bit.

A **UIO TLP** is a TLP that is associated with a UIO Virtual Channel.

Transactions are carried using Requests and Completions. Completions are used only where required, for example, to return read data, or to acknowledge Completion of I/O and Configuration Write Transactions. All UIO Requests require Completions. Completions are associated with their corresponding Requests by the value in the Transaction ID field of the Packet header.

All TLP fields marked Reserved (sometimes abbreviated as R) must be filled with all 0's when a TLP is formed. Values in such fields must be ignored by Receivers and forwarded unmodified by Switches. Note that for certain fields there are both specified and Reserved values - the handling of Reserved values in these cases is specified separately for each case.

There are different header formats for Non-Flit Mode (NFM) and Flit Mode (FM). Routing elements must translate between the FM and NFM TLP formats when the Ingress Port and Egress Port are in different modes. In some cases, translation is not possible, and the handling of such cases is also defined in this chapter.

2.2.1 Common Packet Header Fields §

2.2.1.1 Common Packet Header Fields for Non-Flit Mode §

All TLP prefixes and headers contain the following fields (see § Figure 2-4):

- Fmt[2:0] - Format of TLP (see § Table 2-2) - bits 7:5 of byte 0
- Type[4:0] - Type of TLP - bits 4:0 of byte 0

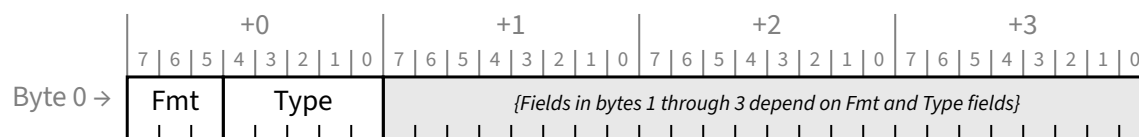


Figure 2-4 Fields Present in All Non-Flit Mode TLPs §

The Fmt field(s) indicates the presence of one or more TLP Prefixes and the Type field(s) indicates the associated TLP Prefix type(s).

The Fmt and Type fields of the TLP Header provide the information required to determine the size of the remaining part of the TLP Header, and if the packet contains a data payload following the header.

The Fmt, Type, TD, and Length fields of the TLP Header contain all information necessary to determine the overall size of the non-prefix portion of the TLP. The Type field, in addition to defining the type of the TLP also determines how the TLP is routed by a Switch. Different types of TLPs are discussed in more detail in the following sections.

- Permitted Fmt[2:0] and Type[4:0] field values are shown in § Table 2-3.
 - All other encodings are Reserved (see § Section 2.3).
- TC[2:0] - Traffic Class (see § Section 2.2.6.6) - bits [6:4] of byte 1
- R (byte 1 bit 1) - Reserved; formerly was the Lightweight Notification (LN) bit, but is now available for reassignment.
- TLP Hints (TH) - 1b indicates the presence of TLP Processing Hints (TPH) in the TLP header and optional TPH TLP Prefix (if present) - bit 0 of byte 1 (see § Section 2.2.7.1.1)
- Attr[1:0] - Attributes (see § Section 2.2.6.3) - bits [5:4] of byte 2
- Attr[2] - Attribute (see § Section 2.2.6.3) - bit 2 of byte 1 (shown as A2 in figures)
- TD - 1b indicates presence of TLP Digest in the form of a single Double Word (DW) at the end of the TLP (see § Section 2.2.3) - bit 7 of byte 2
- Error Poisoned (EP) - indicates the TLP is poisoned (see § Section 2.7) - bit 6 of byte 2
- Length[9:0] - Length of data payload, or of data referenced, in DW (see § Table 2-4) - bits 1:0 of byte 2 concatenated with bits 7:0 of byte 3
 - TLP data must be 4-byte naturally aligned and in increments of 4-byte DW.
 - Reserved for TLPs that do not contain or refer to data payloads, including Cpl, CplLk, and Messages (except as specified)

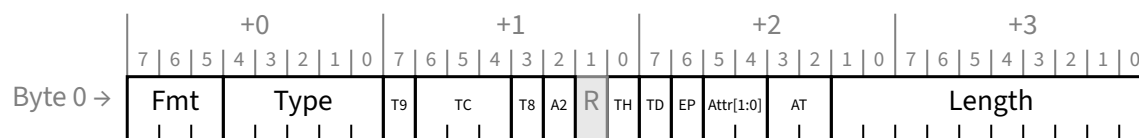


Figure 2-5 Fields Present in All Non-Flit Mode TLP Headers §

Table 2-2 Fmt[2:0] Field Values §

Fmt[2:0]	Corresponding TLP Format
000b	3 DW header, no data
001b	4 DW header, no data
010b	3 DW header, with data
011b	4 DW header, with data
100b	TLP Prefix
	All encodings not shown above are Reserved (see § Section 2.3).

Table 2-3 Non-Flit Mode Fmt[2:0] and Type[4:0] Field Encodings §

Name	Fmt [2:0] 4 (b)	Type [4:0] (b)	Description
MRd	000 001	0 0000	Memory Read Request

4. Requests with two Fmt[2:0] values shown can use either 32 bits (the first value) or 64 bits (the second value) Addressing Packet formats.

Name	Fmt [2:0] (b)	Type [4:0] (b)	Description
MRdLk	000 001	0 0001	Memory Read Request-Locked
MWrr	010 011	0 0000	Memory Write Request
IORd	000	0 0010	I/O Read Request
IOWr	010	0 0010	I/O Write Request
CfgRd0	000	0 0100	Type 0 Configuration Read Request
CfgWr0	010	0 0100	Type 0 Configuration Write Request
CfgRd1	000	0 0101	Type 1 Configuration Read Request
CfgWr1	010	0 0101	Type 1 Configuration Write Request
TCfgRd	000	1 1011	Deprecated TLP Type ⁵
DMWr	010 011	1 1011	Deferrable Memory Write Request ⁶
Msg	001	1 0r ₂ r ₁ r ₀	Message Request - The sub-field r[2:0] specifies the Message routing mechanism (see § Table 2-20).
MsgD	011	1 0r ₂ r ₁ r ₀	Message Request with data payload - The sub-field r[2:0] specifies the Message routing mechanism (see § Table 2-20).
Cpl	000	0 1010	Completion without Data - Used for I/O, Configuration Write, and Deferrable Memory Write Completions with any Completion Status. Also used for AtomicOp Completions and Read Completions (I/O, Configuration, or Memory) with Completion Status other than Successful Completion.
CplD	010	0 1010	Completion with Data - Used for Memory, I/O, and Configuration Read Completions. Also used for AtomicOp Completions.
CplLk	000	0 1011	Completion for Locked Memory Read without Data - Used only in error case.
CplDLk	010	0 1011	Completion for Locked Memory Read - Otherwise like CplD.
FetchAdd	010 011	0 1100	Fetch and Add AtomicOp Request
Swap	010 011	0 1101	Unconditional Swap AtomicOp Request
CAS	010 011	0 1110	Compare and Swap AtomicOp Request
LPrfx	100	0 L ₃ L ₂ L ₁ L ₀	Local TLP Prefix - The sub-field L[3:0] specifies the Local TLP Prefix type (see § Table 2-38).
EPrfx	100	1 E ₃ E ₂ E ₁ E ₀	End-End TLP Prefix - The sub-field E[3:0] specifies the End-End TLP Prefix type (see § Table 2-39).

5. Deprecated TLP Types: previously used for Trusted Configuration Space (TCS), which is no longer supported by this specification. If a Receiver does not implement TCS, the Receiver must treat such Requests as Malformed TLPs.

6. This TLP Type value was previously used for Trusted Configuration Space (TCS) Writes, which are no longer supported by this specification.

Name	Fmt [2:0] (b)	Type [4:0] (b)	Description
			All encodings not shown above are Reserved (see § Section 2.3).

Table 2-4 Length[9:0] Field Encoding §

Length[9:0]	Corresponding TLP Data Payload Size
00 0000 0001b	1 DW
00 0000 0010b	2 DW
...	...
11 1111 1111b	1023 DW
00 0000 0000b	1024 DW

2.2.1.2 Common Packet Header Fields for Flit Mode §

The TLP grammar is defined as:

- zero or more 1DW Local TLP prefixes ⁷
- TLP Header Base with size indicated by Type[7:0] field, followed by zero to 7 DW of Orthogonal Header Content (OHC) as indicated by OHC[4:0] field
- TLP data payload of 0 to 1024DW
- TLP Trailer, if present as indicated by TS[2:0] field

It is required to transmit NOP TLPs while TLP transmission is active if there are no other TLPs to transmit. NOP TLPs must be discarded without effect by the Receiver. All header fields other than the Type field are Reserved for NOP TLPs.

Other notable differences between Flit Mode and Non-Flit Mode TLPs include the following:

- Content that in Non-Flit Mode is included in End-to-End TLP prefixes is now incorporated within the header, as OHC.
- In Flit Mode, Steering Tags are not overlaid with the Byte Enables. The PH, Steering Tags, and AMA/AV fields are consolidated in OHC.

All Flit Mode TLPs contain the same fields in the first DW of the Header Base (see § Figure 2-6).

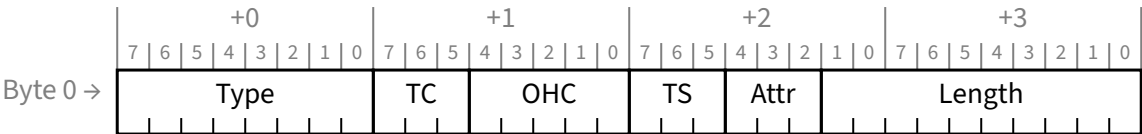


Figure 2-6 First DW of Header Base §

§ Table 2-5 defines the values for the Type[7:0] field for Flit Mode.

7. Sequencing requirements between Local Vendor Defined TLP Prefixes and the Flit Mode Local TLP Prefix are implementation specific.

- The Type[7:0] field must be fully decoded by all Receivers regardless of which specific encodings are supported.
- All Receivers must handle Flow Control for all Type[7:0] field encodings as specified.
 - For TLPs, where the FC Type is none, Receivers are not required to buffer the TLP, and must silently discard the TLP; for other FC Types:
 - Switch Ports must buffer and route TLPs, including Reserved entries, as specified.
 - Endpoint Upstream Ports and Root Ports are required to buffer, including for Header Logging, up to the largest Header Base size plus all OHC content, but are permitted, after accounting for Flow Control, to discard Header Base and OHC content that is not supported by the Port and not include that information in header logging.
- For all Reserved entries, TLP routing must be handled as indicated in the Description field, and the Header Base fields used for routing are at the same location within the Header as with the non-Reserved Header Base formats.
 - Entries marked “Local ... Terminate at Receiver” must be discarded at the Receiving Port.
- A Receiver targeted by a TLP with a Reserved Type[7:0] encoding of FC Type PR or NPR is strongly recommended⁸ to discard the Request following the update of flow control credits, and must handle a TLP with Reserved Type[7:0] encoding of FC Type CPL as an Unexpected Completion.
- A Routing Element that receives a TLP to be forwarded with a Reserved Type[7:0] encoding of FC Type PR or NPR, but is unable to forward it due to a problem like the Egress Port being in DL_Down, is strongly recommended⁹ to discard the Request following the update of flow control credits.
- UIO Requests using FC Type PR are referred to as UIO PR-FC TLPs; UIO Requests using FC Type NPR are referred to as UIO NPR-FC TLPs.

In the Translation Rule column, an entry of “1:1” indicates that there is no change in meaning or behavior when translating between Non-Flit Mode and Flit Mode in either direction. For TLPs that cannot be translated, those not handled by the Ingress Port must be handled by the Egress Port as follows, logging a TLP Translation Egress Blocked error when an error is reported.

Egress Port errors are handled according to one of the following rules:

- **Egress Rule P** – PR FC Type: block at Egress Port; if TLP is UIO, report no error, else handle as Uncorrectable
- **Egress Rule NP** – NPR FC Type: block at Egress Port; report no error
- **Egress Rule CPL** – CPL FC Type: block at Egress Port; handle as Uncorrectable

UIO is defined only for FM, and no translation of UIO TLPs to NFM is permitted. UIO TLPs targeting an Egress Port in NFM must be handled as described in the preceding paragraph. Note that error cases involving UIO VC mis-matches are addressed in § Section 2.5.2.

UIO TLPs are indicated as UIO in the Description column. Entries marked Reserved in the description column do not have an assigned VC restriction. A restriction, if required, will be specified when those entries become defined. Entries #0, #141-143 do not have an assigned VC restriction. All other entries are non-UIO TLPs.

8. For backward compatibility with previous versions of this specification, the Request is permitted to be handled as an Unsupported Request.

9. For backward compatibility with previous versions of this specification, the Request is permitted to be handled as an Unsupported Request.

Table 2-5 Flit Mode TLP Header Type Encodings [§]

#	Type								Description	Name	FC Type	Data Payload?	Header Base Size (DW)	New for Flit Mode	Translation Rule
	7	6	5	4	3	2	1	0							
0	0	0	0	0	0	0	0	0	No Operation – Local TLP – Terminate at Receiver	NOP	none	n	1	y	NFM uses this Type code for MRd (see #3)
1	0	0	0	0	0	0	0	1	Memory Read Request Locked, 32b address routed	MRdLk	NPR	n	3	n	1:1
2	0	0	0	0	0	0	1	0	I/O Read Request	IORd	NPR	n	3	n	1:1
3	0	0	0	0	0	0	1	1	Memory Read Request, 32b address routed	MRd	NPR	n	3	y/n	Requires change of Type field value
4	0	0	0	0	0	1	0	0	Type 0 Configuration Read Request	CfgRd0	NPR	n	3	n	1:1
5	0	0	0	0	0	1	0	1	Type 1 Configuration Read Request	CfgRd1	NPR	n	3	n	1:1
6	0	0	0	0	0	1	1	0	Reserved – ID routed		CPL	Length	4	y	Block at NFM Egress – Uncorrectable
7	0	0	0	0	0	1	1	1			CPL	Length	4	y	
8	0	0	0	0	1	0	0	0	Reserved – ID routed		PR	n	3	y	Block at NFM Egress – if UIO TLP report no error, else handle as Uncorrectable
9	0	0	0	0	1	0	0	1			PR	n	3	y	
10	0	0	0	0	1	0	1	0	Completion without Data	Cpl	CPL	n	3	n	1:1
11	0	0	0	0	1	0	1	1	Completion without Data, Locked (only for error cases)	CplLk	CPL	n	3	n	1:1
12	0	0	0	0	1	1	0	0	UIO Write Completion	UIOWrCpl	CPL	n	3	y	Block at NFM Egress – Uncorrectable
13	0	0	0	0	1	1	0	1	UIO Read Completion – No Data	UIORdCpl	CPL	n	3	y	Block at NFM Egress – Uncorrectable

#	Type								Description	Name	FC Type	Data Payload?	Header Base Size (DW)	New for Flit Mode	Translation Rule
	7	6	5	4	3	2	1	0							
14-15	0	0	0	0	1	1	1	X	Reserved – ID routed		CPL	n	3	y	Block at NFM Egress – Uncorrectable
16-19	0	0	0	1	0	0	X	X	Reserved – 64b address routed		NPR	Length	5	y	Block at NFM Egress – report no error
20-21	0	0	0	1	0	1	0	X	Reserved – 64b address routed		NPR	Length	5	y	Block at NFM Egress – report no error
22-23	0	0	0	1	0	1	1	X	Reserved – 64b address routed		NPR	Length	7	y	Block at NFM Egress – report no error
24	0	0	0	1	1	0	0	0	Reserved – ID routed		CPL	n	7	y	Block at NFM Egress – Uncorrectable
25	0	0	0	1	1	0	0	1			CPL	n	7	y	
26	0	0	0	1	1	0	1	0			CPL	Length	7	y	
27	0	0	0	1	1	0	1	1	Reserved – ID routed <i>{was: Trusted Configuration Read (deprecated)}</i>		CPL	Length	7	y	
28-29	0	0	0	1	1	1	0	X	Reserved – ID routed		NPR	n	3	y	Block at NFM Egress – report no error
30-31	0	0	0	1	1	1	1	X	Reserved – ID routed		NPR	n	6	y	Block at NFM Egress – report no error
32	0	0	1	0	0	0	0	0	Memory Read Request, 64b address routed	MRd	NPR	n	4	n	1:1
33	0	0	1	0	0	0	0	1	Memory Read Request Locked, 64b address routed	MRdLk	NPR	n	4	n	1:1
34	0	0	1	0	0	0	1	0	UIO Memory Read Request	UIOMRd	NPR	n	4	y	Block at NFM Egress – report no error
35	0	0	1	0	0	0	1	1	Reserved – 64b address routed		NPR	n	4	y	Block at NFM Egress – report no error
36-39	0	0	1	0	0	1	X	X			NPR	n	4	y	

#	Type								Description	Name	FC Type	Data Payload?	Header Base Size (DW)	New for Flit Mode	Translation Rule
	7	6	5	4	3	2	1	0							
40-43	0	0	1	0	1	0	X	X	Reserved – ID routed		CPL	n	4	y	Block at NFM Egress – Uncorrectable
44-45	0	0	1	0	1	1	0	X	Reserved – ID routed		PR	n	4	y	Block at NFM Egress – if UIO TLP report no error, else handle as Uncorrectable
46-47	0	0	1	0	1	1	1	X	Reserved – ID routed		PR	n	5	y	Block at NFM Egress – if UIO TLP report no error, else handle as Uncorrectable
48	0	0	1	1	0	0	0	0	Message w/o Data, Routed to Root Complex	Msg	PR	n	4	n	1:1
49	0	0	1	1	0	0	0	1	Message w/o Data, Routed by Address (64b) - NONE DEFINED	Msg	PR	n	4	n	1:1
50	0	0	1	1	0	0	1	0	Message w/o Data, Routed by ID	Msg	PR	n	4	n	1:1
51	0	0	1	1	0	0	1	1	Message w/o Data, Broadcast from Root Complex	Msg	PR	n	4	n	1:1
52	0	0	1	1	0	1	0	0	Message w/o Data, Local - Terminate at Receiver	Msg	PR	n	4	n	1:1
53	0	0	1	1	0	1	0	1	Message w/o Data, Gathered and routed to RC (PME_TO_Ack)	Msg	PR	n	4	n	1:1
54	0	0	1	1	0	1	1	0	Message w/o Data --	Msg	PR	n	4	n	N/A
55	0	0	1	1	0	1	1	1	RESERVED	Msg	PR	n	4	n	
56-59	0	0	1	1	1	0	X	X	Reserved – 64b address routed		NPR	n	4	y	Block at NFM Egress – report no error

#	Type								Description	Name	FC Type	Data Payload?	Header Base Size (DW)	New for Flit Mode	Translation Rule
	7	6	5	4	3	2	1	0							
60-61	0	0	1	1	1	1	0	X	Reserved – ID routed		NPR	n	4	y	
62-63	0	0	1	1	1	1	1	X	Reserved – ID routed		NPR	n	5	y	
64	0	1	0	0	0	0	0	0	Memory Write Request, 32b address routed	MWr	PR	Length	3	n	1:1
65	0	1	0	0	0	0	0	1	Reserved – ID routed		PR	Length	6	y	Block at NFM Egress – if UIO TLP report no error, else handle as Uncorrectable
66	0	1	0	0	0	0	1	0	I/O Write Request	IOWr	NPR	Length	3	n	1:1
67	0	1	0	0	0	0	1	1	Reserved – ID routed		PR	Length	6	y	Block at NFM Egress – if UIO TLP report no error, else handle as Uncorrectable
68	0	1	0	0	0	1	0	0	Type 0 Configuration Write Request	CfgWr0	NPR	Length	3	n	1:1
69	0	1	0	0	0	1	0	1	Type 1 Configuration Write Request	CfgWr1	NPR	Length	3	n	1:1
70	0	1	0	0	0	1	1	0	Reserved – ID routed		NPR	Length	3	y	Block at NFM Egress – report no error
71	0	1	0	0	0	1	1	1			NPR	Length	3	y	
72	0	1	0	0	1	0	0	0	UIO Read Completion with Data	UIORDCplD	CPL	Length	3	y	Block at NFM Egress – Uncorrectable
73	0	1	0	0	1	0	0	1	Reserved – ID routed		CPL	Length	3	y	Block at NFM Egress – Uncorrectable
74	0	1	0	0	1	0	1	0	Completion with Data	CplID	CPL	Length	3	n	1:1
75	0	1	0	0	1	0	1	1	Completion with Data, Locked	CplDLk	CPL	Length	3	n	1:1
76	0	1	0	0	1	1	0	0	Fetch and Add AtomicOp Request, 32b	FetchAdd	NPR	Length	3	n	1:1

#	Type								Description	Name	FC Type	Data Payload?	Header Base Size (DW)	New for Flit Mode	Translation Rule
	7	6	5	4	3	2	1	0							
									address routed						
77	0	1	0	0	1	1	0	1	Unconditional Swap AtomicOp Request, 32b address routed	Swap	NPR	Length	3	n	1:1
78	0	1	0	0	1	1	1	0	Compare and Swap AtomicOp Request, 32b address routed	CAS	NPR	Length	3	n	1:1
79	0	1	0	0	1	1	1	1	Reserved – 64b address routed		PR	n	4	y	Block at NFM Egress – if UIO TLP report no error, else handle as Uncorrectable
80-83	0	1	0	1	0	0	X	X	Reserved – 64b address routed		NPR	Length	6	y	Block at NFM Egress – report no error
84-85	0	1	0	1	0	1	0	X	Reserved – 64b address routed		NPR	Length	6	y	
86-87	0	1	0	1	0	1	1	X	Reserved – 64b address routed		NPR	Length	7	y	
88-89	0	1	0	1	1	0	0	X	Reserved – ID routed		PR	Length	3	y	Block at NFM Egress – if UIO TLP report no error, else handle as Uncorrectable
90	0	1	0	1	1	0	1	0	Reserved – 64b address routed		PR	Length	4	y	
91	0	1	0	1	1	0	1	1	Deferrable Memory Write Request, 32b address routed <i>{was: Trusted Configuration Write (deprecated)}</i>	DMWr	NPR	Length	3	n	1:1
92-93	0	1	0	1	1	1	0	X	Reserved – ID routed		PR	Length	4	y	Block at NFM Egress – if UIO TLP report no

#	Type								Description	Name	FC Type	Data Payload?	Header Base Size (DW)	New for Flit Mode	Translation Rule
	7	6	5	4	3	2	1	0							
94-95	0	1	0	1	1	1	1	X	Reserved – ID routed		PR	Length	5	y	error, else handle as Uncorrectable
96	0	1	1	0	0	0	0	0	Memory Write Request, 64b address routed	MWr	PR	Length	4	n	1:1
97	0	1	1	0	0	0	0	1	UIO Memory Write Request	UIOMWr	PR	Length	4	y	Block at NFM Egress – if UIO TLP report no error, else handle as Uncorrectable
98-99	0	1	1	0	0	0	1	X	Reserved – 64b address routed		PR	Length	4	y	
100-103	0	1	1	0	0	1	X	X			PR	Length	4	y	
104-107	0	1	1	0	1	0	X	X	Reserved – 64b address routed		PR	Length	4	y	Block at NFM Egress – if UIO TLP report no error, else handle as Uncorrectable
108	0	1	1	0	1	1	0	0	Fetch and Add AtomicOp Request, 64b address routed	FetchAdd	NPR	Length	4	n	1:1
109	0	1	1	0	1	1	0	1	Unconditional Swap AtomicOp Request, 64b address routed	Swap	NPR	Length	4	n	1:1
110	0	1	1	0	1	1	1	0	Compare and Swap AtomicOp Request, 64b address routed	CAS	NPR	Length	4	n	1:1
111	0	1	1	0	1	1	1	1	Reserved – 64b address routed		NPR	Length	4	y	Block at NFM Egress – report no error
112	0	1	1	1	0	0	0	0	Message with Data, Routed to Root Complex	MsgD	PR	Length	4	n	1:1
113	0	1	1	1	0	0	0	1	Message with Data, Routed by Address (64b) - NONE DEFINED	MsgD	PR	Length	4	n	1:1

#	Type								Description	Name	FC Type	Data Payload?	Header Base Size (DW)	New for Flit Mode	Translation Rule
	7	6	5	4	3	2	1	0							
114	0	1	1	1	0	0	1	0	Message with Data, Routed by ID	MsgD	PR	Length	4	n	1:1
115	0	1	1	1	0	0	1	1	Message with Data, Broadcast from Root Complex	MsgD	PR	Length	4	n	1:1
116	0	1	1	1	0	1	0	0	Message with Data, Local - Terminate at Receiver	MsgD	PR	Length	4	n	1:1
117	0	1	1	1	0	1	0	1	Message with Data, Gathered and routed to RC (MsgD NOT USED)	MsgD	PR	Length	4	n	1:1
118	0	1	1	1	0	1	1	0	Message with Data -- RESERVED	MsgD	PR	Length	4	n	N/A
119	0	1	1	1	0	1	1	1		MsgD	PR	Length	4	n	
120	0	1	1	1	1	0	0	0	Reserved – 64b address routed		NPR	Length	4	y	Block at NFM Egress – report no error
121	0	1	1	1	1	0	0	1			NPR	Length	4	y	
122	0	1	1	1	1	0	1	0			NPR	Length	4	y	
123	0	1	1	1	1	0	1	1	Deferrable Memory Write Request, 64b address routed	DMWr	NPR	Length	4	n	1:1
124-127	0	1	1	1	1	1	X	X	Reserved – 64b address routed		NPR	Length	4	y	Block at NFM Egress – report no error
128-135	1	0	0	0	0	X	X	X	Reserved – Local TLP Prefix – Terminate at Receiver		none	n	1	n	N/A
136-139	1	0	0	0	1	0	X	X	Reserved – Local TLP Prefix – Terminate at Receiver		none	n	1	n	N/A
140	1	0	0	0	1	1	0	0			none	n	1	n	

#	Type								Description	Name	FC Type	Data Payload?	Header Base Size (DW)	New for Flit Mode	Translation Rule
	7	6	5	4	3	2	1	0							
141	1	0	0	0	1	1	0	1	Flit Mode Local TLP Prefix	FlitModePrefix	none	n	1	n	N/A
142	1	0	0	0	1	1	1	0	1 DW Prefix - Vendor Defined Local 0	VendPrefixL0	none	n	1	n	N/A
143	1	0	0	0	1	1	1	1	1 DW Prefix - Vendor Defined Local 1	VendPrefixL1	none	n	1	n	N/A
144-147	1	0	0	1	0	0	X	X	Reserved – 64b address routed		PR	n	4	y	Strongly Recommended: Block at NFM Egress / Permitted: Terminate at FM Ingress Port. If UIO TLP report no error, else handle as Uncorrectable ¹⁰
148-151	1	0	0	1	0	1	X	X	Reserved – 64b address routed		PR	n	5	y	
152-155	1	0	0	1	1	0	X	X	Reserved – 64b address routed		PR	n	6	y	Strongly Recommended: Block at NFM Egress / Permitted: Terminate at FM Ingress Port. If UIO TLP report no error, else handle as Uncorrectable ¹¹
156-159	1	0	0	1	1	1	X	X	Reserved – 64b address routed		PR	n	7	y	
160-167	1	0	1	0	0	X	X	X	Reserved – 64b address routed		NPR	n	5	y	Block at NFM Egress – report no error
168-169	1	0	1	0	1	0	0	X	Reserved – ID routed		PR	n	6	y	Block at NFM Egress – if UIO TLP report no error, else handle as Uncorrectable
170-171	1	0	1	0	1	0	1	X	Reserved – ID routed		PR	n	7	y	Block at NFM Egress – if UIO TLP report no

10. TLP Translation Egress Blocked at Egress Port, or Unsupported Request at Ingress Port.

11. TLP Translation Egress Blocked at Egress Port, or Unsupported Request at Ingress Port.

#	Type								Description	Name	FC Type	Data Payload?	Header Base Size (DW)	New for Flit Mode	Translation Rule
	7	6	5	4	3	2	1	0							
															error, else handle as Uncorrectable
172-173	1	0	1	0	1	1	0	X	Reserved – ID routed		CPL	n	5	y	Block at NFM Egress – Uncorrectable
174-175	1	0	1	0	1	1	1	X	Reserved – ID routed		CPL	n	6	y	Block at NFM Egress – Uncorrectable
176-183	1	0	1	1	0	X	X	X	Reserved – 64b address routed		PR	Length	5	y	Block at NFM Egress – if UIO TLP report no error, else handle as Uncorrectable
184-191	1	0	1	1	1	X	X	X	Reserved – 64b address routed		PR	Length	5	y	Block at NFM Egress – if UIO TLP report no error, else handle as Uncorrectable
192-199	1	1	0	0	0	X	X	X	Reserved – 64b address routed		NPR	n	6	y	Block at NFM Egress – report no error
200-201	1	1	0	0	1	0	0	X	Reserved – ID routed		NPR	n	7	y	Block at NFM Egress – report no error
202-203	1	1	0	0	1	0	1	X	Reserved – ID routed		CPL	Length	5	y	Block at NFM Egress – Uncorrectable
204-205	1	1	0	0	1	1	0	X	Reserved – ID routed		CPL	Length	6	y	Block at NFM Egress – Uncorrectable
206-207	1	1	0	0	1	1	1	X	Reserved – ID routed		PR	Length	7	y	Block at NFM Egress – if UIO TLP report no error, else handle as Uncorrectable
208-215	1	1	0	1	0	X	X	X	Reserved – 64b address routed		PR	Length	6	y	Block at NFM Egress – if UIO TLP report no error, else handle as Uncorrectable
216-223	1	1	0	1	1	X	X	X	Reserved – 64b address routed		PR	Length	6	y	Block at NFM Egress – if UIO TLP report no

#	Type								Description	Name	FC Type	Data Payload?	Header Base Size (DW)	New for Flit Mode	Translation Rule
	7	6	5	4	3	2	1	0							
															error, else handle as Uncorrectable
224-225	1	1	1	0	0	0	0	X	Reserved – Local TLP – Terminate at Receiver		none	n	4	y	N/A
226-227	1	1	1	0	0	0	1	X	Reserved – Local TLP – Terminate at Receiver		none	n	6	y	N/A
228-229	1	1	1	0	0	1	0	X	Reserved – Local TLP – Terminate at Receiver		none	Length	4	y	N/A
230-231	1	1	1	0	0	1	1	X	Reserved – Local TLP – Terminate at Receiver		none	Length	6	y	N/A
232-239	1	1	1	0	1	X	X	X	Reserved – 64b address routed		NPR	n	7	y	Block at NFM Egress – report no error
240-241	1	1	1	1	0	0	0	X	Reserved – ID routed		NPR	Length	4	y	Block at NFM Egress – report no error
242-243	1	1	1	1	0	0	1	X	Reserved – ID routed		NPR	Length	5	y	Block at NFM Egress – report no error
244-245	1	1	1	1	0	1	0	X	Reserved – ID routed		NPR	Length	6	y	Block at NFM Egress – report no error
246-247	1	1	1	1	0	1	1	X	Reserved – ID routed		NPR	Length	7	y	Block at NFM Egress – report no error
248-255	1	1	1	1	1	X	X	X	Reserved – 64b address routed		PR	Length	7	y	Block at NFM Egress – if UIO TLP report no error, else handle as Uncorrectable

The TS[2:0] field indicates Trailer Size and is encoded as:

- 000b – No Trailer
- 001b – 1DW Trailer – Content Reserved if OHC-C is present and the Sub-Stream field indicates an IDE TLP; Else 1DW Trailer containing an ECRC¹².

- 010b – 1DW Trailer – Content Reserved
- 011b – 2DW Trailer – Content Reserved
- 100b – 2DW Trailer – Content Reserved
- 101b – 3DW Trailer with IDE MAC if an OHC-C is present and the Sub-Stream field indicates an IDE TLP; Else 3DW Trailer – Content Reserved
- 110b – 4DW Trailer with IDE MAC and PCRC if an OHC-C is present and the Sub-Stream field indicates an IDE TLP; Else 4DW Trailer – Content Reserved
- 111b – 5DW Trailer – Content Reserved

The definitions of the TC, Attr and Length fields in Flit Mode are the same as in Non-Flit Mode.

Bit 1 in byte 1 of Non-Flit Mode is now Reserved, but it was the LN bit associated with the now deprecated Lightweight Notification (LN) protocol. This bit is not supported in Flit Mode. Thus, it must be ignored when translating from Non-Flit Mode to Flit Mode, and it must be set to 0b when translating from Flit Mode to Non-Flit Mode.

The OHC[4:0] field indicates the presence of “Orthogonal Header Content” (OHC) encoded as:

- 0 0000b = No OHC present
- x xxx1b = OHC-Ax present
- x xx1xb = OHC-B present
- x x1xxb = OHC-C present
- 0 0xxxb = No OHC-E present
- 0 1xxxb = OHC-E1 present
- 1 0xxxb = OHC-E2 present
- 1 1xxxb = OHC-E4 present

When present, OHC must follow the Header Base. It is permitted for any combination of OHC content to be present, but, when present, must follow the Header Base, in A-B-C-E order. The contents of the OHC in some cases varies depending on the TLP type.

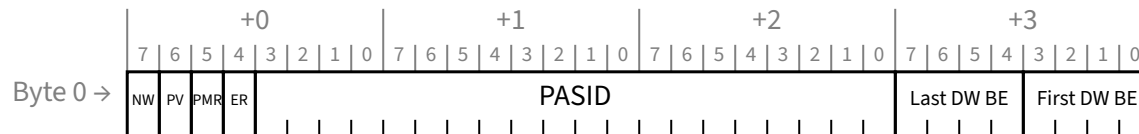
For specific TLP types, as defined in this specification, specific OHC content must be included by the Transmitter. Receivers must check for violations of these rules. If a Receiver determines that a Request violates a rule requiring specific OHC content, the Request must be handled as an Unsupported Request. If a Receiver determines that a Completion violates a rule requiring specific OHC content, the Completion must be handled as an Unexpected Completion.

*Table 2-6 **OHC-Ax** Included Fields for OHC-A1 through OHC-A5 (see § Figure 2-7 through § Figure 2-11) §*

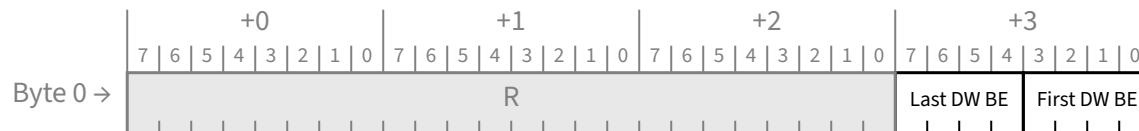
Name	Required for	Byte Enables	PASID, PV	ER, PMR	Destination Segment, DSV	Completer Segment	Completion Status	Lower Address[1:0]	NW Flag
<u>OHC-A1</u>	Memory Requests with explicit Byte Enables and/or PASID Address Routed Messages with PASID and Route to Root Complex Messages with PASID	Y	Y	Y					Y

12. Earlier versions of this specification defined this combination as always containing an ECRC, even for IDE TLPs. Receiver behavior is undefined for implementations compliant to those earlier specifications

Name	Required for	Byte Enables	PASID, PV	ER, PMR	Destination Segment, DSV	Completer Segment	Completion Status	Lower Address[1:0]	NW Flag
	Translation Requests								
<u>OHC-A2</u>	I/O Requests	Y							
<u>OHC-A3</u>	Configuration Requests	Y			Y				
<u>OHC-A4</u>	ID-Routed Messages that require Destination Segment and/or PASID		Y		Y				
<u>OHC-A5</u>	Completions when required as defined in § Section 2.2.9.2				Y	Y	Y	Y	
OHC-Ax	Others	When OHC-A is present on other TLPs, all OHC-A bits are Reserved							

Figure 2-7 **OHC-A1** §In OHC-A1

- The ER bit is Execute Requested, and the PMR bit is Privileged Mode Requested (see § Section 6.20). These bits are Reserved for all Requests other than Translation Requests (see § Section 10.2.2) and Page Requests (see § Section 10.4).
- When OHC-A1 is included with a TLP, if the PASID is not known or has not been assigned, then the PV ("PASID Valid") bit must be Clear.
- The ER and PMR bits are Reserved if PV is Clear.
- The PASID field is Reserved if PV is Clear.
- The NW bit is No Write. This bit is Reserved for all Requests other than Translation Requests.
- OHC-A1 is required as specified in § Section 2.2.5.2 .

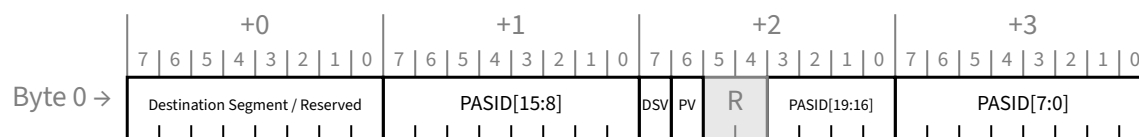
Figure 2-8 **OHC-A2** §In OHC-A2:

1. OHC-A2 is required as specified in § Section 2.2.5.2.

Figure 2-9 **OHC-A3** §

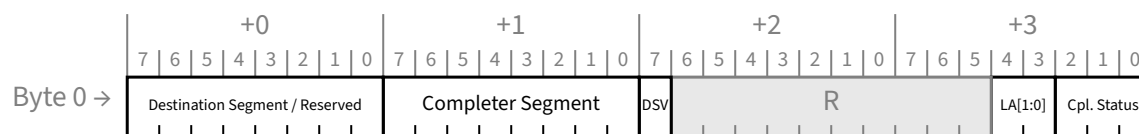
In OHC-A3:

- Destination Segment is Reserved if DSV is Clear.
- OHC-A3 is required as specified in § Section 2.2.5.2.

Figure 2-10 **OHC-A4** §

In OHC-A4:

- When OHC-A4 is included with a TLP, if the PASID is not known or has not been assigned, then the PV ("PASID Valid") bit must be Clear.
- The PASID field is Reserved if PV is Clear.
- The Destination Segment field is Reserved if DSV is Clear.
- OHC-A4 must be included in ID Routed Messages when Destination Segment or PASID is required.

Figure 2-11 **OHC-A5** §

In OHC-A5:

- LA[1:0] is Lower Address[1:0].
- The Destination Segment field is Reserved if DSV is Clear.
- OHC-A5 is required as specified in § Section 2.2.9.2.

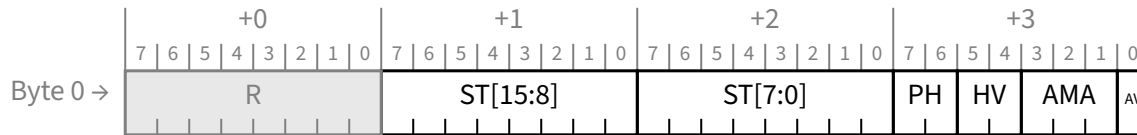


Figure 2-12 OHC-B §

In OHC-B:

- OHC-B is defined for Address Routed Requests only. When OHC-B is present on other TLPs, all OHC-B bits are Reserved.
- When TLP Processing Hints (TPH) are used OHC-B must be included with the appropriate PH and ST values.
- The PH and ST fields are qualified by the HV[1:0] ("Hints Valid") field, are encoded as:
 - 00b: PH[1:0], ST[15:0] are not valid and are Reserved
 - 01b: PH[1:0] and ST[7:0] are valid, ST[15:8] is not valid and is Reserved
 - 10b: Reserved encoding, Receivers must treat as 00b.
 - 11b: PH[1:0] and ST[15:0] are valid
- The AMA field is Reserved when AV is Clear.

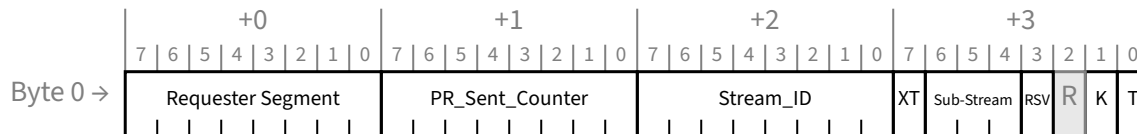


Figure 2-13 OHC-C §

For OHC-C:

- The Requester Segment field is Reserved when RSV is Clear.
- IDE TLPs must include OHC-C.
 - If Sub-Stream is 000b, 001b, or 010b, the PR_Sent_Counter, Stream_ID, K, XT, and T fields are meaningful (see § Section 6.33.5)
 - If Sub-Stream is 011b to 110b, Receiver behavior is undefined.
 - For IDE Completion TLPs, the RSV bit must be Clear.
- Non-IDE Request TLPs must, in some cases, also include OHC-C to indicate the Requester Segment (see Segment Rules below). When a non-IDE Completion TLP includes OHC-C, the RSV bit must be Clear.
 - Non-IDE TLPs with OHC-C are identified by the Sub Stream value of 111b. (see § Section 6.33.5)
 - If Sub-Stream is 111b, the PR_Sent_Counter, Stream_ID, K, XT, and T fields are Reserved.
- Note: OHC-C does not include the M and P bits present in the IDE TLP Prefix. In Flit Mode, the presence of a MAC/PCRC is indicated using the TS field.

Since IDE TLPs cannot be modified between the two Partner Ports, the IDE Partner Ports and the path between them must operate entirely in Non-Flit Mode or in Flit Mode. Root Complexes that support peer-to-peer and Switches cannot modify IDE TLPs associated with Flow-Through Selective IDE Streams, making TLP Translation impossible. If an IDE TLP is directed out an Egress Port operating in a different mode from the Ingress Port, the IDE TLP must be dropped, and the result must be reported as a Misrouted IDE TLP error.

It is permitted to configure a Root Complex or Switch such that the Ingress Port is a terminus for an IDE connection and the Egress Port another terminus, such that the TLP is passed through the RC/Switch unprotected by IDE. Doing this requires that the RC/Switch to be trusted, and requires the Root/Switch Ports to have the ability to act as an IDE Terminus, not simply to support Flow-Through IDE.

In Flit Mode, NOP TLPs must never be transmitted as IDE TLPs. Receivers are not required to check for violations of this rule, but, if checked, Receivers must handle NOP TLPs received as IDE TLPs as Malformed TLPs.

Segment Rules:

In Flit Mode, it is possible, and in some cases required, to include Segment fields in TLPs. One benefit of the Segment fields is to enable routing Route-by-ID TLPs between Hierarchies, which are, by definition, in different Segments. Root Complexes are the only place where peer-to-peer Requests will traverse from one Hierarchy to another. Peer-to-peer Route-by-ID Message Requests can traverse Hierarchies when the Requester includes a valid Destination Segment field. Memory Requests are address routed between Hierarchies, but the associated Completions are ID routed. To aid in Root Complex routing of Completions between Hierarchies, FM Completions can include the Destination Segment field which reflects the value of the Requester Segment field from the associated NP or UIO Memory Request. This allows a Root Complex to route Non-Posted or UIO Memory Requests between Hierarchies without the need to assume ownership of each outstanding transaction for the purpose of routing the associated Completions back to the Hierarchy of the original Requester. This can lead to performance improvements for peer-to-peer transfers between Hierarchies.

A second use of the Segment fields is to improve error logging. When FM TLP headers are logged in the AER Extended Capability structure the Segment fields will be included. The Segment fields improve traceability when identical Requester/Completer IDs exist in different Hierarchies. The rules in this section allow the Segment fields to be omitted in some cases to reduce FM TLP overhead. It should be noted that omitting the Segment fields in these cases could forfeit the improved error-logging traceability benefit. It is permitted to use implementation specific mechanisms to select when optional Segment fields are included (e.g., during debug) while still achieving optimal performance during normal operation by omitting non-required Segment fields.

These fields, which exist only in FM, are used to communicate Segment information:

- The Requester Segment field indicates the Hierarchy in which the Requester is located. This field exists in OHC-C and is sometimes included in Memory and Message Requests.
 - The Requester Segment Valid (RSV) bit, when Set, indicates that the Requester Segment field is valid.
 - When Requester Segment Valid (RSV) is Clear then the Requester Segment field is Reserved.
 - For TLPs with OHC-C that are not IDE TLPs, the Sub-Stream[2:0] field must be 111b, and the Stream_ID, PR_Sent_Counter, K, XT, and T fields/bits are Reserved.
 - In earlier versions of this specification, Sub-Stream was 4 bits in Symbol 3, bits 7:4. See § Section 6.33.5 for additional details.
 - IDE Requests (see § Section 6.33) other than Configuration Requests must include Requester Segment in OHC-C.
- The Completer Segment field indicates the Hierarchy in which the Completer is located. This field exists in OHC-A5 and is sometimes included in Completions.
- The Destination Segment field indicates the Hierarchy to which the TLP should be routed for ID Based Routing. In Configuration Write Requests this field is also used to configure the Segment of the completing Function. Configuration Requests in FM always include this field in OHC-A3 unless the Request had previously traversed a

NFM Link. Route-by-ID Message Requests sometimes include this field in OHC-A4. Completions sometimes include this field in OHC-A5.

- The Destination Segment Valid (DSV) bit, when Set, indicates that the Destination Segment field is valid.
- When Destination Segment Valid (DSV) is Clear then the Destination Segment field is Reserved.

In addition to the following rules that apply specifically to Root Complexes, Requesters and Completers within Root Complexes must also follow the rules later in this section that apply to Requesters and Completers.

- All Configuration Requests transmitted by a Root Port in Flit Mode, including those initiated through the SFI Configuration Access Method, must include OHC-A3 with the DSV bit set and a valid Destination Segment. The Destination Segment is necessary for the Completer to capture its Segment as described in § Section 2.2.6.2
 - The Root Complex must indicate the correct Segment value in the Destination Segment field, even if only one Segment is implemented.
 - By definition, a Root Port and its hierarchy domain share the same Segment number.
- Completions associated with Configuration Requests must be identifiable solely by Transaction ID when received at a RP. Such Completions will not include a Destination Segment field because Configuration Requests do not include a Requester Segment field.

IMPLEMENTATION NOTE: ROOT COMPLEX SUPPORT FOR PEER-TO-PEER NON-POSTED MEMORY TRANSACTIONS THAT TRAVERSE HIERARCHIES §

Since Segment fields aren't communicated across Links in NFM, Root Complexes take on an additional burden for peer-to-peer non-UIO NP Memory Requests that cross from one Hierarchy to another. With the loss of the Requester Segment field when a Request is translated to NFM, the Requester ID that remains in the original NP Memory Request might be indistinguishable from that of other Requesters within the hierarchy domain. Unless all Links along the path from the egress RP to the Completer are known to be in FM, Root Complexes must replace the Requester ID in peer-to-peer NP Memory Requests that cross from one Hierarchy to another. The Requester ID supplied by the Root Complex must be an ID associated with the Root Complex itself. This action is sometimes called "taking ownership" of the NP Request. It is necessary for the Root Complex to take ownership of such Requests to ensure that the Requester ID remains unique within the hierarchy domain of the egress RP, and that the associated Completions can be routed correctly by any Switches within that hierarchy domain. The egress RP also must track all such outstanding NP Memory Requests in order to route the associated Completion(s) to the Hierarchy of the original Requester within the Root Complex, as well as to restore the original Requester ID (Destination BDF/BF in FM) within the Completion(s).

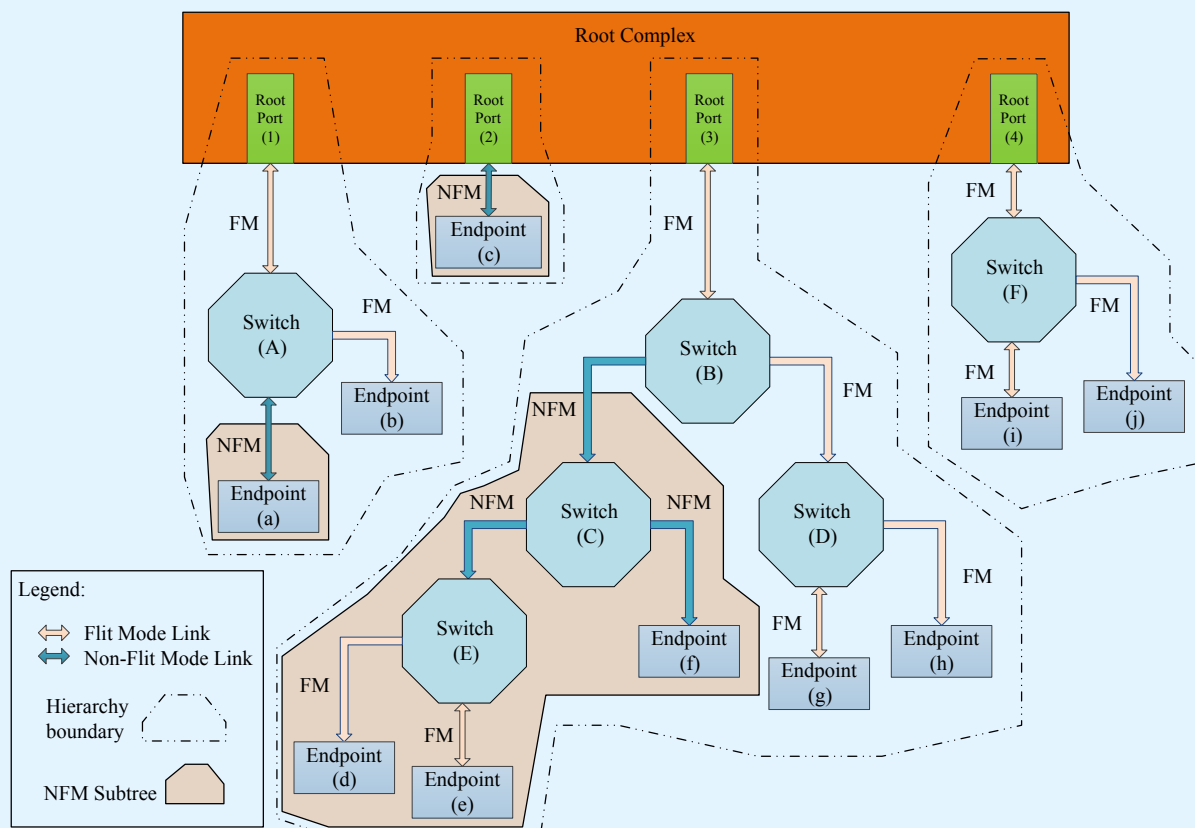


Figure 2-14 Example Topology Illustrating Multiple Segments and NFM Subtrees §

The No NFM Subtree Below This Root Port bit defaults to Clear to indicate that one or more NFM subtree(s) may exist below a Root Port. Referring to the example shown in § Figure 2-14, each Root Port is in a unique Segment/Hierarchy, and RP 1 through RP 3 have NFM subtrees below the Root Port. For RP 2, the Link immediately below the Root Port is in NFM, but for RP 1 and RP 3 the Root Port cannot directly determine the existence of a NFM subtree within its hierarchy domain, and so the default value of the No NFM Subtree Below This Root Port bit ensures that the Root Complex will take ownership of NP Requests egressing from those Root Ports. In all cases, it is necessary that system software ensure the No NFM Subtree Below This Root Port bit for a Root Port is Clear in cases where the Root Port has one or more NFM Links or subtrees below it.

Since RP 4 does not have any NFM Links below it, and therefore it is not necessary for the Root Complex to take ownership of NP Requests egressing that Root Port. It is strongly recommended that system software Set the No NFM Subtree Below This Root Port bit in such cases, and it is strongly recommended that Root Complex implementations use the value in the No NFM Subtree Below This Root Port bit to avoid taking ownership of NP Requests when it is not necessary to do so.

Note that for non-IDE Requests directed Upstream to the RC, the existence of a NFM Link between the original Requester and the Root Port is not a factor, because the RC inherently knows the Hierarchy of the Requester based on the Ingress RP of the Request, and can add the Requester Segment if needed.

Regardless of the value of the No NFM Subtree Below This Root Port bit, a Root Complex need not apply NP Memory Request tracking mechanisms for peer-to-peer Selective IDE Stream transactions that cross from one Hierarchy to another, and IDE TLPs cannot in any case be modified between the two IDE Partner Ports. When a Selective IDE Stream is established passing peer-to-peer between Hierarchies, software must ensure that the RC supports such routing, and that the entire path between the two Partner Ports is entirely in FM.

A NFM device could be hot-added into a subtree for which the No NFM Subtree Below This Root Port bit had previously been Set. In such cases it is necessary for system software to Clear the No NFM Subtree Below This Root Port bit prior to allowing the hot-added NFM device to act as a Completer for any NP Memory Request passing peer-to-peer through the RC.

- It is not permitted to configure a Selective IDE Stream passing peer-to-peer between different Hierarchies unless it is known that the RC supports flow-through IDE between the two Root Ports, and that all Links on the path between the two Partner Ports, including both the Root Ports, are in FM.
 - Root Complexes are not required to support Selective IDE Streams passing peer-to-peer through the RC.
 - If a condition exists that precludes the RC from passing an IDE TLP associated with a Selective IDE Stream configured to flow-through the RC, then the RC must treat the TLP as a Misrouted IDE TLP error at either the Ingress Port or the Egress Port.
- If a Message or Memory Request received at a RP includes a Requester Segment that does not match the Hierarchy associated with the receiving RP, the Request must be handled by the RP as an Unsupported Request.
- A RP is permitted to add a Requester Segment indication to a non-IDE Memory Write Request, or a non-IDE Route by Address Message Request, passing peer-to-peer through the RC if that TLP did not include a Requester Segment at the ingress RP, where the Requester Segment must correspond to the Segment of the Ingress RP.
- Route by ID Message Requests received at a RP without a Destination Segment, or received in NFM, are implied to be destined for a Completer within the same Hierarchy as the Ingress RP.
- When taking ownership of an NP or UIO Memory Request passing peer-to-peer through the RC:
 - The Requester ID in the Request must be replaced with one associated with the Root Complex.

- The Request must either use the Requester Segment value associated with the hierarchy domain of the Egress RP, or must not include a Requester Segment.
- The RC is permitted to replace the Tag in the Request, and when doing so must ensure that the Transaction ID satisfies uniqueness requirements for Requests associated with the same Requester ID used for taking ownership.
 - For non-UIO Requests, the RC is permitted to change the size of the Tag. If this is done, it is permitted to use implementation specific means to determine what size of Tag is appropriate.
 - The Tag in the Completion(s) must be restored to the Tag from the original Request, as received at the Ingress RP, before returning those Completion(s).
- Completions associated with the Request must be identifiable solely by Transaction ID when received at the RP. Such Completions will not include a Destination Segment if the RP did not include a Requester Segment in the Request or if a NFM Link exists between the RP and the Completer.
- The Requester ID value in the Completion(s) must be restored to the Requester ID from the original Request, as received at the Ingress RP, before returning those Completion(s).
- If the RP that received the Request is in FM and OHC-A5 is returned to the Requester with the Completion(s):
 - The Destination Segment must be set to 00h and the DSV bit must be Clear in OHC-A5 that is returned to the original Requester.
 - The Completer Segment field must not be modified if the RP receiving the Completion is in FM and OHC-A5 was received with the Completion. The Completer Segment in OHC-A5 returned to the Requester must be set to 00h if the RP receiving the Completion is in NFM or if OHC-A5 was not received with the Completion.
- When passing an NP or UIO Memory Request peer-to-peer through the RC without taking ownership:
 - The Requester ID and Tag in the Request must not be modified.
 - For non-IDE NP Memory Requests passing peer-to-peer through the RC that do not include a Requester Segment at the Ingress RP, the RC must add a Requester Segment indication at the Egress RP, using the Segment value associated with the Ingress RP.
 - Any Completion received with the DSV bit set and a Destination Segment not matching the value associated with the hierarchy domain of the receiving RP must be routed through the RC to the specified Hierarchy.
 - The Requester ID and Tag fields returned to the Requester must not be modified from the values received with the Completion in the destination hierarchy domain.
 - If the RP that received the Request is in FM and OHC-A5 is returned to the Requester with the Completion(s) the DSV bit, Destination Segment, and Completer Segment fields must not be modified from the values received with OHC-A5 in the destination hierarchy domain.

RP Segment Exceptions – There are specific cases where a RP is not required to include Segment information:

- A RP is not required to include the Requester Segment field in any non-IDE Memory Request initiated by a Requester within the Root Complex.
- A RP is not required to include a Requester Segment field with Memory Write Requests passing peer-to-peer through the RC.
- A RP is not required to include a Requester Segment field with NP Memory Requests passing peer-to-peer through the RC if the Egress RP is taking ownership of the Request.
- A RP is not required to include the Completer Segment or Destination Segment fields in Completions associated with NP Memory Requests targeting system memory or another element of the Root Complex itself. OHC-A5 must be included if required as described in § Section 2.2.9.2 .

- A RP is not required to include the Completer Segment or Destination Segment fields in Completions associated with NP Memory Requests passing peer-to-peer through the RC. OHC-A5 must be included if required as described in § Section 2.2.9.2.

Each Switch exists entirely within a single Hierarchy by definition. However, Switches are required to comprehend Segment fields in some TLP types for routing purposes. The following rules apply to Switches:

- For TLPs in FM for both the Ingress and Egress Ports, Switches must never modify, add, or remove any Segment field or the DSV/RSV bit(s) within the TLP.
- For Configuration Requests initiated in FM through the SFI Configuration Access Method on a Switch Downstream Port, the Destination Segment and DSV fields must reflect the values received in the associated Configuration Write or Read Request to the SFI CAM Data Register.
- A Switch for which the Segment Captured bit is Set must handle as a TLP Translation Egress Blocked error an NP Memory Request received at the Upstream Port destined for a Downstream Port in NFM that includes a Requester Segment that does not match the Switch's captured Segment value.
 - The Request must not be forwarded to the Downstream Port.
- If a condition exists that precludes the Switch from passing an IDE TLP associated with a Selective IDE Stream configured to flow-through the Switch without modification, then the Switch must handle the TLP as a Misrouted IDE TLP error at either the Ingress Port or the Egress Port.
- When a Switch must translate a TLP from NFM to FM:
 - If the Segment Captured bit is Clear, OHC-C must not be added to a Request.
 - If the Segment Captured bit is Set, the Switch is permitted to add OHC-C to Memory and Message Requests with the Requester Segment containing the value established when the Switch itself was configured.
 - OHC-C must not be added to Configuration Requests.
 - If any OHC-A with DSV and Destination Segment fields is added, the DSV bit must be Clear and the Destination Segment must be 00h.
 - For a Completion that requires OHC-A5 (see § Section 2.2.9.2),
 - if the Segment Captured bit is Set, then the Switch must set the Completer Segment field to the Segment value established when the Switch itself was configured,
 - if the Segment Captured bit is Clear, then the Switch must set the Completer Segment field to 00h.
- Switches must route Configuration Requests solely by the BDF fields (Destination BDF/BF in FM); the Destination Segment field must not be considered for routing.
- A Switch for which the Segment Captured bit is Set must route Completions and Route by ID Message Requests Upstream if DSV is Set and the Destination Segment does not match the Switch's captured Segment value.
- Completions and Route by ID Message Requests must be routed solely by Requester ID / Destination BDF / Destination Device ID if the Ingress Port is in NFM, a Destination Segment is not included (DSV bit is clear), or the included Destination Segment matches the Switch's captured Segment value.
- A Switch for which the Segment Captured bit is Clear must signal a TLP Translation Egress Blocked error if a Completion or Route by ID Message Request is received with DSV Set, and the TLP must not be forwarded.
- A Switch for which the Segment Captured bit is Clear must signal a TLP Translation Egress Blocked error if a received Message or Memory Request includes a Requester Segment. The Request must not be forwarded.
- A Switch for which the Segment Captured bit is Set must signal a TLP Translation Egress Blocked error if a Message or Memory Request received on a Downstream Port includes a Requester Segment that does not match the Switch's captured Segment value. The Request must not be forwarded.

- When reordering Completions with other Completions, Switches are permitted to consider Destination Segment fields included in the Completions as effectively part of the Transaction ID. When not included, the Destination Segment is implied to be the same Segment where the Completion exists.
- When reordering TLPs based on ID Based Ordering (IDO), Switches must consider Requester Segment fields included in Requests, and Destination Segment fields included in Completions, as effectively part of the Transaction ID. When the Destination Segment is not included, for reordering purposes the Destination Segment must be considered to be the same Segment where the Completion exists. When the Requester Segment is not included in a Request, Switches must assume a matching value for IDO purposes.

The following rules apply to Requesters:

- When the Requester Segment field is included in a Request it must be set to the value captured from a Configuration Write Request as described in § Section 2.2.6.2 .
- When the Segment Captured bit is Clear all non-IDE Message and Memory Requests initiated by the Requester must not include OHC-C.
- When the Segment Captured bit is Set all Message Requests initiated by the Requester must include OHC-C with Requester Segment.
- When the Segment Captured bit is Set a Requester is permitted to include OHC-C with Requester Segment in Memory Requests.
- When the Segment Captured bit is Clear, Route by ID Message Requests initiated by the Requester must not include a Destination Segment (the DSV bit must be Clear).
- When the Segment Captured bit is Set a Requester is required to include a Destination Segment, and set the DSV bit, in Route by ID Message Requests destined for a different Hierarchy. Requesters use implementation specific means to determine the Hierarchy to which a Route by ID Message Request should be routed. When the Segment Captured bit is Set the Destination Segment is required in ATS Invalidate Request, Invalidate Completion, and PRG Response Messages even if the target is in the same Hierarchy. For other Route by ID Message Requests the Destination Segment is optional when the Segment Captured bit is Set and the Requester knows, by definition or through programming, that the target of the Request is in the same Hierarchy.
- Requesters must accept any value in the Destination Segment field (if present) in received Completions.
- A Requester is not required to include the Requester Segment field in any non-IDE Memory Request.

The following rules apply to Completers:

- Completers must capture their Segment value from Configuration Write Requests as described in § Section 2.2.6.2 .
- When the Segment Captured bit is Clear, Completers must set the Completer Segment field to 00h in any OHC-A5 that is included in a Completion.
- When the Segment Captured bit is Set, Completers must set the Completer Segment field in any OHC-A5 that is included in a Completion to the Segment value that was captured as described in § Section 2.2.6.2 .
 - If the Completion associated with the first Configuration Write Request includes OHC-A5, the Completer Segment field must be set to the value captured from that Request.
- Completers must clear the DSV bit and set the Destination Segment field to 00h in any OHC-A5 that is included with a Completion associated with a Configuration Request.
- For an NP or UIO Memory Request received without a Requester Segment field, Completers must clear the DSV bit and set the Destination Segment field to 00h in any OHC-A5 that is included with the associated Completion(s).
- For an NP or UIO Memory Request received with a Requester Segment field, Completers must set the DSV bit and set the Destination Segment field to the value of the received Requester Segment in any OHC-A5 that is

included with the associated Completion(s). See RP Segment Exceptions for cases where RPs are not required to include Segment information.

- When the Segment Captured bit is Set and an NP Memory Request is received with a Requester Segment value not matching the Completer's captured Segment value, all Completions associated with the Request must include OHC-A5.
- Completers must not qualify the acceptance of a Route by ID Message Request based on the value of the Destination Segment field in the Request.
- Completers must include OHC-A5 with a Completion if required as described in § Section 2.2.9.2 and § Section 6.33.4.
- Completers must not include OHC-A5 with a Completion when all of the following are true:
 - Completion Status is Successful Completion.
 - Lower Address[1:0] equal to 00b.
 - The Completer's Segment Captured bit is Clear.
- Completers are permitted to not include OHC-A5 with a Completion when all of the following are true:
 - Completion Status is Successful Completion.
 - Lower Address[1:0] equal to 00b.
 - The Completer's Segment Captured bit is Set.
 - The associated Request either did not include a Requester Segment field or included a Requester Segment field matching the Completer's captured Segment value.
 - The TEE-IO Supported bit is Clear or Completion is not on a Selective IDE Stream. See § Section 6.33.4.

2.2.2 TLPs with Data Payloads - Rules §

- Length is specified as an integral number of DW
- Length[9:0] is Reserved for all Messages except those that explicitly refer to a data length
 - Refer to the Message Code tables in § Section 2.2.8.
- A Function transmitting a TLP with a data payload must not allow the data payload length as indicated by the TLP's Length field to exceed the Function's applicable MPS setting. If the Function's Mixed_MPS_Supported bit is Clear or the target is host memory, the applicable MPS setting must be the Function's computed Tx_MPS_Limit, as defined below. If the Mixed_MPS_Supported bit is Set, the Function must have an implementation specific mechanism capable of supporting different MPS settings for different targets, and must handle both Request and Completion TLPs. Target-specific MPS settings are permitted to be above or below the Function's Tx_MPS_Limit, but they must never exceed the Function's Max_Payload_Size Supported field value. The Function's Tx_MPS_Limit is determined as follows:
 - For a Single-Function Device, the Tx_MPS_Limit must be its Max_Payload_Size field value, its "MPS setting".
 - Otherwise, for an ARI Device, the Tx_MPS_Limit must be the MPS setting in Function 0. The MPS settings in other Functions of an MFD must be ignored.
 - Otherwise, for a Function in a non-ARI MFD whose MPS settings are identical across all Functions, the Tx_MPS_Limit must be the common MPS setting.
 - Otherwise, for a Function in a non-ARI MFD whose MPS settings are not identical across all Functions, the Tx_MPS_Limit must be the MPS setting in an implementation specific Function.
 - Transmitter implementations are encouraged to use the MPS setting from the Function that generated the transaction, or else the smallest MPS setting across all Functions.

- Software should not configure the MPS setting in different Functions to different values unless software is aware of the specific implementation.
- MPS settings apply only to TLPs with data payloads; Memory Read Requests are not restricted in length by MPS settings. The size of the Memory Read Request is controlled by the TLP's Length field.
- The data payload size in a Received TLP as indicated by the TLP's Length field must not exceed a computed Rx_MPS_Limit for the receiving Function, as determined by MPS-related parameters as indicated below.
 - Receivers must check for violations of this rule. If a Receiver determines that a TLP violates this rule, the TLP must be handled as a Malformed TLP.
 - This is a reported error associated with the Receiving Port (see § Section 6.2).
 - In Flit Mode, Receivers must handle the full range of the Length field for the purpose of determining the total size of each TLP and deciding which symbol is the start of the next TLP.
 - In the receiving Function, if the Rx_MPS_Fixed bit is Set, the Rx_MPS_Limit must be the Max_Payload_Size Supported field. Otherwise, the Rx_MPS_Limit must be determined by the Max_Payload_Size field (the "MPS setting") in one or more Functions as follows:
 - For a Single-Function Device, the Rx_MPS_Limit must be its MPS setting.
 - Otherwise, for an ARI Device, the Rx_MPS_Limit must be the MPS setting in Function 0. MPS settings in other Functions must be ignored.
 - Otherwise, for an Upstream Port associated with a non-ARI MFD whose MPS settings are identical across all Functions, the Rx_MPS_Limit must be the common MPS setting.
 - Otherwise, for an Upstream Port associated with a non-ARI MFD whose MPS settings are not identical across all Functions, the Rx_MPS_Limit must be the MPS setting in an implementation specific Function.
 - Receiver implementations are encouraged to use the MPS setting from the Function targeted by the transaction, or else the largest MPS setting across all Functions.
 - Software should not configure the MPS setting in different Functions to different values unless software is aware of the specific implementation.
- For TLPs that include data, the value in the Length field and the actual amount of data included in the TLP must match.
 - In NFM, Receivers must check for violations of this rule. If a Receiver determines that a TLP violates this rule, the TLP is a Malformed TLP.
 - This is a Reported Error associated with the Receiving Port (see § Section 6.2).
- The value in the Length field applies only to data - the TLP Digest is not included in the Length.
- When a data payload associated with a byte address is included in a TLP other than an AtomicOp Request or an AtomicOp Completion, the first byte of data following the header corresponds to the byte address closest to zero and the succeeding bytes are in increasing byte address sequence.
 - Example: For a 16-byte write to location 100h, the first byte following the header would be the byte to be written to location 100h, and the second byte would be written to location 101h, and so on, with the final byte written to location 10Fh.
- The data payload in AtomicOp Requests and AtomicOp Completions must be formatted such that the first byte of data following the TLP header is the least significant byte of the first data value, and subsequent bytes of data are strictly increasing in significance. With Compare And Swap (CAS) Requests, the second data value immediately follows the first data value, and must be in the same format.
 - The endian format used by AtomicOp Completers to read and write data at the target location is implementation specific, and is permitted to be whatever the Completer determines is appropriate for the target memory (e.g., little endian, big endian, etc.). Endian format capability reporting and controls for AtomicOp Completers are outside the scope of this specification.

- Little endian example: For a 64-bit (8-byte) Swap Request targeting location 100h with the target memory in little endian format, the first byte following the header is written to location 100h, the second byte is written to location 101h, and so on, with the final byte written to location 107h. Note that before performing the writes, the Completer first reads the target memory locations so it can return the original value in the Completion. The byte address correspondence to the data in the Completion is identical to that in the Request.
- Big endian example: For a 64-bit (8-byte) Swap Request targeting location 100h with the target memory in big endian format, the first byte following the header is written to location 107h, the second byte is written to location 106h, and so on, with the final byte written to location 100h. Note that before performing the writes, the Completer first reads the target memory locations so it can return the original value in the Completion. The byte address correspondence to the data in the Completion is identical to that in the Request.
- § Figure 2-15 shows little endian and big endian examples of Completer target memory access for a 64-bit (8-byte) FetchAdd. The bytes in the operands and results are numbered 0-7, with byte 0 being least significant and byte 7 being most significant. In each case, the Completer fetches the target memory operand using the appropriate endian format. Next, AtomicOp compute logic in the Completer performs the FetchAdd operation using the original target memory value and the “add” value from the FetchAdd Request. Finally, the Completer stores the FetchAdd result back to target memory using the same endian format used for the fetch.

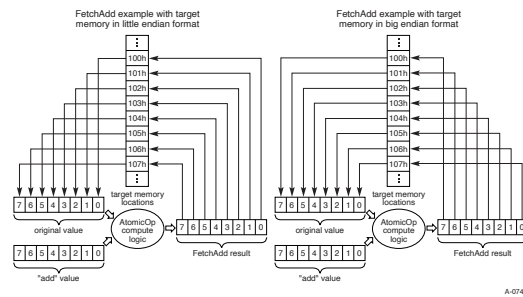


Figure 2-15 Examples of Completer Target Memory Access for FetchAdd §

IMPLEMENTATION NOTE:

ENDIAN FORMAT SUPPORT BY RC ATOMICOP COMPLETERS §

One key reason for permitting an AtomicOp Completer to access target memory using an endian format of its choice is so that PCI Express devices targeting host memory with AtomicOps can interoperate with host software that uses atomic operation instructions (or instruction sequences). Some host environments have limited endian format support with atomic operations, and by supporting the “right” endian format(s), an RC AtomicOp Completer may significantly improve interoperability.

For an RC with AtomicOp Completer capability on a platform supporting little-endian-only processors, there is little envisioned benefit for the RC AtomicOp Completer to support any endian format other than little endian. For an RC with AtomicOp Completer capability on a platform supporting bi-endian processors, there may be benefit in supporting both big endian and little endian formats, and perhaps having the endian format configurable for different regions of host memory.

There is no PCI Express requirement that an RC AtomicOp Completer support the host processor's “native” format (if there is one), nor is there necessarily significant benefit to doing so. For example, some processors can use load-link/store-conditional or similar instruction sequences to do atomic operations in non-native endian formats and thus not need the RC AtomicOp Completer to support alternative endian formats.

IMPLEMENTATION NOTE:

MAINTAINING ALIGNMENT IN DATA PAYLOADS §

§ Section 2.3.1.1 discusses rules for forming Read Completions respecting certain natural address boundaries. Memory Write performance can be significantly improved by respecting similar address boundaries in the formation of the Write Request. Specifically, forming Write Requests such that natural address boundaries of 64 or 128 bytes are respected will help to improve system performance.

2.2.3 TLP Digest Rules - Non-Flit Mode Only §

- For any TLP, a value of 1b in the TD bit indicates the presence of the TLP Digest field including an end-to-end CRC (ECRC) value at the end of the TLP.
 - A TLP where the TD bit value does not correspond with the observed size (accounting for the data payload, if present) is a Malformed TLP.
 - This is a reported error associated with the Receiving Port (see § Section 6.2).
- If an intermediate or ultimate PCI Express Receiver of the TLP does not support ECRC checking, the Receiver must ignore the TLP Digest¹³.
 - If the Receiver of the TLP supports ECRC checking, the Receiver interprets the value in the TLP Digest field as an ECRC value, according to the rules in § Section 2.7.1.

13. An exception is an Intermediate Receiver forwarding a Multicast TLP out an Egress Port with MC_Overlay enabled. See § Section 6.14.5.

2.2.4 Routing and Addressing Rules §

There are three principal mechanisms for TLP routing: address, ID, and implicit. This section defines the rules for the address and ID routing mechanisms. Implicit routing is used only with Message Requests, and is covered in § Section 2.2.8.

2.2.4.1 Address Based Routing Rules §

- Address routing is used with Memory, I/O Requests and Address Routed Messages.
- in NFM, two address formats are specified:
 - a 32-bit format with a 3 DW header (see § Figure 2-16)
 - a 64-bit format with a 4 DW header (see § Figure 2-17)
- In FM, five address formats are specified:
 - a 32-bit format with a 3 DW header (see § Figure 2-18)
 - a 64-bit format with a 4 DW header (see § Figure 2-19)
 - a 64-bit format with a 5 DW header (see § Figure 2-20)
 - a 64-bit format with a 6 DW header (see § Figure 2-21)
 - a 64-bit format with a 7 DW header (see § Figure 2-22)

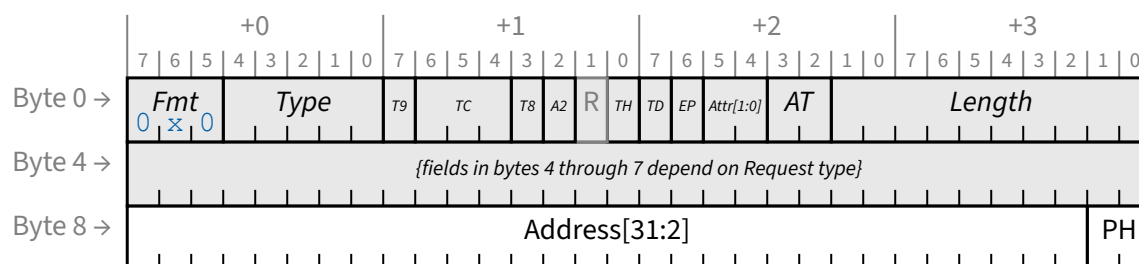


Figure 2-16 32-bit Address Routing - Non-Flit Mode §

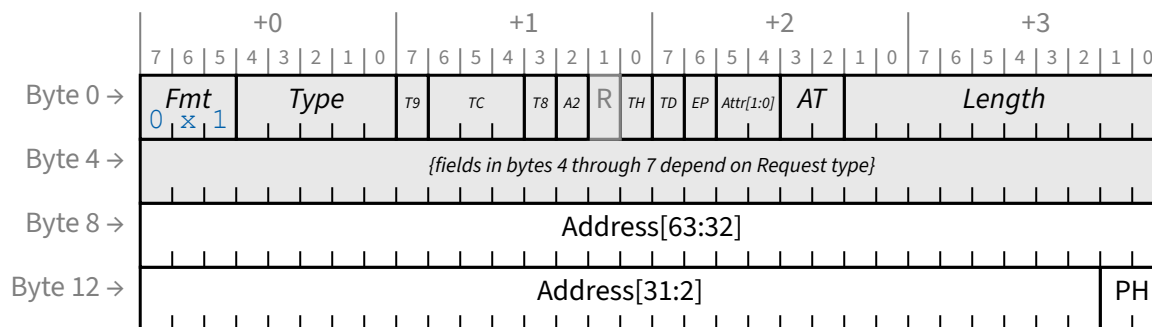


Figure 2-17 64-bit Address Routing - Non-Flit Mode §

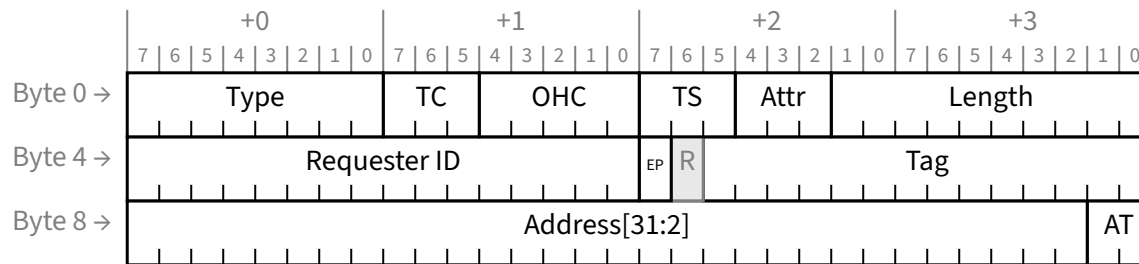


Figure 2-18 32-bit Address Routing - Flit Mode §

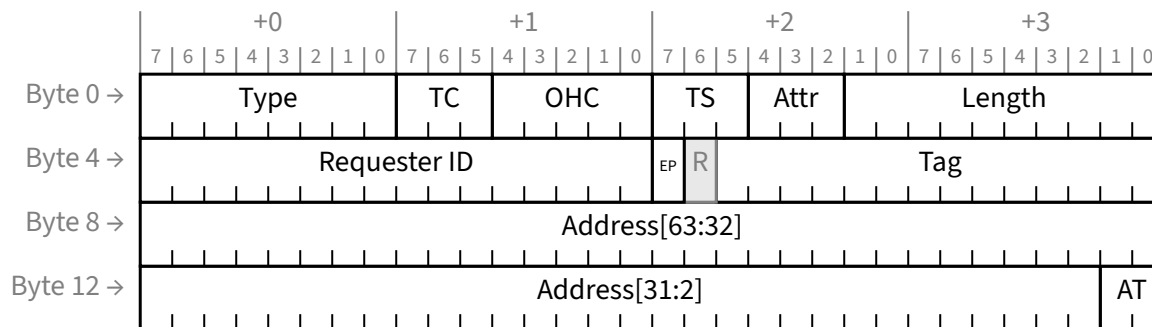


Figure 2-19 64-bit Address Routing - Flit Mode - 4 DW §

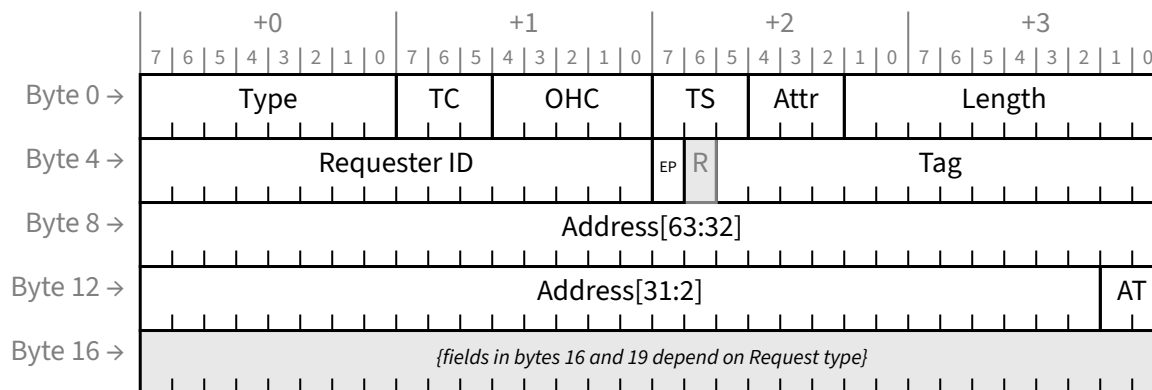


Figure 2-20 64-bit Address Routing - Flit Mode - 5 DW §

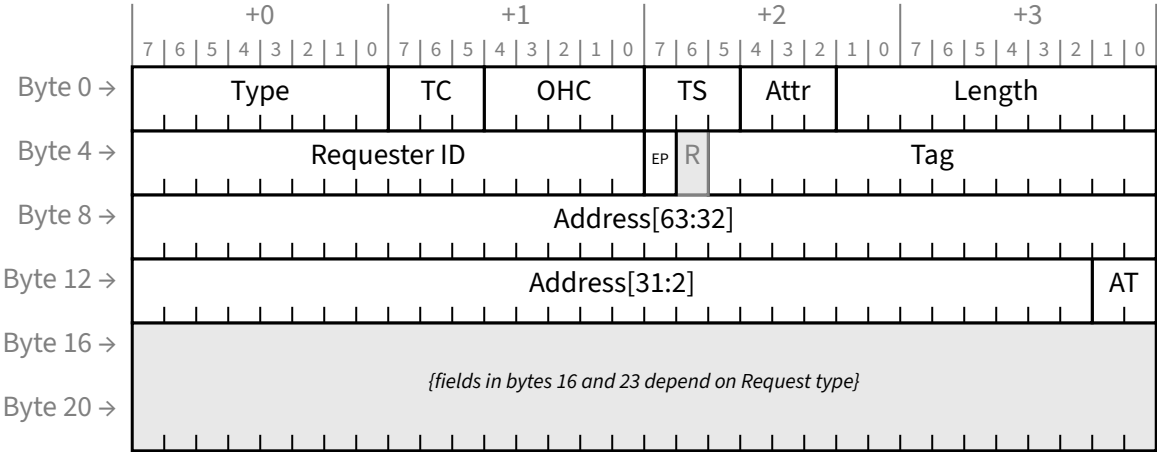


Figure 2-21 64-bit Address Routing - Flit Mode - 6 DW §

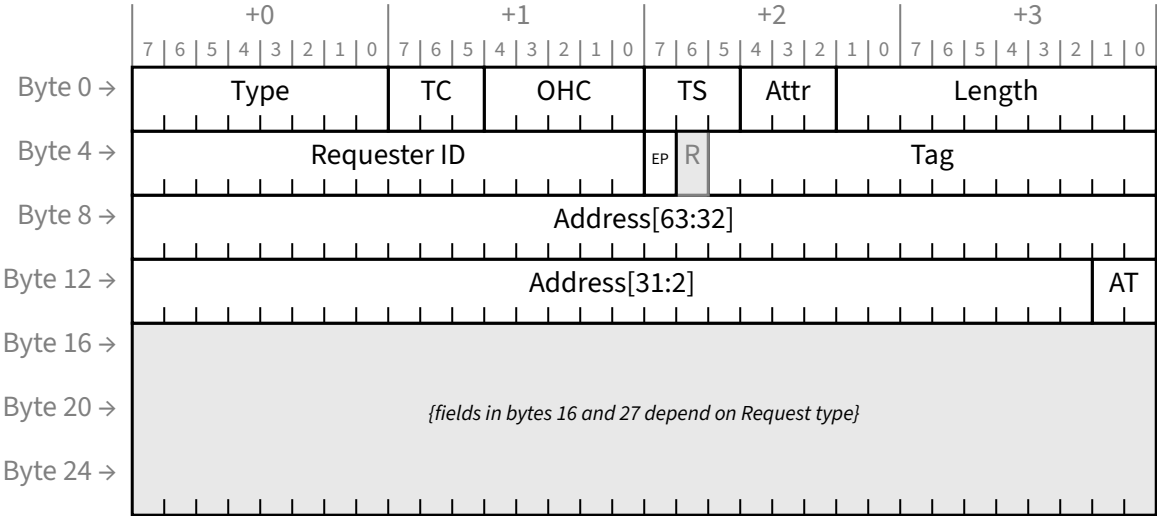


Figure 2-22 64-bit Address Routing - Flit Mode - 7 DW §

- For Memory Read, Memory Write, DMWr, and AtomicOp Requests, the Address Type (AT) field is encoded as shown in § Table 10-1. For Address Routed Messages in Flit Mode, the Address Type (AT) field is encoded as shown in § Table 10-1 with the exception that the value of 01b is reserved. For all other Requests, the AT field is Reserved unless explicitly stated otherwise.
- If TH is Set, the PH field is encoded as shown in § Table 2-18. If TH is Clear, the PH field is Reserved.
- Address mapping to the TLP header is shown in § Table 2-7.

Table 2-7 Address Field Mapping §

Address Bits	32-bit Addressing	64-bit Addressing
63:56	Not Applicable	Bits 7:0 of Byte 8

Address Bits	32-bit Addressing	64-bit Addressing
55:48	Not Applicable	Bits 7:0 of Byte 9
47:40	Not Applicable	Bits 7:0 of Byte 10
39:32	Not Applicable	Bits 7:0 of Byte 11
31:24	Bits 7:0 of Byte 8	Bits 7:0 of Byte 12
23:16	Bits 7:0 of Byte 9	Bits 7:0 of Byte 13
15:8	Bits 7:0 of Byte 10	Bits 7:0 of Byte 14
7:2	Bits 7:2 of Byte 11	Bits 7:2 of Byte 15

- The following address routed Requests must use 64-bit addressing (when addressing below 4 GB the upper 32 address bits must be to 0000 0000h):
 - All Address Routed UIO Requests
 - IDE TLPs with partial header encryption
 - This *MUST@FLIT* include Address Routed Messages ¹⁴.
- Except when explicitly required otherwise, non-UIO Memory Read, Memory Write, DMW_r, and AtomicOp Requests use both formats.
 - For Addresses below 4 GB, Requesters must use the 32-bit format. The behavior of the Receiver is not specified if a 64-bit format Request addressing below 4 GB (i.e., with the upper 32 bits of address all 0) is received.
- I/O Read Requests and I/O Write Requests use the 32-bit format.
- All agents must decode all address bits in the header - address aliasing is not allowed.

IMPLEMENTATION NOTE: PREVENTION OF ADDRESS ALIASING §

For correct software operation, full address decoding is required even in systems where it may be known to the system hardware architect/designer that fewer than 64 bits of address are actually meaningful in the system.

2.2.4.2 ID Based Routing Rules §

- ID routing is used with Configuration Requests, with ID Routed Messages, and with Completions. This specification defines several Messages that are ID Routed (see § Table F-1). Other specifications are permitted to define additional ID Routed Messages.
- ID routing uses the Bus, Device, and Function Numbers (as applicable) to specify the destination for the TLP:
 - For non-ARI Routing IDs, Bus, Device, and (3-bit) Function Number to TLP header mapping is shown in § Table 2-8.
 - For ARI Routing IDs, the Bus and (8-bit) Function Number to TLP header mapping is shown in § Table 2-9.

14. Earlier versions of this specification did not specify Address Routed Message behavior when the address was below 4 GB.

- In FM, Completions and ID Routed Messages with a different destination Hierarchy than the Hierarchy in which they originate must be routed to the destination Hierarchy using the Destination Segment field and then routed within the destination Hierarchy by the destination Bus, Device, and Function Numbers.
- In NFM, two ID routing formats are specified, one used with a 4 DW header (see § Figure 2-23 and § Figure 2-24) and one used with a 3 DW header (see § Figure 2-25 and § Figure 2-26).
 - Header field locations are the same for both formats (see § Table 2-8 and § Table 2-9).
- In FM, five ID routing formats are specified:
 - One with a 3 DW header (see § Figure 2-27)
 - One with a 4 DW header (see § Figure 2-28)
 - One with a 5 DW Header (see § Figure 2-29)
 - One with a 6 DW Header (see § Figure 2-30)
 - One with a 7 DW Header (see § Figure 2-31)

Table 2-8 Header Field Locations for non-ARI ID Routing - Non-Flit Mode §

Field	Header Location
Bus Number[7:0]	Bits 7:0 of Byte 8
Device Number[4:0]	Bits 7:3 of Byte 9
Function Number[2:0]	Bits 2:0 of Byte 9

Table 2-9 Header Field Locations for ARI ID Routing - Non-Flit Mode §

Field	Header Location
Bus Number[7:0]	Bits 7:0 of Byte 8
Function Number[7:0]	Bits 7:0 of Byte 9

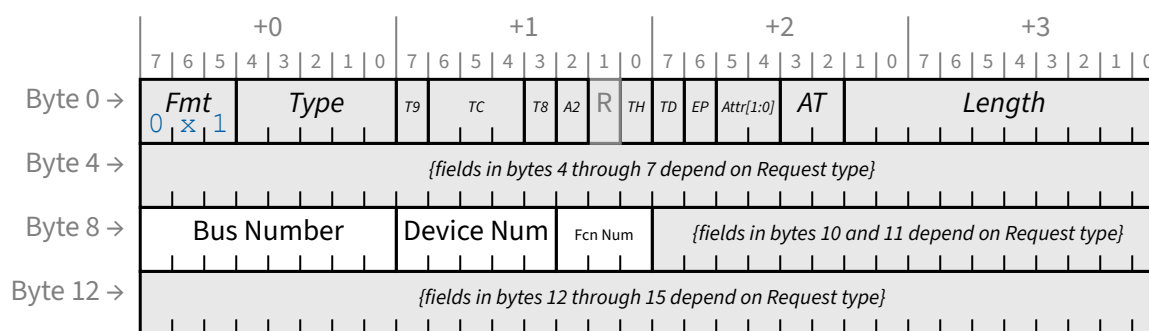


Figure 2-23 Non-ARI ID Routing with 4 DW Header - Non-Flit Mode §

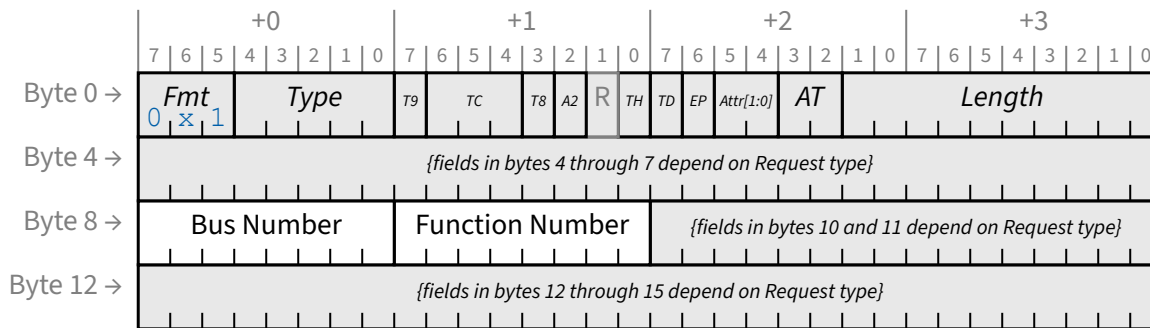


Figure 2-24 ARI ID Routing with 4 DW Header - Non-Flit Mode §

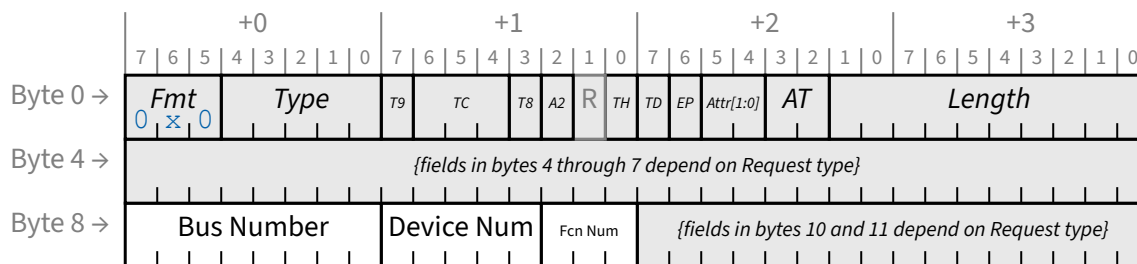


Figure 2-25 Non-ARI ID Routing with 3 DW Header - Non-Flit Mode §

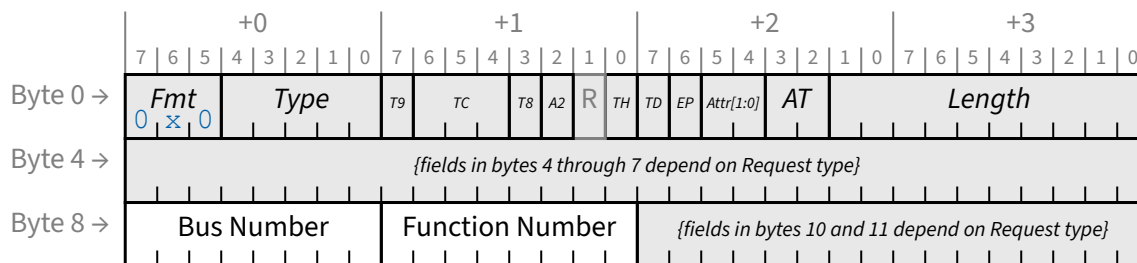


Figure 2-26 ARI ID Routing with 3 DW Header - Non-Flit Mode §

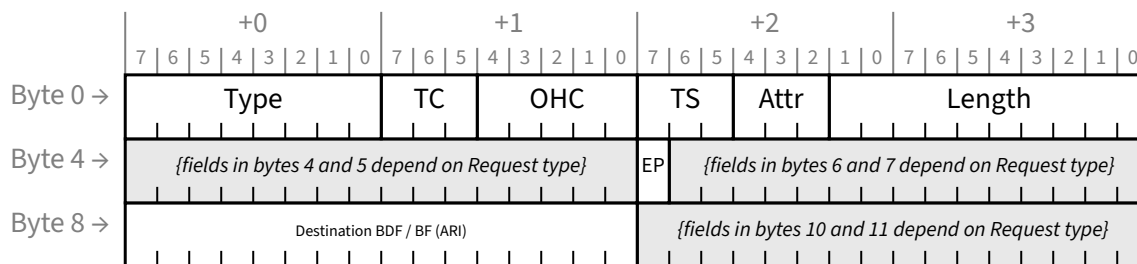


Figure 2-27 ID Routing with 3 DW Header - Flit Mode §

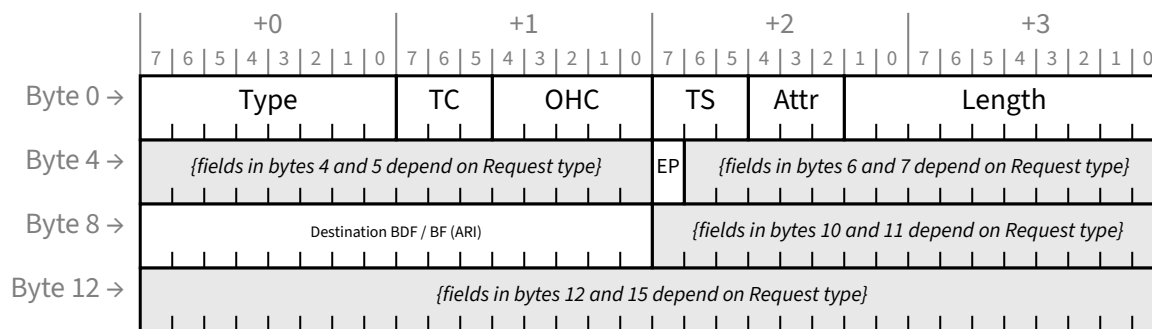


Figure 2-28 ID Routing with 4 DW Header - Flit Mode §

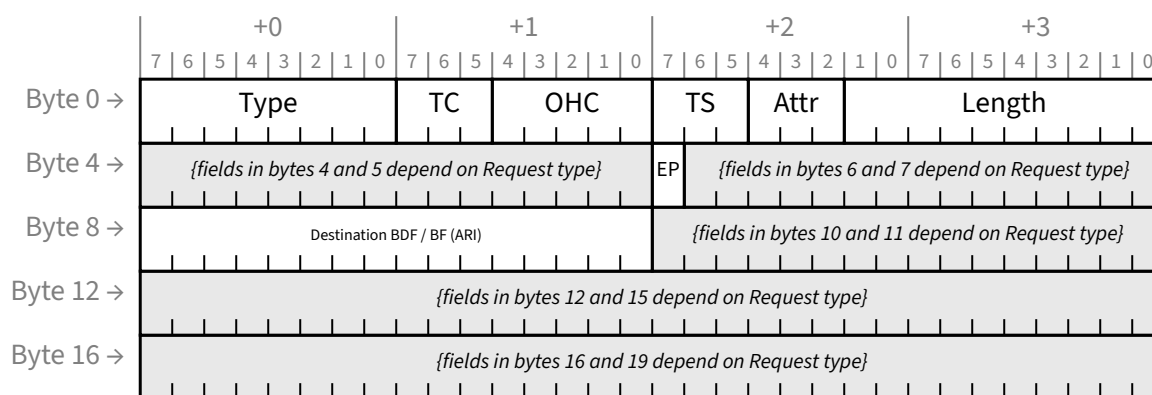


Figure 2-29 ID Routing with 5 DW Header - Flit Mode §

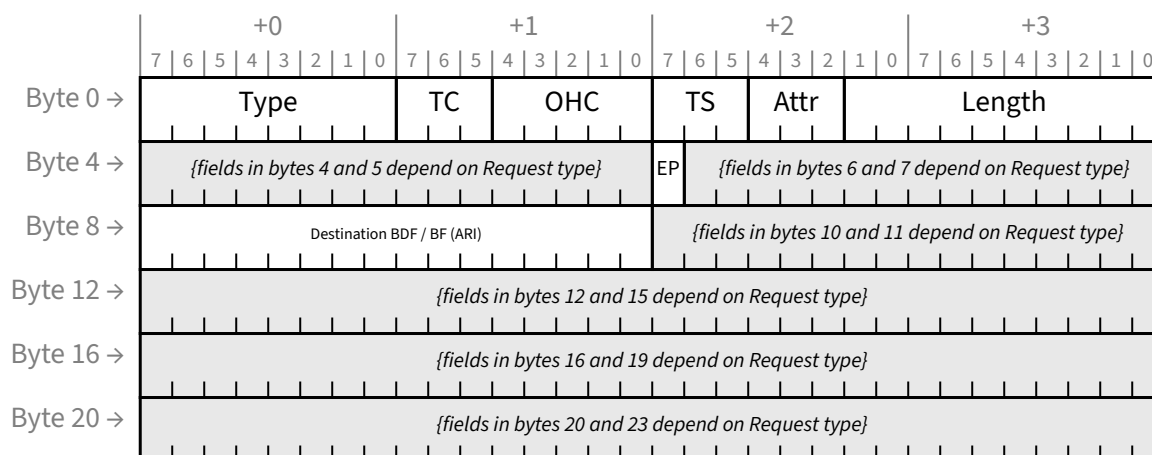


Figure 2-30 ID Routing with 6 DW Header - Flit Mode §

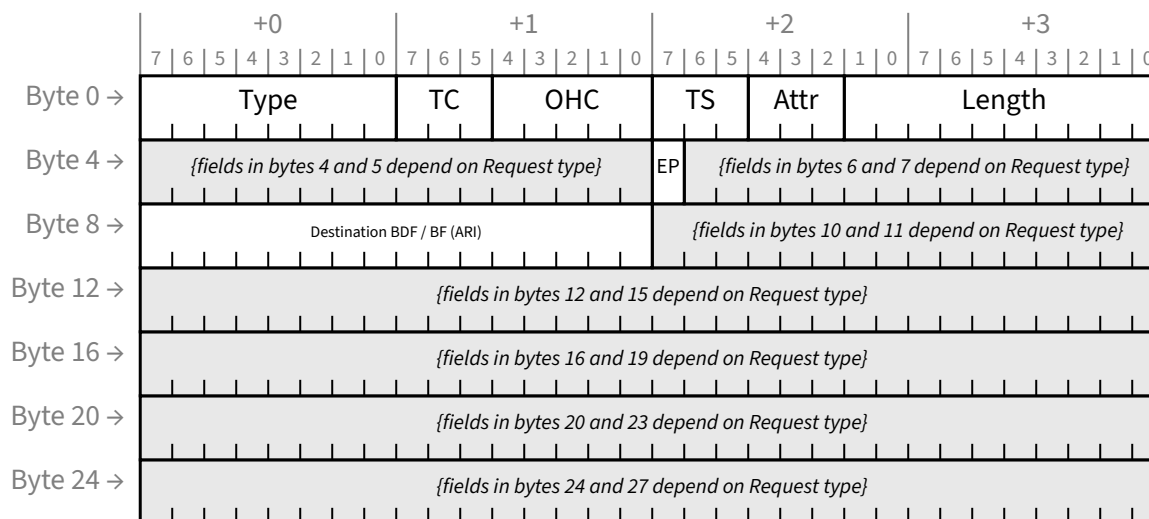


Figure 2-31 ID Routing with 7 DW Header - Flit Mode §

2.2.5 First/Last DW Byte Enables Rules §

The general function of TLP Byte Enables is similar in Non-Flit Mode and Flit Mode, however the detailed rules differ.

IMPLEMENTATION NOTE:

SECURITY ISSUES ASSOCIATED WITH NON-ENABLED BYTES §

The data included with a Write or Read Completion necessarily is DW aligned, and so in cases where some bytes are not enabled, the content of the non-enabled bytes is undefined. To optimize platform security, it is strongly recommended that non-enabled bytes be filled with zeros to avoid data being inadvertently leaked (“leaky bytes”).

As a best practice, it is strongly recommended that devices receiving non-enabled bytes also ensure that the values provided in those bytes are discarded by hardware, such that the values cannot be visible to software. Hardware that fails to do so can provide a path for an attacker to observe confidential data without the need for physical access to a system.

2.2.5.1 Byte Enable Rules for Non-Flit Mode §

Byte Enables are included with Memory, I/O, and Configuration Requests. This section defines the corresponding rules. Byte Enables, when present in the Request header, are located in byte 7 of the header (see § Figure 2-32). For Memory Read Requests and DMWr Requests that have the TH bit Set, the Byte Enable fields are repurposed to carry the ST[7:0] field (refer to § Section 2.2.7.1.1 for details), and values for the Byte Enables are implied as defined below. The TH bit must only be Set in Memory Read Requests and DMWr Requests when it is acceptable to complete those Requests as if all bytes for the requested data were enabled.

- For Memory Read Requests and DMWr Requests that have the TH bit Set, the following values are implied for the Byte Enables. See § Section 2.2.7 for additional requirements.
 - If the Length field for this Request indicates a length of 1 DW, then the value for the First DW Byte Enables is implied to be 1111b and the value for the Last DW Byte Enables is implied to be 0000b.
 - If the Length field for this Request indicates a length of greater than 1 DW, then the value for the First DW Byte Enables and the Last DW Byte Enables is implied to be 1111b.

IMPLEMENTATION NOTE:

READ REQUEST WITH TPH TO DWORDS WITH SIDE EFFECTS §

Memory Read Requests with the TH bit Set and that target DWORDs with side effects should only be issued when the Requester knows that completion of such reads will not create unintended side effects due to implied Byte Enable values.

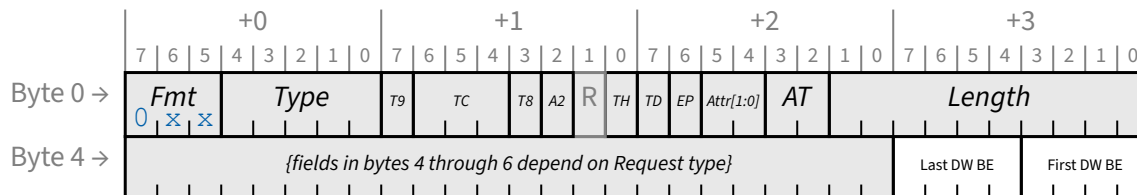


Figure 2-32 Location of Byte Enables in TLP Header - Non-Flit Mode §

- The First DW BE[3:0] field contains Byte Enables for the first (or only) DW referenced by a Request.
 - If the Length field for a Request indicates a length of greater than 1 DW, this field must not equal 0000b.
- The Last DW BE[3:0] field contains Byte Enables for the last DW of a Request.
 - If the Length field for a Request indicates a length of 1 DW, this field must equal 0000b.
 - If the Length field for a Request indicates a length of greater than 1 DW, this field must not equal 0000b.
- For each bit of the Byte Enables fields:
 - a value of 0b indicates that the corresponding byte of data must not be written or, if Read Side Effects exist, must not be read at the Completer.
 - a value of 1b indicates that the corresponding byte of data must be written or read at the Completer.
 - See special rules in this section regarding Memory Read Requests and DMWr Requests that have the TH bit Set.
- Non-contiguous Byte Enables (enabled bytes separated by non-enabled bytes) are permitted in the First DW BE field for all Requests with length of 1 DW.
 - Non-contiguous Byte Enable examples: 1010b, 0101b, 1001b, 1011b, 1101b
- Non-contiguous Byte Enables are permitted in both Byte Enables fields for Quad Word (QW) aligned Memory Requests with length of 2 DW (1 QW).

- All non-QW aligned Memory Requests with length of 2 DW (1 QW) and Memory Requests with length of 3 DW or more must enable only bytes that are contiguous with the data between the first and last DW of the Request.
 - Contiguous Byte Enables examples:
First DW BE: 1100b, Last DW BE: 0011b

First DW BE: 1000b, Last DW BE: 0111b
- § Table 2-10 shows the correspondence between the bits of the Byte Enables fields, their location in the Request header, and the corresponding bytes of the referenced data.

Table 2-10 Byte Enables Location and Correspondence §

Byte Enables	Header Location	Affected Data Byte ¹⁵
First DW BE[0]	Bit 0 of Byte 7	Byte 0
First DW BE[1]	Bit 1 of Byte 7	Byte 1
First DW BE[2]	Bit 2 of Byte 7	Byte 2
First DW BE[3]	Bit 3 of Byte 7	Byte 3
Last DW BE[0]	Bit 4 of Byte 7	Byte N-4
Last DW BE[1]	Bit 5 of Byte 7	Byte N-3
Last DW BE[2]	Bit 6 of Byte 7	Byte N-2
Last DW BE[3]	Bit 7 of Byte 7	Byte N-1

- A Write Request with a length of 1 DW with no bytes enabled is permitted, and has no effect at the Completer unless otherwise specified.

IMPLEMENTATION NOTE: ZERO-LENGTH WRITE §

A Memory Write Request of 1 DW with no bytes enabled, or “zero-length Write,” may be used by devices under certain protocols, in order to achieve an intended side effect.

- If a Read Request of 1 DW specifies that no bytes are enabled to be read (First DW BE[3:0] field = 0000b), the corresponding Completion must specify a length of 1 DW, and include a data payload of 1 DW.
The contents of the data payload within the Completion packet is unspecified and may be any value.
- Receiver/Completer behavior is undefined for a TLP violating the Byte Enables rules specified in this section.
- Receivers are permitted to check for violations of the Byte Enable rules specified in this section. If a Receiver implementing such checks determines that a TLP violates one or more Byte Enable rules, the TLP is a Malformed TLP. These checks are independently optional (see § Section 6.2.3.4).
 - If Byte Enable rules are checked, a violation is a reported error associated with the Receiving Port (see § Section 6.2).

15. Assuming the data referenced is N bytes in length (Byte 0 to Byte N-1). Note that Last DW Byte Enables are used only if the data length is greater than one DW.

- Byte Enable rules cannot be meaningfully checked by intermediate Receivers, and therefore must not be performed on Flow-Through IDE TLPs.

IMPLEMENTATION NOTE: ZERO-LENGTH READ §

A Memory Read Request of 1 DW with no bytes enabled, or “zero-length Read,” may be used by devices as a type of flush Request. For a Requester, the flush semantic allows a device to ensure that previously issued Posted Writes have been completed at their PCI Express destination. To be effective in all cases, the address for the zero-length Read must target the same device as the Posted Writes that are being flushed. One recommended approach is using the same address as one of the Posted Writes being flushed.

The flush semantic has wide application, and all Completers must implement the functionality associated with this semantic. Since a Requester may use the flush semantic without comprehending the characteristics of the Completer, Completers must ensure that zero-length reads do not have side-effects. Note that the flush applies only to traffic in the same Traffic Class as the zero-length Read.

2.2.5.2 Byte Enable Rules for Flit Mode §

Except as defined in this section, all Byte Enable Rules in Flit Mode are the same as in Non-Flit Mode.

For all Memory Requests, it is permitted for OHC-A1 (see § Figure 2-7) to be present. OHC-A1 must be included for Requests that require any of the fields included in OHC-A1. For a Memory Request without OHC-A1 and when the Request’s Byte Enable fields are not Reserved, the implied field values for a 1 DW Request are 1111b for First DW Byte Enable and 0000b for Last DW Byte Enable, and for a greater than 1 DW Request is 1111b for both First DW Byte Enable and Last DW Byte Enable. If a Request requires non-Reserved BE field values other than these, then OHC-A1 must be present. When OHC-A1 is present, the PASID, PMR and ER fields are valid if and only if the PV bit is Set.

As defined in § Section 2.2.7.1, the Byte Enable fields for AtomicOp Requests are Reserved or implied to be Reserved. If an AtomicOp Request includes OHC-A1, its Byte Enable fields must be Reserved.

OHC-A2 must be included for all I/O Requests.

OHC-A3 must be included for all Configuration Requests.

In all cases where OHC-Ax is present, with Byte Enable fields, these fields must be handled as defined in § Section 2.2.5.1.

If a FM Requester uses ST[7:0] and also sets the Byte Enables to values that do not match the implied Byte Enable values specified in § Section 2.2.5.1, the Request will not be translatable into NFM. If translation is necessary by any Routing Element between the Requester and Completer, the result will usually be a TLP Translation Egress Blocked error, subject to architected error handling rules. FM Requesters are permitted not to match the implied Byte Enable values, but are strongly recommended to consider the resulting configuration limitations.

2.2.6 Transaction Descriptor §

2.2.6.1 Overview §

The Transaction Descriptor is a mechanism for carrying Transaction information between the Requester and the Completer. Transaction Descriptors are composed of three fields:

- Transaction ID - identifies outstanding Transactions
- Attributes field - specifies characteristics of the Transaction
- Traffic Class (TC) field - associates Transaction with type of required service

§ Figure 2-33 shows the fields of the Transaction Descriptor. Note that these fields are shown together to highlight their relationship as parts of a single logical entity. The fields are not contiguous in the packet header.

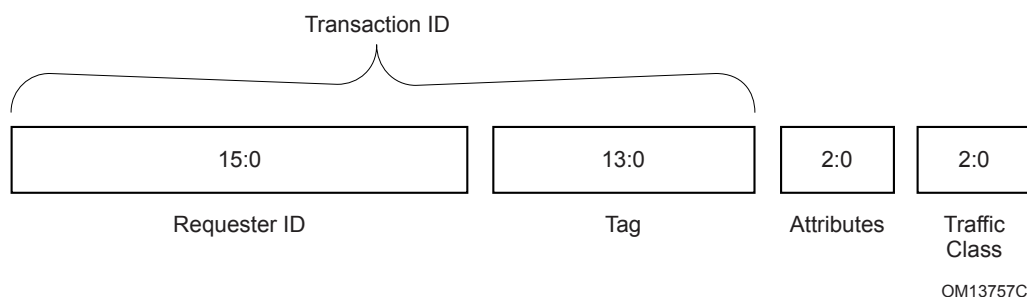


Figure 2-33 Transaction Descriptor §

2.2.6.2 Transaction Descriptor - Transaction ID Field §

The Transaction ID field consists of two major sub-fields: Requester ID and Tag as shown in § Figure 2-34.

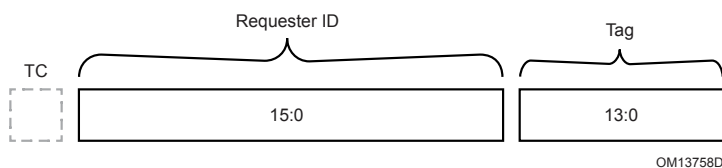


Figure 2-34 Transaction ID §

In some cases (defined below) the Traffic Class (TC) is also included in the Transaction ID.

The Transaction ID is used to associate Completions with Requests. There are three groups of Request/Completion types for which the Transaction ID has differing rules. The groups are distinguished by the Completion Type expected for the Request type(s) in that group. Each group forms a distinct namespace, and there is no requirement for Transaction ID uniqueness between groups. These groups and their high-level requirements are:

- Group I: Cpl / CplD, which apply to Non-UIO Requests:
 - The Transaction ID consists of Requester ID[15:0] and Tag[13:0]¹⁶
 - Requesters must assign Tag values such that Transaction ID values are unique for all outstanding Non-Posted Requests in Group I, without regard to TC or any other field.
- Group II: UIOWrCpl, which applies to UIO Memory Write (UIOMWr) Requests:

16. Not all Tag bits are used in all cases – detailed rules follow.

- The Transaction ID consists of the TC[2:0], Requester ID[15:0] and Tag[13:0].
- Requesters are permitted to assign Tag values such that multiple outstanding Requests in Group II have the same Transaction ID (see § Section 2.2.9.2)
- Group III: UIORdCplD and UIORdCpl, which apply to UIO Memory Read (UIOMRd) Requests:
 - The Transaction ID consists of the TC[2:0], Requester ID[15:0] and Tag[13:0].
 - Requesters must assign Tag values such that Transaction ID values are unique for all outstanding Requests in Group III.

Four Tag sizes are architected for non-UIO channel operation: 14-bit, 10-bit, 8-bit, and 5-bit. One Tag size is architected for UIO channel operation: 14-bit. § Section 2.2.6.2.1 contains rules for Tags under non-UIO channel operation. § Section 2.2.6.2.2 contains rules for Tags under UIO channel rules.

2.2.6.2.1 Tag Rules for Non-UIO Channel Operation §

A given Function may support different Tag sizes when operating as a Requester versus operating as a Completer. Below are the rules regarding operational Tag sizes. Also see the “Considerations for Implementing Larger-Tag Capabilities” Implementation Note later in this section.

- 14-Bit Tags and 10-Bit Tags are referred to as “larger” Tags.
 - A Request containing a 10-bit or 14 bit Tag is called **larger-Tag Request**.
 - The 10-bit Tag Requester Enable and 14-bit Tag Requester Enable bits are called the **larger-Tag Requester Enable bits**.
 - The VF 10-bit Tag Requester Enable and VF 14-bit Tag Requester Enable bits are called the **VF larger-Tag Requester Enable bits**.
- 8-Bit Tags and 5-Bit Tags are referred to as “smaller” Tags.
- All Functions must support 8-Bit Tag Completer capability.
- A Function that supports Flit Mode must support 14-Bit Tag Completer capability, and thus it automatically supports 10-Bit Tag Completer capability.
- Functions¹⁷ (including those in Switches) that support 16.0 GT/s or higher data rates must support 10-Bit Tag Completer capability.
- A Function must not support 14-Bit Tag Requester capability unless it supports 14-Bit Tag Completer capability.
- A Function must not support 10-Bit Tag Requester capability unless it supports 10-Bit Tag Completer capability.
- In Non-Flit Mode, Tag[8] and Tag[9], are not contiguous with other Tag field bits in the TLP Header. These bits were Reserved prior to 10-Bit Tags being architected. Requesters in Non-Flit Mode that do not support 10-Bit Tag Requester capability must set Tag[9:8] to 00b.
- RCs containing elements that indicate support for 14-Bit Tag Completer capability or 10-Bit Tag Completer capability must handle supported Tag-sized Requests correctly by all registers and memory regions supported as targets of PCIe Requesters (e.g., host memory targeted by DMA Requests or MMIO regions in RCiEPs).
 - Each RP indicating support must handle such Requests received by its Ingress Port.
 - Each RCiEP indicating support must handle such Requests coming from supported internal paths, including those coming through RPs.
- If an RC contains RCiEPs that indicate support for 14-Bit Tag Requester capability or 10-Bit Tag Requester capability, the RC must handle Requests from those RCiEPs correctly by all registers and memory regions supported as targets of those RCiEPs (e.g., host memory targeted by DMA Requests or MMIO regions in RCiEPs).

17. An exception is PCI Express to PCI/PCI-X Bridges, since 10-Bit Tag capability is not architected for these Functions.

- Receivers/Completers must handle 8-bit Tag values correctly regardless of the setting of their Extended Tag Field Enable bit (see § Section 7.5.3.4). Refer to [PCIe-to-PCI-PCI-X-Bridge] for details on the bridge handling of Extended Tags.
- Receivers/Completers that support 14-Bit Tag Completer capability or 10-Bit Tag Completer capability must handle the supported Tag-size values correctly, regardless of the setting of their corresponding Tag Requester Enable bits (see § Section 7.5.3.16 and § Section 7.7.9.3).
- 14-Bit Tag capability and 10-Bit Tag capability are not architected for PCI Express to PCI/PCI-X Bridges, and they must not indicate the associated Tag Requester capability or Tag Completer capability.
- If one or both larger-Tag Requester Enable bits are Set, the following rules apply.
 - If both larger-Tag Requester Enable bits are Set in an Endpoint¹⁸, then 14-Bit Tags are permitted for Requests that target host memory. An implementation specific hardware mechanism in the Endpoint is permitted to limit those Requests to 10-Bit Tags or smaller Tags, but generic software or firmware should not Set the 14-Bit Tag Requester Enable bit unless the host supports 14-Bit Tag Completer capability for host memory.
 - If an Endpoint¹⁹ supports sending Requests to other Endpoints (as opposed to host memory), the Endpoint must not send larger-Tag Requests to another given Endpoint unless an implementation specific mechanism determines that the Endpoint supports the corresponding larger Tag Completer capability. Not sending larger-Tag Requests to other Endpoints at all may be acceptable for some implementations. More sophisticated mechanisms are outside the scope of this specification.
 - If a PIO Requester has larger-Tag Requester capability, how the Requester determines when to use larger Tags versus smaller Tags is outside the scope of this specification. One example approach is to use smaller Tags for all PIO Requests and use larger Tags for integrated data-mover engines that use the same Requester ID. A similar approach might be used for integrated hardware that takes ownership of P2P requests.
 - With 14-Bit Tags, determination of valid Tag values is complicated by inconsistencies in previous versions of this specification. The strongly recommended behavior is for all Tag[13:10] values except 0000b to be valid, and for 14-Bit Requesters not to generate Tag values with Tag[13:10] equal to 0000b. This enables a Requester to determine if a Completion it receives that should have a 14-Bit Tag contains an invalid Tag value. However, for backward compatibility with previous versions of this specification, 14-bit Requesters are permitted to generate any Tag[13:8] values except 00 0000b, and such Tag values are valid.
 - With 10-Bit Tags, all Tag[9:8] values except 00b are valid. 10-Bit Tag values with Tag[9:8] equal to 00b are invalid, and must not be generated by the Requester. This enables a Requester to determine if a Completion it receives that should have a 10-Bit Tag contains an invalid Tag value, usually caused by the Completer not supporting 10-Bit Tag Completer capability.
 - If a Requester sends a larger-Tag Request to a Completer that lacks the associated larger-Tag Completer capability, the returned Completion(s) will have Tags with invalid Tag values. Such Completions will be handled as Unexpected Completions²⁰, which by default are Advisory Non-Fatal Errors. The Requester must follow standard PCI Express error handling requirements.
 - When a Requester handles a Completion with an invalid Tag as an Unexpected Completion, the original Request will likely incur a Completion Timeout. If the Requester handles the Completion Timeout condition in some device-specific manner that avoids data corruption, the Requester is permitted to suppress handling the Completion Timeout by standard PCI Express error handling mechanisms as required otherwise.
 - If a Requester supports sending larger-Tag Requests to some Completers and smaller-Tag Requests to other Completers concurrently, the Requester must honor the Extended Tag Field Enable bit setting for

18. This includes PCI Express Endpoints, Legacy PCI Express Endpoints, and Root Complex Integrated Endpoints.

19. This includes PCI Express Endpoints, Legacy PCI Express Endpoints, and Root Complex Integrated Endpoints.

20. If a Completion has a higher precedence error, that error should be reported instead.

the smaller-Tag Requests. That is, if the bit is Clear, only the lower 5 bits of the Tag field may be non-Zero; if the bit is Set, only the lower 8 bits of the Tag field may be non-Zero.

- If a Requester supports sending larger-Tag Requests to some Completers and smaller-Tag Requests to other Completers concurrently, the Requester must ensure that no outstanding larger Tags can alias to an outstanding smaller Tag if any larger-Tag Request is completed by a Completer that lacks larger-Tag Completer capability. See the "Using Larger Tags and Smaller Tags Concurrently" Implementation Note later in this section.
- The default value of the Extended Tag Field Enable bit is implementation specific. The default value of the 14-Bit Tag Requester Enable bit and the 10-Bit Tag Requester Enable bit is 0b.
- Receiver/Completer behavior is undefined if multiple uncompleted Requests, are issued from the same Requester with non-unique Transaction ID values. In FM, Completers must be designed to handle simultaneous uncompleted Requests with non-unique Transaction ID values from Requesters that reside in different Hierarchies, as indicated by implied or explicit Segment numbers associated with each Request.
- If Phantom Function Numbers are used to extend the number of outstanding Requests, the combination of the Phantom Function Number and the Tag field must be unique for all outstanding Requests that require a Completion for that Requester, without regard to TC or any other field.
- If Shadow Functions are used to extend the number of outstanding Requests, the combination of the Shadow Function Number and the Tag field must be unique for all outstanding Requests that require a Completion for that Requester, without regard to TC or any other field.
- § Table 2-11 indicates how the three tag enable bits determine the maximum tag size and permitted tag value ranges a Requester must use for different Completers and their associated paths. For a given combination of Tag enable settings, a Requester must use a Tag size within its enabled maximum and within the Tag capabilities of the Completer and its associated path. For each Request, the Requester is permitted to use a Tag size smaller than the greatest common Tag size supported by the Completer/path, but the Requester must still abide by the permitted Tag value range for the Tag size that it uses.

Table 2-11 Tag Enables, Sizes, and Permitted Ranges for non-UIO Transactions §

14-bit Tag Requester Enable	10-bit Tag Requester Enable	Extended Tag Field Enable	Maximum Request Tag size	Permitted range for an 8-bit Tag Completer/path	Permitted range for a 10-bit Tag Completer/path	Permitted range for a 14-bit Tag Completer/path
0	0	0	5 bits	0 to 31	0 to 31	0 to 31
0	0	1	8 bits	0 to 255	0 to 255	0 to 255
0	1	0	10 bits	0 to 31	256 to 1023	256 to 1023
0	1	1	10 bits	0 to 255	256 to 1023	256 to 1023
1	0	0	14 bits	0 to 31	0 to 31	1024 to 16383
1	0	1	14 bits	0 to 255	0 to 255	1024 to 16383
1	1	0	14 bits	0 to 31	256 to 1023	1024 to 16383 ²¹
1	1	1	14 bits	0 to 255	256 to 1023	1024 to 16383 ²²

Notes:

21. The permitted range of 1024 to 16383 is strongly recommended, but for backward compatibility with previous versions of this specification, 256 to 16383 is permitted.

22. The permitted range of 1024 to 16383 is strongly recommended, but for backward compatibility with previous versions of this specification, 256 to 16383 is permitted.

1. The permitted range for a 5-bit Tag Completer/path is always 0 to 31, so there is no column in the table to indicate this.
2. The "X-bit Tag Completer/path" is the greatest common Tag size capability of the Completer and all routing elements along the path between the Requester and the targeted Completer. If a routing element is not the targeted Completer, but detects an Uncorrectable Error with a Request, the routing element may serve as the Completer for the Request.
3. If a Requester supports sending larger-Tag Requests to some Completers and smaller-Tag Requests to other Completers concurrently, the Requester must ensure that no outstanding larger Tags can alias to an outstanding smaller Tag if any larger-Tag Request is completed by a Completer that lacks larger-Tag Completer capability.
 - For Posted Requests, the Tag[13:8] field is Reserved in Non-Flit Mode, and Tag[13:0] is Reserved in Flit Mode.
 - An exception to this rule is allowed for the uses defined in [MCTP-VDM].
 - In Non-Flit Mode, for Posted Requests with the TH bit Set, the Tag[7:0] field is repurposed for the ST[7:0] field (refer to § Section 2.2.7.1.1 for details). For Posted Requests with the TH bit Clear, the Tag[7:0] field is undefined and may contain any value. (Refer to § Table F-1 for exceptions to this rule for certain Vendor-Defined Messages.)
 - For Posted Requests with the TH field Clear, the value in the Tag[7:0] field must not affect Receiver processing of the Request.
 - For Posted Requests with the TH bit Set, the value in the ST[7:0] field may affect Completer processing of the Request (refer to § Section 2.2.7.1 for details).
 - A Transaction ID must be unique for each pending Transaction within a Hierarchy.
 - Transaction ID is included with all Requests and Completions.
 - The Requester ID is a 16-bit value that is unique for every PCI Express Function within a Hierarchy.
 - Functions must capture the Bus and Device Numbers²³ supplied with all Type 0 Configuration Write Requests completed by the Function and supply these numbers in the Bus and Device Number fields of the Requester ID²⁴ for all Requests initiated without the use of Shadow Functions by the Device/Function. See § Section 7.9.25, for details of how the Requester ID may be modified by the use of Shadow Functions. It is recommended that Numbers are captured for successfully completed Requests only.

Exception: The assignment of Bus and Device Numbers to the Devices within a Root Complex, and Device Numbers to the Downstream Ports within a Switch, may be done in an implementation specific way.

Note that the Bus Number and Device Number²⁵ may be changed at run time, and so it is necessary to re-capture this information with each and every Type 0 Configuration Write Request to the Device.

Configuration Write Requests addressed to unimplemented Functions *MUST@FLIT* not affect captured Bus and Device Numbers for implemented Functions.

- When generating Requests on their own behalf (for example, for error reporting), Switches must use the Requester ID associated with the primary side of the bridge logically associated with the Port (see § Section 7.1) causing the Request generation.
- Prior to the initial Configuration Write to a Function, the Function is not permitted to initiate Non-Posted Requests. (A valid Requester ID is required to properly route the resulting completions.)
 - Exception: Functions within a Root Complex are permitted to initiate Requests prior to software-initiated configuration for accesses to system boot device(s).

23. In ARI Devices, Functions are only required to capture the Bus Number. ARI Devices are permitted to retain the captured Bus Number on either a per-Device or a per-Function basis. If the captured Bus Number is retained on a per-Device basis, all Functions are required to update and use the common Bus Number.

24. An ARI Requester ID does not contain a Device Number field. See § Section 2.2.4.2.

25. With ARI Devices, only the Bus Number can change.

Note that this rule and the exception are consistent with the existing PCI model for system initialization and configuration.

- Each Function associated with a Device must be designed to respond to a unique Function Number for Configuration Requests addressing that Device. Note: Each non-ARI Device may contain up to eight Functions. Each ARI Device may contain up to 256 Functions.
- A Switch must forward Requests without modifying the Transaction ID, except when this is not possible due to any non-zero Tag[13:10] bits. For a Request from an Ingress Port operating in FM targeting an Egress Port operating in NFM, the presence of any non-zero Tag[13:10] bits must be handled by the Egress Port first by blocking the TLP and then reporting a TLP Translation Egress Blocked error for a Posted Request or reporting no error for a Non-Posted Request or a UIO Request. Such Tag bits cannot be conveyed in NFM.
- In some circumstances, a PCI Express to PCI/PCI-X Bridge is required to generate Transaction IDs for Requests it forwards from a PCI or PCI-X bus.
- In Flit Mode, Functions must capture the value of the Destination Segment supplied with all Type 0 Configuration Write Requests successfully completed by the Function. It is permitted for each Function of a Device to independently capture the Destination Segment value, or for all Functions of a Device to use the value captured by Function 0. All Functions within a Switch share a common Segment value that is captured by Functions associated with the Upstream Port. Functions also must capture the DSV bit in Type 0 Configuration Write Requests as described in the Segment Captured bit description in § Section 7.7.9.4 .
 - The Segment is effectively an extension of the Requester ID, but is formally defined as a distinct field to avoid confusion with the use of the term Transaction ID in Non-Flit Mode operation.
 - In systems that support multiple Segments, each Hierarchy must be associated with a single Segment. It is permitted for multiple hierarchy domains to be associated with a single Segment.
 - The Segment Number in the Configuration Write Request must be set to the Downstream Port's Segment Number.
- In Flit-Mode, in some circumstances, the captured Segment is also explicitly indicated in a TLP, which enables the Completer to distinguish identical Transaction IDs in Requests coming from different Hierarchies.

IMPLEMENTATION NOTE: INCREASING THE NUMBER OF OUTSTANDING REQUESTS USING PHANTOM FUNCTIONS OR SHADOW FUNCTIONS §

To increase the maximum possible number of outstanding Requests requiring Completion beyond that possible using Tag bits alone, a device may, if the Phantom Functions Enable bit is Set (see § Section 7.5.3.4), or the Shadow Functions Enable bit is Set (see § Section 7.9.25.3), use Function Numbers not assigned to implemented Functions to logically extend the Tag identifier. For a Single-Function Device, this can allow a significant increase in the maximum number of outstanding Requests.

When the Phantom Functions Enable bit is Set, unclaimed Function Numbers are referred to as Phantom Function Numbers.

Phantom Functions have a number of architectural limitations, including a lack of support by ARI Devices, Virtual Functions (VFs), and Physical Functions (PFs) when VFs are enabled. In addition, Address Translation Services (ATS) and ID-Based Ordering (IDO) do not comprehend Phantom Functions. Shadow Functions have fewer limitations. Thus, for many implementations, the use of larger Tags and Shadow Functions are better ways to increase the number of outstanding Non-Posted Requests.

IMPLEMENTATION NOTE: CONSIDERATIONS FOR IMPLEMENTING LARGER-TAG CAPABILITIES §

The use of "larger" (i.e., 10-bit or 14-bit) Tags enables a Requester to increase its number of outstanding Non-Posted Requests (NPRs) substantially, which for very high rates of NPRs or very large round-trip times can avoid Tag availability from becoming a bottleneck. The following formula gives the basic relationship between payload bandwidth, number of outstanding NPRs, and other factors:

$BW = S * N / RTT$, where

BW = payload bandwidth

S = transaction payload size

N = number of outstanding NPRs

RTT = transaction round-trip time

Generally, only high-speed Requesters on high-speed Links using relatively small transactions will benefit from increasing their number of outstanding NPRs beyond 256, although this can also help maintain performance in configurations where the transaction round-trip time is high.

In configurations where a Requester with larger-Tag Requester capability needs to target multiple Completers, one needs to ensure that the Requester sends larger-Tag Requests only to Completers that have sufficient larger-Tag Completer capability. This is greatly simplified if all Completers have larger-Tag capability.

For general industry enablement of larger Tags, it is strongly recommended that all Functions²⁶ support larger-Tag Completer capability. With new implementations, Completers that don't need to operate on higher numbers of NPRs concurrently themselves can generally track larger Tags internally and return them in Completions with modest incremental investment.

Completers that actually process higher numbers of NPRs concurrently may require substantial additional hardware resources, but the full performance benefits of larger Tags generally can't be realized unless Completers actually do process higher numbers of NPRs concurrently.

For platforms where the RC supports larger-Tag Completer capability, it is strongly recommended for platform firmware or operating system software that configures PCIe hierarchies to Set one of the larger-Tag Requester Enable bits automatically in Endpoints with larger-Tag Requester capability. This enables the important class of larger-Tag capable adapters that send Memory Read Requests only to host memory.

For Endpoints other than RCiEPs, one can determine if the RC supports larger-Tag Completer capability for each one by checking the larger-Tag Completer Supported bits in its associated RP. RCiEPs have no associated RP, so for this reason they are not permitted to have one of their larger-Tag Requester Supported bits Set unless the RC supports sufficient larger-Tag Completer capability for them. Thus, software does not need to perform a separate check for RCiEPs.

Non-Flit Mode Switches that lack 10-bit Tag Completer capability are still able to forward NPRs and Completions carrying 10-bit Tags correctly, since the two new Tag bits are in TLP Header bits that were formerly Reserved, and Switches are required to forward Reserved TLP Header bits without modification. However, if such a Switch detects an error with an NPR carrying a 10-bit Tag, and that Switch handles the error by acting as the Completer for the NPR, the resulting Completion will have an invalid 10-bit Tag. Thus, it is strongly recommended that Non-Flit Mode Switches between any components using 10-bit Tags support 10-bit Completer capability. Note that Switches supporting 16.0 GT/s or higher data rates must support 10-bit Tag Completer capability.

26. An exception is PCI Express to PCI/PCI-X Bridges, since larger-Tag capability is not architected for these Functions.

For configurations where a Requester with larger-Tag Requester capability targets Completers where some do and some do not have sufficient larger-Tag Completer capability, how the Requester determines which NPRs include larger Tags is outside the scope of this specification.

IMPLEMENTATION NOTE: USING LARGER TAGS AND SMALLER TAGS CONCURRENTLY §

As stated earlier in this section, if a Requester supports sending larger-Tag Requests to some Completers and smaller-Tag Requests to other Completers concurrently, the Requester must ensure that no outstanding larger Tags can alias to an outstanding smaller Tag if any larger-Tag Request is completed by a Completer that lacks sufficient larger-Tag Completer capability.

For 10-bit Tags, one implementation approach is to have the Requester partition its 8-bit Tag space into 2 regions: one that will only be used for smaller Tags (8-bit or 5-bit Tags), and one that will only be used for the lower 8 bits of 10-bit Tags. Note that this forces a tradeoff between the Tag space available for 10-bit Tags and smaller Tags.

For example, if a Requester partitions its 8-bit Tag space to use only the lowest 4 bits for smaller Tags, this supports up to 16 outstanding smaller Tags, and it reduces the 10-bit Tag space by 3×16 values, supporting $768 - 48 = 720$ outstanding 10-bit Tags. Many other partitioning options are possible, all of which reduce the total number of outstanding Requests. In general, reserving N values for smaller Tags reduces 10-bit Tag space by $3 \times N$ values, and the total for smaller Tags plus 10-bit Tags ends up being $768 - 2 \times N$.

Similar implementation approaches for 14-Bit Tags are possible, and they are straight-forward if only 14-Bit and 8-Bit/5-Bit Tags are supported. If a Requester implementation needs to handle 14-Bit, 10-Bit, and 8-Bit/5-Bit Tag sizes concurrently, the general approach of partitioning the Requester's Tag spaces still works, but the complexity increases significantly.

2.2.6.2.2 Tag Rules for UIO Channel Operation §

UIO transactions must follow the Tag namespace requirements for Group II and Group III in § Section 2.2.6.2 . Additional Tag rules under UIO channel operation are much simpler compared to non-UIO channel operation:

- UIO Completers must support 14-Bit Tags, i.e., returning each UIO Request's entire 14-bit Tag value in any UIO Completions resulting from the Request.
- UIO Requesters are permitted to use 14-bit Tags including all Tag[13:0] values, regardless of Tag size enable settings. See Extended Tag Field Enable, 10-Bit Tag Requester Enable, and 14-Bit Tag Requester Enable.
- UIO Completers must be designed to handle simultaneous uncompleted Requests with identical Transaction ID values coming from Requesters that reside in different Hierarchies, as indicated by implied or explicit Segment numbers associated with each Request.

2.2.6.3 Transaction Descriptor - Attributes Field §

The Attributes field is used to provide additional information that allows modification of the default handling of Transactions. These modifications apply to different aspects of handling the Transactions within the system, such as:

- Ordering
- Hardware coherency management (snoop)

Attributes are hints that allow, but do not require, optimizations in the handling of traffic. The level of optimization support is dependent on the target applications of particular PCI Express peripherals and platform building blocks. In Flit Mode the Attributes Field is contiguous in the TLP Header. In Non-Flit Mode, attribute bit 2 is sometimes labeled A2 and is not adjacent to bits 1 and 0 (see § Figure 2-36 and § Figure 2-37).

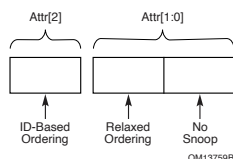


Figure 2-35 Attributes Field of Transaction Descriptor §

2.2.6.4 Relaxed Ordering and ID-Based Ordering Attributes §

§ Table 2-12 defines the states of the Relaxed Ordering and ID-Based Ordering attribute Bit-wise. These attributes are discussed in § Section 2.4 . Note that Relaxed Ordering and ID-Based Ordering attributes are not adjacent in location (see § Figure 2-5).

Table 2-12 Ordering Attributes §

Attribute Bit [2]	Attribute Bit [1]	Ordering Type	Ordering Model
0	0	Default Ordering	PCI Strongly Ordered Model
0	1	Relaxed Ordering	PCI-X Relaxed Ordering Model
1	0	ID-Based Ordering	Independent ordering based on Requester/Completer ID
1	1	Relaxed Ordering plus ID-Based Ordering	Logical “OR” of Relaxed Ordering and IDO

Attribute bit [1] is not applicable and must be Clear for Configuration Requests, I/O Requests, Memory Requests that are Message Signaled Interrupts, and Message Requests (except where specifically permitted).

Attribute bit [2], IDO, is Reserved for Configuration Requests and I/O Requests. IDO is not Reserved for all Memory Requests, including Message Signaled Interrupts (MSI/MSI-X). IDO is not Reserved for Message Requests unless specifically prohibited. A Requester is permitted to Set IDO only if the IDO Request Enable bit in the Device Control 2 register is Set.

The value of the IDO bit must not be considered by Receivers when determining if a TLP is a Malformed TLP.

A Completer is permitted to Set IDO only if the IDO Completion Enable bit in the Device Control 2 register is Set. It is not required to copy the value of IDO from the Request into the Completion(s) for that Request. If the Completer has IDO enabled, it is recommended that the Completer set IDO for all Completions, unless there is a specific reason not to (see § Appendix E.).

A Root Complex that supports forwarding TLPs peer-to-peer between Root Ports is not required to preserve the IDO bit from the Ingress to Egress Port.

2.2.6.5 No Snoop Attribute §

§ Table 2-13 defines the states of the No Snoop attribute field. Note that the No Snoop attribute does not alter Transaction ordering.

Table 2-13 Cache Coherency Management Attribute §

No Snoop Attribute (b)	Cache Coherency Management Type	Coherency Model
0	Default	Hardware enforced cache coherency expected
1	No Snoop	Hardware enforced cache coherency not expected

This attribute is not applicable and must be Clear for Configuration Requests, I/O Requests, Memory Requests that are Message Signaled Interrupts, and Message Requests (except where specifically permitted).

2.2.6.6 Transaction Descriptor - Traffic Class Field §

The Traffic Class (TC) is a 3-bit field that allows differentiation of transactions into eight traffic classes.

Together with the PCI Express Virtual Channel support, the TC mechanism is a fundamental element for enabling differentiated traffic servicing. Every PCI Express Transaction Layer Packet uses TC information as an Invariant label that is carried end to end within the PCI Express fabric. As the packet traverses across the fabric, this information is used at every Link and within each Switch element to make decisions with regards to proper servicing of the traffic. A key aspect of servicing is the routing of the packets based on their TC labels through corresponding Virtual Channels. § Section 2.5 covers the details of the VC mechanism.

§ Table 2-14 defines the TC encodings.

Table 2-14 Definition of TC Field Encodings §

TC Field Value (b)	Definition
000	TC0: Best Effort service class (General Purpose I/O) (Default TC - must be supported by every PCI Express device)
001 to 111	TC1 to TC7: Differentiated service classes (Differentiation based on Weighted-Round-Robin (WRR) and/or priority)

It is up to the system software to determine TC labeling and TC/VC mapping in order to provide differentiated services that meet target platform requirements.

The concept of Traffic Class applies only within the PCI Express interconnect fabric. Specific requirements of how PCI Express TC service policies are translated into policies on non-PCI Express interconnects is outside of the scope of this specification.

2.2.7 Memory, I/O, and Configuration Request Rules §

The general requirements for Memory, I/O, and Configuration Requests are similar in Non-Flit Mode and Flit Mode, however some specific rules differ. Rules that are common between Non-Flit Mode and Flit Mode follow, with rules that are specific to each in subsequent sub-sections.

2.2.7.1 Non-Flit Mode §

The following rule applies to all Memory, I/O, and Configuration Requests. Additional rules specific to each type of Request follow.

- All Memory, I/O, and Configuration Requests include the following fields in addition to the common header fields:
 - Requester ID[15:0] and Tag[9:0], forming the Transaction ID. In Non-Flit Mode, the Tag field is 10 bits.
 - Last DW BE[3:0] and First DW BE[3:0]. For Memory Read Requests, DMWr Requests, and AtomicOp Requests with the TH bit Set, the byte location for the Last DW BE[3:0] and First DW BE [3:0] fields in the header are repurposed to carry ST[7:0] field.
 - For Memory Read Requests and DMWr Requests with the TH bit Clear, see § Section 2.2.5 for First/Last DW Byte Enable Rules.
 - For AtomicOp Requests and DMWr Requests with TH bit Set, the values for the DW BE fields are implied to be Reserved. For AtomicOp Requests with TH bit Clear, the DW BE fields are Reserved.

For Memory Requests, the following rules apply:

- Memory Requests route by address, using either 64-bit or 32-bit Addressing (see § Figure 2-36 and § Figure 2-37).
- For Memory Read Requests, Length must not exceed the value specified by Max_Read_Request_Size (see § Section 7.5.3.4).
- For AtomicOp Requests, architected operand sizes and their associated Length field values are specified in § Table 2-15. If a Completer supports AtomicOps, the following rules apply. The Completer must check the Length field value. If the value does not match an architected value, the Completer must handle the TLP as a Malformed TLP. Otherwise, if the value does not match an operand size that the Completer supports, the Completer must handle the TLP as an Unsupported Request (UR). This is a reported error associated with the Receiving Port (see § Section 6.2).

Table 2-15 Length Field Values for AtomicOp Requests §

AtomicOp Request	Length Field Value for Architected Operand Sizes		
	32 Bits	64 Bits	128 Bits
FetchAdd, Swap	1 DW	2 DW	N/A
CAS	2 DW	4 DW	8 DW

- A FetchAdd Request contains one operand, the “add” value.
- A Swap Request contains one operand, the “swap” value.
- A CAS Request contains two operands. The first in the data area is the “compare” value, and the second is the “swap” value.

- For AtomicOp Requests, the Address must be naturally aligned with the operand size. The Completer must check for violations of this rule. If a TLP violates this rule, the TLP is a Malformed TLP. This is a reported error associated with the Receiving Port (see § Section 6.2).
- Requests must not specify an Address/Length combination that causes a Memory Space access to cross a 4-KB boundary.
 - Receivers may optionally check for violations of this rule. If a Receiver implementing this check determines that a TLP violates this rule, the TLP is a Malformed TLP.
 - If checked, this is a reported error associated with the Receiving Port (see § Section 6.2).
 - It is recommended that this optional check only occur in Completers and never in intermediate Receivers.
 - Intermediate Receivers are not permitted to implement this check for TLPs with Reserved Type values (see § Table 2-5). The relationship between the TLP Length field and the length of the affected memory range depends on the Request Type (for an example where they are different, see AtomicOp CAS Request).
 - For AtomicOp Requests, the mandatory Completer check for natural alignment of the Address (see above) already guarantees that the access will not cross a 4-KB boundary, so a separate 4-KB boundary check is not necessary.
 - If a 4-KB boundary check is performed for AtomicOp CAS Requests, this check must comprehend that the TLP's Length field value is based on the size of two operands, whereas the access to Memory Space is based on the size of one operand.

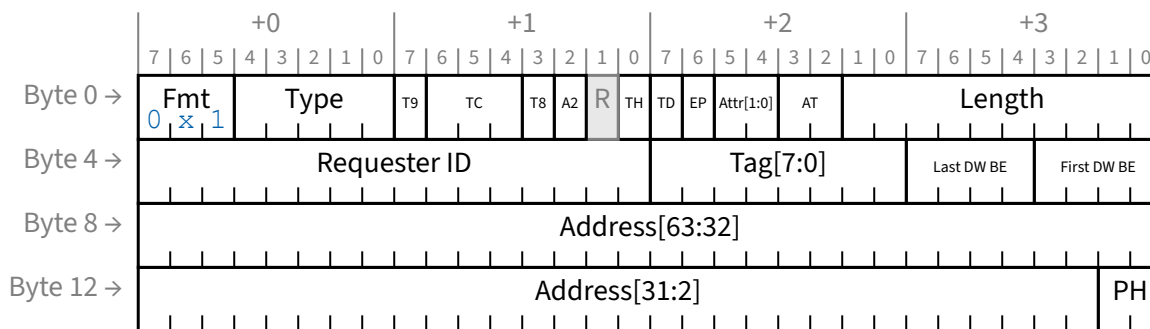


Figure 2-36 Memory Request for 64-bit Addressing Header - Non-Flit Mode §

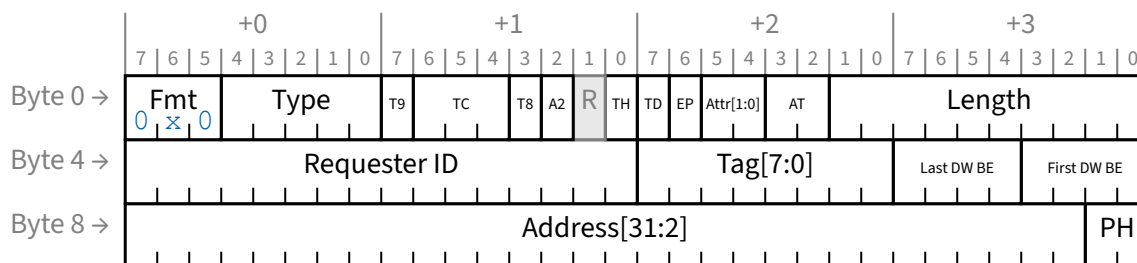


Figure 2-37 Memory Request for 32-bit Addressing Header - Non-Flit Mode §

IMPLEMENTATION NOTE: GENERATION OF 64-BIT ADDRESSES §

It is strongly recommended that PCI Express Endpoints be capable of generating the full range of 64-bit addresses. However, if a PCI Express Endpoint supports a smaller address range, and is unable to reach the full address range required by a given platform environment, the corresponding device driver must ensure that all Memory Transaction target buffers fall within the address range supported by the Endpoint. The exact means of ensuring this is platform and operating system specific, and beyond the scope of this specification.

For I/O Requests, the following rules apply:

- I/O Requests route by address, using 32-bit Addressing (see § Figure 2-38)
- I/O Requests have the following restrictions:
 - TC[2:0] must be 000b
 - TH is not applicable to I/O Request and the bit is Reserved
 - Attr[2] is Reserved
 - Attr[1:0] must be 00b
 - AT[1:0] must be 00b. Receivers are not required or encouraged to check this.
 - Length[9:0] must be 00 0000 0001b
 - Last DW BE[3:0] must be 0000b

Receivers may optionally check for violations of these rules (but must not check Reserved bits). These checks are independently optional (see § Section 6.2.3.4). If a Receiver implementing these checks determines that a TLP violates these rules, the TLP is a Malformed TLP.

- If checked, this is a reported error associated with the Receiving Port (see § Section 6.2).

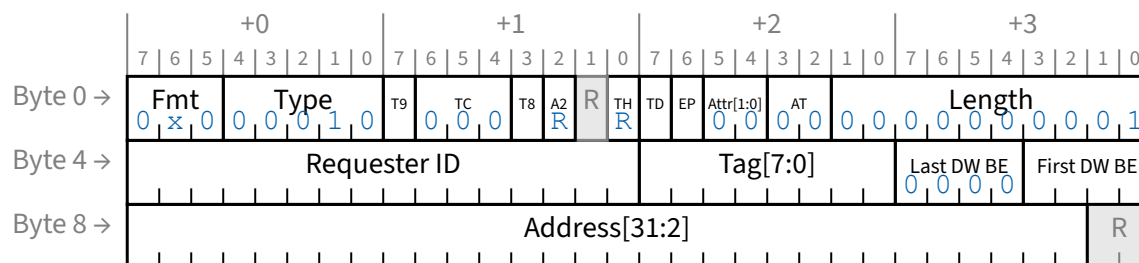


Figure 2-38 I/O Request Header - Non-Flit Mode §

For Configuration Requests, the following rules apply:

- Configuration Requests route by ID, and use a 3 DW header.
- In addition to the header fields included in all Memory, I/O, and Configuration Requests and the ID routing fields, Configuration Requests contain the following additional fields (see § Figure 2-39).
 - Register Number[5:0]

- Extended Register Number[3:0]
- Configuration Requests have the following restrictions:
 - TC[2:0] must be 000b
 - TH is not applicable to Configuration Requests and the bit is Reserved
 - Attr[2] is Reserved
 - Attr[1:0] must be 00b
 - AT[1:0] must be 00b. Receivers are not required or encouraged to check this.
 - Length[9:0] must be 00 0000 0001b
 - Last DW BE[3:0] must be 0000b

Receivers may optionally check for violations of these rules (but must not check Reserved bits). These checks are independently optional (see § Section 6.2.3.4). If a Receiver implementing these checks determines that a TLP violates these rules, the TLP is a Malformed TLP.

- If checked, this is a reported error associated with the Receiving Port (see § Section 6.2).

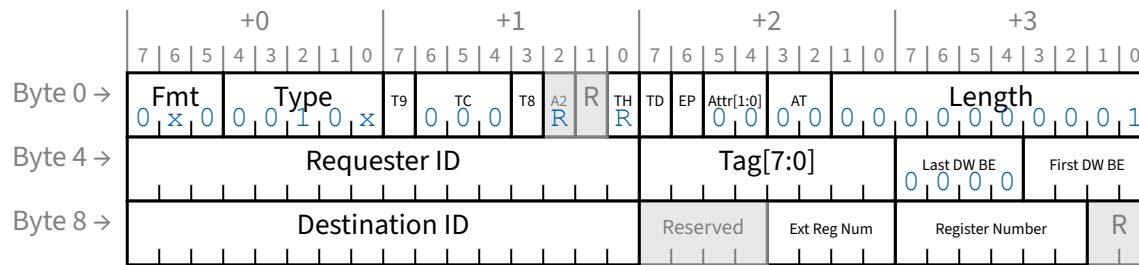


Figure 2-39 Configuration Request Header - Non-Flit Mode §

MSI/MSI-X mechanisms use Memory Write Requests to represent interrupt Messages (see § Section 6.1.4). The Request format used for MSI/MSI-X transactions is identical to the Memory Write Request format defined above, and MSI/MSI-X Requests are indistinguishable from memory writes with regard to ordering, Flow Control, and data integrity.

2.2.7.1.1 TPH Rules – Non-Flit Mode §

- Two formats are specified for TPH. The baseline TPH format without the TPH TLP Prefix (see § Figure 2-41, § Figure 2-42, § Figure 2-43, and § Figure 2-44) must be used for all Requests that provide TPH. The Extended TPH format with the TPH TLP Prefix extends the TPH fields (see § Figure 2-40) to provide additional bits for the Steering Tag (ST) field.

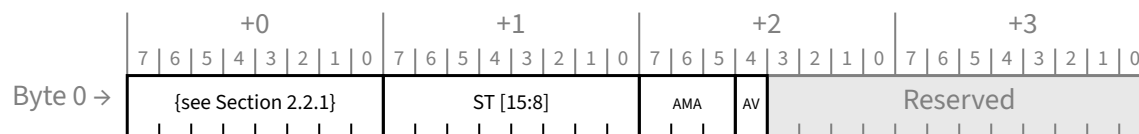


Figure 2-40 TPH TLP Prefix - Non-Flit Mode §

- The optional TPH TLP Prefix is used to provide additional TPH information.
 - The presence of a TPH TLP Prefix is determined by decoding byte 0.

*Table 2-16 TPH TLP Prefix
Bit Mapping - Non-Flit Mode* §

Fields	TPH TLP Prefix
ST[15:8]	Bits 7:0 of byte 1
AMA	Bits 7:5 of byte 2
AV	Bit 4 of byte 2
Reserved	Bits 3:0 of byte 2
Reserved	Bits 7:0 of byte 3

- The TPH TLP Prefix is used to send a non-Zero value for any of:
 - AMA
 - ST[15:8]
- For Requests that target Memory Space, a value of 1b in the TH bit indicates the presence of TPH in the TLP header and optional TPH TLP Prefix (if present).
 - The TH bit must be Set for Requests that provide TPH.
 - The TH bit is permitted to be Set for Requests with a TPH TLP Prefix. When the TH bit is 1b, then ST[15:8] is present and meaningful in the TPH TLP Prefix.
 - When the TH bit is Clear, the PH field is Reserved.
 - The TH bit and the PH field are not applicable and are Reserved for all other Requests.
- For Requests that target Memory Space, the TPH TLP Prefix may be present if the value of the TH bit is 0b. When the AV bit is 1b and the TPH TLP Prefix is present, AMA is present and meaningful in the TPH TLP Prefix.
- For Requests that target Memory Space with the AT field not set to 10b, the AMA field in the TPH TLP Prefix is Reserved.
- The Processing Hints (PH) fields mapping is shown in § Figure 2-41, § Figure 2-42 and § Table 2-17.

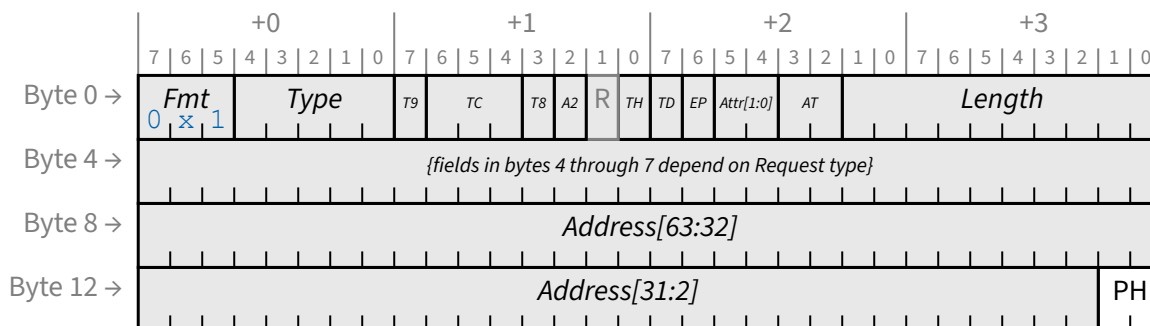


Figure 2-41 Location of PH[1:0] in a 4 DW Request Header - Non-Flit Mode §

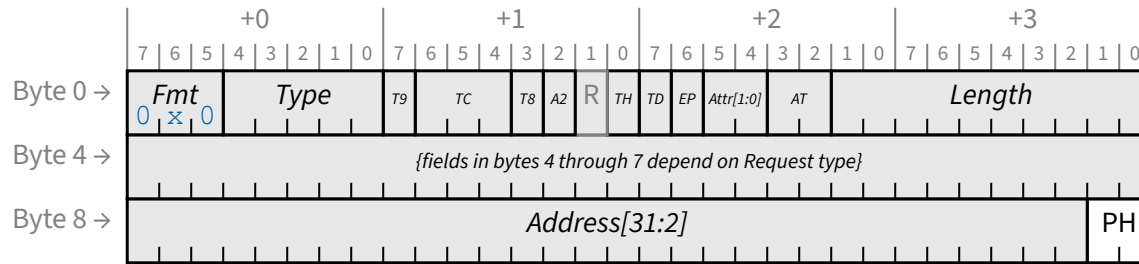


Figure 2-42 Location of PH[1:0] in a 3 DW Request Header - Non-Flit Mode §

Table 2-17 Location of PH[1:0] in TLP Header - Non-Flit Mode §

PH	32-bit Addressing	64-bit Addressing
1:0	Bits 1:0 of Byte 11	Bits 1:0 of Byte 15

- The PH[1:0] field provides information about the data access patterns and is defined as described in § Table 2-18.

Table 2-18 Processing Hint Encoding §

PH[1:0] (b)	Processing Hint	Description
00	Bi-directional data structure	Indicates frequent read and/or write access to data by Host and device
01	Requester	Indicates frequent read and/or write access to data by device
10	Target	Indicates frequent read and/or write access to data by Host
11	Target with Priority	Indicates frequent read and/or write access by Host and indicates high temporal locality for accessed data

The Steering Tag (ST) fields are mapped to the TLP header as shown in § Figure 2-43, § Figure 2-44 and § Table 2-19.

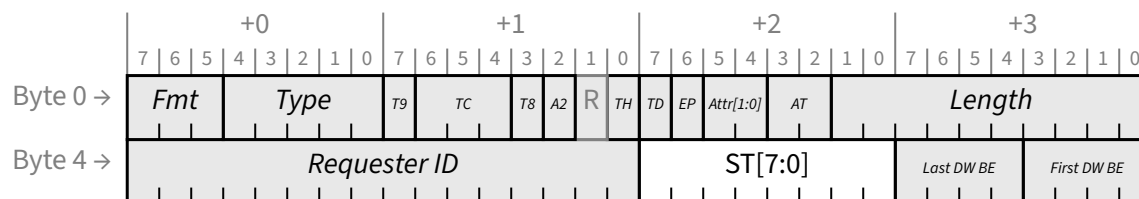


Figure 2-43 Location of ST[7:0] in the Memory Write Request Header - Non-Flit Mode §

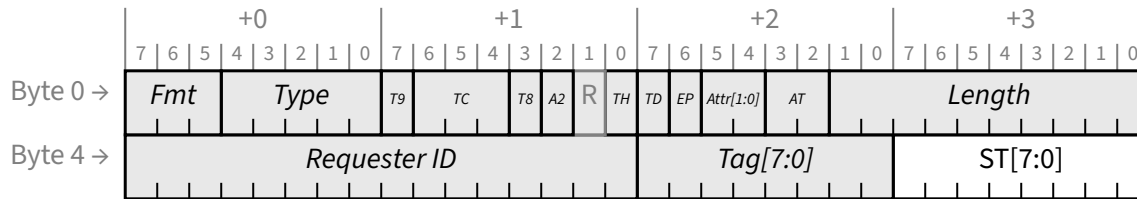


Figure 2-44 Location of ST[7:0] in Memory Read, DMWr, and AtomicOp Request Headers - Non-Flit Mode §

Table 2-19 Location of ST[7:0] in TLP Headers – Non-Flit Mode §

ST Bits	Memory Write Request	Memory Read Request or AtomicOp Request
7:0	Bits 7:0 of Byte 6	Bits 7:0 of Byte 7

- ST[7:0] field carries the Steering Tag value
 - A value of Zero indicates no Steering Tag preference
 - A total of 255 unique Steering Tag values are provided
- A Function that does not support the TPH Completer or Routing capability and receives a transaction with the TH bit Set is required to ignore the TH bit and handle the Request in the same way as Requests of the same transaction type without the TH bit Set.

2.2.7.2 Flit Mode §

Except as stated, rules that apply in Non-Flit Mode also apply in Flit Mode.

- All Memory, I/O, and Configuration Requests include the following fields in addition to the common header fields:
 - A Transaction ID, consisting of Requester ID[15:0] and Tag[13:0], and, for Memory Requests, sometimes also including the Requester Segment[7:0]
- Byte Enable rules for Flit Mode are covered in § Section 2.2.5.2 . There are several notable differences from the Byte Enable rules for Non-Flit Mode covered in § Section 2.2.5.1 .
- For non-UIO Memory Requests, including AtomicOp and DMWr, the rules for the formation and processing of Header Fields are the same as in Non-Flit Mode.
- For UIO Requests, the rules for the formation and processing of Header Fields are the same as in Non-Flit Mode with the following exceptions:
 - Attr[2:1] are Reserved
 - AT[1:0] value of 01b is Reserved (See § Section 10.2.2)
 - When multiple outstanding Group II UIO Requests are issued using the same Transaction ID (see § Section 2.2.6.2), such Requests must have the same value for Attr[0] (i.e., No Snoop).
- For I/O Requests, the rules for the formation and processing of Header Fields are the same as in Non-Flit Mode.
- Configuration Requests must include OHC-A3.
- Configuration Requests must only include OHC-C when they are associated with an IDE stream.
- UIO Requests are only defined for Flit Mode.

- The TH bit, present in Non-Flit Mode Requests, is not supported in Flit Mode.
- The following figures illustrate currently defined Flit Mode Request Headers:
 - Reserved Requests (as indicated in § Table 2-5), are defined in § Section 2.2.4.1 and § Section 2.2.4.2 .

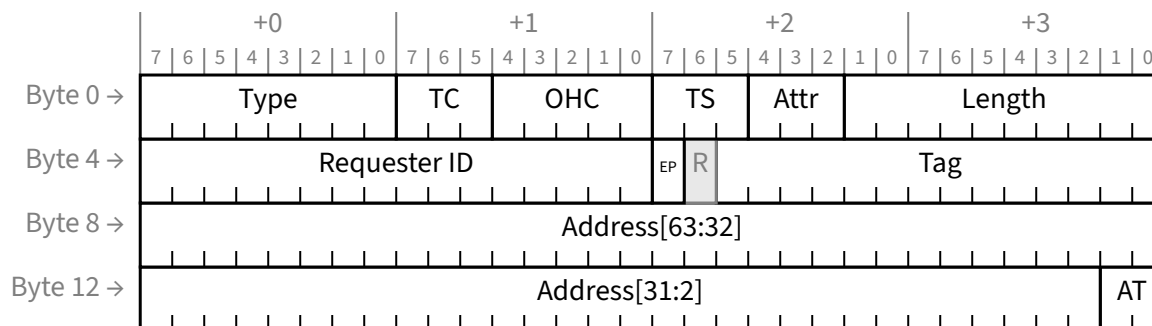


Figure 2-45 Memory Request with 64-bit Addressing Header - Flit Mode §

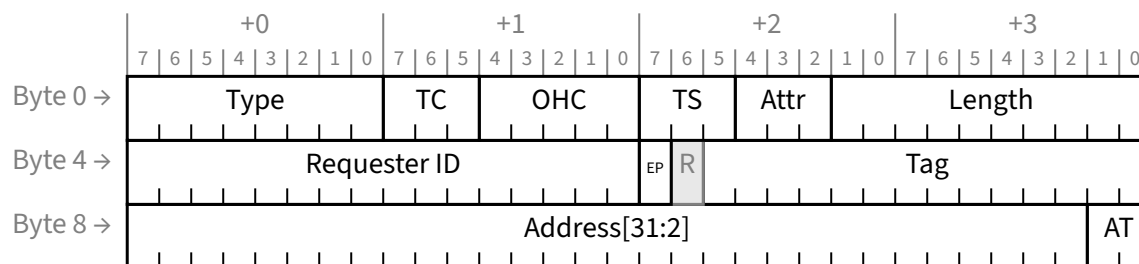


Figure 2-46 Memory Request with 32-bit Addressing Header - Flit Mode §

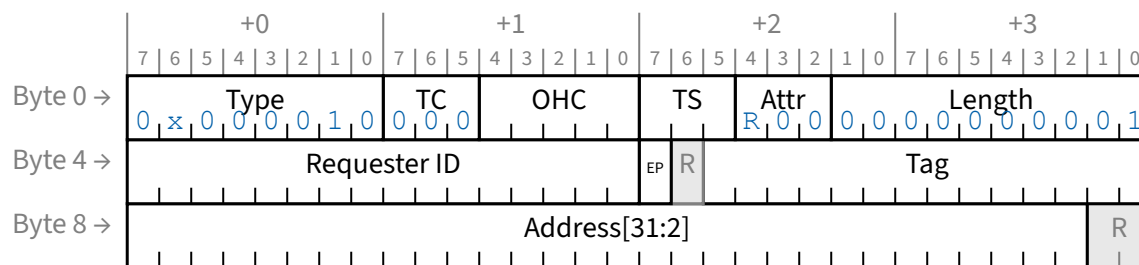


Figure 2-47 I/O Request Header - Flit Mode §

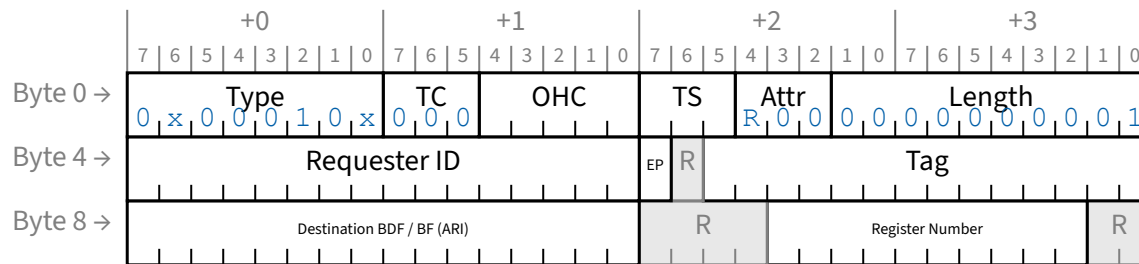


Figure 2-48 Configuration Request Header - Flit Mode §

2.2.8 Message Request Rules §

This document defines the following groups of Messages:

- INTx Interrupt Signaling
- Power Management
- Error Signaling
- Locked Transaction Support
- Slot Power Limit Support
- Vendor-Defined Messages
- Latency Tolerance Reporting (LTR) Messages
- Optimized Buffer Flush/Fill (OBFF) Messages
- Device Readiness Status (DRS) Messages
- Function Readiness Status (FRS) Messages
- Hierarchy ID Messages
- Precision Time Measurement (PTM) Messages
- Integrity and Data Encryption (IDE) Messages

The following rules apply to all Message Requests. Additional rules specific to each type of Message follow.

- All Message Requests include the following fields in addition to the common header fields (see § Figure 2-49 and § Figure 2-50):
 - Requester ID[15:0]
 - Message Code[7:0] - Indicates the particular Message embodied in the Request.
 - EP - For Messages with data only, indicates data payload is poisoned (see § Section 2.7); Reserved for Messages without data.
- All Message Requests use the Msg or MsgD TLP Type.
- The Message Code field must be fully decoded (Message aliasing is not permitted).
- The Attr[2] field is not Reserved unless specifically indicated as Reserved.
- Except as noted, the Attr[1:0] field is Reserved.
- Except as noted, TH is not applicable to Message Requests and the bit is Reserved.

- AT[1:0] must be 00b except for Routed by Address Messages in Flit Mode (see § Table 2-20). Receivers are not required or encouraged to check this.
- Bytes 8 through 15 are Reserved unless specifically defined.
- Bytes 8 through 15 must be copied intact during Translation between Flit Mode and Non-Flit Mode, regardless of Message Code.
- Byte 6, bits 6:0 must be copied intact during Translation between Flit Mode and Non-Flit Mode, regardless of Message Code.
- Message Requests are posted and do not require Completion.
- Message Requests follow the same ordering rules as Memory Write Requests.

Many types of Messages, including Vendor-Defined Messages, are potentially usable in non-D0 states, and it is strongly recommended that the handling of Messages by Ports be the same when the Port's Bridge Function is in D1, D2, and D3_{Hot} as it is in D0. It is strongly recommended that Type 0 Functions support the generation and reception of Messages in non-D0 states.

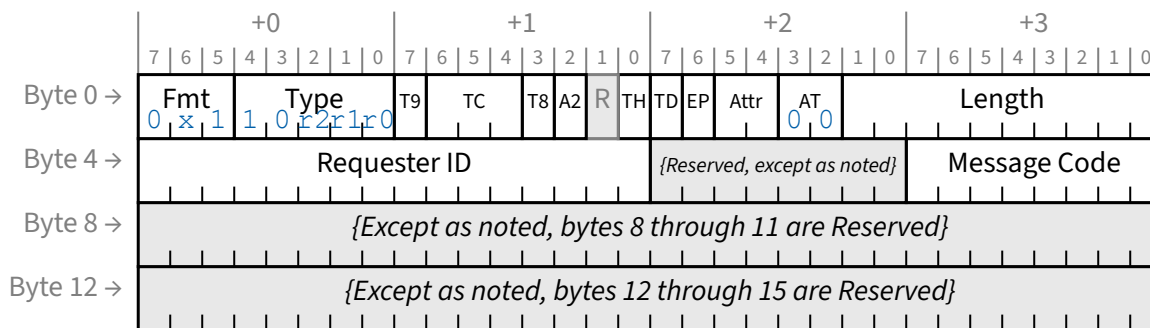


Figure 2-49 Message Request Header - Non-Flit Mode §

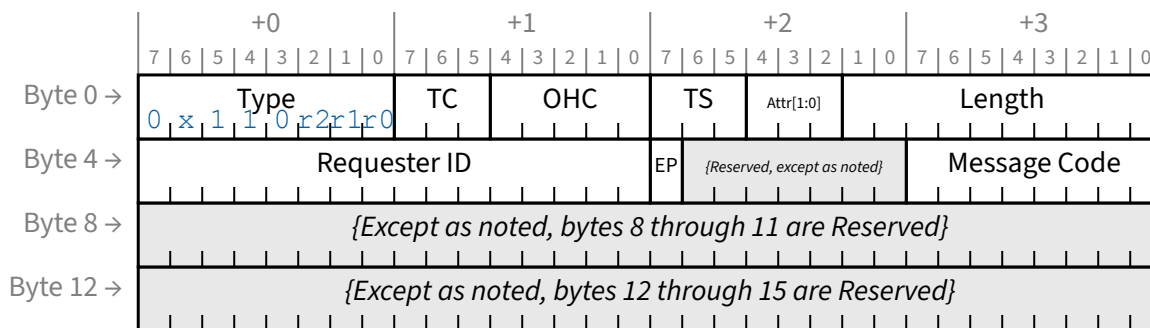


Figure 2-50 Message Request Header - Flit Mode §

In addition to address and ID routing, Messages support several other routing mechanisms. These mechanisms are referred to as “implicit” because no address or ID specifies the destination, but rather the destination is implied by the routing type. The following rules cover Message routing mechanisms:

- Message routing is determined using the r[2:0] sub-field of the Type field

- Message Routing r[2:0] values are defined in § Table 2-20
- Permitted values are defined in the following sections for each Message

Table 2-20 Message Routing §

r[2:0] (b)	Description	Bytes 8 to 15 ²⁷
000	Routed to Root Complex	Reserved
001	Routed by Address + AT, in Flit Mode ²⁸	Address/AT
010	Routed by ID	See § Section 2.2.4.2
011	Broadcast from Root Complex	Reserved
100	Local - Terminate at Receiver	Reserved
101	Gathered and routed to Root Complex ²⁹	Reserved
110 to 111	Reserved - Terminate at Receiver	Reserved

In Flit Mode, when Route by ID is used and the Destination Segment is different from the Requester Segment, OHC-A4 must be present and include the Destination Segment in byte 0 and DSV must be Set. DSV is permitted to be Set when the Destination Segment is the same as the Requester Segment. DSV must be Clear when Route by ID is not used. When DSV is clear, the Destination Segment field must be set to 00h. OHC-A4 must be present for Route by ID Messages that require PASID. OHC-A1 must be present for Routed to Root Complex Messages that require PASID, ER or PMR.

2.2.8.1 INTx Interrupt Signaling - Rules §

A Message Signaled Interrupt (MSI or MSI-X) is the preferred interrupt signaling mechanism in PCI Express (see § Section 6.1). However, in some systems, there may be Functions that cannot support the MSI or MSI-X mechanisms, or it is possible that system firmware/software does not enable MSI or MSI-X. The INTx virtual wire interrupt signaling mechanism, when implemented, can be used to support cases where the MSI or MSI-X mechanisms cannot be used. Switches must support passing interrupts via this mechanism. The following rules apply to the INTx Interrupt Signaling mechanism:

- The INTx mechanism uses eight distinct Messages (see § Table 2-21).
- Assert_INTx/Deassert_INTx Messages do not include a data payload (TLP Type is Msg).
- The Length field is Reserved.
- With Assert_INTx/Deassert_INTx Messages, the Function Number field in the Requester ID must be 0. Note that the Function Number field is a different size for non-ARI and ARI Requester IDs.
- Assert_INTx/Deassert_INTx Messages are only issued by Upstream Ports.
 - Receivers may optionally check for violations of this rule. If a Receiver implementing this check determines that an Assert_INTx/Deassert_INTx violates this rule, it must handle the TLP as a Malformed TLP.
 - This is a reported error associated with the Receiving Port (see § Section 6.2).
- Assert_INTx and Deassert_INTx interrupt Messages must use the default Traffic Class designator (TC0). Receivers must check for violations of this rule. If a Receiver determines that a TLP violates this rule, it must handle the TLP as a Malformed TLP.

27. Except as noted, e.g., Vendor-Defined Messages.

28. Note that no Messages defined in this document use Address routing.

29. This routing type is used only for PME_TO_Ack, and is described in § Section 5.3.3.2.1 .

- This is a reported error associated with the Receiving Port (see § Section 6.2).

Table 2-21 INTx Mechanism Messages §

Name	Code[7:0] (b)	Routing r[2:0] (b)	Support ³⁰				Description/Comments
			RC	Ep	Sw	Br	
Assert_INTA	0010 0000	100	All:				Assert INTA virtual wire Note: These Messages are used for Conventional PCI-compatible INTx emulation.
			r		tr		
			As Required:				
				t		t	
Assert_INTB	0010 0001	100	All:				Assert INTB virtual wire
			r		tr		
			As Required:				
				t		t	
Assert_INTC	0010 0010	100	All:				Assert INTC virtual wire
			r		tr		
			As Required:				
				t		t	
Assert_INTD	0010 0011	100	All:				Assert INTD virtual wire
			r		tr		
			As Required:				
				t		t	
Deassert_INTA	0010 0100	100	All:				Deassert INTA virtual wire
			r		tr		
			As Required:				
				t		t	
Deassert_INTB	0010 0101	100	All:				Deassert INTB virtual wire
			r		tr		
			As Required:				
				t		t	
Deassert_INTC	0010 0110	100	All:				Deassert INTC virtual wire
			r		tr		
			As Required:				

30. Abbreviations: RC = Root Complex Sw = Switch (only used with “Link” routing) Ep = Endpoint Br = PCI Express (primary) to PCI/PCI-X (secondary) Bridge r = Supports as Receiver t = Supports as Transmitter

Name	Code[7:0] (b)	Routing r[2:0] (b)	Support				Description/Comments
			RC	Ep	Sw	Br	
				t		t	
Deassert_INTD	0010 0111	100	All:				Deassert INTD virtual wire
			r		tr		
			As Required:				
				t		t	

The Assert_INTx/Deassert_INTx Message pairs constitute four “virtual wires” for each of the legacy PCI interrupts designated A, B, C, and D. The following rules describe the operation of these virtual wires:

- The components at both ends of each Link must track the logical state of the four virtual wires using the Assert_INTx/Deassert_INTx Messages to represent the active and inactive transitions (respectively) of each corresponding virtual wire.
 - An Assert_INTx represents the active going transition of the INTx (x = A, B, C, or D) virtual wire
 - A Deassert_INTx represents the inactive going transition of the INTx (x = A, B, C, or D) virtual wire
- When the local logical state of an INTx virtual wire changes at an Upstream Port, the Port must communicate this change in state to the Downstream Port on the other side of the same Link using the appropriate Assert_INTx or Deassert_INTx Message.

Note: Duplicate Assert_INTx/Deassert_INTx Messages have no effect, but are not errors.

- INTx Interrupt Signaling is disabled when the Interrupt Disable bit of the Command register (see [§ Section 7.5.1.1.3](#)) is Set.
 - Any INTx virtual wires that are active when the Interrupt Disable bit is Set must be deasserted by transmitting the appropriate Deassert_INTx Message(s).
- Virtual and actual PCI to PCI Bridges must map the virtual wires tracked on the secondary side of the Bridge according to the Device Number of the device on the secondary side of the Bridge, as shown in [§ Table 2-22](#).
- Switches must track the state of the four virtual wires independently for each Downstream Port, and present a “collapsed” set of virtual wires on its Upstream Port.
- If a Switch Downstream Port goes to DL_Down status, the INTx virtual wires associated with that Port must be deasserted, and the Switch Upstream Port virtual wire state updated accordingly.
 - If this results in deassertion of any Upstream INTx virtual wires, the appropriate Deassert_INTx Message(s) must be sent by the Upstream Port.
- The Root Complex must track the state of the four INTx virtual wires independently for each of its Downstream Ports, and map these virtual signals to system interrupt resources.
 - Details of this mapping are system implementation specific.
- If a Downstream Port of the Root Complex goes to DL_Down status, the INTx virtual wires associated with that Port must be deasserted, and any associated system interrupt resource request(s) must be discarded.

Table 2-22 Bridge Mapping for INTx Virtual Wires §

Requester ID[7:3] from the Assert_INTx/Deassert_INTx Message received on Secondary Side of Bridge (Interrupt Source ³¹) If ARI Forwarding is enabled, the value 0 must be used instead of Requester ID[7:3].	INTx Virtual Wire on Secondary Side of Bridge	Mapping to INTx Virtual Wire on Primary Side of Bridge
0,4,8,12,16,20,24,28	INTA	INTA
	INTB	INTB
	INTC	INTC
	INTD	INTD
1,5,9,13,17,21,25,29	INTA	INTB
	INTB	INTC
	INTC	INTD
	INTD	INTA
2,6,10,14,18,22,26,30	INTA	INTC
	INTB	INTD
	INTC	INTA
	INTD	INTB
3,7,11,15,19,23,27,31	INTA	INTD
	INTB	INTA
	INTC	INTB
	INTD	INTC

IMPLEMENTATION NOTE: SYSTEM INTERRUPT MAPPING §

Note that system software (including BIOS and operating system) needs to comprehend the remapping of legacy interrupts (INTx mechanism) in the entire topology of the system (including hierarchically connected Switches and subordinate PCI Express/PCI Bridges) to establish proper correlation between PCI Express device interrupt and associated interrupt resources in the system interrupt controller. The remapping described by § Table 2-22 is applied hierarchically at every Switch. In addition, PCI Express/PCI and PCI/PCI Bridges perform a similar mapping function.

31. The Requester ID of an Assert_INTx/Deassert_INTx Message will correspond to the Transmitter of the Message on that Link, and not necessarily to the original source of the interrupt.

IMPLEMENTATION NOTE: VIRTUAL WIRE MAPPING FOR INTX INTERRUPTS FROM ARI DEVICES §

The implied Device Number for an ARI Device is 0. When ARI-aware software (including BIOS and operating system) enables ARI Forwarding in the Downstream Port immediately above an ARI Device in order to access its Extended Functions, software must comprehend that the Downstream Port will use Device Number 0 for the virtual wire mappings of INTx interrupts coming from all Functions of the ARI Device. If non-ARI-aware software attempts to determine the virtual wire mappings for Extended Functions, it can come up with incorrect mappings by examining the traditional Device Number field and finding it to be non-0.

2.2.8.2 Power Management Messages §

These Messages are used to support PCI Express power management, which is described in detail in § Chapter 5. . The following rules define the Power Management Messages:

- § Table 2-23 defines the Power Management Messages.
- Power Management Messages do not include a data payload (TLP Type is Msg).
- The Length field is Reserved.
- With PM_Active_State_Nak Messages, the Function Number field in the Requester ID must contain the Function Number of the Downstream Port that sent the Message, or else 000b for compatibility with earlier revisions of this specification.
- With PME_TO_Ack Messages, the Function Number field in the Requester ID must be Reserved, or else for compatibility with earlier revisions of this specification must contain the Function Number of one of the Functions associated with the Upstream Port. Note that the Function Number field is a different size for non-ARI and ARI Requester IDs.
- Power Management Messages must use the default Traffic Class designator (TC0). Receivers must check for violations of this rule. If a Receiver determines that a TLP violates this rule, it must handle the TLP as a Malformed TLP.
 - This is a reported error associated with the Receiving Port (see § Section 6.2).

Table 2-23 Power Management Messages §

Name	Code[7:0] (b)	Routing r[2:0] (b)	Support				Description/Comments
			RC	Ep	Sw	Br	
<i>PM_Active_State_Nak</i>	0001 0100	100	t	r	tr	r	Terminate at Receiver
<i>PM_PME</i>	0001 1000	000	All:				Sent Upstream by PME-requesting component. Propagates Upstream.
			r		tr	t	
			If PME supported:				
				t			

Name	Code[7:0] (b)	Routing r[2:0] (b)	Support				Description/Comments
			RC	Ep	Sw	Br	
PME_Turn_Off	0001 1001	011	t	r		r	Broadcast Downstream
PME_TO_Ack	0001 1011	101	r	t		t	Sent Upstream by Upstream Port. See § <u>Section 5.3.3.2.1</u> .
			(Note: Switch handling is special)				

2.2.8.3 Error Signaling Messages §

Error Signaling Messages are used to signal errors that occur on specific transactions and errors that are not necessarily associated with a particular transaction. These Messages are initiated by the agent that detected the error.

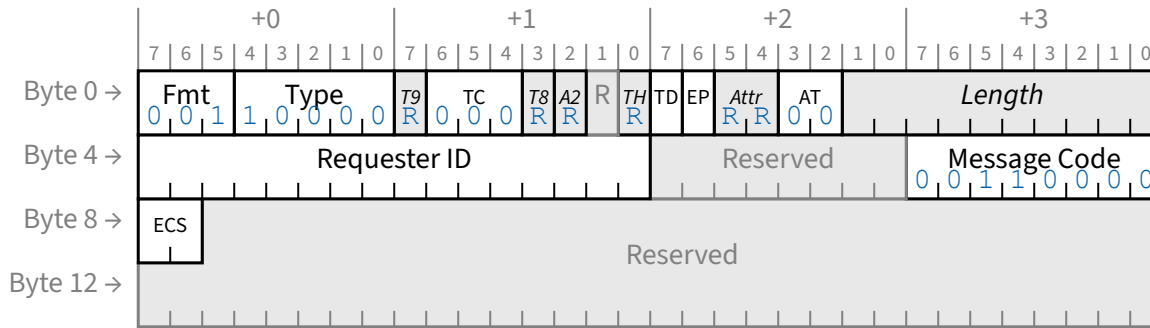
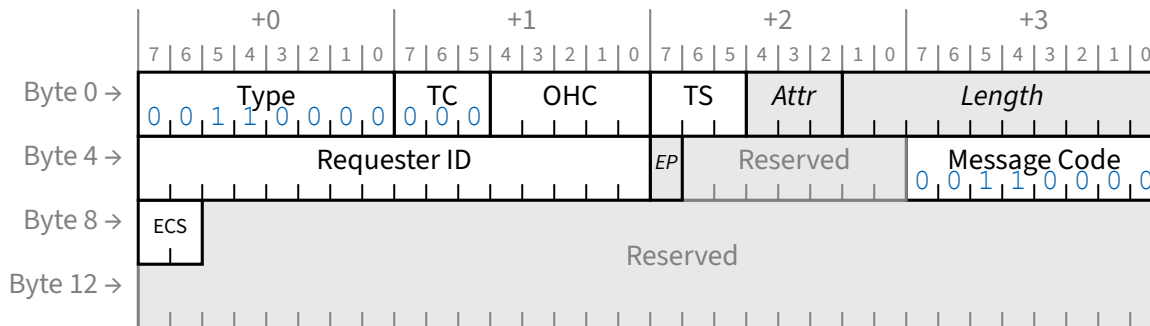
- § Table 2-24 defines the Error Signaling Messages.
- Error Signaling Messages do not include a data payload (TLP Type is Msg).
- The Length field is Reserved.
- With Error Signaling Messages, the Function Number field in the Requester ID must indicate which Function is signaling the error. Note that the Function Number field is a different size for non-ARI and ARI Requester IDs.
- Error Signaling Messages must use the default Traffic Class designator (TC0) Receivers must check for violations of this rule. If a Receiver determines that a TLP violates this rule, it must handle the TLP as a Malformed TLP.
 - This is a reported error associated with the Receiving Port (see § Section 6.2).

Table 2-24 Error Signaling Messages §

Name	Code[7:0] (b)	Routing r[2:0] (b)	Support				Description/Comments
			RC	Ep	Sw	Br	
ERR_COR	0011 0000	000	r	t	tr	t	This Message is issued when the Function or Device detects a correctable error on the PCI Express interface.
ERR_NONFATAL	0011 0001	000	r	t	tr	t	This Message is issued when the Function or Device detects a Non-Fatal, uncorrectable error on the PCI Express interface.
ERR_FATAL	0011 0011	000	r	t	tr	t	This Message is issued when the Function or Device detects a Fatal, uncorrectable error on the PCI Express interface.

The initiator of the Message is identified with the Requester ID of the Message header. The Root Complex translates these error Messages into platform level events. Refer to § Section 6.2 for details on uses for these Messages.

- ERR_COR Messages have an ERR_COR Subclass (ECS) field in the Message header that enables different subclasses to be distinguished from each other. See § Figure 2-51 and § Figure 2-52. ERR_NONFATAL and ERR_FATAL Messages do not have the ECS field.

Figure 2-51 **ERR_COR Message - Non-Flit Mode** §Figure 2-52 **ERR_COR Message - Flit Mode** §

- The ERR_COR Subclass (ECS) field is encoded as shown in § Table 2-25, indicating the ERR_COR Message subclass.

Table 2-25 **ERR_COR Subclass (ECS) Field Encodings** §

ECS Coding (b)	Description
00	ECS Legacy - The value inherently used if a Requester does not support ECS capability. ECS-capable Requesters must not use this value. See § Section 7.5.3.3.
01	ECS SIG_SFW - Must be used by an ECS-capable Requester when signaling a DPC or SFI event with an ERR_COR Message.
10	ECS SIG_OS - Must be used by an ECS-capable Requester when signaling an AER or RP PIO event with an ERR_COR Message.
11	ECS Extended - Intended for possible future use. Requesters must not use this value. Receivers must handle the signal internally the same as ECS SIG_OS.

2.2.8.4 Locked Transactions Support §

The Unlock Message is used to support Lock Transaction sequences. Refer to § Section 6.5 for details on Lock Transaction sequences. The following rules apply to the formation of the Unlock Message:

- § Table 2-26 defines the Unlock Messages.
- The Unlock Message does not include a data payload (TLP Type is Msg).
- The Length field is Reserved.
- With Unlock Messages, the Function Number field in the Requester ID is Reserved.
- The Unlock Message must use the default Traffic Class designator (TC0). Receivers must check for violations of this rule. If a Receiver determines that a TLP violates this rule, it must handle the TLP as a Malformed TLP.
 - This is a reported error associated with the Receiving Port (see § Section 6.2).

Table 2-26 Unlock Message §

Name	Code[7:0] (b)	Routing r[2:0] (b)	Support				Description/Comments
			RC	Ep	Sw	Br	
Unlock	0000 0000	011	t	r	tr	r	Unlock Completer

2.2.8.5 Slot Power Limit Support §

This Message is used to convey a slot power limitation value from a Downstream Port (of a Root Complex or a Switch) to an Upstream Port of a component (with Endpoint, Switch, or PCI Express-PCI Bridge Functions) attached to the same Link.

- § Table 2-27 defines the Set_Slot_Power_Limit Message.
- The Set_Slot_Power_Limit Message includes a 1 DW data payload (TLP Type is MsgD).
- The Set_Slot_Power_Limit Message must use the default Traffic Class designator (TC0). Receivers must check for violations of this rule. If a Receiver determines that a TLP violates this rule, it must handle the TLP as a Malformed TLP.
 - This is a reported error associated with the Receiving Port (see § Section 6.2).

Table 2-27 Set_Slot_Power_Limit Message §

Name	Code[7:0] (b)	Routing r[2:0] (b)	Support				Description/Comments
			RC	Ep	Sw	Br	
Set_Slot_Power_Limit	0101 0000	100	t	r	tr	r	Set Slot Power Limit in Upstream Port

The Set_Slot_Power_Limit Message includes a one DW data payload. The data payload is copied from the Slot Capabilities register of the Downstream Port and is written into the Device Capabilities register of the Upstream Port on the other side of the Link. Bits 1:0 of Byte 1 of the data payload map to the Slot Power Limit Scale field and bits 7:0 of Byte 0 map to the Slot Power Limit Value field. Bits 7:0 of Byte 3, 7:0 of Byte 2, and 7:2 of Byte 1 of the data payload must all be set to zero by the Transmitter and ignored by the Receiver. This Message must be sent automatically by the Downstream Port (of a Root Complex or a Switch) when one of the following events occurs:

- On a Configuration Write to the Slot Capabilities register (see § Section 7.5.3.9) when the Data Link Layer reports DL_Up status.
- Any time when a Link transitions from a non-DL_Up status to a DL_Up status (see § Section 2.9.2) and the Auto Slot Power Limit Disable bit is Clear in the Slot Control Register. This transmission is optional if the Slot Capabilities register has not yet been initialized.

The component on the other side of the Link (with Endpoint, Switch, or Bridge Functions) that receives Set_Slot_Power_Limit Message must copy the values in the data payload into the Device Capabilities register associated with the component's Upstream Port. PCI Express components that are targeted exclusively for integration on the system planar (e.g., system board) as well as components that are targeted for integration on an adapter where power consumption of the entire adapter is below the lowest power limit specified for the adapter form factor (as defined in the corresponding form factor specification) are permitted to hardwire the value of all 0's in the Captured Slot Power Limit Scale and Captured Slot Power Limit Value fields of the Device Capabilities Register, and are not required to copy the Set_Slot_Power_Limit Message payload into that register.

For more details on Power Limit control mechanism see § Section 6.9 .

2.2.8.6 Vendor-Defined Messages §

The Vendor-Defined Messages allow expansion of PCI Express messaging capabilities, either as a general extension to [PCIe] or a vendor-specific extension. This section defines the rules associated with these Messages generically.

- The Vendor-Defined Messages (see § Table 2-28) use the header format shown in § Figure 2-53 and § Figure 2-54.
 - The Requester ID is implementation specific. The Requester ID field *MUST*³² contain the value associated with the Requester.
 - If the Route by ID routing is used, bytes 8 and 9 form a 16-bit field for the destination ID
 - otherwise these bytes are Reserved.
 - Bytes 10 and 11 form a 16-bit field for the Vendor ID, as defined by PCI-SIG[®], of the vendor defining the Message.
 - Bytes 12 through 15 are available for vendor definition.
 - The low 7 bits of byte 6 is available for vendor definition. Byte 6, bit 7 is Reserved in Non-Flit Mode and is the EP bit in Flit Mode.

Table 2-28 Vendor-Defined Messages §

Name	Code[7:0] (b)	Routing r[2:0] (b)	Support				Description/Comments
			RC	Ep	Sw	Br	
Vendor-Defined Type 0	0111 1110	000, 010, 011, 100	See Note 1.				Triggers detection of UR by Completer if not implemented.
Vendor-Defined Type 1	0111 1111	000, 010, 011, 100	See Note 1.				Silently discarded by Completer if not implemented.

Notes:

1. Transmission by Endpoint/Root Complex/Bridge is implementation specific. Switches must forward received Messages using Routing r[2:0] field values of 000b, 010b, and 011b.

32. ACS Source Validation (see § Section 6.12.1.1) checks the Requester ID on all Requests, including Vendor-Defined Messages. This validation depends on the Requester ID properly identifying the Requester.

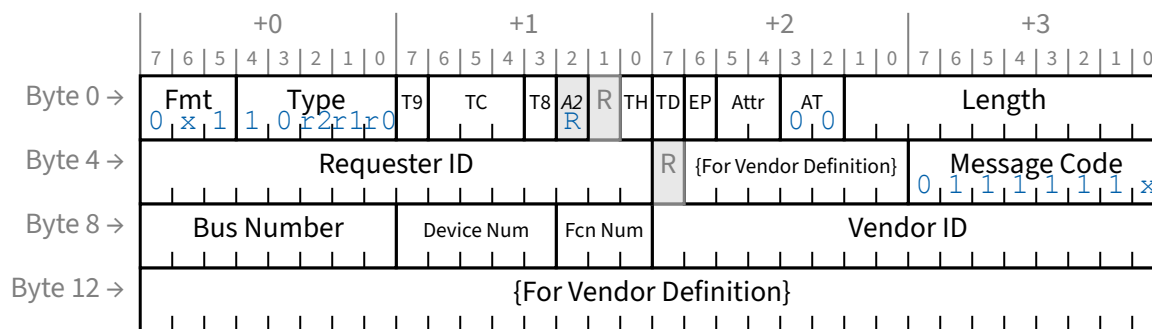


Figure 2-53 Header for Vendor-Defined Messages - Non-Flit Mode §

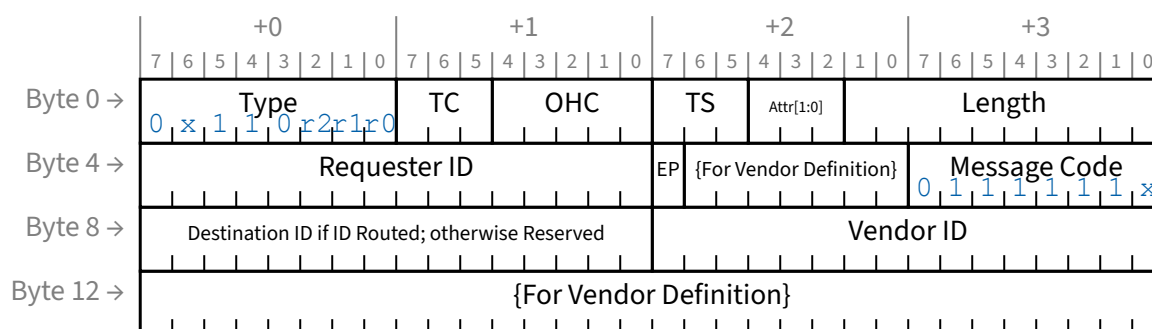


Figure 2-54 Header for Vendor-Defined Messages - Flit Mode §

- A data payload may be included with either type of Vendor-Defined Message (TLP type is Msg if no data payload is included or MsgD if a data payload is included).
- For both types of Vendor-Defined Messages, the Attr[1:0] and Attr[2] fields are not Reserved.
- Messages defined by different vendors or by PCI-SIG are distinguished by the value in the Vendor ID field.
 - The further differentiation of Messages defined by a particular vendor is beyond the scope of this document.
 - Support for Messages defined by a particular vendor is implementation specific, and beyond the scope of this document.
- Completers silently discard Vendor-Defined Type 1 Messages that they are not designed to receive - this is not an error condition.
 - When an ID Routed Message targeting a Function that is not implemented is detected, it is implementation specific whether that message is silently discarded or signals Unsupported Request.
- Completers handle the receipt of an unsupported Vendor-Defined Type 0 Message as an Unsupported Request, and the error is reported according to § Section 6.2.

[PCIe-to-PCI-PCI-X-Bridge] defines additional requirements for Vendor-Defined Messages that are designed to be interoperable with PCI-X Device ID Messages. This includes restrictions on the contents of the Tag[7:0] field and the Length[9:0] field as well as specific use of Bytes 12 through 15 of the message header. Vendor-Defined Messages intended for use solely within a PCI Express environment (i.e., not intended to address targets behind a PCI Express to

PCI/PCI-X Bridge) are not subject to the additional rules. Refer to [PCIe-to-PCI-PCI-X-Bridge] for details. Refer to § Section 2.2.6.2 for considerations regarding larger-Tag capabilities.

2.2.8.6.1 PCI-SIG Defined VDMs §

PCI-SIG-Defined VDMs are Vendor-Defined Type 1 Messages that use the PCI-SIG® Vendor ID (0001h). As a Vendor-Defined Type 1 Message, each is silently discarded by a Completer if the Completer does not implement it.

Beyond the rules for other Vendor-Defined Type 1 Messages, the following rules apply to the formation of the PCI-SIG-Defined VDMs:

- PCI-SIG-Defined VDMs use the Header format shown in § Figure 2-55 and § Figure 2-56.
- The Requester ID field must contain the value associated with the Requester.
- The Message Code must be 0111 1111b.
- The Vendor ID must be 0001h, which is assigned to the PCI-SIG.
- The Subtype field distinguishes the specific PCI-SIG-Defined VDMs. See § Appendix F. for a list of PCI-SIG-Defined VDMs.

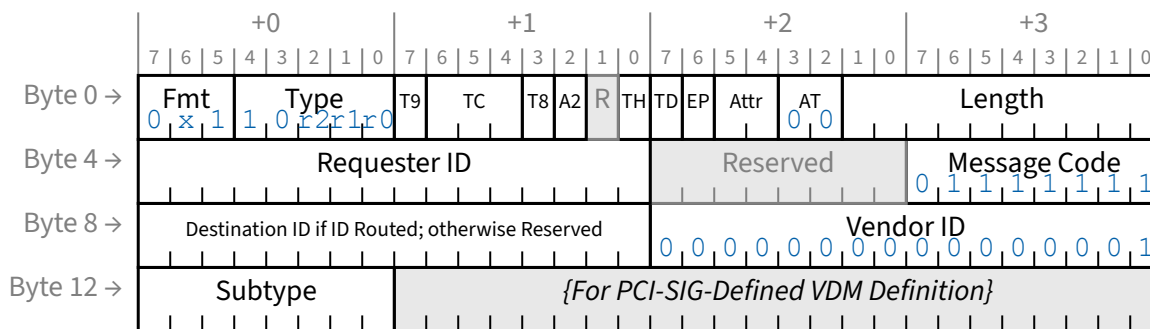


Figure 2-55 Header for PCI-SIG-Defined VDMs - Non-Flit Mode §

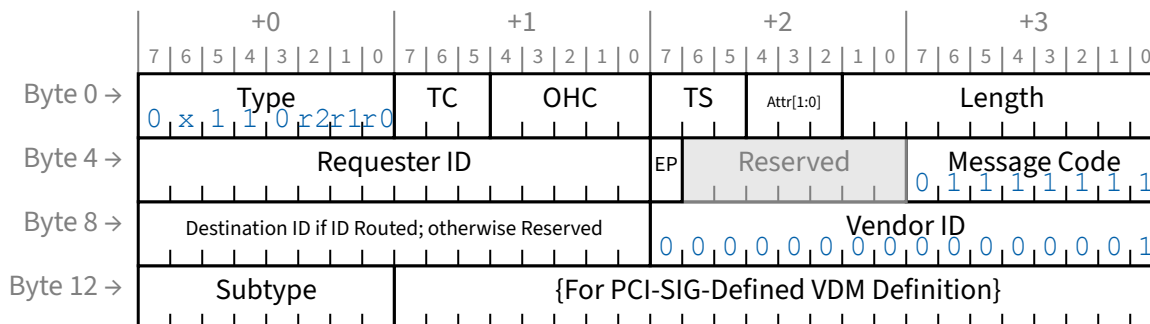


Figure 2-56 Header for PCI-SIG-Defined VDMs - Flit Mode §

2.2.8.6.2 Device Readiness Status (DRS) Message §

The Device Readiness Status (DRS) protocol (see § Section 6.22.1) uses the PCI-SIG-Defined VDM mechanism (see § Section 2.2.8.6.1). The DRS Message is a PCI-SIG-Defined VDM (Vendor-Defined Type 1 Message) with no data payload.

Beyond the rules for other PCI-SIG-Defined VDMs, the following rules apply to the formation of DRS Messages:

- § Table 2-29, § Figure 2-57, and § Figure 2-58 illustrate and define the DRS Message.
- The TLP Type must be Msg.
- The TC[2:0] field must be 000b.
- The Attr[2:0] field is Reserved.
- The Tag field is Reserved.
- The Subtype field must be 08h.
- The Message Routing field must be 100b - Local - Terminate at Receiver.

Receivers may optionally check for violations of these rules (but must not check Reserved bits). These checks are independently optional (see § Section 6.2.3.4). If a Receiver implementing these checks determines that a TLP violates these rules, the TLP is a Malformed TLP.

- If checked, this is a reported error associated with the Receiving Port (see § Section 6.2).

Table 2-29 DRS Message §

Name	Code[7:0] (b)	Routing r[2:0] (b)	Support				Description/Comments
			RC	Ep	Sw	Br	
DRS Message	0111 1111	100	r	t	tr		Device Readiness Status

The format of the DRS Message is shown in § Figure 2-57 and § Figure 2-58:

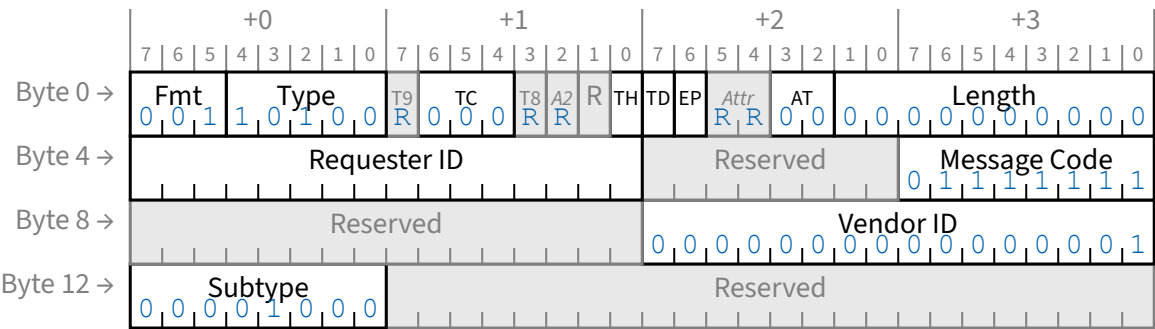


Figure 2-57 DRS Message - Non-Flit Mode §

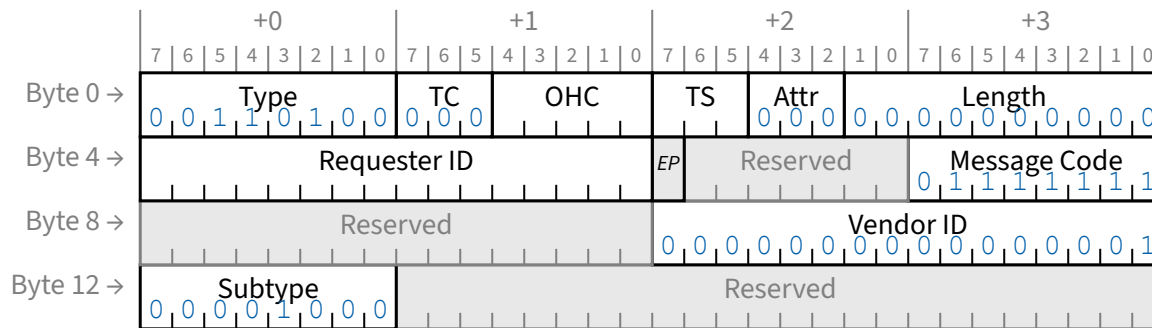


Figure 2-58 DRS Message - Flit Mode §

2.2.8.6.3 Function Readiness Status Message (FRS Message) §

The Function Readiness Status (FRS) protocol (see § Section 6.22.2) uses the PCI-SIG-Defined VDM mechanism (see § Section 2.2.8.6.1). The FRS message is a PCI-SIG-Defined VDM (Vendor-Defined Type 1 Message) with no data payload.

Beyond the rules for other PCI-SIG-Defined VDMs, the following rules apply to the formation of FRS Messages:

- § Table 2-30, § Figure 2-59, and § Figure 2-60 illustrate and define the FRS Message.
- The TLP Type must be Msg.
- The TC[2:0] field must be 000b.
- The Attr[2:0] field is Reserved.
- The Tag field is Reserved.
- The Subtype field must be 09h.
- The FRS Reason[3:0] field indicates why the FRS Message was generated. Encodings are:

0001b: DRS Message Received

The Downstream Port indicated by the Message Requester ID received a DRS Message and has the DRS Signaling Control field in the Link Control Register set to DRS to FRS Signaling Enabled

0010b: D3_{Hot} to D0 Transition Completed

A D3_{Hot} to D0 transition has completed, and the Function indicated by the Message Requester ID is now Configuration-Ready and has returned to the D0_{uninitialized} or D0_{active} state depending on the setting of the No_Soft_Reset bit (see § Section 7.5.2.2)

0011b: FLR Completed

An FLR has completed, and the Function indicated by the Message Requester ID is now Configuration-Ready

1000b: VF Enabled

The Message Requester ID indicates a Physical Function (PF) - All Virtual Functions (VFs) associated with that PF are now Configuration-Ready

1001b: VF Disabled

The Message Requester ID indicates a PF - All VFs associated with that PF have been disabled and the Single Root I/O Virtualization (SR-IOV) data structures in that PF may now be accessed.

Others:

All other encodings are Reserved

- The Message Routing field must be 000b - Routed to Root Complex

Receivers may optionally check for violations of these rules (but must not check Reserved bits). These checks are independently optional (see § Section 6.2.3.4). If a Receiver implementing these checks determines that a TLP violates these rules, the TLP is a Malformed TLP.

- If checked, this is a reported error associated with the Receiving Port (see § Section 6.2).

Table 2-30 FRS Message §

Name	Code[7:0] (b)	Routing r[2:0] (b)	Support				Description/Comments
			RC	Ep	Sw	Br	
FRS Message	0111 1111	000	r	t	tr		Function Readiness Status

The format of the FRS Message is shown in § Figure 2-59 and § Figure 2-60 below:

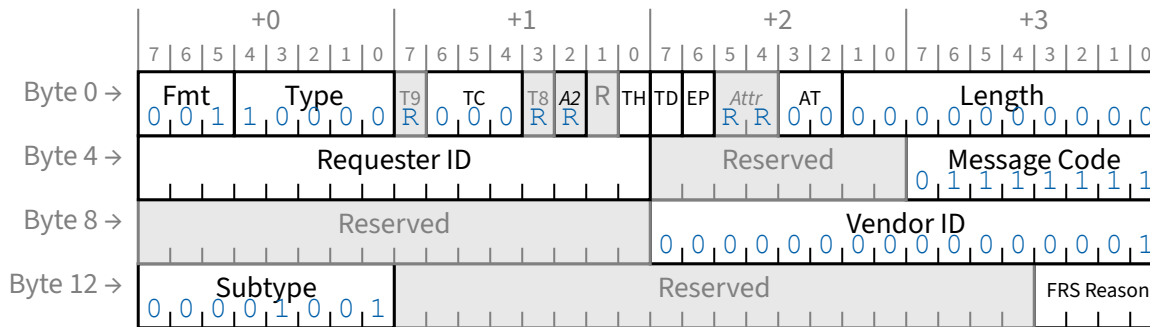


Figure 2-59 FRS Message - Non-Flit Mode §

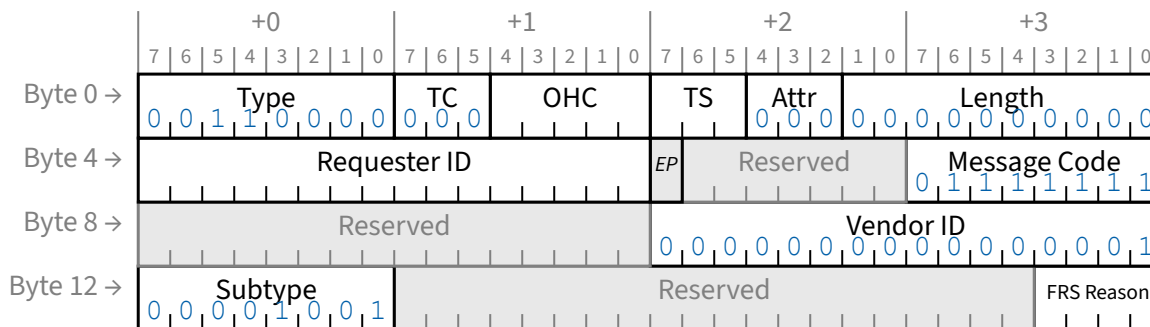


Figure 2-60 FRS Message - Flit Mode §

2.2.8.6.4 Hierarchy ID Message §

Hierarchy ID uses the PCI-SIG-Defined VDM mechanism (see § Section 2.2.8.6.1). The Hierarchy ID Message is a PCI-SIG-Defined VDM (Vendor-Defined Type 1 Message) with payload (MsgD).

Beyond the rules for other PCI-SIG-Defined VDMs, the following rules apply to the formation of Hierarchy ID Messages:

- § Table 2-31, § Figure 2-61, and § Figure 2-62 illustrate and define the Hierarchy ID Message.
- The TLP Type must be MsgD.
- Each Message must include a 4-DWORD data payload.
- The Length field must be 4.
- The TC[2:0] field must be 000b.
- The Attr[2:0] field is Reserved.
- The Tag field is Reserved.
- The Subtype field is 01h.
- The Message Routing field must be 011b - Broadcast from Root Complex.

Receivers may optionally check for violations of these rules (but must not check Reserved bits). These checks are independently optional (see § Section 6.2.3.4). If a Receiver implementing these checks determines that a TLP violates these rules, the TLP is a Malformed TLP.

- If checked, this is a reported error associated with the Receiving Port (see § Section 6.2).

The payload of each Hierarchy ID Message contains the lower 128-bits of the System GUID.

For details of the Hierarchy ID, GUID Authority ID, and System GUID fields see § Section 6.25 .

Table 2-31 Hierarchy ID Message §

Name	Code[7:0] (b)	Routing r[2:0] (b)	Support				Description/Comments
			RC	Ep	Sw	Br	
Hierarchy ID Message	0111 1111	011	t	r	tr		Hierarchy ID

The format of the Hierarchy ID Message is shown in § Figure 2-61 and § Figure 2-62 below:

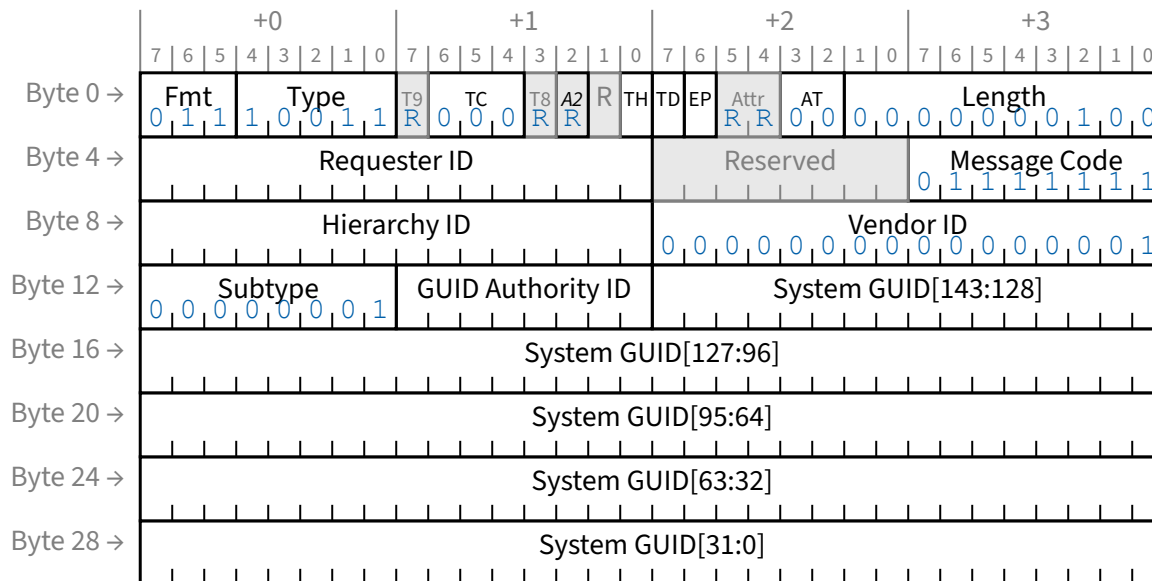


Figure 2-61 Hierarchy ID Message - Non-Flit Mode §

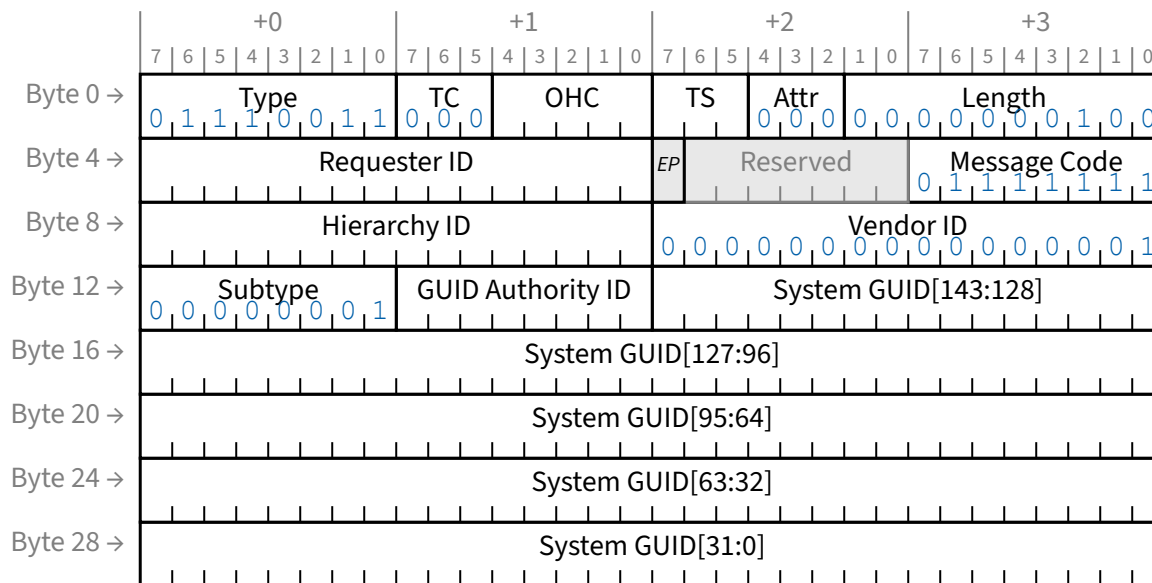


Figure 2-62 Hierarchy ID Message - Flit Mode §

2.2.8.7 Ignored Messages §

The messages listed in § Table 2-32 were previously used for a mechanism (Hot-Plug Signaling) that is no longer supported. Transmitters *MUST@FLIT* not transmit these messages. If message transmission is implemented, it must conform to the requirements of [PCIe-1.0a].

Beyond normal Link-Layer processing and mandatory checking for properly-formed TLPs, Receivers *MUST@FLIT* not process these messages further (i.e., carry out their originally architected Transaction-Layer functionality). If complete processing of these messages is implemented, Receivers must process these messages in conformance with the requirements [PCIe-1.0a].

Ignored messages listed in § Table 2-32 are handled by the Receiver as follows:

- The Physical and Data Link Layers must handle these messages identical to handling any other TLP.
- The Transaction Layer must account for flow control credit but take no other action in response to these messages.

Table 2-32 Ignored Messages §

Name	Code[7:0] (b)	Routing r[2:0] (b)	Support				Description/Comments
			RC	Ep	Sw	Br	
Ignored Message	0100 0001	100					
Ignored Message	0100 0011	100					
Ignored Message	0100 0000	100					
Ignored Message	0100 0101	100					
Ignored Message	0100 0111	100					
Ignored Message	0100 0100	100					
Ignored Message	0100 1000	100					

2.2.8.8 Latency Tolerance Reporting (LTR) Message §

The LTR Message is optionally used to report device behaviors regarding its tolerance of Read/Write service latencies. Refer to § Section 6.18 for details on LTR. The following rules apply to the formation of the LTR Message:

- § Table 2-33, § Figure 2-63, and § Figure 2-64 defines the LTR Message.
- The LTR Message does not include a data payload (the TLP Type is Msg).
- The Length field is Reserved.
- The LTR Message must use the default Traffic Class designator (TC0). Receivers that implement LTR support must check for violations of this rule. If a Receiver determines that a TLP violates this rule, it must handle the TLP as a Malformed TLP.
 - This is a reported error associated with the Receiving Port (see § Section 6.2).

Table 2-33 LTR Message §

Name	Code[7:0] (b)	Routing r[2:0] (b)	Support ¹				Description/Comments
			RC	Ep	Sw	Br	
LTR	0001 0000	100	r	t	tr		Latency Tolerance Reporting

Notes:

Name	Code[7:0] (b)	Routing r[2:0] (b)	Support ¹				Description/Comments
			RC	Ep	Sw	Br	

- Support for LTR is optional. Functions that support LTR must implement the reporting and enable mechanisms described in § Chapter 7. .

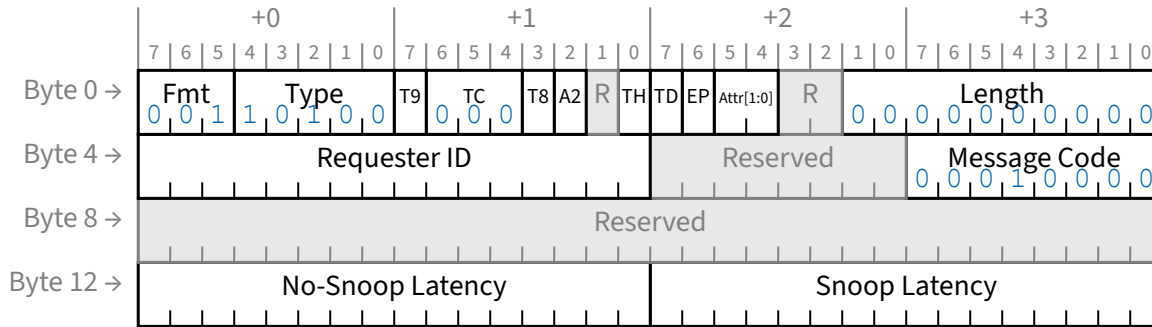


Figure 2-63 LTR Message - Non-Flit Mode §

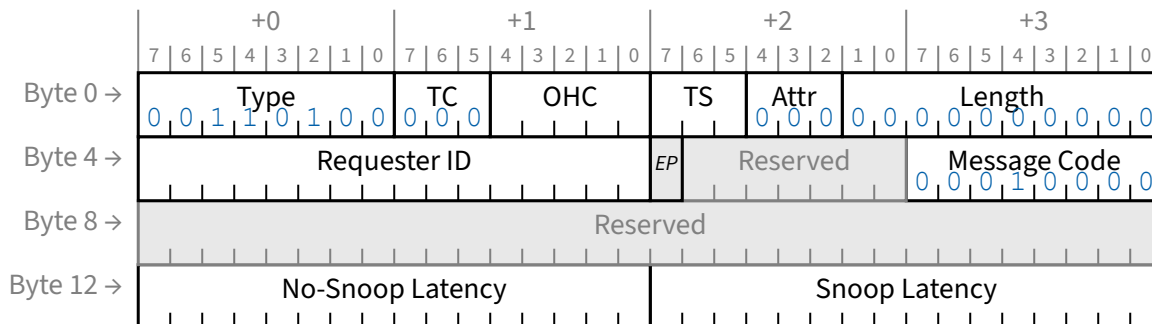


Figure 2-64 LTR Message - Flit Mode §

2.2.8.9 Optimized Buffer Flush/Fill (OBFF) Message §

The OBFF Message is optionally used to report platform central resource states to Endpoints. This mechanism is described in detail in § Section 6.19 .

The following rules apply to the formation of the OBFF Message:

- § Table 2-34, § Figure 2-65, and § Figure 2-66 defines the OBFF Message.
- The OBFF Message does not include a data payload (TLP Type is Msg).
- The Length field is Reserved.
- The Requester ID must be set to the Transmitting Port's ID.

- The OBFF Message must use the default Traffic Class designator (TC0). Receivers that implement OBFF support must check for violations of this rule. If a Receiver determines that a TLP violates this rule, it must handle the TLP as a Malformed TLP.
 - This is a reported error associated with the Receiving Port (see § Section 6.2).

Table 2-34 OBFF Message §

Name	Code[7:0] (b)	Routing r[2:0] (b)	Support ¹				Description/Comments
			RC	Ep	Sw	Br	
OBFF	0001 0010	100	t	r	tr		Optimized Buffer Flush/Fill

Notes:

- Support for OBFF is optional. Functions that support OBFF must implement the reporting and enable mechanisms described in § Chapter 7.

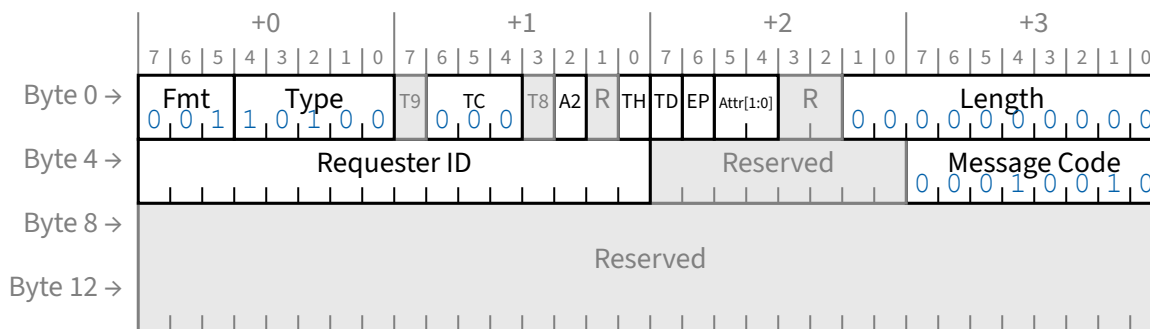


Figure 2-65 OBFF Message - Non-Flit Mode §

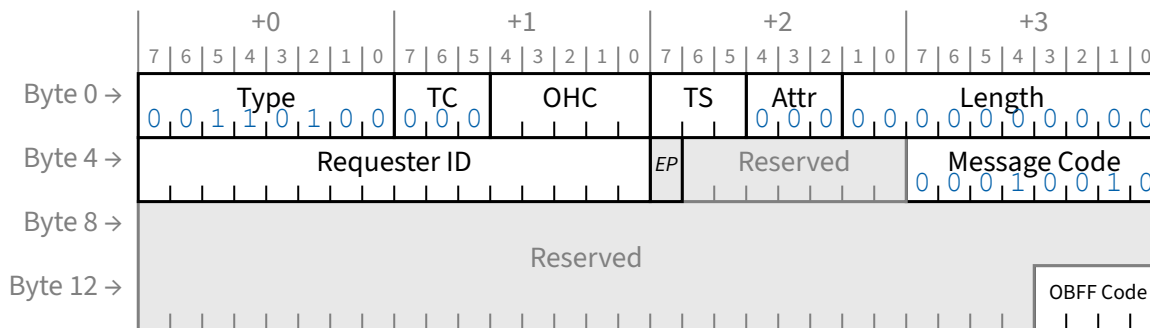


Figure 2-66 OBFF Message - Flit Mode §

2.2.8.10 Precision Time Measurement (PTM) Messages §

§ Table 2-35, § Figure 2-67, § Figure 2-68, § Figure 2-69, and § Figure 2-70 define the PTM Messages.

- The PTM Request and PTM Response Messages must use a TLP Type of Msg, and must not include a data payload. The Length field is reserved.
 - § Figure 2-67 illustrates the format of the PTM Request and Response Messages.
- The PTM ResponseD Message must use a TLP Type of MsgD, and must include a 64 bit PTM Master Time field in bytes 8 through 15 of the TLP header and a 1 DW data payload containing the 32 bit Propagation Delay field.
 - § Figure 2-68 illustrates the format of the PTM ResponseD Message.
 - Refer to § Section 6.21.3.2 for details regarding how to populate the PTM ResponseD Message.
- The Requester ID must be set to the Transmitting Port's ID.
- A PTM dialog is defined as a matched pair of messages consisting of a PTM Request and the corresponding PTM Response or PTM ResponseD message.
- The PTM Messages must use the default Traffic Class designator (TC0). Receivers implementing PTM must check for violations of this rule. If a Receiver determines that a TLP violates this rule, it must handle the TLP as a Malformed TLP.
 - This is a reported error associated with the Receiving Port (see § Section 6.2).

Table 2-35 Precision Time Measurement Messages §

Name	TLP Type	Code[7:0] (b)	Routing r[2:0] (b)	Support				Description/Comments
				RC	EP	Sw	Br	
PTM Request	Msg	0101 0010	100	r	t	tr		Initiates PTM dialog
PTM Response	Msg	0101 0011	100	t	r	tr		Completes current PTM dialog - does not carry timing information
PTM ResponseD	MsgD	0101 0011	100	t	r	tr		Completes current PTM dialog - carries timing information

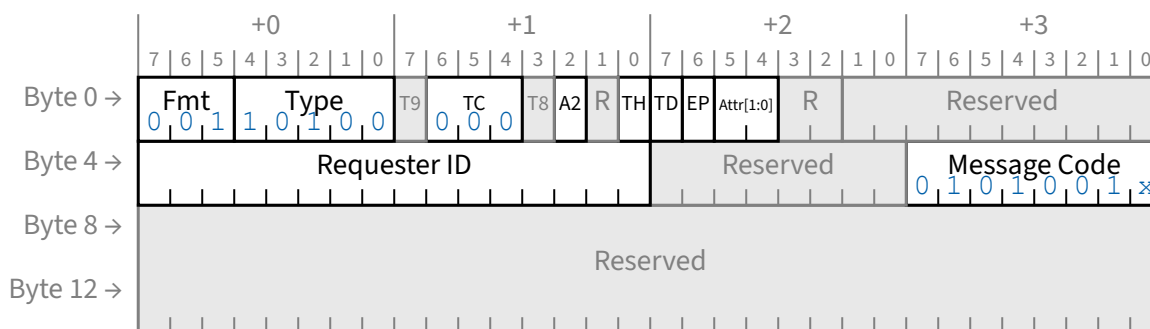


Figure 2-67 PTM Request/Response Message - Non-Flit Mode §

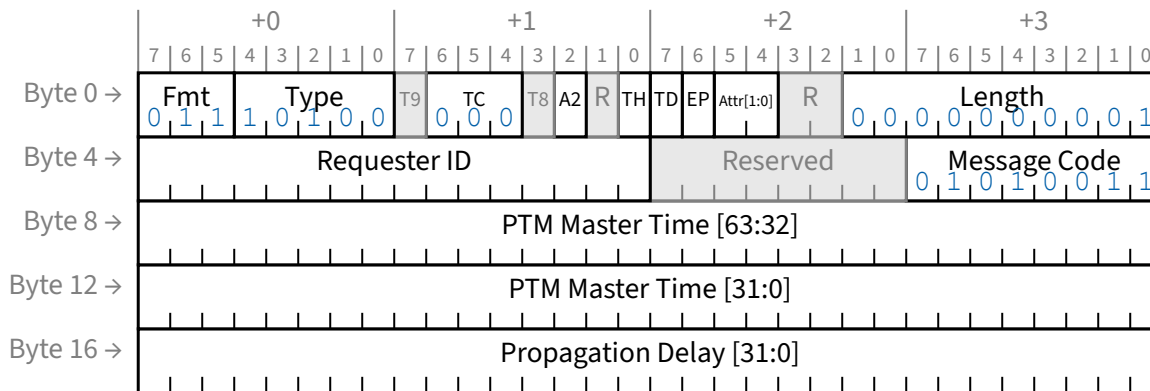


Figure 2-68 PTM ResponseD Message - Non-Flit Mode §

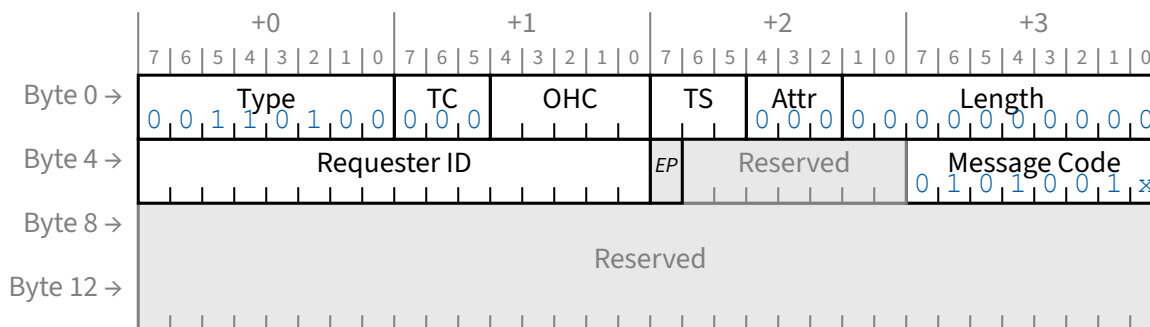


Figure 2-69 PTM Request/Response Message - Flit Mode §

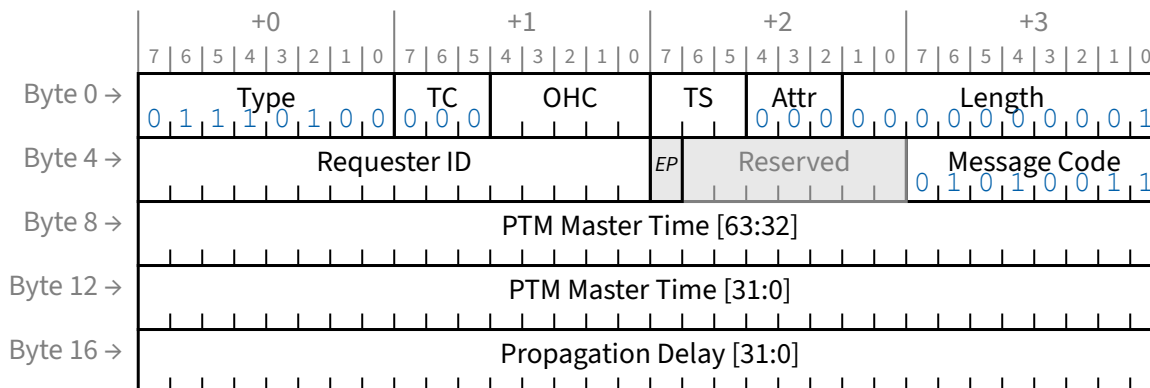


Figure 2-70 PTM ResponseD Message - Flit Mode §

IMPLEMENTATION NOTE: PROPAGATION DELAY[31:0] ENDIANNESS §

The bytes within the Propagation Delay[31:0] field (shown in § Figure 2-68) are such that:

- Data Byte 0 contains Propagation Delay [31:24]
- Data Byte 1 contains Propagation Delay [23:16]
- Data Byte 2 contains Propagation Delay [15:8]
- Data Byte 3 contains Propagation Delay [7:0]

Due to ambiguity in previous versions of this document, some implementations made this interpretation:

- Data Byte 0 contains Propagation Delay [7:0]
- Data Byte 1 contains Propagation Delay [15:8]
- Data Byte 2 contains Propagation Delay [23:16]
- Data Byte 3 contains Propagation Delay [31:24]

As a result, it is recommended that implementations provide mechanisms for adapting to either byte interpretation. One such mechanism is the optional PTM Propagation Delay Adaptation Capability.

2.2.8.11 Integrity and Data Encryption (IDE) Messages §

IDE Messages are used with the optional Integrity and Data Encryption (IDE) mechanism (see § Section 6.33). The following rules apply to the formation of IDE Messages:

- § Table 2-36 defines the IDE Messages.
- The IDE Messages do not include a data payload (TLP Type is Msg).
- The Length field is Reserved.
- The Requester ID must be set to the RID of the Function implementing IDE at the Transmitting Port.
- IDE Sync and IDE Fail Messages associated with a Link IDE Stream must use Local - Terminate at Receiver routing (100b).
- IDE Sync and IDE Fail Messages associated with a Selective IDE Stream must use Routed by ID (010b), and the Destination ID must contain the value in the RID Base field of the Selective IDE RID Association Register Block.

These Messages must only be Transmitted if the Valid bit is Set in the Selective IDE RID Association Register for the Selective IDE Stream.

- IDE Sync and IDE Fail Messages must use the same Traffic Class designator as the associated IDE Stream, if the Traffic Class designator maps to a non-UIO VC. IDE Fail and IDE Sync messages are not architected for Traffic Class designators that map to a UIO VC.
- IDE Sync Messages are implicitly associated with the same IDE Stream as indicated in the IDE Prefix applied to the IDE Sync Message.

Table 2-36 IDE Messages §

Name	TLP Type	Code[7:0] (b)	Routing r[2:0] (b)	Support ¹				Description/Comments
				RC	EP	Sw	Br	
IDE Sync	Msg	0101 0100	010 / 100	tr	tr	tr		Synchronization of IDE PR Count for the associated IDE Stream
IDE Fail	Msg	0101 0101	010 / 100	tr	tr	tr		Notification of IDE failure for a specific IDE Stream from the detecting Port to the IDE Partner Port

Notes:

- Support for these messages is required when the optional IDE mechanism is implemented

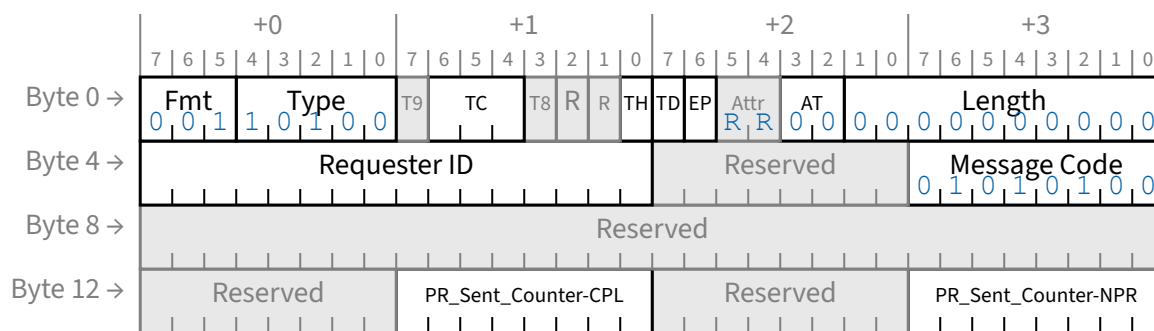


Figure 2-71 IDE Sync Message for Link IDE Stream - Non-Flit Mode §

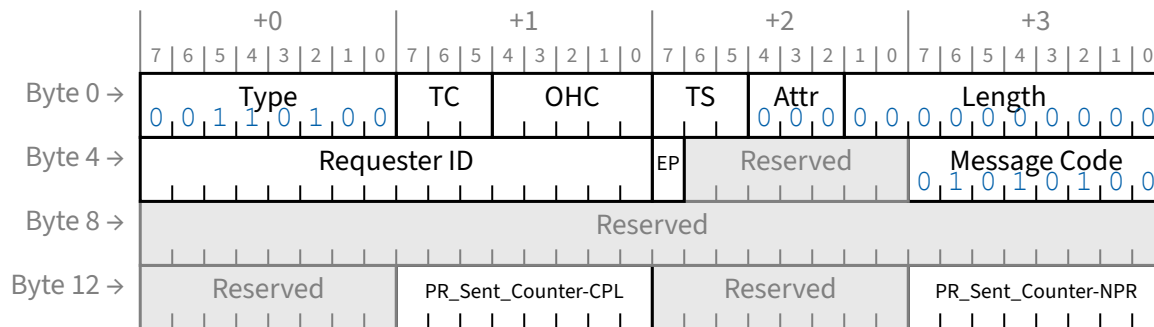


Figure 2-72 IDE Sync Message for Link IDE Stream - Flit Mode §

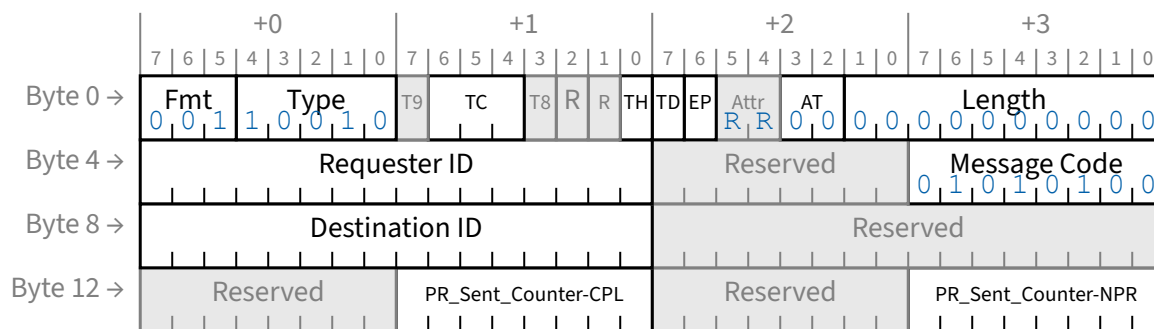


Figure 2-73 IDE Sync Message for Selective IDE Stream - Non-Flit Mode §

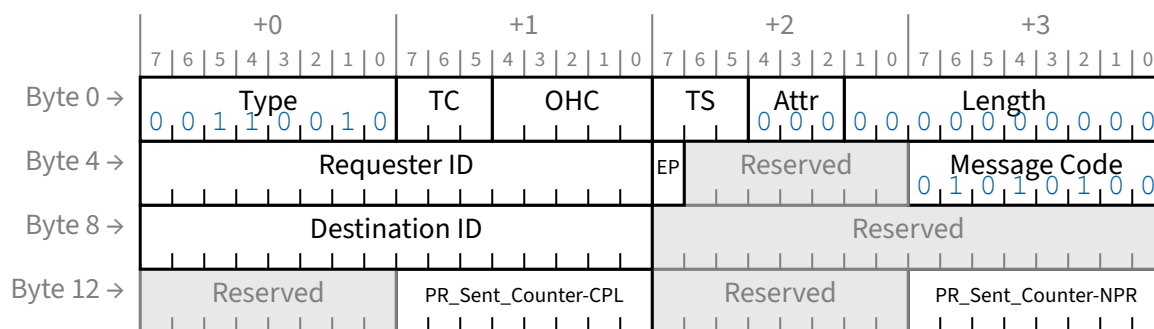


Figure 2-74 IDE Sync Message for Selective IDE Stream - Flit Mode §

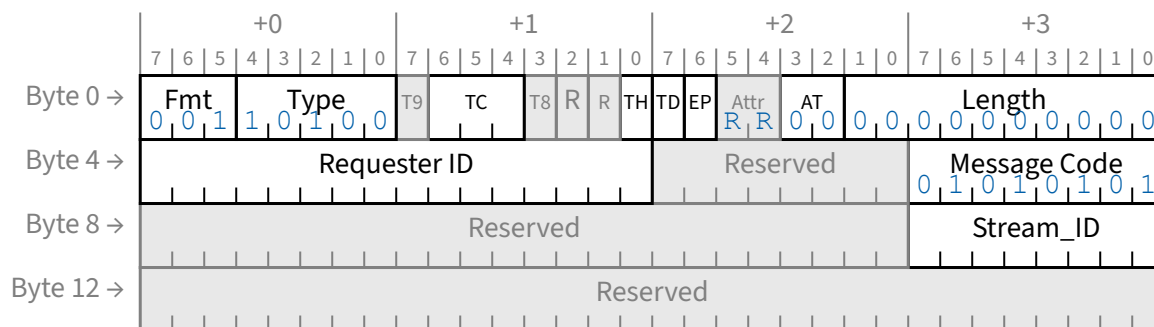


Figure 2-75 IDE Fail Message for Link IDE Stream - Non-Flit Mode §

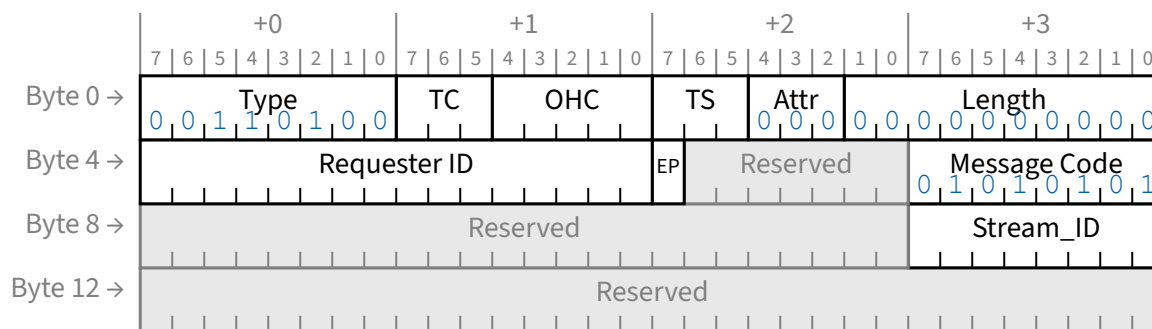


Figure 2-76 IDE Fail Message for Link IDE Stream - Flit Mode §

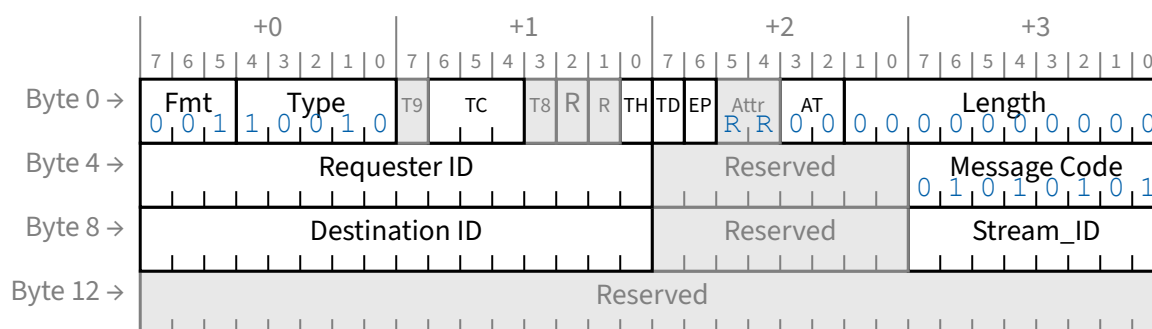


Figure 2-77 IDE Fail Message for Selective IDE Stream - Non-Flit Mode §

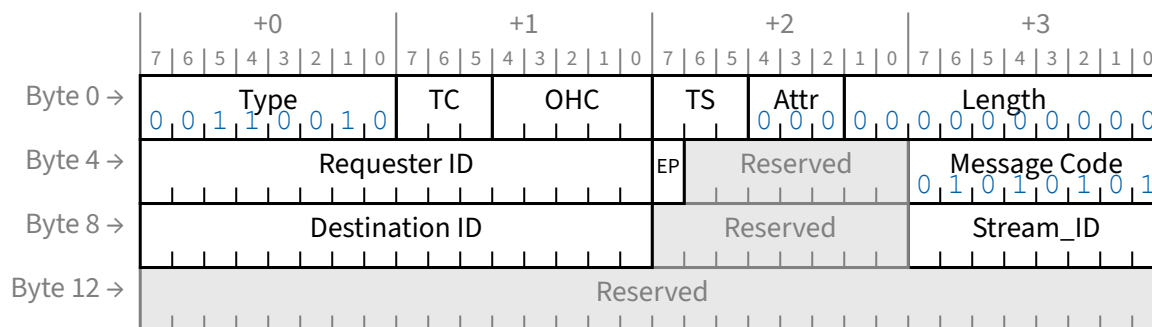


Figure 2-78 IDE Fail Message for Selective IDE Stream - Flit Mode §

2.2.9 Completion Rules §

All Read, Non-Posted Write, UIO, DMWR, and AtomicOp Requests require Completion. Completions include a Completion header that, for some types of Completions, will be followed by some number of DWs of data. The rules for each of the fields of the Completion header are defined in the following sections.

2.2.9.1 Completion Rules for Non-Flit Mode §

- Completions route by ID, and use a 3 DW header.
 - Note that the routing ID fields correspond directly to the Requester ID supplied with the corresponding Request. Thus, for Completions these fields will be referred to collectively as the Requester ID instead of the distinct fields used generically for ID routing.
- In addition to the header fields included in all TLPs and the ID routing fields, Completions contain the following additional fields (see § Figure 2-79):
 - Completer ID[15:0] - Identifies the Completer - described in detail below
 - Completion Status[2:0] - Indicates the status for a Completion (see § Table 2-37)
 - Rules for determining the value in the Completion Status[2:0] field are in § Section 2.3.1 and § Section 10.2.3
 - BCM - Byte Count Modified - this bit must not be set by PCI Express Completers, and may only be set by PCI-X completers
 - Byte Count - The remaining Byte Count for Request
 - The Byte Count value is specified as a binary number, with 0000 0000 0001b indicating 1 byte, 1111 1111 1111b indicating 4095 bytes, and 0000 0000 0000b indicating 4096 bytes.
 - For Memory Read Completions, Byte Count is set according to the rules in § Section 2.3.1.1 .
 - For AtomicOp Completions, the Byte Count value must equal the associated AtomicOp operand size in bytes.
 - For all other types of Completions, the Byte Count value must be 4.
 - Tag[9:0] - in combination with the Requester ID field, corresponds to the Transaction ID. In Non-Flit Mode, the Tag field is 10 bits.
 - Lower Address[6:0] - lower byte address for starting byte of Completion
 - For Memory Read Completions, the value in this field is the byte address for the first enabled byte of data returned with the Completion (see the rules in § Section 2.3.1.1).
 - For AtomicOp Completions, the Lower Address field is Reserved.
 - This field is set to all 0's for all remaining types of Completions. Receivers may optionally check for violations of this rule. See § Section 2.3.2 , second bullet, for details.

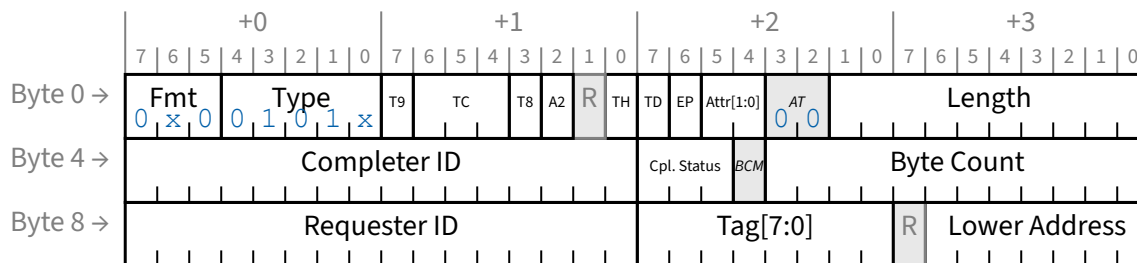
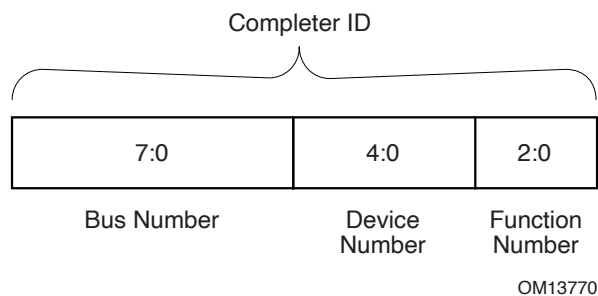
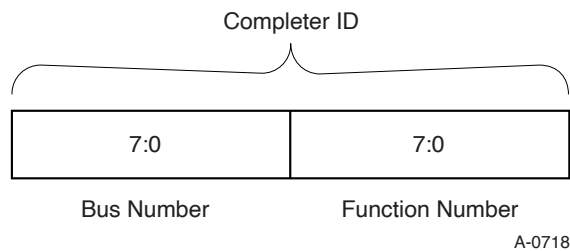


Figure 2-79 Completion Header Format - Non-Flit Mode §

Table 2-37 Completion Status Field Values §

Cpl. Status[2:0] Field Value (b)	Completion Status
000	Successful Completion (SC)
001	Unsupported Request (UR)
010	Request Retry Status (RRS)
100	Completer Abort (CA)
all others	Reserved

- The Completer ID[15:0] is a 16-bit value that is unique for every PCI Express Function within a Hierarchy (see § Figure 2-80 and § Figure 2-81)

*Figure 2-80 (Non-ARI) Completer ID* §*Figure 2-81 ARI Completer ID* §

- Functions must capture the Bus and Device Numbers³³ supplied with all Type 0 Configuration Write Requests completed by the Function, and supply these numbers in the Bus and Device Number fields of the Completer ID³⁴ for all Completions generated by the Device/Function.
 - If a Function must generate a Completion prior to the initial device Configuration Write Request, 0's must be entered into the Bus Number and Device Number fields
 - Note that Bus Number and Device Number may be changed at run time, and so it is necessary to re-capture this information with each and every Configuration Write Request.

33. With ARI Devices, Functions are only required to capture the Bus Number. ARI Devices are permitted to retain the captured Bus Number on either a per-Device or a per-Function basis. See § Section 2.2.6.2 .

34. An ARI Completer ID does not contain a Device Number field. See § Section 2.2.4.2 .

- Exception: The assignment of Bus Numbers to the Devices within a Root Complex may be done in an implementation specific way.
- In some cases, a Completion with UR Completion Status may be generated by an MFD without associating the Completion with a specific Function within the device - in this case, the Function Number field³⁵ is Reserved.
 - Example: An MFD receives a Read Request that does not target any resource associated with any of the Functions of the device - the device generates a Completion with UR status and sets a value of all 0's in the Function Number field of the Completer ID.
- Completion headers must supply the same values for the Requester ID, Tag, and Traffic Class as were supplied in the header of the corresponding Request.
- Completion headers must supply the same values for the Attribute as were supplied in the header of the corresponding Request, except as explicitly allowed:
 - when IDO is used (see § Section 2.2.6.4)
 - when RO is used in a Translation Completion (see § Section 10.2.3)
- For Completions, the TH bit is Reserved.
- AT[1:0] must be 00b. Receivers are not required or encouraged to check this.
- The Completer ID field is not meaningful prior to the software initialization and configuration of the completing device (using at least one Configuration Write Request), and for this case the Requester must ignore the value returned in the Completer ID field.
- A Completion including a data payload must specify the actual amount of data returned in that Completion, and must include the amount of data specified.
 - It is a TLP formation error to include more or less data than specified in the Length field, and the resulting TLP is a Malformed TLP.

Note: This is simply a specific case of the general rule requiring the TLP data payload length to match the value in the Length field.

2.2.9.2 Completion Rules for Flit Mode §

In Flit Mode, the rules for non-UIO Completions are the same as in Non-Flit Mode, except as defined in this section. In Flit Mode, non-UIO Completions must use the Completion Header Base Format shown in § Figure 2-82. UIO Write Completions and UIO Read Completions with Completion Status other than Successful Completion (i.e., without Data) must use the Completion Header Base Format shown in § Figure 2-83. UIO Read Completions with Data must use the UIO Completion Header Base Format shown in § Figure 2-84.

In Flit Mode, the Tag field is 14 bits.

In Flit Mode, Lower Address[6], Lower Address[5:2], and Lower Address[1:0], are not contiguous field bits in the TLP. In § Figure 2-82 to § Figure 2-84, LA6 is Lower Address[6] and LA[5:2] is Lower Address[5:2].

Reserved Completions (as indicated in § Table 2-5), are ID Routed TLPs as defined in § Section 2.2.4.2 .

35. With an ARI Completer ID, the Function Number field is 8 bits.

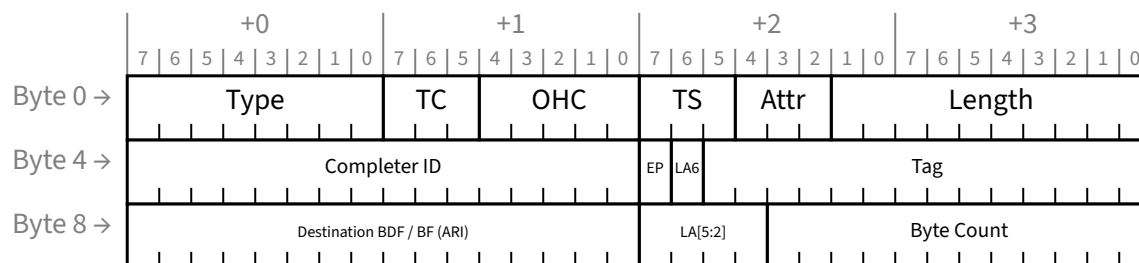


Figure 2-82 Completion Header Base Format - Non-UIO Flit Mode §

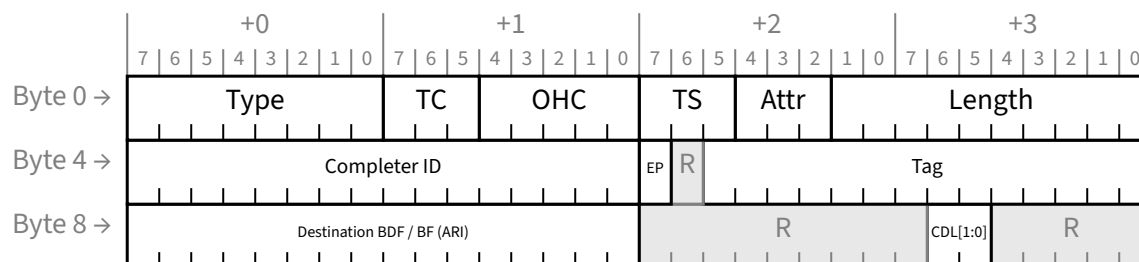


Figure 2-83 Completion Header Base Format – UIOWrCpl and UIORdCpl - Flit Mode §

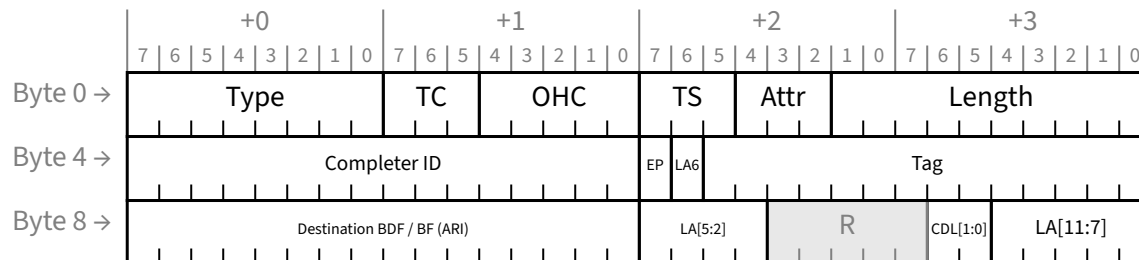


Figure 2-84 Completion Header Base Format - UIORdCplD - Flit Mode §

OHC-A5 (see § Figure 2-11) is required for all:

- Unsuccessful Completions
- Non-UIO Completions with Lower Address[1:0] not equal to 00b
- Completions that require the Destination Segment due to the associated Non-Posted Request or UIO Request containing a Requester Segment that does not match the Completer's captured Segment.

When OHC-A5 is not present it is implied that the Completion Status is Successful Completion, that Completer Segment and Destination Segment need not be explicitly indicated (see Segment rules in § Section 2.2.1.2), and that, for non-UIO Completions, the Lower Address[1:0] = 00b.

When OHC-A5 is present:

- The Completion Status and, for non-UIO Completions, Lower Address[1:0] fields must contain valid values.
 - For UIO Completions, Lower Address[1:0] is Reserved.
- If the Segment Captured bit is Set, the Completer Segment field must contain the Segment value captured by the Function as described in § Section 2.2.6.2 ; if the Segment Captured bit is Clear, the Completer Segment field must be 00h.
- if the associated Request did not include a Requester Segment, the Destination Segment field must be 00h and the DSV bit must be Clear. If the associated Request included a Requester Segment, the Destination Segment field must reflect the value of the Requester Segment and the DSV bit must be Set. See RP Segment Exceptions for cases where RPs are not required to include Segment information.

The BCM field, present in Non-Flit Mode Completions, is not supported in Flit Mode.

For all UIO Completions:

- The Read Completion Boundary and Write Completion Boundary rules defined in § Section 2.3.1.2 and § Section 2.3.1.3 , respectively, must be followed.
- Length[9:0] indicates the total number of DW represented by this Completion. See § Table 2-4 for values.
 - Regardless of Completion Status, Completers must return Completions corresponding to all DW in a UIO Request.
 - Byte Enables must not be considered when determining the Length value for UIO Completions.
 - For a Zero Length UIO Write (where in the Request, Length is 00_0000_0001b and First Byte Enable 0000b), one DW must be considered to have been written.
- The Tag field value must match the Tag field value for the corresponding UIO Request(s)
 - UIO Write Completions are permitted to be coalesced or split, for a given Transaction ID, provided all DW Completion accounting remains accurate (see § Section 2.3.1.3).
 - UIO Read Completions are permitted to be split, for a given Transaction ID, provided all DW Completion accounting remains accurate (see § Section 2.3.1.2).
- For UIO Completions without Data (see § Figure 2-83)
 - Lower Address is Reserved.
- For UIO Completions with Data (see § Figure 2-84):
 - Lower Address[11:2] must contain valid values.
 - Lower Address[1:0] is Reserved.
- The CDL[1:0] field is assigned for use by [CXL]; this field must be treated as Reserved for use cases not covered by [CXL].
- UIO Requesters must accept UIO Completions in any order.
- UIO Memory Request(s) associated with a Transaction ID are considered completed only when the sum of all DW completed, as indicated by the Length[9:0] field value(s) in the Completion(s), equals the sum of all DW expected for the associated Request(s).
 - Only UIO Memory Writes are allowed to have multiple outstanding Requests with the same Transaction ID. If it is necessary for a Requester to ensure that UIO Memory Write Requests issued with a given Transaction ID have completed, the Requester must delay issuing additional Requests with that Transaction ID until it has received Completions accounting for all outstanding UIO Memory Write Requests using that Transaction ID.
- When fully completed, all Requests associated with the same Transaction ID are represented by the same Completion Status. However, individual Completions of a UIO Request may indicate different Completion

Status values. At any point where UIO Completions are coalesced, including at the Requester, the coalesced Completion Status is determined according to the following rules:

- UR if any of the UIO Completions have UR Completion Status
- CA if none of the UIO Completions have UR Completions Status, and any have CA Completion Status
- RRS if none of the UIO Completions have UR or CA Completions Status, and any have RRS Completion Status
- SC if all UIO Completions have Successful Completion Status
- Any completion status not defined in § Table 2-37 should be treated as a UR (see § Section 2.3.2)
- UIO Completions for UIO Read Requests that have a Completion Status other than Successful Completion must use TLP Type UIORdCpl
 - The Length value for UIORdCpl is not constrained by Max Payload Size or Max Read Request Size.
- Attr[2:0], corresponding to ID0, RO and NS in non-UIO Memory Requests, are Reserved
- The EP bit is Reserved for UIOWrCpl and UIORdCpl.

2.2.10 TLP Prefix Rules §

2.2.10.1 TLP Prefix General Rules - Non-Flit Mode §

In NFM, the following rules apply to any TLP that contains a TLP Prefix:

- For any TLP, a value of 100b in the Fmt[2:0] field in byte 0 of the TLP indicates the presence of a TLP Prefix and the Type[4] bit indicates the type of TLP Prefix.
 - A value of 0b in the Type[4] bit indicates the presence of a Local TLP Prefix
 - A value of 1b in the Type[4] bit indicates the presence of an End-End TLP Prefix
- The format for bytes 1 through 3 of a TLP Prefix is defined by its TLP Prefix type.
- A TLP that contains a TLP Prefix must have an underlying TLP Header. A received TLP that violates this rule is handled as a Malformed TLP. This is a reported error associated with the Receiving Port (see § Section 6.2).
- It is permitted for a TLP to contain more than one TLP Prefix of any type
 - When a combination of Local and End-End TLP Prefixes are present in TLP, it is required that all the Local TLP Prefixes precede any End-End TLP Prefixes. A received TLP that violates this rule is handled as a Malformed TLP. This is a reported error associated with the Receiving Port (see § Section 6.2).
- The size of each TLP Prefix is 1 DW. A TLP Prefix may be repeated to provide space for additional data.
- If the value in the Fmt and Type field indicates the presence of a Local TLP Prefix, handle according to the Local TLP Prefix handling (see § Section 2.2.10.2).
- If the value in the Fmt and Type field indicates the presence of an End-End TLP Prefix, handle according to the End-End TLP Prefix handling (see § Section 2.2.10.4).

2.2.10.2 Local TLP Prefix Processing §

The following rules apply to Local TLP Prefixes:

- In Flit Mode, TLP Prefix types are determined using the Type[7:0] field (see § Table 2-5)
- In Non-Flit Mode, Local TLP Prefix types are determined using the L[3:0] sub-field of the Type field

- Type[4] must be 0b

Local TLP Prefix L[3:0] values are defined in § Table 2-38.

Table 2-38 Local TLP Prefix Types §

Local TLP Prefix Type	L[3:0] (b)	Description
MR-IOV	0000	MR-IOV TLP Prefix - Refer to [MR-IOV] for details.
FlitModePrefix	1101	Flit Mode Local TLP Prefix - See § Section 2.2.10.3
VendPrefixL0	1110	Vendor Defined Local TLP Prefix - Refer to § Section 2.2.10.2.1 for further details.
VendPrefixL1	1111	Vendor Defined Local TLP Prefix - Refer to § Section 2.2.10.2.1 for further details.
		All other encodings are Reserved.

- The size, routing, and flow control rules are specific to each Local TLP Prefix type.
- It is an error to receive a TLP with a Local TLP Prefix type not supported by the Receiver. If the Extended Fmt Field Supported bit is Set, TLPs in violation of this rule are handled as a Malformed TLP unless explicitly stated differently in another specification. This is a reported error associated with the Receiving Port (see § Section 6.2). If the Extended Fmt Field Supported bit is Clear, behavior is device specific.
- No Local TLP Prefixes are protected by ECRC even if the underlying TLP is protected by ECRC.

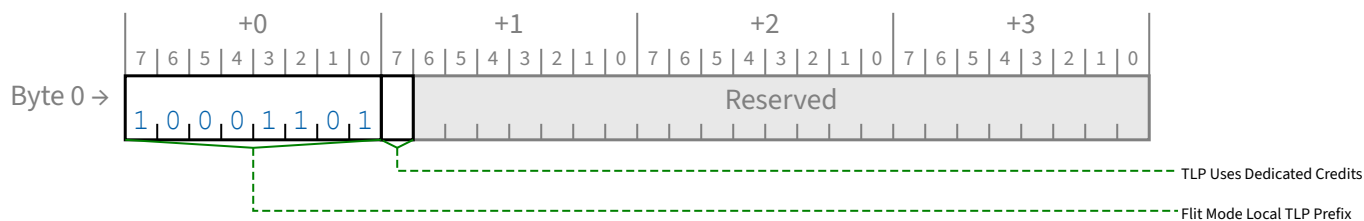
2.2.10.2.1 Vendor Defined Local TLP Prefix §

As described in § Table 2-38, Types VendPrefixL0 and VendPrefixL1 are defined for use as Vendor Defined Local TLP Prefixes. To maximize interoperability and flexibility the following rules are applied to such prefixes:

- Components must not send TLPs containing Vendor Defined Local TLP Prefixes unless this has been explicitly enabled (using vendor-specific mechanisms).
- Components that support any usage of Vendor Defined Local TLP Prefixes must support the 3-bit definition of the Fmt field and have the Extended Fmt Field Supported bit Set (see § Section 7.5.3.15).
- It is recommended that components be configurable (using vendor-specific mechanisms) so that all vendor defined prefixes can be sent using either of the two Vendor Defined Local TLP Prefix encodings. Such configuration need not be symmetric (for example each end of a Link could transmit the same Prefix using a different encoding).

2.2.10.3 Flit Mode Local TLP Prefix §

This prefix (see § Figure 2-85) is only permitted when operating in Flit Mode.

Figure 2-85 **Flit Mode Local TLP Prefix** §

If the Flit Mode Local TLP Prefix is applied to a NFM TLP, this is an error that *MUST@FLIT* be handled as a Malformed TLP. It is permitted to apply the Flit Mode Local TLP Prefix to any FM TLP, but it is strongly recommended that the Flit Mode Local TLP Prefix is only applied to TLPs that specifically require the Prefix to be present.

The Flit Mode Local TLP Prefix includes:

- **TLP Uses Dedicated Credits** – This bit when Set indicates that the associated TLP must be handled using dedicated flow control credits. If this bit is Clear, or if the Flit Mode Local TLP Prefix is not present, the associated TLP must be handled using shared flow control credits.

2.2.10.4 End-End TLP Prefix Processing - Non-Flit Mode §

The following rules apply to End-End TLP Prefixes

- End-End TLP Prefix types are determined using the E[3:0] sub-field of the Type field
 - Type[4] must be 1b
 - End-End TLP Prefix E[3:0] values are defined in § Table 2-39

Table 2-39 **End-End TLP Prefix Types** §

End-End TLP Prefix Type	E[3:0] (b)	Description
TPH	0000	TPH - Refer to § Section 2.2.7.1.1 and § Section 6.17 for further details.
PASID	0001	PASID - Refer to § Section 6.20.2.1 for further details.
IDE	0010	Identifies an IDE TLP - Refer to § Section 6.33 for further details.
VendPrefixE0	1110	Vendor Defined End-End TLP Prefix - Refer to § Section 2.2.10.4.1 for further details.
VendPrefixE1	1111	Vendor Defined End-End TLP Prefix - Refer to § Section 2.2.10.4.1 for further details.
		All other encodings are Reserved.

- The maximum number of End-End TLP Prefixes permitted in a TLP is 4:
 - A Receiver supporting TLP Prefixes must check this rule. If a Receiver determines that a TLP violates this rule, the TLP is a Malformed TLP. This is a reported error associated with the Receiving Port (see § Section 6.2).
- The presence of an End-End TLP Prefix does not alter the routing of a TLP. TLPs are routed based on the routing rules covered in § Section 2.2.4.

- Functions indicate how many End-End TLP Prefixes they support by the Max End-End TLP Prefixes field in the Device Capabilities 2 register (see § Section 7.5.3.15).
 - For Root Ports, the Max End-End TLP Prefixes field is permitted to return a value indicating support for fewer End-End TLP Prefixes than what the Root Port hardware actually implements; however, the error handling semantics must still be based on the value contained in the field. TLPs received that contain more End-End TLP Prefixes than are supported by the Root Port must be handled as follows. It is recommended that Requests be handled as Unsupported Requests, but otherwise they must be handled as Malformed TLPs. It is recommended that Completions be handled as Unexpected Completions, but otherwise they must be handled as Malformed TLPs. For TLPs received by the Ingress Port, this is a reported error associated with the Ingress Port. For TLPs received internally to be transmitted out the Egress Port, this is a reported error associated with the Egress Port. See § Section 6.2 .
 - For all other Function types, TLPs received that contain more End-End TLP Prefixes than are supported by a Function must be handled as Malformed TLPs. This is a reported error associated with the Receiving Port (see § Section 6.2).

Advanced Error Reporting (AER) logging (if supported) occurs as specified in § Section 6.2.4.4 .

- Switches must support forwarding of TLPs with up to 4 End-End TLP Prefixes if the End-End TLP Prefix Supported bit is Set.
- Different Root Ports with the End-End TLP Prefix Supported bit Set are permitted to report different values for Max End-End TLP Prefixes.
- All End-End TLP Prefixes are protected by ECRC if the underlying TLP is protected by ECRC.
- It is an error to receive a TLP with an End-End TLP Prefix by a Receiver that does not support End-End TLP Prefixes. A TLP in violation of this rule is handled as a Malformed TLP. This is a reported error associated with the Receiving Port (see § Section 6.2).
- Software should ensure that TLPs containing End-End TLP Prefixes are not sent to components that do not support them. Components where the Extended Fmt Field Supported bit is Clear may misinterpret TLPs containing TLP Prefixes.
- If one Function of an Upstream Port has the End-End TLP Prefix Supported bit Set, all Functions of that Upstream Port must handle the receipt of a Request addressed to them that contains an unsupported End-End TLP Prefix type as an Unsupported Request. This is a reported error associated with the Receiving Port (see § Section 6.2).
- If one Function of an Upstream Port has the End-End TLP Prefix Supported bit Set, all Functions of that Upstream Port must handle the receipt of a Completion addressed to them that contains an unsupported End-End TLP Prefix type as an Unexpected Completion. This is a reported error associated with the Receiving Port (see § Section 6.2).
- For Routing Elements, the End-End TLP Prefix Blocking bit in each Egress Port determines whether TLPs containing End-End TLP Prefixes can be transmitted via that Egress Port (see § Section 7.5.3.16). If forwarding is blocked the entire TLP is dropped and a TLP Prefix Blocked Error is reported. If the blocked TLP is a Non-Posted Request, the Egress Port returns a Completion with Unsupported Request Completion Status. The TLP Prefix Blocked Error is a reported error associated with the Egress Port (see § Section 6.2).
- For routing elements where Multicast is enabled (see § Section 6.14). End-End TLP Prefixes are replicated in all Multicast copies of a TLP. TLP Prefix Egress Blocking of Multicast packets is performed independently at each Egress Port.

2.2.10.4.1 Vendor Defined End-End TLP Prefix §

As described in § Table 2-39, Types VendPrefixE0 and VendPrefixE1 are defined for use as Vendor Defined End-End TLP Prefixes. To maximize interoperability and flexibility the following rules are applied to such prefixes:

- Components must not send TLPs containing Vendor Defined End-End TLP Prefixes unless this has been explicitly enabled (using vendor-specific mechanisms).
- It is recommended that components be configurable (using vendor-specific mechanisms) to use either of the two Vendor Defined End-End TLP Prefix encodings. Doing so allows two different Vendor Defined End-End TLP Prefixes to be in use simultaneously within a single PCI Express topology while not requiring that every source understand the ultimate destination of every TLP it sends.

2.2.10.4.2 Root Ports with End-End TLP Prefix Supported §

Support for peer-to-peer routing of TLPs containing End-End TLP Prefixes between Root Ports is optional and implementation dependent. If an RC supports End-End TLP Prefix routing capability between two or more Root Ports, it must indicate that capability in each associated Root Port via the End-End TLP Prefix Supported bit in the Device Capabilities 2 register.

An RC is not required to support End-End TLP Prefix routing between all pairs of Root Ports that have the End-End TLP Prefix Supported bit Set. A Request with End-End TLP Prefixes that would require routing between unsupported pairs of Root Ports must be handled as a UR. A Completion with End-End TLP Prefixes that would require routing between unsupported pairs of Root Ports must be handled as an Unexpected Completion (UC). In both cases, this error is reported by the “sending” Port.

The End-End TLP Prefix Supported bit must be Set for any Root Port that supports forwarding of TLPs with End-End TLP Prefixes initiated by host software or Root Complex Integrated Endpoints (RCiEPs). The End-End TLP Prefix Supported bit must be Set for any Root Ports that support forwarding of TLPs with End-End TLP Prefixes received on their Ingress Port to RCiEPs.

Different Root Ports with the End-End TLP Prefix Supported bit Set are permitted to report different values for Max End-End TLP Prefixes.

An RC that splits a TLP into smaller TLPs when performing peer-to-peer routing between Root Ports must replicate the original TLP's End-End TLP Prefixes in each of the smaller TLPs (see § Section 1.3.1).

2.2.11 OHC-E Rules - Flit Mode §

End-End TLP Prefixes in Non-Flit Mode are replaced by OHC-E in Flit Mode (see § Figure 2-86, § Figure 2-87, and § Figure 2-88).

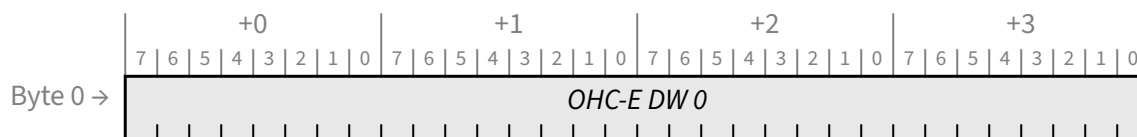


Figure 2-86 OHC-E1 §

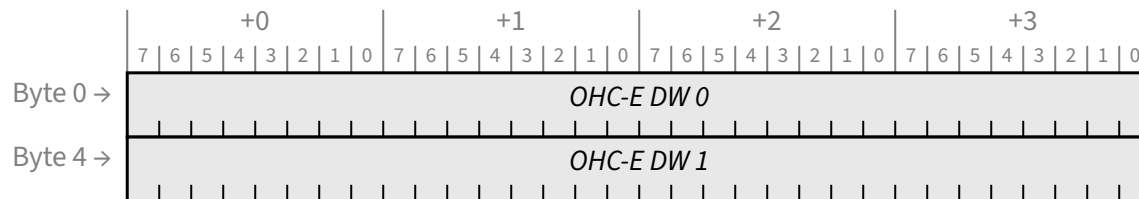


Figure 2-87 OHC-E2 §

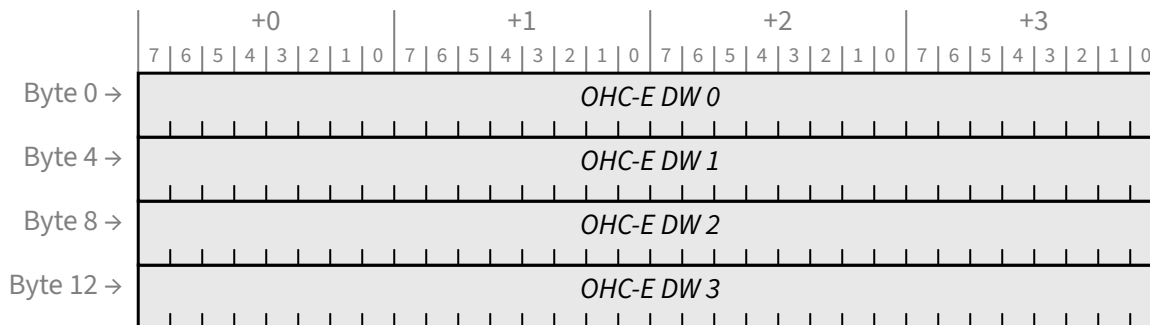


Figure 2-88 OHC-E4 §

OHC-E is used to convey content that would otherwise use End-End TLP Prefixes 0011b to 1111b.

- For each DW of OHC-E, Byte 0, bits [7:4] indicate the format of the remainder of the DW and are encoded:
 - 0000b - No Entry - The remainder of the DW is Reserved
 - 0001b - End-End TLP Prefix DW - The remainder of the DW is defined as follows:
 - Byte 0, bits [3:0] take the value of E[3:0] in the corresponding End-End TLP Prefix (see § Table 2-39), with the exception that encodings 0000b-0010b are Reserved.
 - Bytes 1, 2 and 3 take the value of bytes 1, 2 and 3 in the corresponding End-End TLP Prefix.
 - 0010b-1111b - Reserved - Receivers must handle as No Entry

OHC-E must be populated without gaps, starting with the first DW. Any No Entry DWs must be populated at the end. Transmitters must use the smallest possible OHC-E and avoid unnecessary No Entry DWs. When translating VendPrefixE0 or VendPrefixE1 from NFM to FM or vice-versa, the same relative sequence must be preserved.

RC support for peer-to-peer routing of TLPs containing OHC-E content between Root Ports is optional and implementation dependent.

If a Function sets does not support OHC-E, and it is the targeted Completer for a received TLP that has OHC-E, it must handle the TLP as an Unsupported Request or an Unexpected Completion. Such a Function is permitted to drop OHC-E content during header logging for the error. This behavior is consistent with the rules stated in § Section 2.2.1.2 for Endpoint Upstream Ports and Root Ports, but with an extension of the rule for Switch Ports as well when they don't support OHC-E.

If a Switch Function or RP Function does not support OHC-E and it receives a TLP with OHC-E to be forwarded, the TLP must be handled as below.

- PR FC Type: Block at Ingress; if TLP is UIO, report no error, else handle as a TLP Prefix Blocked Error
- NPR FC Type: Block at Ingress; report no error
- CPL FC Type: Block at Ingress; handle as a TLP Prefix Blocked Error

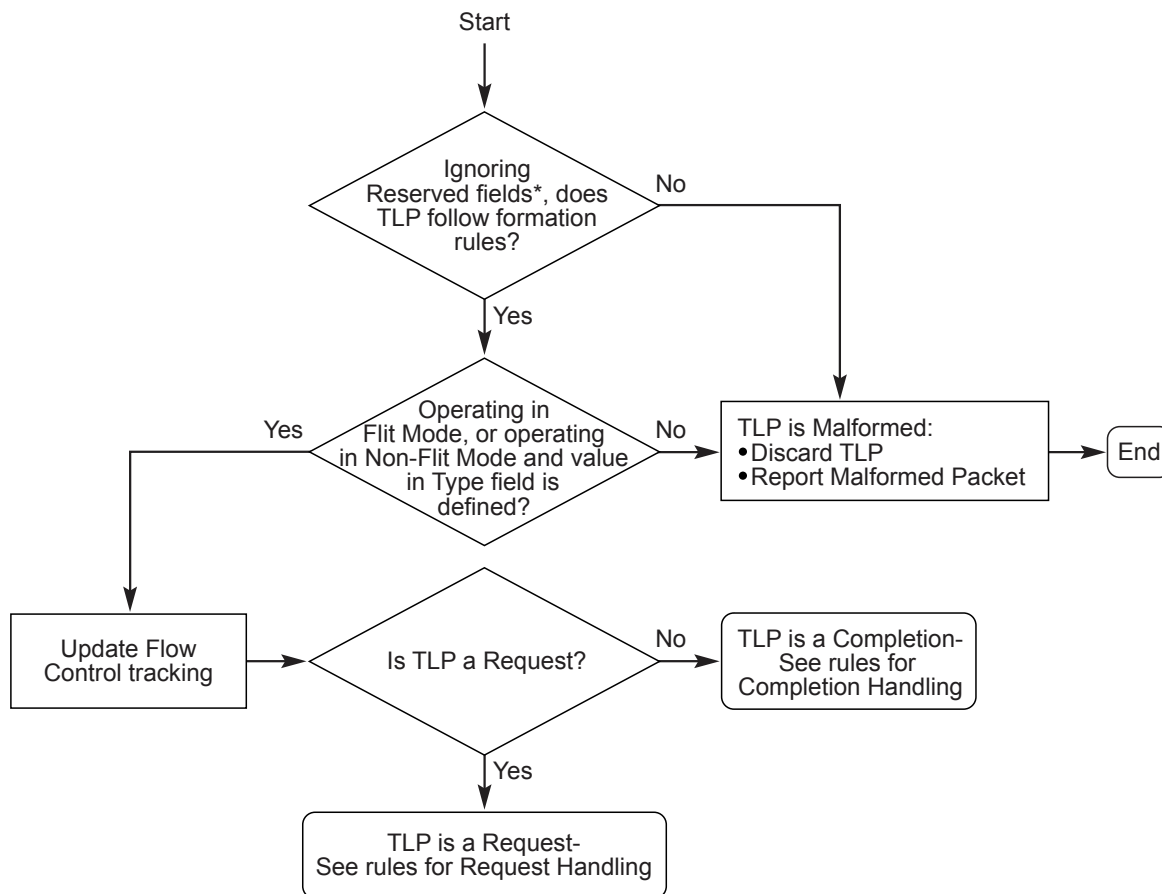
If a Function sets its OHC-E Support field to 001b or 010b, it must handle a received TLP that targets it and that has OHC-E containing more DWs than it supports as an Unsupported Request or an Unexpected Completion.

2.3 Handling of Received TLPs §

This section describes how all Received TLPs are handled when they are delivered to the Receive Transaction Layer from the Receive Data Link Layer, after the Data Link Layer has validated the integrity of the received TLP. The rules are diagrammed in the flowchart shown in § Figure 2-89.

- Values in Reserved fields must be ignored by the Receiver.
- In Non-Flit Mode, if the value in the Fmt field indicates the presence of at least one TLP Prefix:
 - Detect if additional TLP Prefixes are present in the header by checking the Fmt field in the first byte of subsequent DWs until the Fmt field does not match that of a TLP Prefix.
 - Handle all received TLP Prefixes according to TLP Prefix Handling Rules (see § Section 2.2.10.1).
- In Flit Mode, if the value in the Type field indicates the presence of at least one Local TLP Prefix:
 - Detect if additional Local TLP Prefixes are present in subsequent DWs.
 - Handle all received Local TLP Prefixes according to TLP Prefix Handling Rules (see § Section 2.2.10.3).
- In Non-Flit Mode, if the Extended Fmt Field Supported bit is Set, Received TLPs that use encodings of Fmt and Type that are Reserved are Malformed TLPs (see § Table 2-2 and § Table 2-3).
 - This is a reported error associated with the Receiving Port (see § Section 6.2).
- In Non-Flit Mode, if the Extended Fmt Field Supported bit is Clear, processing of Received TLPs that have Fmt[2] Set is undefined.³⁶
- In Non-Flit Mode, all Received TLPs with Fmt[2] Clear and that use undefined Type field values are Malformed TLPs.
 - This is a reported error associated with the Receiving Port (see § Section 6.2).
- All Received Malformed TLPs must be discarded.
 - Received Malformed TLPs that are ambiguous with respect to which buffer to release or are mapped to an uninitialized or disabled Virtual Channel must be discarded without updating Receiver Flow Control information.
 - All other Received Malformed TLPs must be discarded, optionally not updating Receiver Flow Control information.
- Otherwise, update Receiver Flow Control tracking information (see § Section 2.6).
- If the value in the Type field indicates the TLP is a Request, handle according to Request Handling Rules, otherwise the TLP is a Completion so handle according to Completion Handling Rules (see § Section 2.3.2).

36. An earlier version of this specification reserved the bit now defined for Fmt[2].



*TLP fields which are marked Reserved are not checked at the Receiver

OM13771C

Figure 2-89 Flowchart for Handling of Received TLPs §

Switches must process both TLPs that address resources within the Switch as well as TLPs that address resources residing outside the Switch. Switches handle all TLPs that address internal resources of the Switch according to the rules above. TLPs that pass through the Switch, or that address the Switch as well as passing through it, are handled according to the following rules (see § Figure 2-90):

- If the value in the Type field indicates the TLP is not a Msg or MsgD Request, the TLP must be routed according to the routing mechanism used (see § Section 2.2.4.1 and § Section 2.2.4.2).
- Switches route Completions using the information in the Requester ID field of the Completion.
- If the value in the Type field indicates the TLP is a Msg or MsgD Request, route the Request according to the routing mechanism indicated in the r[2:0] sub-field of the Type field.
 - If the value in r[2:0] indicates the Msg/MsgD is Routed to Root Complex (000b), the Switch must route the Msg/MsgD to the Upstream Port of the Switch.
 - It is an error to receive a Msg/MsgD Request specifying 000b routing at the Upstream Port of a Switch. Switches may check for violations of this rule - TLPs in violation are Malformed TLPs. If checked, this is a reported error associated with the Receiving Port (see § Section 6.2).

- If the value in r[2:0] indicates the Msg/MsgD is routed by address (001b), the Switch must route the Msg/MsgD in the same way it would route a Memory Request by address.
- If the value in r[2:0] indicates the Msg/MsgD is routed by ID (010b), the Switch must route the Msg/MsgD in the same way it would route a Completion by ID.
- If the value in r[2:0] indicates the Msg/MsgD is a broadcast from the Root Complex (011b), the Switch must route the Msg/MsgD to all Downstream Ports of the Switch.
 - It is an error to receive a Msg/MsgD Request specifying 011b routing at the Downstream Port of a Switch. Switches may check for violations of this rule - TLPs in violation are Malformed TLPs. If checked, this is a reported error associated with the Receiving Port (see § Section 6.2).
- If the value in r[2:0] indicates the Msg/MsgD terminates at the Receiver (100b or a Reserved value), or if the Message Code field value is defined and corresponds to a Message that must be comprehended by the Switch, the Switch must process the Message according to the Message processing rules.
- If the value in r[2:0] indicates Gathered and routed to Root Complex (101b), see § Section 5.3.3.2.1 for Message handling rules.
 - It is an error to receive any Msg/MsgD Request other than a PME_TO_Ack that specifies 101b routing.
 - It is an error to receive a PME_TO_Ack at the Upstream Port of a Switch. Switches may optionally check for violations of these rules. These checks are independently optional (see § Section 6.2.3.4). If checked, violations are Malformed TLPs, and are reported errors associated with the Receiving Port (see § Section 6.2).

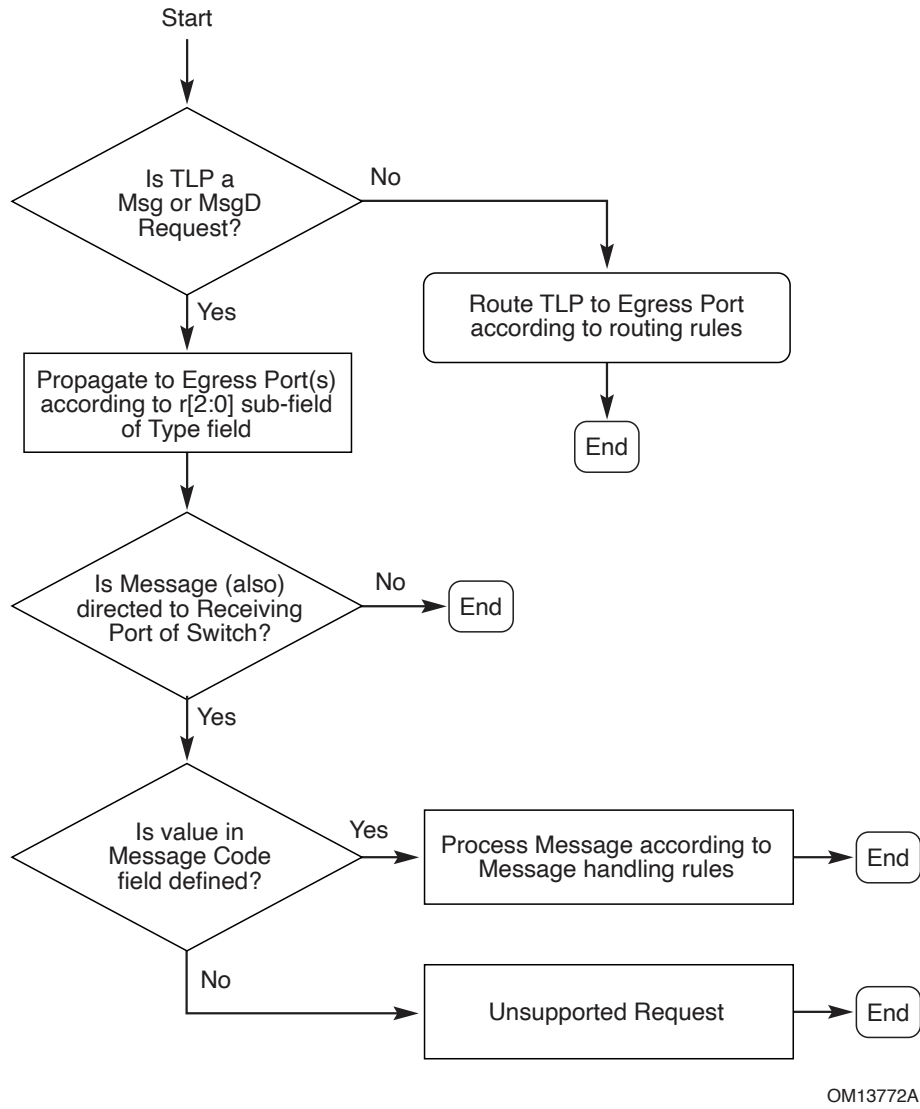


Figure 2-90 Flowchart for Switch Handling of TLPs §

2.3.1 Request Handling Rules §

This section describes how Received Requests are handled, following the initial processing done with all TLPs. The rules are diagrammed in the flowchart shown in § Figure 2-91.

- If the Request Type is defined in § Table 2-2, and § Table 2-3 (NFM), or is a non-Reserved value in § Table 2-5 (FM), but is not supported (by design or because of configuration settings) by the device, the Request must be handled as an Unsupported Request (UR). This is an error associated with the Receiving Port (see § Section 6.2)
 - If the Request requires a Completion, a Completion Status of UR must be returned (see § Section 2.2.9).
- For a Receiver that decodes UIO Type values, if the Request is a UIO Request, but the Receiver cannot forward or process the Request, then the Request must be dropped.

- In Flit Mode, if a Receiver is targeted by a Request TLP, and the Type value is defined in § Table 2-5 as a Reserved value, it is strongly recommended³⁷ for the Request to be discarded following the update of flow control credits.
- In Flit Mode, if a Routing Element receives a Request TLP to be forwarded with the Type value defined in § Table 2-5 as a Reserved value, but is unable to forward it due to a problem like the Egress Port being in DL_Down, it is strongly recommended³⁸ for the Request to be discarded following the update of flow control credits.

IMPLEMENTATION NOTE: WHEN REQUESTS ARE TERMINATED USING UNSUPPORTED REQUEST §

In Conventional PCI, a device “claims” a request on the bus by asserting DEVSEL#. If no device claims a request after a set number of clocks, the request is terminated as a Master Abort. Since PCI Express is a point to point interconnect, there is no equivalent mechanism for claiming a request on a Link, since all transmissions by one component are always sent to the other component on the Link. Therefore, it is necessary for the receiver of a request to determine if the request should be claimed. If the request is not claimed, then it is handled as an Unsupported Request, which is the PCI Express equivalent of Conventional PCI's Master Abort termination. In general, one can determine the correct behavior by asking the question: *Would the device assert DEVSEL# for this request in conventional PCI?*

For device Functions with Type 0 headers (all types of Endpoints), it is relatively simple to answer this question. For Memory and I/O Requests, this determination is based on the address ranges the Function has been programmed to respond to. For Configuration requests, the Type 0 request format indicates the device is by definition the “target”, although the device will still not claim the Configuration Request if it addresses an unimplemented Function.

For device Functions with Type 1 headers (Root Ports, Switches and Bridges), the same question can generally be applied, but since the behavior of a conventional PCI bridge is more complicated than that of a Type 0 Function, it is somewhat more difficult to determine the answers. One must consider Root Ports and Switch Ports as if they were actually composed of conventional PCI to PCI bridges, and then at each stage consider the configuration settings of the virtual bridge to determine the correct behavior.

PCI Express Messages do not exist in conventional PCI, so the above guideline cannot be applied. This specification describes specifically for each type of Message when a device must handle the request as an Unsupported Request. Messages pass through Root and Switch Ports unaffected by conventional PCI control mechanisms including the Bus Master Enable bit and power state setting.

Note that CA, which is the PCI Express equivalent to Target Abort, is used only to indicate a serious error that makes the Completer permanently unable to respond to a request that it would otherwise have normally responded to. Since Target Abort is used in conventional PCI only when a target has asserted DEVSEL#, it is incorrect to use a CA for any case where a Conventional PCI target would have ignored a request by not asserting DEVSEL#.

- If the Request is a Message, and the Message Code, routing field, or Msg / MsgD indication corresponds to a combination that is undefined, or that corresponds to a Message not supported by the device Function (other than Vendor-Defined Type 1, which is not treated as an error - see § Table F-1), the Request is an Unsupported

37. For backward compatibility with previous versions of this specification, the Request is permitted to be handled as an Unsupported Request.

38. For backward compatibility with previous versions of this specification, the Request is permitted to be handled as an Unsupported Request.

Request (UR). This is a reported error associated with the Receiving Port and is reported according to [§ Section 6.2](#)

- If the Message Code is a supported value, process the Message according to the corresponding Message processing rules; if the Message Code is an Ignored Message and the Receiver is ignoring it, ignore the Message without reporting any error (see [§ Section 2.2.8.7](#))
- If the Request is a Message with a routing field that indicates Routed by ID, and if the Request is received by a device Function with Type 0 headers, the device *MUST@FLIT* be treated as the target of the Message regardless of the Bus Number and, for non-ARI Devices, the Device Number specified in the Destination ID field of the Request
 - If the Function specified in the Destination ID field is unimplemented, then it is implementation specific whether that Message is either silently discarded or else handled as an Unsupported Request (UR) and reported as an error associated with the Receiving Port (see [§ Section 6.2](#)).
 - Earlier versions of this specification recommended ignoring the Bus Number and Device Number fields with the Destination ID comparison in certain cases. While this behavior is still permitted, it is no longer recommended.

If the Request is not a Message, and is a supported Type, specific implementations may be optimized based on a defined programming model that ensures that certain types of (otherwise legal) Requests will never occur. Such implementations may take advantage of the following rule:

- If the Request violates the programming model of the device Function, the Function may optionally treat the Request as a Completer Abort, instead of handling the Request normally
 - If the Request is treated as a Completer Abort, this is a reported error associated with the Function (see [§ Section 6.2](#))
 - If the Request requires Completion, a Completion Status of CA is returned (see [§ Section 2.2.9](#))

IMPLEMENTATION NOTE: OPTIMIZATIONS BASED ON RESTRICTED PROGRAMMING MODEL §

When a device's programming model restricts (versus what is otherwise permitted in PCI Express) the characteristics of a Request, that device is permitted to return a UR or a CA Completion Status, or to terminate the Request in a suitable implementation specific way for any Request that violates the programming model. Examples include unaligned or wrong-size access to a register block and unsupported size of request to a Memory Space.

Generally, devices are able to rely on a restricted programming model when all communication will be between the device's driver software and the device itself. Devices directly accessed via other software (e.g., operating system, application software) may not be able to rely on a restricted programming.

Devices that implement legacy capabilities should be designed to support all types of Requests that are possible in the existing usage model for the device. If this is not done, the device may fail to operate with existing software.

If the Request arrives between the time an FLR has been initiated and the completion of the FLR by the targeted Function, the Request is permitted to be silently discarded (following update of flow control credits) without logging or signaling it as an error. It is recommended that the Request be handled as an Unsupported Request (UR).

- For DMWr Requests, refer to the rules in [§ Section 6.32](#).

- Otherwise (supported Request Type, not a DMWr, not a Message), process the Request
 - If the Completer is permanently unable to process the Request due to a device-specific error condition the Completer must, if possible, handle the Request as a Completer Abort
 - This is a reported error associated with the Receiving Function, if the error can be isolated to a specific Function in the component, or to the Receiving Port if the error cannot be isolated (see § Section 6.2)
 - For Configuration Requests, if Device Readiness Status (DRS) is supported³⁹, then:
 - Following any DRS Event (see § Section 6.22), once the Link is in L0, a Function associated with an Upstream Port *MUST@FLIT* return a Completion Status of RRS until the Upstream Port has transmitted a Device Readiness Status Message.
 - Once the Upstream Port has transmitted a Device Readiness Status Message, all non-VF Functions of the Upstream Port must respond to all properly formed Configuration Requests with a Completion Status of Successful Completion.
 - For Configuration Requests only, if Device Readiness Status is not supported, following reset it is permitted for a Function to terminate the request and indicate that it is temporarily unable to process the Request, but will be able to process the Request in the future - in this case, the Request Retry Status (RRS) Completion Status must be used (see § Section 6.6). Valid reset conditions after which a device/Function is permitted to return RRS in response to a Configuration Request are:
 - Cold, Warm, and Hot Resets
 - FLRs
 - A reset initiated in response to a D3_{Hot} to D0_{uninitialized} device state transition
 - A device Function is explicitly not permitted to return RRS in response to a Configuration Request following a software-initiated reset (other than an FLR) of the device, e.g., by the device's software driver writing to a device-specific reset bit. A device Function is not permitted to return RRS in response to a Configuration Request after it has indicated that it is Configuration-Ready (see § Section 6.22) without an intervening valid reset (i.e., FLR or Conventional Reset) condition, or if the Immediate Readiness bit in the Function's Status register is Set. Additionally, a device Function is not permitted to return RRS in response to a Configuration Request after having previously returned a Successful Completion without an intervening valid reset (i.e., FLR or Conventional Reset) condition.
 - A Function that implements the Readiness Time Reporting Extended Capability must not return RRS in response to Configuration Requests that are received after the relevant times reported in that Extended Capability.
 - In the process of servicing the Request, the Completer may determine that the (otherwise acceptable) Request must be handled as an error, in which case the Request is handled according to the type of the error.
 - Example: A PCI Express/PCI Bridge may initially accept a Request because it specifies a Memory Space range mapped to the secondary side of the Bridge, but the Request may Master Abort or Target Abort on the PCI side of the Bridge. From the PCI Express perspective, the status of the Request in this case is UR (for Master Abort) or CA (for Target Abort). If the Request requires Completion on PCI Express, the corresponding Completion Status is returned.
- If the Request is a type that requires a Completion to be returned, generate a Completion according to the rules for Completion formation (see § Section 2.2.9)
 - The Completion Status is determined by the result of handling the Request

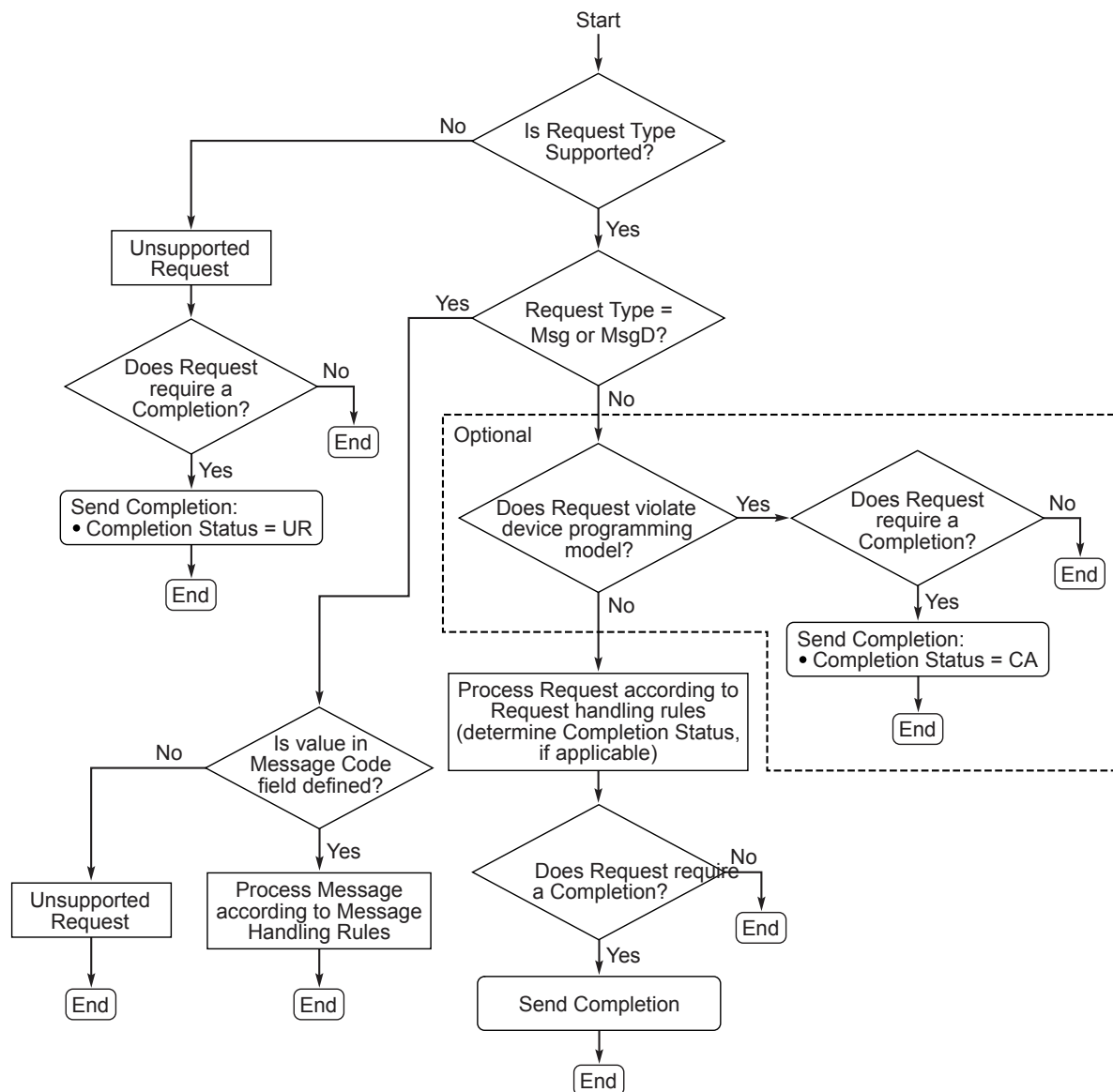
39. If Flit Mode is supported then Device Readiness Status must be supported.

- If the Request has an ECRC Check Failed error, then it is implementation specific whether to return a Completion or not. If a Completion is returned, the Completion *MUST* have a UR Completion Status.
- Under normal operating conditions, PCI Express Endpoints and Legacy Endpoints must never delay the acceptance of a Posted Request for more than 10 μ s, which is called the **Posted Request Acceptance Limit**. The device must either (a) be designed to process received Posted Requests and return associated Flow Control credits within the necessary time limit, or (b) rely on a restricted programming model to ensure that a Posted Request is never initiated to the Endpoint either by software or by other devices while the Endpoint is unable to accept a new Posted Request within the necessary time limit.
 - The following are not considered normal operating conditions under which the Posted Request Acceptance Limit applies:
 - The period immediately following a Fundamental Reset (see § Section 6.6)
 - TLP retransmissions or Link retraining
 - One or more dropped Flow Control Packets (FCPs)
 - The device being in a diagnostic mode
 - The device being in a device-specific mode that is not intended for normal use
 - The following are considered normal operating conditions, but any delays they cause do not count against the Posted Request Acceptance Limit:
 - Upstream TLP traffic delaying Upstream FCPs
 - The Link coming out of a low-power state
 - Arbitration with traffic on other VCs
 - Though not a requirement, it is strongly recommended that RCiEPs also honor the Posted Request Acceptance Limit.
- If the device/Function supports being a target for I/O Write Requests, which are Non-Posted Requests, it is strongly recommended that each associated Completion be returned within the same time limit as for Posted Request acceptance, although this is not a requirement.
- If the device/Function supports being a target for DMWr Requests, each associated Completion must be returned within the Posted Request Acceptance Limit⁴⁰

IMPLEMENTATION NOTE: RESTRICTED PROGRAMMING MODEL FOR MEETING THE POSTED REQUEST ACCEPTANCE LIMIT §

Some hardware designs may not be able to process every DMWr or Posted Request within the required Posted Request Acceptance Limit. An example is writing to a command queue where commands can take longer than the acceptance time limit to complete. Subsequent writes to such a device when it is currently processing a previous write could experience acceptance delays that exceed the limit. Such devices may rely on a restricted programming model, where the device driver limits the rate of DMWr/memory writes issued to the device, the driver polls the device to determine buffer availability before issuing the transaction, or the driver implements some other software-based flow control mechanism.

40. Although DMWr is a Non-Posted Request, the Posted Request Acceptance Limit is applied because many (but not all) of the same concerns apply as with Posted Requests. The name Posted Request Acceptance Limit is retained for historical reasons.



OM13773A

Figure 2-91 Flowchart for Handling of Received Requests §

IMPLEMENTATION NOTE:

REQUEST RETRY STATUS FOR CONFIGURATION REQUESTS §

Some devices require a lengthy self-initialization sequence to complete before they are able to service Configuration Requests. In specified circumstances it is permitted for a device/Function to “hold off” initial configuration via the Request Retry Status (RRS) Completion Status mechanism. A device in receipt of a Configuration Request following a valid reset condition may respond with an RRS Completion Status to terminate the Request, and thus effectively stall the Configuration Request until such time that the subsystem has completed local initialization and is ready to communicate with the host. Note that it is only legal in specified circumstances to respond with an RRS Completion Status in response to a Configuration Request. Readiness Notifications (see § Section 6.22) and Immediate Readiness (see § Section 7.5.1.1.4) and Immediate_Readiness_on_Return_to_D0 (see § Section 7.5.2.1) also forbid the use of RRS Completion Status in response to a Configuration Request in certain situations.

Receipt by the Requester of a Completion with RRS Completion Status terminates the Configuration Request. Further action by the Root Complex regarding the original Configuration Request is specified in § Section 2.3.2 .

Root Complexes that implement Configuration RRS Software Visibility have the ability to report the receipt of RRS Completion Status for a Configuration Request to software, enabling software to attend to other tasks rather than being stalled while the device completes its self-initialization. Software that intends to take advantage of this mechanism must ensure that the first access made to a device following a valid reset condition is a Configuration Read Request accessing both bytes of the Vendor ID field in the device's Configuration Space header. For this case only, the Root Complex, if enabled, will synthesize a special read-data value for the Vendor ID field to indicate to software that RRS Completion Status has been returned by the device in response to a Configuration Request. For Configuration Requests to other addresses, or when Configuration RRS Software Visibility is not enabled, the Root Complex will generally re-issue the Configuration Request until it completes with a status other than RRS as described in § Section 2.3.2 .

Systems that contain PCIe components whose self-initialization time may require them to return a RRS Completion Status in response to a Configuration Request (by the rules in § Section 6.6) should provide some mechanism for re-issuing Configuration Requests terminated with RRS status. In systems running legacy PCI/PCI-X based software, the Root Complex must re-issue the Configuration Request using a hardware mechanism to ensure proper enumeration of the system.

Refer to § Section 6.6 for more information on reset.

2.3.1.1 Data Return for Non-UIO Read Requests §

- Individual Completions for Memory Read Requests may provide less than the full amount of data Requested so long as all Completions for a given Request when combined return exactly the amount of data Requested in the Read Request.
 - Completions for different Requests cannot be combined.
 - I/O and Configuration Reads must be completed with exactly one Completion.
 - A Completion with Completion Status other than Successful Completion must:
 - be of type Cpl or CplLk,
 - for a Read Request, including a Translation Request, be the final Completion returned.
- Completions must not include more data than permitted by the Transmitting Function's Tx_MPS_Limit.
 - A Receiving Function must check for violations of its Rx_MPS_Limit.

- See [§ Section 2.2.2](#) for important details with both Transmitters and Receivers.

Note: This is simply a specific case of the rules that apply to all TLPs with [data payloads](#)

- Memory Read Requests may be completed with one, or in some cases, multiple Completions
- [Read Completion Boundary \(RCB\)](#) determines the naturally aligned address boundaries on which a Completer is permitted to break up the response for a single Read Request into multiple Completions.
 - For a Root Complex, RCB is 64 bytes or 128 bytes.
 - This value is reported in the [Read Completion Boundary](#) bit in the [Link Control Register](#) (see [§ Section 7.5.3.7](#)).

Note: Bridges and Endpoints may implement a corresponding command bit that may be set by system software to indicate the RCB value for the Root Complex, allowing the Bridge or Endpoint to optimize its behavior when the Root Complex's RCB is 128 bytes.

- For all other System Elements, RCB is 128 bytes.
- Completions for Requests that do not cross the naturally aligned address boundaries at integer multiples of RCB bytes must include all data specified in the Request.
- Requests that do cross the address boundaries at integer multiples of RCB bytes are permitted to be completed using more than one Completion subject to the following rules:
 - The first Completion must start with the address specified in the Request, and if successful must end at one of the following:
 - The address that satisfies the entire Request
 - An address boundary between the start and end of the Request at an integer multiple of RCB bytes
 - If the final Completion is successful, it must end at the address that satisfies the entire Request
 - All Completions between, but not including, the first and final Completions must be an integer multiple of RCB bytes in length
- Receivers may optionally check for violations of RCB. If a Receiver implementing this check determines that a Completion violates this rule, it must handle the Completion as a Malformed TLP.
 - This is a reported error associated with the Receiving Port (see [§ Section 6.2](#)).
- Multiple Memory Read Completions for a single Read Request must return data in increasing address order.
- If all the Memory Read Completions for a single Read Request have a Successful Completion Status, the sum of their payloads must equal the size requested. See [§ Section 10.2.4](#) for an exception for Memory Reads that are ATS Translation Requests.
- For each Memory Read Completion, the Byte Count field must indicate the remaining number of bytes required to complete the Request including the number of bytes returned with the Completion, except when the BCM bit is Set.⁴¹
 - The total number of bytes required to complete a Memory Read Request is calculated as shown in [§ Table 2-40](#).
 - If a Memory Read Request is completed using multiple Completions, the Byte Count value for each successive Completion is the value indicated by the preceding Completion minus the number of bytes returned with the preceding Completion.

41. Only PCI-X completers Set the BCM bit. PCI Express completers are not permitted to set the BCM bit. In Flit Mode, the BCM bit is deprecated. When translating from Non-Flit Mode to Flit Mode, if the BCM bit is 1b a TLP Translation Egress Blocked error must be indicated. When translating from Flit Mode to Non-Flit Mode the BCM bit must be Clear in the Non-Flit Mode Header.

- The Completion Data area begins at the DW address specified by the Request. In the first or only Data DW of the first or only Completion, only the bytes configured as active in the First DW BE field in the Request contain valid data. Bytes configured as inactive in the First DW BE field in the Request will return undefined content.
- In the last Data DW of the last successful Completion, only the bytes configured as active in the Last DW BE field in the Request contain valid data. Bytes configured as inactive in the Last DW BE field in the Request will return undefined content.
- All the Completion Data bytes, including those with undefined content, are included in all CRC calculations.
- § Figure 2-92 presents an example of the above. The example assumes a single Completion TLP is returned.

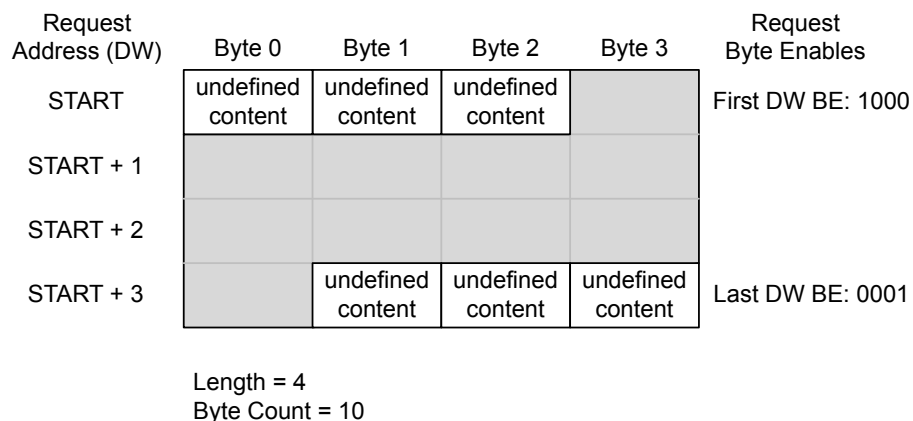


Figure 2-92 Example Completion Data when some Byte Enables are 0b §

IMPLEMENTATION NOTE:

BCM BIT USAGE §

To satisfy certain PCI-X protocol constraints, a PCI-X Bridge or PCI-X Completer for a PCI-X burst read in some cases will set the Byte Count field in the first PCI-X transaction of the Split Completion sequence to indicate the size of just that first transaction instead of the entire burst read. When this occurs, the PCI-X Bridge/PCI-X Completer will also Set the BCM bit in that first PCI-X transaction, to indicate that the Byte Count field has been modified from its normal usage. Refer to the [PCI-X] for further details.

A PCI Express Memory Read Requester needs to correctly handle the case when a PCI-X Bridge/PCI-X Completer sets the BCM bit. When this occurs, the first Read Completion packet returned to the Requester will have the BCM bit Set, indicating that the Byte Count field reports the size of just that first packet instead of the entire remaining Byte Count. The Requester should not conclude at this point that other packets of the Read Completion are missing.

The BCM bit will never be Set in subsequent packets of the Read Completion, so the Byte Count field in those subsequent packets will always indicate the remaining Byte Count in each instance. Thus, the Requester can use the Byte Count field in these packets to determine if other packets of the Read Completion are missing.

PCI Express Completers will never Set the BCM bit.

The BCM bit is not present in Flit Mode.

*Table 2-40 Calculating Byte Count from Length and First/
Last Byte Enables §*

First DW BE[3:0] (b)	Last DW BE[3:0] (b)	Total Byte Count
1xx1	0000 ⁴²	4
01x1	0000	3
1x10	0000	3
0011	0000	2
0110	0000	2
1100	0000	2
0001	0000	1
0010	0000	1
0100	0000	1
1000	0000	1
0000	0000	1
xxx1	1xxx	Length ⁴³ * 4
xxx1	01xx	(Length * 4) - 1
xxx1	001x	(Length * 4) - 2
xxx1	0001	(Length * 4) - 3
xx10	1xxx	(Length * 4) - 1
xx10	01xx	(Length * 4) - 2
xx10	001x	(Length * 4) - 3
xx10	0001	(Length * 4) - 4
x100	1xxx	(Length * 4) - 2
x100	01xx	(Length * 4) - 3
x100	001x	(Length * 4) - 4
x100	0001	(Length * 4) - 5
1000	1xxx	(Length * 4) - 3
1000	01xx	(Length * 4) - 4
1000	001x	(Length * 4) - 5
1000	0001	(Length * 4) - 6

42. Note that Last DW BE of 0000b is permitted only with a Length of 1 DW.

43. Length is the number of DW as indicated by the value in the Length field, and is multiplied by 4 to yield a number in bytes.

- For all Memory Read Completions, the Lower Address field must indicate the lower bits of the byte address for the first enabled byte of data returned with the Completion.
 - For the first (or only) Completion, the Completer can generate this field from the least significant 5 bits of the address of the Request concatenated with 2 bits of byte-level address formed as shown in § Table 2-41.
 - For any subsequent Completions, the Lower Address field will always be zero except for Completions generated by a Root Complex with an RCB value of 64 bytes. In this case the least significant 6 bits of the Lower Address field will always be zero and the most significant bit of the Lower Address field will toggle according to the alignment of the 64-byte data payload.

Table 2-41 Calculating Lower Address from First DW Byte Enables §

First DW BE[3:0] (b)	Lower Address[1:0] (b)
0000	00
xxx1	00
xx10	01
x100	10
1000	11

- When a Read Completion is generated with a Completion Status other than Successful Completion:
 - No data is included with the Completion
 - The Cpl (or CplLk) encoding is used instead of CplD (or CplDLk)
 - This Completion is the final Completion for the Request
 - The Completer must not transmit additional Completions for this Request
 - Example: Completer split the Request into four parts for servicing; the second Completion had a Completer Abort Completion Status; the Completer terminated servicing for the Request, and did not Transmit the remaining two Completions.
 - The Byte Count field must indicate the remaining number of bytes that would be required to complete the Request (as if the Completion Status were Successful Completion)
 - The Lower Address field must indicate the lower bits of the byte address for the first enabled byte of data that would have been returned with the Completion if the Completion Status were Successful Completion

IMPLEMENTATION NOTE:

RESTRICTED PROGRAMMING MODEL §

When a device's programming model restricts (vs. what is otherwise permitted in PCI Express) the size and/or alignment of Read Requests directed to the device, that device is permitted to use a Completer Abort Completion Status for Read Requests that violate the programming model. An implication of this is that such devices, generally devices where all communication will be between the device's driver software and the device itself, need not necessarily implement the buffering required to generate Completions of length RCB. However, in all cases, the boundaries specified by RCB must be respected for all reads that the device will complete with Successful Completion status.

Examples:

1. Memory Read Request with Address of 1 0000h and Length of C0h bytes (192 decimal) could be completed by a Root Complex with an RCB value of 64 bytes with one of the following combinations of Completions (bytes):
192 -or- 128, 64 -or- 64, 128 -or- 64, 64
2. Memory Read Request with Address of 1 0000h and Length of C0h bytes (192 decimal) could be completed by a Root Complex with an RCB value of 128 bytes in one of the following combinations of Completions (bytes):
192 -or- 128, 64
3. Memory Read Request with Address of 1 0020h and Length of 100h bytes (256 decimal) could be completed by a Root Complex with an RCB value of 64 bytes in one of the following combinations of Completions (bytes):
256 -or-
32, 224 -or- 32, 64, 160 -or- 32, 64, 64, 96 -or- 32, 64, 64, 64, 32 -or-
32, 64, 128, 32 -or- 32, 128, 96 -or- 32, 128, 64, 32 -or-
96, 160 -or- 96, 128, 32 -or- 96, 64, 96 -or- 96, 64, 64, 32 -or-
160, 96 -or- 160, 64, 32 -or- 224, 32
4. Memory Read Request with Address of 1 0020h and Length of 100h bytes (256 decimal) could be completed by an Endpoint in one of the following combinations of Completions (bytes):
256 -or- 96, 160 -or- 96, 128, 32 -or- 224, 32

2.3.1.2 UIO Read Completions §

- UIO Read Completions must follow the same rules as non-UIO Read Completions, with the following exceptions:
 - Multiple UIO Read Completions for a single UIO Read Request are permitted to be returned in any address order.

2.3.1.3 UIO Write Completions §

- UIO Write Completions must follow these rules:
 - Write Completion Boundary (WCB) is 64B, and indicates the naturally aligned address boundaries on which a Completer is permitted to break up the response for a single UIO Write Request into multiple Completions.
 - Multiple UIO Write Completions for a single UIO Write Request are permitted to be returned in any address order.
- It is permitted for UIO Completers to coalesce UIO Write Completions with the same Transaction ID.
 - Completion coalescing must not exceed the number of DWORDs that can be represented in the Length[9:0] field.
 - It is recommended that UIO Completers coalesce opportunistically, and, in most cases, it is recommended not to delay the return of a UIO Completion in order to coalesce it with a subsequent UIO Completion. Specific policies for UIO Completion coalescing are implementation-specific.
- Switches are not permitted to coalesce UIO Completions.

2.3.2 Completion Handling Rules §

- When a device receives a Completion that does not match the Transaction ID for any of the outstanding Requests issued by that device, the Completion is called an “Unexpected Completion”.
- If a received Completion matches the Transaction ID of an outstanding Request, but in other TLP fields does not match the corresponding Request, the Receiver *MUST@FLIT* handle the Completion as an Unexpected Completion.
 - The Requester must not check the IDO Attribute (Attribute Bit 2) in the Completion, since the Completer is not required to copy the value of IDO from the Request into the Completion for that request as stated in [§ Section 2.2.6.4](#) and [§ Section 2.2.9](#).
 - However, if the Completion is otherwise properly formed, it is permitted for the Receiver to handle the Completion as a Malformed TLP.
- When an Ingress Port of a Switch receives a Completion that cannot be forwarded, that Ingress Port must handle the Completion as an Unexpected Completion. This includes Completions that target:
 - a non-existent Function in the Device associated with the Upstream Port,
 - a non-existent Device on the Bus associated with the Upstream Port,
 - a non-existent Device or Function on the internal switching fabric, or
 - a Bus Number within the Upstream Port's Bus Number aperture but not claimed by any Downstream Port.
- Receipt of an Unexpected Completion is an error and must be handled according to the following rules:
 - The agent receiving an Unexpected Completion must discard the Completion.
 - An Unexpected Completion is a reported error associated with the Receiving Port (see [§ Section 6.2](#)).

Note: Unexpected Completions are assumed to occur mainly due to Switch misrouting of the Completion. The Requester of the Request may not receive a Completion for its Request in this case, and the Requester's Completion Timeout mechanism (see [§ Section 2.8](#)) will terminate the Request.

- Completions with a Completion Status other than Successful Completion or Request Retry Status (in response to Configuration Request) must cause the Requester to:
 - Free Completion buffer space and other resources associated with the Request.
 - Handle the error via a Requester-specific mechanism (see § Section 6.2.3.2.5).

If the Completion arrives between the time an FLR has been initiated and the completion of the FLR by the targeted Function, the Completion is permitted to be handled as an Unexpected Completion or to be silently discarded (following update of flow control credits) without logging or signaling it as an error. Once the FLR has completed, received Completions corresponding to Requests issued prior to the FLR must be handled as Unexpected Completions, unless the Function has been re-enabled to issue Requests.

- Root Complex handling of a Completion with Request Retry Status for a Configuration Request is implementation specific, except for the period following system reset (see § Section 6.6). For Root Complexes that support Configuration RRS Software Visibility, the following rules apply:
 - If Configuration RRS Software Visibility is not enabled, the Root Complex must re-issue the Configuration Request as a new Request.
 - If Configuration RRS Software Visibility is enabled (see below):
 - For a Configuration Read Request that includes both bytes of the Vendor ID field of a device Function's Configuration Space Header, the Root Complex must complete the Request to the host by returning a read-data value of 0001h for the Vendor ID field and all 1's for any additional bytes included in the request. This read-data value has been reserved specifically for this use by the PCI-SIG and does not correspond to any assigned Vendor ID.
 - For a Configuration Write Request or for any other Configuration Read Request, the Root Complex must re-issue the Configuration Request as a new Request.

A Root Complex implementation may choose to limit the number of Configuration Request/RRS Completion Status loops before determining that something is wrong with the target of the Request and taking appropriate action (e.g., complete the Request to system software as a failed transaction).

Configuration RRS Software Visibility may be enabled through the Configuration RRS Software Visibility Enable bit in the Root Control Register (see § Section 7.5.3.12) to control Root Complex behavior on an individual Root Port basis. Alternatively, Root Complex behavior may be managed through the Configuration RRS Software Visibility Enable bit in the Root Complex Register Block (RCRB) Control register as described in § Section 7.9.7.4 , permitting the behavior of one or more Root Ports or RCiEPs to be controlled by a single Enable bit. For this alternate case, each Root Port or RCiEP declares its association with a particular Enable bit via an RCRB header association in a Root Complex Link Declaration Capability (see § Section 7.9.8). Each Root Port or RCiEP is permitted to be controlled by at most one Enable bit. Thus, for example, it is prohibited for a Root Port whose Root Control register contains an Enable bit to declare an RCRB header association to an RCRB that also includes an Enable bit in its RCRB Header Capability. The presence of an Enable bit in a Root Port or RCRB Header Capability is indicated by the corresponding Configuration RRS Software Visibility bit (see § Section 7.5.3.13 and § Section 7.9.7.3 , respectively).

- Completions with a Reserved Completion Status value are treated as if the Completion Status was Unsupported Request (UR).
- Completions with a Completion Status of Unsupported Request or Completer Abort are reported using the conventional PCI reporting mechanisms (see § Section 7.5.1.1.4).
 - Note that the error condition that triggered the generation of such a Completion is reported by the Completer as described in § Section 6.2 .
- When a Completion for a Read, AtomicOp, or DMWr Request is received with a Completion Status other than Successful Completion:

- No data is included with the Completion
 - The Cpl (or CplLk) encoding is used instead of CplD (CplDLk)
- This Completion is the final Completion for the Request
 - The Requester must consider the Request terminated, and not expect additional Completions
 - Handling of partial Completions Received earlier is implementation specific

Example: The Requester received 32 bytes of Read data for a 128-byte Read Request it had issued, then it receives a Completion with the Completer Abort Completion Status. The Requester then must free the internal resources that had been allocated for that particular Read Request.

IMPLEMENTATION NOTE:

READ DATA VALUES WITH UR COMPLETION STATUS §

Some system configuration software depends on reading a data value of all 1's when a Configuration Read Request is terminated as an Unsupported Request, particularly when probing to determine the existence of a device in the system. A Root Complex intended for use with software that depends on a read-data value of all 1's must synthesize this value when UR Completion Status is returned for a Configuration Read Request.

2.4 Transaction Ordering §

2.4.1 Transaction Ordering Rules for TLPs not using UIO or Flow-Through IDE Streams §

The rules defined in this section apply uniformly to all types of Transactions on PCI Express including Memory, I/O, Configuration, and Messages, except for UIO Requests/Completions (see § Section 2.4.2) and Flow-Through IDE Streams which have modified ordering requirements (see § Section 6.33.7). The ordering rules defined in this table apply within a single Traffic Class (TC). There is no ordering requirement among transactions with different TC labels. Note that this also implies that there is no ordering required between traffic that flows through different Virtual Channels since transactions with the same TC label are not allowed to be mapped to multiple VCs on any PCI Express Link.

For § Table 2-42, the columns represent a first issued transaction and the rows represent a subsequently issued transaction. The table entry indicates the ordering relationship between the two transactions. The table entries are defined as follows:

Yes

The second transaction (row) must be allowed to pass the first (column) to avoid deadlock. (When blocking occurs, the second transaction is required to pass the first transaction. Fairness must be comprehended to prevent starvation.)

Y/N

There are no requirements. The second transaction may optionally pass the first transaction or be blocked by it.

No

The second transaction must not be allowed to pass the first transaction. This is required to support the producer/consumer strong ordering model.

Table 2-42 Non-UIO Ordering Rules §

Row Pass Column?		Posted Request (Col 2)	Non-Posted Request		Completion (Col 5)
			Read Request (Col 3)	NPR with Data (Col 4)	
Posted Request (Row A)		a) No b) <u>Y/N</u>	<u>Yes</u>	<u>Yes</u>	a) <u>Y/N</u> b) <u>Yes</u>
Non-Posted Request	Read Request (Row B)	a) No b) <u>Y/N</u>	<u>Y/N</u>	<u>Y/N</u>	<u>Y/N</u>
	NPR with Data (Row C)	a) No b) <u>Y/N</u>	<u>Y/N</u>	<u>Y/N</u>	<u>Y/N</u>
Completion (Row D)		a) No b) <u>Y/N</u>	<u>Yes</u>	<u>Yes</u>	a) <u>Y/N</u> b) <u>No</u>

Explanation of the row and column headers in § Table 2-42:

A **Posted Request** is a Memory Write Request or a Message Request.

A **Read Request** is a Configuration Read Request, an I/O Read Request, or a Memory Read Request.

An **NPR** (Non-Posted Request) **with Data** is a Configuration Write Request, an I/O Write Request, an AtomicOp Request, or a DMWr.

A **Non-Posted Request** is a Read Request or an NPR with Data.

Explanation of the entries in § Table 2-42:

A2a

A Posted Request must not pass another Posted Request unless A2b applies.

A2b

A Posted Request with RO⁴⁴ Set is permitted to pass another Posted Request.⁴⁵ A Posted Request with IDO Set is permitted to pass another Posted Request if the two Requester IDs (including Requester Segment when in FM) are different. Additionally, a Posted Request with IDO Set is permitted to pass another Posted Request with the same Requester ID if both Requests contain a PASID and the two PASID values are different.

A3, A4

A Posted Request must be able to pass Non-Posted Requests to avoid deadlocks.

A5a

A Posted Request is permitted to pass a Completion, but is not required to be able to pass Completions unless A5b applies.

A5b

Inside a PCI Express to PCI/PCI-X Bridge whose PCI/PCI-X bus segment is operating in conventional PCI mode, for transactions traveling in the PCI Express to PCI direction, a Posted Request must be able to pass Completions to avoid deadlock.

B2a

A Read Request must not pass a Posted Request unless B2b applies.

44. In this section, “RO” is an abbreviation for the Relaxed Ordering Attribute field.

45. Some usages are enabled by not implementing this passing (see the No RO-enabled PR-PR Passing bit in § Section 7.5.3.15).

B2b

A Read Request with IDO Set is permitted to pass a Posted Request if the two Requester IDs (including Requester Segment in FM) are different. Additionally, a Read Request with IDO Set is permitted to pass a Posted Request with the same Requester ID if both Requests contain a PASID and the two PASID values are different.

C2a

An NPR with Data must not pass a Posted Request unless C2b applies.

C2b

An NPR with Data and with RO Set ⁴⁶ is permitted to pass Posted Requests. An NPR with Data and with IDO Set is permitted to pass a Posted Request if the two Requester IDs (including Requester Segment in FM) are different. Additionally, an NPR with Data and with IDO Set is permitted to pass a Posted Request with the same Requester ID if both Requests contain a PASID and the two PASID values are different.

B3, B4, C3, C4

A Non-Posted Request is permitted to pass another Non-Posted Request.

B5, C5

A Non-Posted Request is permitted to pass a Completion.

D2a

A Completion must not pass a Posted Request unless D2b applies.

D2b

An I/O or Configuration Write Completion ⁴⁷ is permitted to pass a Posted Request. A Completion with RO Set is permitted to pass a Posted Request. A Completion with IDO Set is permitted to pass a Posted Request if the Completer ID of the Completion is different from the Requester ID of the Posted Request.

D3, D4

A Completion must be able to pass Non-Posted Requests to avoid deadlocks.

D5a

Completions with different Transaction IDs are permitted to pass each other.

D5b

Completions with the same Transaction ID must not pass each other. This ensures that multiple Completions associated with a single Memory Read Request will remain in ascending address order.

Additional Rules:

- PCI Express Switches are permitted to allow a Memory Write or Message Request with the Relaxed Ordering bit set to pass any previously posted Memory Write or Message Request moving in the same direction. Switches must forward the Relaxed Ordering attribute unmodified. The Root Complex is also permitted to allow data bytes within the Request to be written to system memory in any order. (The bytes must be written to the correct system memory locations. Only the order in which they are written is unspecified).
- For Root Complex and Switch, Memory Write combining (as defined in the [PCI]) is prohibited.
 - Note: This is required so that devices can be permitted to optimize their receive buffer and control logic for Memory Write sizes matching their natural expected sizes, rather than being required to support the maximum possible Memory Write payload size.
- For Root Complex and Switch, Memory Write collapsing (as defined in the [PCI]) is prohibited.

46. Note: Not all NPR with Data transactions are permitted to have RO Set.

47. Note: Not all components can distinguish I/O and Configuration Write Completions from other Completions. In particular, routing elements not serving as the associated Requester or Completer generally cannot make this distinction. A component must not apply this rule for I/O and Configuration Write Completions unless it is certain of the associated Request type.

- Combining of Memory Read Requests, and/or Completions for different Requests is prohibited.
- The No Snoop bit does not affect the required ordering behavior.
- For Root Ports and Switch Downstream Ports, acceptance of a Posted Request or Completion must not depend upon the transmission of a Non-Posted Request within the same traffic class.⁴⁸
- For Switch Upstream Ports, acceptance of a Posted Request or Completion must not depend upon the transmission on a Downstream Port of Non-Posted Request within the same traffic class.⁴⁹
- For Endpoint, Bridge, and Switch Upstream Ports, the acceptance of a Posted Request must not depend upon the transmission of any TLP from that same Upstream Port within the same traffic class.⁵⁰
- For Endpoint, Bridge, and Switch Upstream Ports, the acceptance of a Non-Posted Request must not depend upon the transmission of a Non-Posted Request from that same Upstream Port within the same traffic class.⁵¹
- For Endpoint, Bridge, and Switch Upstream Ports, the acceptance of a Completion must not depend upon the transmission of any TLP from that same Upstream Port within the same traffic class.⁵²

Note that Endpoints are never permitted to block acceptance of a Completion.

- Completions issued for Non-Posted Requests must be returned in the same Traffic Class as the corresponding Non-Posted Request.
- Root Complexes that support peer-to-peer operation and Switches must enforce these transaction ordering rules for all forwarded traffic.

To ensure deadlock-free operation, devices should not forward traffic from one Virtual Channel to another. The specification of constraints used to avoid deadlock in systems where devices forward or translate transactions between Virtual Channels is outside the scope of this document (see § Appendix D. for a discussion of relevant issues).

48. Satisfying the above rules is a necessary, but not sufficient condition to ensure deadlock free operation. Deadlock free operation is dependent upon the system topology, the number of Virtual Channels supported and the configured Traffic Class to Virtual Channel mappings. Specification of platform and system constraints to ensure deadlock free operation is outside the scope of this specification (see § Appendix D. for a discussion of relevant issues).

49. Satisfying the above rules is a necessary, but not sufficient condition to ensure deadlock free operation. Deadlock free operation is dependent upon the system topology, the number of Virtual Channels supported and the configured Traffic Class to Virtual Channel mappings. Specification of platform and system constraints to ensure deadlock free operation is outside the scope of this specification (see § Appendix D. for a discussion of relevant issues).

50. Satisfying the above rules is a necessary, but not sufficient condition to ensure deadlock free operation. Deadlock free operation is dependent upon the system topology, the number of Virtual Channels supported and the configured Traffic Class to Virtual Channel mappings. Specification of platform and system constraints to ensure deadlock free operation is outside the scope of this specification (see § Appendix D. for a discussion of relevant issues).

51. Satisfying the above rules is a necessary, but not sufficient condition to ensure deadlock free operation. Deadlock free operation is dependent upon the system topology, the number of Virtual Channels supported and the configured Traffic Class to Virtual Channel mappings. Specification of platform and system constraints to ensure deadlock free operation is outside the scope of this specification (see § Appendix D. for a discussion of relevant issues).

52. Satisfying the above rules is a necessary, but not sufficient condition to ensure deadlock free operation. Deadlock free operation is dependent upon the system topology, the number of Virtual Channels supported and the configured Traffic Class to Virtual Channel mappings. Specification of platform and system constraints to ensure deadlock free operation is outside the scope of this specification (see § Appendix D. for a discussion of relevant issues).

IMPLEMENTATION NOTE:

DEADLOCKS CAUSED BY PORT ACCEPTANCE DEPENDENCIES §

With certain configurations and communication paradigms, systems whose Ports have acceptance dependencies may experience deadlocks. In this context, Port acceptance dependencies refer to the Ingress Port making the acceptance of a Posted Request or Completion dependent upon the Egress Port first being able to transmit one or more TLPs. As stated earlier in this section, Endpoints, Bridges, and Switch Upstream Ports are forbidden to have these dependencies. However, Downstream Ports are allowed to have these dependencies.

In certain cases, Downstream Port acceptance dependencies are unavoidable. For example, the ACS P2P Request Redirect mechanism may be redirecting some peer-to-peer Posted Requests Upstream through an RP for validation in the Root Complex. Validated Posted Requests are then reflected back down through the same RP so they can make their way to their original targets. The validated Posted Requests set up the acceptance dependency due to this internal looping. For traffic within one system, Downstream Port acceptance dependencies do not contribute to deadlocks. However, for certain types of traffic between systems, Downstream Port acceptance dependencies can contribute to deadlocks.

One general case where this may contribute to deadlocks is when two or more systems have an interconnect that enables each host to map host memory in other hosts for Programmed I/O (PIO) access, as shown on the left side of § Figure 2-93. A specific example is when two systems each have a PCIe Switch with one or more integrated by Non-Transparent Bridges (NTBs), and the two systems are connected as shown on the right side of the figure.

Deadlock can occur if each host CPU is doing a heavy stream of Posted Requests to host memory in the opposite host. If Posted Request traffic in each direction gets congested, and the RP in each host stops accepting Posted Requests because the RP can't transmit outbound TLPs, deadlock occurs. The root cause of deadlock in this case is actually the adapter to the system-to-system interconnect setting up an acceptance dependency, which is forbidden for Endpoints. For the example case of PCIe Switches with integrated NTBs, the NTBs are Endpoints, and the Switch Upstream Port has the acceptance dependency. While the RP's acceptance dependency is not the root cause of the deadlock, it contributes to the deadlock.

Solutions using this paradigm for intersystem communications will either need to determine that their systems don't have these acceptance dependencies or rely on other mechanisms to avoid these potential deadlocks. Such mechanisms are outside the scope of this specification.

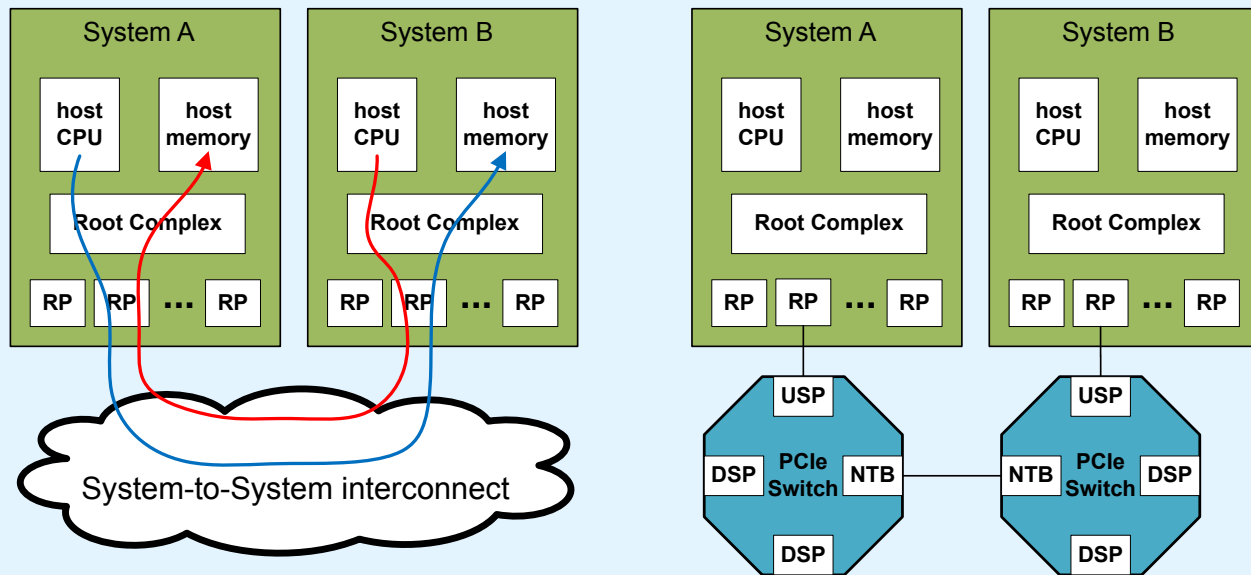


Figure 2-93 Deadlock Examples with Intersystem Interconnects §

IMPLEMENTATION NOTE: LARGE MEMORY READS VS. MULTIPLE SMALLER MEMORY READS §

Note that the rule associated with entry D5b in § Table 2-42 ensures that for a single Memory Read Request serviced with multiple Completions, the Completions will be returned in address order. However, the rule associated with entry D5a permits that different Completions associated with distinct Memory Read Requests may be returned in a different order than the issue order for the Requests. For example, if a device issues a single Memory Read Request for 256 bytes from location 1000h, and the Request is returned using two Completions (see § Section 2.3.1.1) of 128 bytes each, it is guaranteed that the two Completions will return in the following order:

1st Completion returned: Data from 1000h to 107Fh.

2nd Completion returned: Data from 1080h to 10FFh.

However, if the device issues two Memory Read Requests for 128 bytes each, first to location 1000h, then to location 1080h, the two Completions may return in either order:

1st Completion returned: Data from 1000h to 107Fh.

2nd Completion returned: Data from 1080h to 10FFh.

- or -

1st Completion returned: Data from 1080h to 10FFh.

2nd Completion returned: Data from 1000h to 107Fh.

2.4.2 Ordering Rules for UIO §

The rules defined in this section apply to UIO TLPs. UIO and non-UIO TLPs are never mixed within a TC/VC, and ordering dependencies between UIO and non-UIO TLPs are not permitted.

The ordering rules for UIO TLPs are (see § Table 2-43):

- UIO Completions must be allowed to pass UIO Requests
- UIO allows arbitrary reordering for all other cases

Table 2-43 UIO TLP Ordering Rules §

Row Pass Col?	UIO PR-FC TLP (Col U1)	UIO NPR-FC TLP (Col U2)	UIO Completion (Col U3)
UIO PR-FC TLP (Row UA)	Yes/No	Yes/No	Yes/No
UIO NPR-FC TLP (Row UB)	Yes/No	Yes/No	Yes/No
UIO Completion (Row UC)	Yes	Yes	Yes/No

It is recommended that permitted UIO reordering cases be supported and implemented.

The deadlock avoidance rules for UIO TLPs are covered in § Table 2-44 and § Table 2-45.

- For considerations on acceptance dependencies, see IMPLEMENTATION NOTE: DEADLOCKS CAUSED BY PORT ACCEPTANCE DEPENDENCIES
- Downstream Ports include Root Ports and Switch Downstream Ports
- Upstream Ports include Switch Upstream Ports and Endpoint Upstream Ports

Table 2-44 UIO Acceptance Dependency Rules – Downstream Ports §

Row Independent of Col?		Egress transmission		
		UIO Memory Write Request (Col 1)	Other UIO Memory Request (Col 2)	UIO Completion (Col 3)
Ingress Acceptance	UIO Memory Write Request (Row A)	Yes/No	Yes/No	Yes/No
	Other UIO Memory Request (Row B)	Yes/No	Yes/No	Yes/No
	UIO Completion (Row C)	Yes	Yes	Yes/No

Table 2-45 UIO Acceptance Dependency Rules – Upstream Ports §

Row Independent of Col?		Egress transmission		
		UIO Memory Write Request (Col 1)	Other UIO Memory Request (Col 2)	UIO Completion (Col 3)
Ingress Acceptance	UIO Memory Write Request (Row A)	Yes	Yes	Yes/No
	Other UIO Memory Request (Row B)	Yes	Yes	Yes/No
	UIO Completion (Row C)	Yes	Yes	Yes

2.4.3 Update Ordering and Granularity Observed by a Read Transaction §

2.4.3.1 Ordering and Granularity for Non-UIO Reads §

If a Requester using a single transaction reads a block of data from a Completer, and the Completer's data buffer is concurrently being updated, the ordering of multiple updates and granularity of each update reflected in the data returned by the read is outside the scope of this specification, unless otherwise specified (see § Section 6.32). This applies both to updates performed by PCI Express write transactions and updates performed by other mechanisms such as host CPUs updating host memory.

If a Requester using a single transaction reads a block of data from a Completer, and the Completer's data buffer is concurrently being updated by one or more entities not on the PCI Express fabric, the ordering of multiple updates and granularity of each update reflected in the data returned by the read is outside the scope of this specification, unless otherwise specified (see § Section 6.32).

As an example of update ordering, assume that the block of data is in host memory, and a host CPU writes first to location A and then to a different location B. A Requester reading that data block with a single read transaction is not guaranteed to observe those updates in order. In other words, the Requester may observe an updated value in location B and an old value in location A, regardless of the placement of locations A and B within the data block. Unless a Completer makes its own guarantees (outside this specification) with respect to update ordering, a Requester that relies on update ordering must observe the update to location B via one read transaction before initiating a subsequent read to location A to return its updated value.

As an example of update granularity, if a host CPU writes a QW to host memory, a Requester reading that QW from host memory may observe a portion of the QW updated and another portion of it containing the old value.

While not required by this specification, it is strongly recommended that host platforms guarantee that when a host CPU writes aligned DWs or aligned QWs to host memory, the update granularity observed by a PCI Express read will not be smaller than a DW.

IMPLEMENTATION NOTE: NO ORDERING REQUIRED BETWEEN CACHELINES §

A Root Complex serving as a Completer to a single Memory Read that requests multiple cachelines from host memory is permitted to fetch multiple cachelines concurrently, to help facilitate multi-cacheline completions, subject to Tx_MPS_Limit. No ordering relationship between these cacheline fetches is required.

2.4.3.2 Ordering and Granularity for UIO Reads §

If a Requester uses a single UIO Read Request to read a block of data from a Completer, and the Completer's data buffer is concurrently being updated using UIO Write Requests, the granularity of each update reflected in the data returned by the read must, within each 64B aligned block, be observed such that each UIO Write Request is either fully completed or not-at-all completed.

Once a Completer transmits a UIO Read Completion reflecting the updated value resulting from a UIO Write Request to a 64B aligned block, subsequently received UIO Requests must observe the UIO Write as fully completed for that 64B aligned block. Thus, at any other Link or Port between a Requester and Completer, observation of a UIO Read Completion at that Link/Port reflecting the updated value resulting from a UIO Write Request to that 64B aligned block implies that all other UIO Requests passing on that Link/Port will observe the UIO Write as fully completed for that 64B aligned block.

The observed sequence is permitted to be different for each 64B aligned block.

A Completer is permitted to implement, through a restricted programming model, an update granularity of less than 64B for some or all of a resource mapped by means of a BAR. In order to maintain the 64B ordering/granularity requirements, such a Completer is permitted to terminate Requests that exceed this smaller update granularity in an implementation specific way.

2.4.4 Update Ordering and Granularity Provided by a Write Transaction §

2.4.4.1 Ordering and Granularity for Non-UIO Writes §

If a single write transaction containing multiple DWs and the Relaxed Ordering bit Clear is accepted by a Completer, the observed ordering of the updates to locations within the Completer's data buffer must be in increasing address order. This semantic is required in case a PCI or PCI-X Bridge along the path combines multiple write transactions into the single one. However, the observed granularity of the updates to the Completer's data buffer is outside the scope of this specification.

While not required by this specification, it is strongly recommended that host platforms guarantee that when a PCI Express write updates host memory, the update granularity observed by a host CPU will not be smaller than a DW.

As an example of update ordering and granularity, if a Requester writes a QW to host memory, in some cases a host CPU reading that QW from host memory could observe the first DW updated and the second DW containing the old value.

2.4.4.2 Ordering and Granularity for UIO Writes §

For each 64B naturally aligned block updated in-full or in-part by a single UIO Write Request, all bytes updated within the 64B aligned block must be fully observable or not-at-all observable to any read, whether the read is performed by a PCI Express transaction or another mechanism. The observed sequence is permitted to be different for each 64B aligned block but the observed sequence must be consistent for all readers of a particular 64B aligned block.

Once a Completer has Transmitted a UIO Write Completion for a 64B aligned block, the Completer must ensure that all UIO Requests it receives must observe the UIO Write as fully completed for that 64B aligned block. Thus, at any other Link or Port between a Requester and Completer, observation of a UIO Write Completion at that Link/Port implies that all other UIO Requests passing on that Link/Port will observe the UIO Write as fully completed for that 64B aligned block.

The observed sequence is permitted to be different for each 64B aligned block.

A Completer is permitted to implement, through a restricted programming model, an update granularity of less than 64B for some or all of a resource mapped by means of a BAR. In order to maintain the 64B ordering/granularity requirements, such a Completer is permitted to terminate Requests that exceed this smaller update granularity in an implementation specific way.

2.5 Virtual Channel (VC) Mechanism §

The Virtual Channel (VC) mechanism provides support for carrying, throughout the fabric, traffic that is differentiated using Traffic Class (TC) labels. The foundations of VCs are independent fabric resources (queues/buffers and associated control logic). These resources are used to move information across Links with fully independent Flow Control between different VCs (Link Flow Control is defined in § Section 2.6). This is key to solving the problem of flow-control induced blocking where a single traffic flow may create a bottleneck for all traffic within the system.

As Link speed increases, the buffer space required to support fully independent Flow Control between different VCs while also supporting full Link bandwidth on any given VC also increases. In Flit Mode, the Shared Flow Control (FC) mechanism can be used to reduce this resource requirement. Flow Control is defined in § Section 2.6 .

Traffic is associated with VCs by mapping packets with particular TC labels to their corresponding VCs. The Streamlined Virtual Channel (SVC), Virtual Channel (VC), and Multi-Function Virtual Channel (MFVC) mechanisms allow flexible mapping of TCs onto the VCs. In the simplest form, TCs can be mapped to VCs on a 1:1 basis. To allow performance/cost tradeoffs, PCI Express provides the capability of mapping multiple TCs onto a single VC. § Section 2.5.2 covers details of TC to VC mapping.

A Virtual Channel is established when one or multiple TCs are associated with a physical VC resource designated by Virtual Channel Identification (VC ID). This process is controlled by configuration software as described in § Section 6.3 , § Section 7.9.1 , and § Section 7.9.2 .

In Flit Mode, initially, VC0 is initialized automatically by hardware with a dedicated FC credit pool, and a shared FC pool. As system software enables other VCs, the enabled VCs are also initialized with a dedicated FC credit pool and a Shared FC pool per VC. The Shared FC credits for additional VCs expand the Shared FC pool available to all VCs (and are permitted to be zero when the appropriate Shared FC credits were granted earlier). When only a single VC is supported and merged credits are not used, there is a single “shared” credit pool and VC0 is initialized with zero dedicated credits. When only a single VC is supported and merged credits are used, there is a single “shared” non-Posted credit pool and VC0 is initialized with zero non-Posted dedicated credits (posted and completion do have dedicated credits).

Once system software has completed enabling all VCs that are to be enabled, it is recommended that system software Set, as appropriate, VC Enablement Completed in the SVC Port Control Register, the All VCs Enabled bit in the Port VC Control Register or the MFVC Port VC Control Register to indicate that VC initialization is completed. Once this bit has been Set, behavior is undefined if additional VCs are enabled or disabled.

The Shared Flow Control Usage Limit mechanism allows system software to manage the allocation of Shared FC by Transmitters, for example to support Quality of Service (QoS) policies.

Support for TCs and VCs beyond the default TC0/VC0 pair is optional although some optional mechanisms also require support for additional TC/VC. The association of TC0 with VC0 is fixed (i.e., hardwired), and must be supported by all components. Therefore, the baseline TC/VC setup does not require any VC-specific hardware or software configuration. In order to ensure interoperability where possible, components that do not implement any of the optional SVC, VC, or MFVC Extended Capability structures must obey the following rules:

- A Requester must only generate requests with TC0 label. (Note that if the Requester initiates requests with a TC label other than TC0, the requests may be treated as malformed by the component on the other side of the Link that implements the extended VC Extended Capability and applies TC Filtering.)
- A Completer must accept requests with TC label other than TC0, and must preserve the TC label. That is, any completion that it generates must have the same TC label as the label of the request.

- A Switch must map all TCs to VC0 and must forward all transactions regardless of the TC label.

Even with the above rules, in some cases interoperability may not be possible, such as when TC/VC mechanisms are used to implement protocols that cannot be mapped onto TC0/VC0. The SVC mechanism and its associated requirements provide a framework for interoperable hardware and software, and it is strongly recommended that hardware implementing support for VCs beyond VC0 support SVC.

A Port containing Functions capable of generating Requests with TC labels other than TC0 must implement suitable SVC, VC, or MFVC Extended Capability structures (as applicable), even if it only supports the default VC. Example Function types are Endpoints and Root Ports. This is required in order to enable mapping of TCs beyond the default configuration. It must follow the TC/VC mapping rules according to the software programming of the extended capability structure used to configure the VC.

SVC provides explicit support for architecturally-defined TC/VC applications via a combination of hardware requirements and software guidance. Ports supporting UIO must implement the SVC Extended Capability. The TC/VC default HW assignments in § Table 2-46 are mandatory for SVC.

Table 2-46 Streamlined Virtual Channel (SVC) TC/VC Default Assignments §

TC	VC	Description
TC0	VC0	TC0 VC0 Default TC/VC- configured automatically by hardware – required to be used for certain mechanisms as defined in this specification
TC1	VC1	Reserved
TC2	VC2	Reserved
TC3	VC3	UIO TC/VC (Required if UIO is supported)
TC4	VC4	UIO TC/VC (Optional if UIO is supported)
TC5	VC5	Reserved
TC6	VC6	Reserved
TC7	VC7	Reserved

IMPLEMENTATION NOTE: MULTI-HOST FABRICS AND STREAMLINED VIRTUAL CHANNEL (SVC) TC/VC DEFAULT ASSIGNMENTS §

When PCIe and/or related switching fabrics support multiple hosts, and one or more fabric Links carry traffic from multiple hosts concurrently, the use of TCs across the fabric and TC/VC mappings on each fabric Link may conflict. For example, if one host relies on TC3 mapping to a UIO VC while another host relies on TC3 mapping to a non-UIO VC, the resulting behavior is undefined.

To avoid TC conflicts on multi-host fabric Links, it is recommended that system software that configures TC/VC mappings:

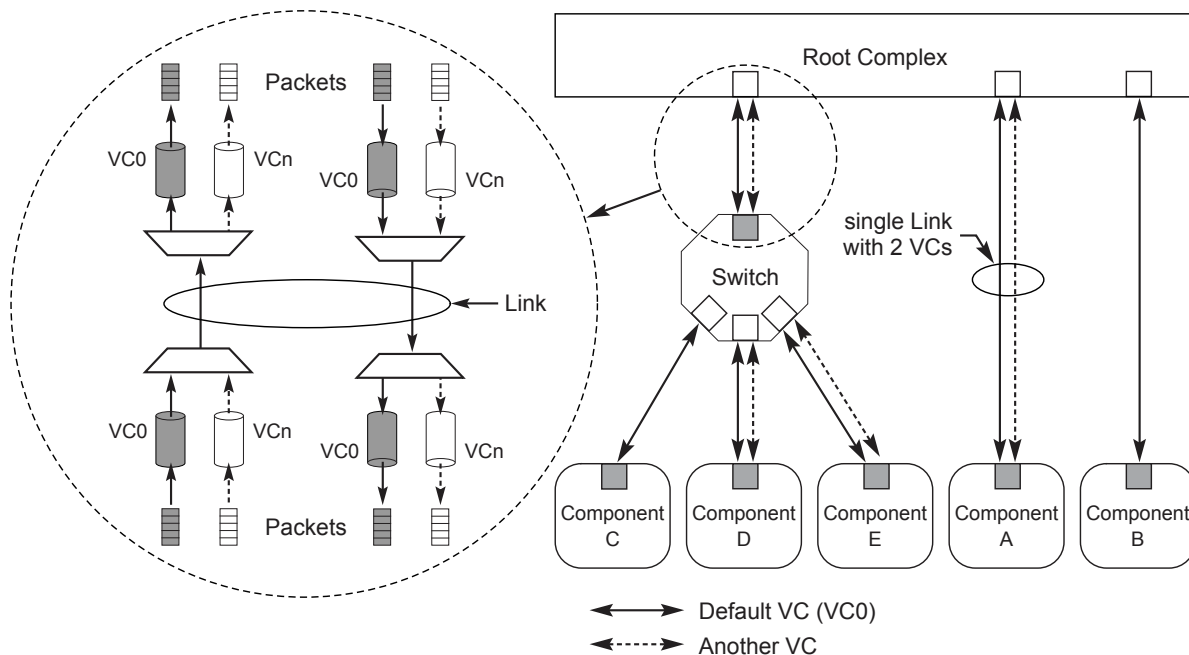
- supports the SVC Extended Capability,
- preserves any TC/VC assignments already configured in Switch Ports, whether such VCs are enabled or not, and
- configures and enables TC/VC assignments on each RP to match such Switch Ports, to the extent permitted by RP hardware capability.

This allows a Fabric Manager to preconfigure TC/VC assignments fabric-wide, and rely on OS cooperation.

For cases where a Fabric Manager does not preconfigure TC/VC assignments, § Table 2-46 provides reasonable defaults that can work in currently envisioned multi-host fabrics.

System software should use the SVC Extended Capability for TC/VC configuration in all hardware that supports SVC. If it enables VCs other than VC0 in hardware that supports only the VC and/or MFVC Extended Capabilities, it should configure enabled TC/VCs to match the SVC default assignments.

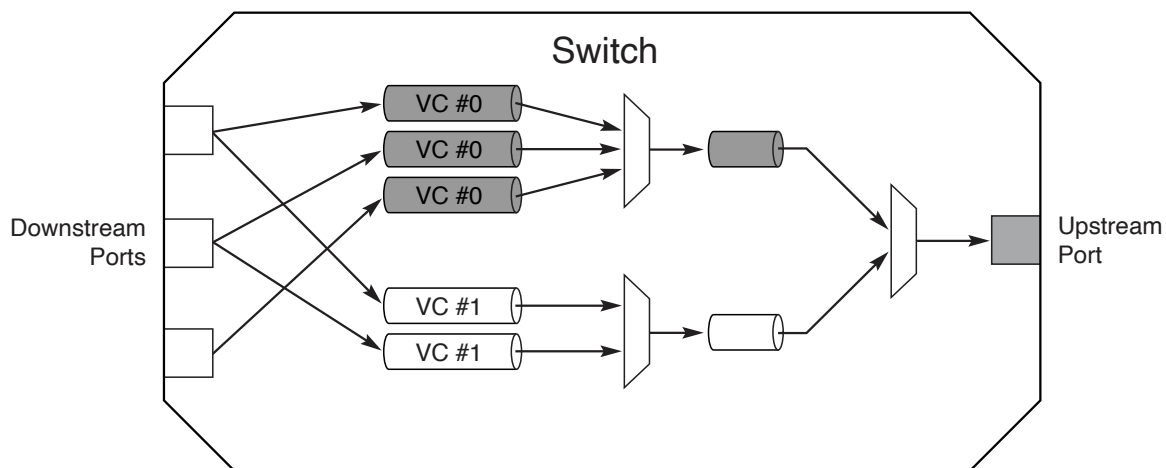
§ Figure 2-94 illustrates the concept of Virtual Channel. Conceptually, traffic that flows through VCs is multiplexed onto a common physical Link resource on the Transmit side and de-multiplexed into separate VC paths on the Receive side.



OM13760A

Figure 2-94 Virtual Channel Concept - An Illustration §

Internal to the Switch, every Virtual Channel requires dedicated physical resources (queues/buffers and control logic) that support independent traffic flows inside the Switch. § Figure 2-95 shows conceptually the VC resources within the Switch (shown in § Figure 2-94) that are required to support traffic flow in the Upstream direction.



OM13761

Figure 2-95 Virtual Channel Concept - Switch Internals (Upstream Flow) §

An MFD may implement Virtual Channel resources similar to a subset of those in a Switch, for the purpose of managing the Quality of Service (QoS) for Upstream requests from the different Functions to the device's Upstream Egress Port.

IMPLEMENTATION NOTE:

VC AND VC BUFFERING CONSIDERATIONS §

The amount of buffering beyond the architectural minimums per supported VC is implementation specific.

Buffering beyond the architectural minimums is not required to be identical across all VCs on a given Link. That is, an implementation may provide greater buffer depth for selected VCs as a function of implementation usage models and other Link attributes (e.g., Link width and signaling).

Implementations may adjust their buffering per VC based on implementation specific policies derived from configuration and VC enablement. For example, if a four VC implementation has only two VCs enabled, the implementation may assign the non-enabled VC buffering to the enabled VCs to improve fabric efficiency/performance by reducing the probability of fabric backpressure due to Link-level flow control.

The number of VCs supported, and the associated buffering per VC per Port, are not required to be the same for all Ports of a multi-Port component (e.g., a Switch or Root Complex).

2.5.1 Virtual Channel Identification (VC ID) §

PCI Express Ports can support 1 to 8 Virtual Channels - each Port is independently configured/managed therefore allowing implementations to vary the number of VCs supported per Port based on usage model-specific requirements. These VCs are uniquely identified using the VC ID mechanism.

Note that while DLLPs contain VC ID information for Flow Control accounting, TLPs do not. The association of TLPs with VC ID for the purpose of Flow Control accounting is done at each Port of the Link using TC to VC mapping as discussed in § Section 2.5.2 .

Rules for assigning VC ID to VC hardware resources within a Port are as follows:

- VC ID assignment must be unique per Port - The same VC ID cannot be assigned to different VC hardware resources within the same Port.
- VC ID assignment must be the same (matching in the terms of numbers of VCs and their IDs) for the two Ports on both sides of a Link.
- If an MFD implements an MFVC Extended Capability structure, its VC hardware resources are distinct from the VC hardware resources associated with any VC Extended Capability structures of its Functions. The VC ID uniqueness requirement (first bullet above) still applies individually for the MFVC and any VC Extended Capability structures. In addition, the VC ID cross-Link matching requirement (second bullet above) applies for the MFVC Extended Capability structure, but not the VC Extended Capability structures of the Functions.
- VC ID 0 is assigned and fixed to the default VC.
- It is permitted to implement VCs that support only specific protocols and/or use models
 - If software maps such VCs in a way that is incompatible with their protocol/use model requirements, the resulting hardware behavior is undefined

2.5.2 TC to VC Mapping §

Every Traffic Class that is supported must be mapped to one of the Virtual Channels. The mapping of TC0 to VC0 is fixed.

The mapping of TCs other than TC0 must obey the following rules:

- One or multiple TCs can be mapped to a VC.
- One TC must not be mapped to multiple VCs in any Port or Endpoint Function.
- TC/VC mapping must be identical for Ports on both sides of a Link.
- If UIO is supported, VC3 must be supported, and it must support UIO, and enabling VC3 is required to use UIO.
- If UIO is supported, and if a second UIO VC is supported, then the second UIO VC must be VC4 (and so VC4 must support UIO traffic); if UIO is enabled using only one VC it must be VC3, if UIO is enabled using two VCs they must be VC3 and VC4.
- If a UIO TLP targets an Egress Port where the TC maps to a non-UIO VC, or a non-UIO TLP targets an Egress Port where the TC maps to a UIO VC, such TLPs must be handled as specified in § Section 2.2.1.2 for other TLPs that cannot be translated. Note that this rule partially overlaps with the rule in § Section 2.2.1.2 regarding a UIO TLP targeting an Egress Port in NFM.

§ Table 2-47 provides an example of TC to VC mapping.

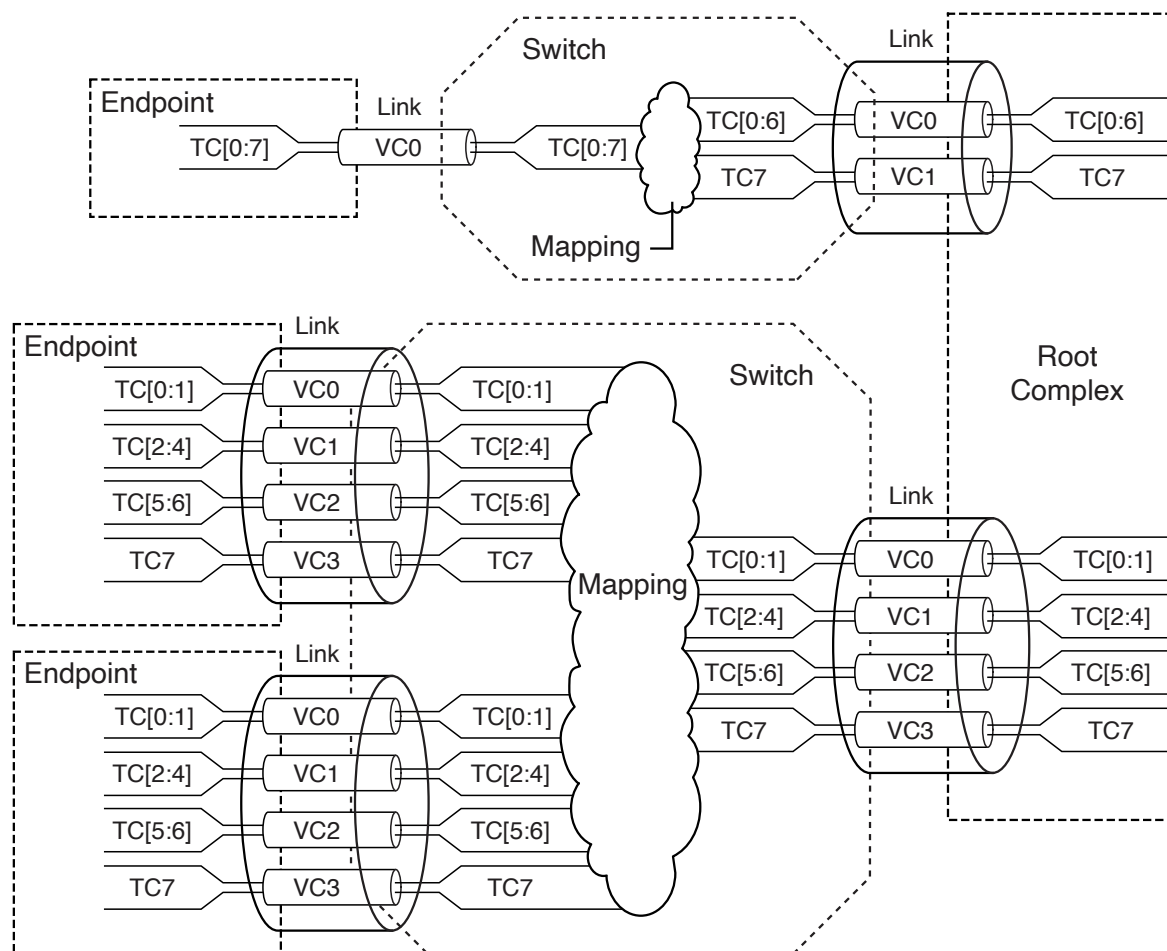
Table 2-47 TC to VC Mapping Example §

Supported VC Configurations	TC/VC Mapping Options
VC0	TC(0-7)/VC0
VC0, VC1	TC(0-6)/VC0, TC7/VC1
VC0-VC3	TC(0-1)/VC0, TC(2-4)/VC1, TC(5-6)/VC2, TC7/VC3
VC0-VC7	TC[0:7]/VC[0:7]

Notes on conventions:

TCn/VCK	TCn mapped to VCk
TC(n-m)/VCK	all TCs in the range n-m mapped to VCk (i.e., to the same VC)
TC[n:m]/VC[n:m]	TCn/VCn, TCn+1/VCn+1, ..., TCm/VCm

§ Figure 2-96 provides a graphical illustration of TC to VC mapping in several different Link configurations. For additional considerations on TC/VC, refer to § Section 6.3 .



OM13762

Figure 2-96 An Example of TC/VC Configurations §

2.5.3 VC and TC Rules §

Here is a summary of key rules associated with the TC/VC mechanism:

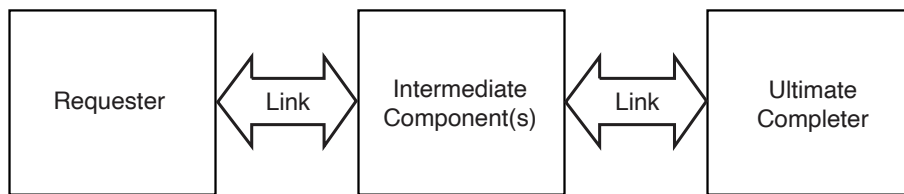
- All devices must support the general purpose I/O Traffic Class (i.e., TC0 and must implement the default VC0).
- Each Virtual Channel (VC) has independent Flow Control.
- There are no ordering relationships required between different TCs.
- There are no ordering relationships required between different VCs.
- A Switch's peer-to-peer capability applies to all Virtual Channels supported by the Switch.
- An MFD's peer-to-peer capability between different Functions applies to all Virtual Channels supported by the MFD.
- Transactions with a TC that is not mapped to any enabled VC in an Ingress Port are treated as Malformed TLPs by the receiving device.
- For Switches, transactions with a TC that is not mapped to any of the enabled VCs in the target Egress Port are treated as Malformed TLPs.

- For a Root Port, transactions with a TC that is not mapped to any of the enabled VCs in the target RCRB are treated as Malformed TLPs.
- For MFDs with an MFVC Extended Capability structure, any transaction with a TC that is not mapped to an enabled VC in the MFVC Extended Capability structure is treated as a Malformed TLP.
- Switches must support independent TC/VC mapping configuration for each Port.
- A Root Complex must support independent TC/VC mapping configuration for each RCRB, the associated Root Ports, and any RCIEPs.

For more details on the VC and TC mechanisms, including configuration, mapping, and arbitration, refer to § Section 6.3.

2.6 Ordering and Receive Buffer Flow Control §

Flow Control (FC) is used to prevent overflow of Receiver buffers and to enable compliance with the ordering rules defined in § Section 2.4. Note that the Flow Control mechanism is used by the Requester to track the queue/buffer space available in the agent across the Link as shown in § Figure 2-97. That is, Flow Control is point-to-point (across a Link) and not end-to-end. Flow Control does not imply that a Request has reached its ultimate Completer.



OM13776

Figure 2-97 Relationship Between Requester and Ultimate Completer §

Flow Control is orthogonal to the data integrity mechanisms used to implement reliable information exchange between Transmitter and Receiver. Flow Control can treat the flow of TLP information from Transmitter to Receiver as perfect, since the data integrity mechanisms ensure that corrupted and lost TLPs are corrected through retransmission (see § Section 3.6).

In Non-Flit Mode each Virtual Channel (VC) maintains an independent FC credit pool.

In Flit Mode, the Shared FC mechanism can be used to reduce VC resource requirements. There are two sets of resources associated with each VC: a (typically small) pool of dedicated resources associated independently with each FC/VC (to avoid deadlock by allowing that the Transmitter to transmit at least one TLP in that VC/FC using only dedicated credit(s)), and a portion of the (typically larger) pool of shared resources. The Transmitter gate function (defined later in this section) uses the sum of all Shared FC returned across all VCs. The transmitter gate function also provides a Usage Limit mechanism to avoid over-consumption of buffers by stalled VCs. This Usage Limit mechanism is configured by software and defaults to disabled. To support Usage Limit, credits are returned to the Transmitter indicating the VC for the TLP(s) that, by making forward progress, freed those credits. The FC information is conveyed between two sides of the Link using DLLPs. The VC ID field of the DLLP is used to carry the VC ID that is required for proper Flow Control credit accounting. Additionally, [Merged] FC enables the sharing of buffers for Posted Requests and Completions, further reducing resource requirements.

Flow Control mechanisms used internally within an MFD are outside the scope of this specification.

Flow Control is handled by the Transaction Layer in cooperation with the Data Link Layer. The Transaction Layer performs Flow Control accounting functions for Received TLPs and “gates” TLP Transmissions based on available credits for transmission even if those TLPs are eventually nullified.

Note: Flow Control is a function of the Transaction Layer and, therefore, the following types of information transmitted on the interface are not associated with Flow Control Credits: LCRC, Packet Framing Symbols, other Special Symbols, and Data Link Layer to Data Link Layer inter-communication packets. An implication of this fact is that these types of information must be processed by the Receiver at the rate they arrive (except as explicitly noted in this specification).

Also, any TLPs transferred from the Transaction Layer to the Data Link and Physical Layers must have first passed the Flow Control gate. Thus, both Transmit and Receive Flow Control mechanisms are unaware if the Data Link Layer transmits a TLP repeatedly due to errors on the Link.

2.6.1 Flow Control (FC) Rules §

In this and other sections of this specification, rules are described using conceptual “registers” that a device could use in order to implement a compliant implementation. This description does not imply or require a particular implementation and is used only to clarify the requirements.

- Flow Control (FC) information is transferred using Flow Control Packets (FCPs), which are a type of DLLP (see § Section 3.5), and, in some cases, the Optimized_Update_FC.
- **FC Unit Size** indicates the number of DW covered by one flow control credit:
 - For Data, the FC Unit Size is 4 DW.
 - For Headers in Non-Flit Mode:
 - For Receivers that do not support TLP Prefixes, FC Unit Size is the sum of one maximum-size Header and TLP Digest.
 - For Receivers that support End-End TLP Prefixes, FC Unit Size is the sum of one maximum-size Header, TLP Digest, and the maximum number of End-End TLP Prefixes permitted in a TLP.
 - The management of FC for Receivers that support Local TLP Prefixes is dependent on the Local TLP Prefix type.
 - For Headers in Flit Mode:
 - For Switch Port Receivers, FC Unit Size is the sum of one maximum-size Base Header, OHC-A, OHC-B, OHC-C, OHC-E if supported, and one maximum-size TLP Trailer.
 - For Endpoint Upstream Port and Root Port Receivers, FC Unit Size is the sum of one Base Header of the largest supported size, OHC-A, OHC-B, OHC-C, OHC-E if supported, and one TLP Trailer of the largest size supported.
- For NFM and for dedicated credits in FM, each Virtual Channel has independent FC.
- In FM, each Virtual Channel has some amount of independent FC referred to as dedicated credits, and some amount of shared FC.
 - When only one single VC is implemented and [Merged] is not used, there are no dedicated credits, all flow control uses shared credits (see § Table 3-3, Notes 3 and 4).
- It is permitted for a Transmitter to use dedicated credits when Transmitting a TLP, even when sufficient shared credits are available.
- The Transmitter indicates the use of dedicated credits for a specific TLP by applying the Flit Mode Local TLP Prefix with the TLP Uses Dedicated Credits bit Set.
- Flow Control distinguishes three types of TLPs (note relationship to ordering rules - see § Section 2.4):
 - Posted Requests (P) - Messages and Memory Writes
 - Non-Posted Requests (NP) - All Reads, I/O Writes, Configuration Writes, AtomicOps, and DMWrs.
 - Completions (Cpl) - Associated with corresponding NP Requests

- In addition, Flow Control distinguishes the following types of TLP information within each of the three types:
 - Headers (H)
 - Data (D)
- Thus, there are six types of information tracked by Flow Control for each Virtual Channel, as shown in § Table 2-48.

Table 2-48 Flow Control Credit Types §

Credit Type	Applies to This Type of TLP Information
PH	Posted Request headers
PD	Posted Request Data payload
NPH	Non-Posted Request headers
NPD	Non-Posted Request Data payload
CplH	Completion headers
CplD	Completion Data payload

- TLPs consume Flow Control credits as shown in § Table 2-49.

Table 2-49 TLP Flow Control Credit Consumption §

TLP	Credit Consumed ⁵³
Memory, I/O, Configuration Read Request	1 NPH unit
Memory Write Request	1 PH + n PD units ⁵⁴
I/O, Configuration Write Request	1 NPH + 1 NPD Note: size of data written is never more than 1 (aligned) DW
AtomicOp, DMWr Request	1 NPH + n NPD units
Message Requests without data	1 PH unit
Message Requests with data	1 PH + n PD units
Memory Read Completion	1 CplH + n CplD units
I/O, Configuration Read Completions	1 CplH unit + 1 CplD unit
I/O, Configuration Write, and DMWr Completions	1 CplH unit
AtomicOp Completion	1 CplH unit + 1 CplD unit Note: size of data returned is never more than 4 (aligned) DWs.

- FC must be initialized autonomously by hardware only for the default Virtual Channel (VC0).

53. Each header credit implies the ability to accept a TLP Digest along with the corresponding TLP.

54. For all cases where “n” appears, $n = \text{Roundup}(\text{Length}/\text{FC unit size})$. Where Length is the size of the Payload in DW.

- VC0 is initialized when the Data Link Layer is in the DL_Init state following reset (see § Section 3.2 and § Section 3.4).
- When Virtual Channels other than VC0 are enabled by software, each newly enabled VC must follow the Flow Control initialization protocol (see § Section 3.4).
 - Software enables a Virtual Channel by setting the VC Enable bits for that Virtual Channel in both components on a Link (see § Section 7.9.1 and § Section 7.9.2).

Note: It is possible for multiple VCs to be following the Flow Control initialization protocol simultaneously - each follows the initialization protocol as an independent process.

- Software disables a Virtual Channel by clearing the VC Enable bits for that Virtual Channel in both components on a Link.
 - Disabling a Virtual Channel for a component resets the Flow Control tracking mechanisms for that Virtual Channel in that component.
 - In Flit Mode, disabling a Virtual Channel resets the Flow Control tracking mechanisms for dedicated credits for that Virtual Channel in that component and has no effect on Shared Flow Control credit tracking.
 - In Flit Mode, behavior is undefined if a VC is disabled and subsequently re-enabled while the link remains up.
- InitFC1 and InitFC2 FCPs are used only for Flow Control initialization (see § Section 3.4).
- An InitFC1, InitFC2, UpdateFC FCP, or Optimized_Update_FC that specifies a Virtual Channel that is disabled must be discarded without effect.
- During FC initialization for any Virtual Channel, including the default VC initialized as a part of Link initialization, Receivers must initially advertise VC credit values equal to or greater than those shown in § Table 2-50.
 - Scaled Flow Control is activated when both Ports on a Link perform the Data Link Feature mechanism with the Scaled Flow Control Supported bit Set (i.e., Local Scaled Flow Control Supported and Remote Scaled Flow Control Supported are both Set, see § Section 3.3).
 - If Scaled Flow Control is not supported or supported but not activated, use the values in the "Scale Factor 1" column.
- If Scaled Flow Control is supported and activated, use the values in the column for the scaling factor associated with that credit type (see § Section 3.4.2).
- For a Multi-Function Device where different Functions have different Rx_MPS_Limit values, the largest Rx_MPS_Limit value across all Functions must be used.
- In Flit Mode, for each Credit Type, shared credit advertisement during initialization must either:
 - Advertise [Infinite.3] on all VCs, or
 - Advertise a combination of [Zero] and non-[Zero] on all VCs.
 - When multiple VCs are supported, it is permitted for all VCs to advertise [Zero] shared credits.
 - All VCs advertising non-[Zero] shared credits must have the same Scale Factor.
 - If any VC advertised non-[Zero] shared credits:
 - All VCs advertising [Zero] shared credits must use that Scale Factor in subsequent UpdateFCs.
 - The sum of the advertisements across all VCs must be greater than or equal to the value in § Table 2-50 multiplied by the number of enabled VCs. The § Table 2-50 minimum values do not apply to an individual VC as long as this rule applies.

- When [Merged] flow control is used, the sum of the Posted advertisements across all VCs must be greater than or equal to the value in § Table 2-50 multiplied by two times the number of enabled VCs (i.e., Posted minimum must account for Completions as well).
- See § Table 3-3 Note 6.
- In Flit Mode, dedicated credits are permitted to use any Scale Factor on any VC for any Credit Type.

Table 2-50 Minimum Initial Flow Control Advertisements⁵⁵ §

Credit Type	Minimum Advertisement		
	No Scaling or Scale Factor 1	Scale Factor 4	Scale Factor 16
PH	Shared credits in Flit Mode: 4 units – credit value of 04h. Otherwise: 1 unit - credit value of 01h.	4 Units - credit value of 01h.	16 Units - credit value of 01h.
PD	Shared credits in Flit Mode: $\text{Ceiling}(\text{Rx_MPS_Limit} / \text{FC Unit Size}) + 4$. Otherwise: $\text{Rx_MPS_Limit} \text{ divided by FC Unit Size}$. Example: If the Rx_MPS_Limit is 1024 bytes, the smallest permitted initial credit value would be 040h (44h for shared credits in Flit Mode).	$\text{Ceiling}(\text{Rx_MPS_Limit} / (\text{FC Unit Size} * 4)) + 1$. Example: If the Rx_MPS_Limit is 1024 bytes, the smallest permitted initial credit value would be 011h.	$\text{Ceiling}(\text{Rx_MPS_Limit} / (\text{FC Unit Size} * 16)) + 1$. Example: If the Rx_MPS_Limit is 1024 bytes, the smallest permitted initial credit value would be 005h.
NPH	Shared credits in Flit Mode: 4 units – credit value of 04h. Otherwise: 1 unit - credit value of 01h.	4 Units - credit value of 01h.	16 Units - credit value of 01h.
NPD	Shared credits in Flit Mode: $\text{Max}(\text{Rx_NP_MPS_Limit} / \text{FC Unit Size}, 4) + 4$ (Note 3) Otherwise: $\text{Rx_NP_MPS_Limit} \text{ divided by FC Unit Size}$ (Note 3)	$\text{Ceiling}(\text{Rx_NP_MPS_Limit} / (\text{FC Unit Size} * 4)) + 1$ (Note 3)	$\text{Ceiling}(\text{Rx_NP_MPS_Limit} / (\text{FC Unit Size} * 16)) + 1$ (Note 3)
CplH	Root Complex (supporting peer-to-peer traffic between all Root Ports) and Switch: for shared credits in Flit Mode: 4 units – credit value of 04h, otherwise 1 FC unit - credit value of 01h Root Complex (not supporting peer-to-peer traffic between all Root Ports) and Endpoint: infinite FC units (Note 1). In Flit Mode, if [Merged] is enabled (see § Section 3.4.1), PH credits are used for Completions in the shared pool.	Root Complex (supporting peer-to-peer traffic between all Root Ports) and Switch: 4 FC units - credit value of 01h Root Complex (not supporting peer-to-peer traffic between all Root Ports) and Endpoint: infinite FC units (Note 1). In Flit Mode, if [Merged] is enabled (see § Section 3.4.1), PH credits are used for Completions in the shared pool.	Root Complex (supporting peer-to-peer traffic between all Root Ports) and Switch: 16 FC units - credit value of 01h Root Complex (not supporting peer-to-peer traffic between all Root Ports) and Endpoint: infinite FC units (Note 1). In Flit Mode, if [Merged] is enabled (see § Section 3.4.1), PH credits are used for Completions in the shared pool.

55. PCI Express to PCI/PCI-X Bridge requirements are addressed in [PCIe-to-PCI-PCI-X-Bridge].

Credit Type	Minimum Advertisement		
	No Scaling or Scale Factor 1	Scale Factor 4	Scale Factor 16
CplD	Root Complex (supporting peer-to-peer traffic between all Root Ports) and Switch: for shared credits in Flit Mode $\text{Max}(\text{Rx_MPS_Limit} / \text{FC Unit Size}, 4) + 4$ (Note 3), otherwise Rx_MPS_Limit divided by FC Unit Size .	Root Complex (supporting peer-to-peer traffic between all Root Ports) and Switch: $\text{Ceiling}(\text{Rx_MPS_Limit} / (\text{FC Unit Size} * 4)) + 1$.	Root Complex (supporting peer-to-peer traffic between all Root Ports) and Switch: $\text{Ceiling}(\text{Rx_MPS_Limit} / (\text{FC Unit Size} * 16)) + 1$.
	Root Complex (not supporting peer-to-peer traffic between all Root Ports) and Endpoint: infinite FC units (Note 2).	Root Complex (not supporting peer-to-peer traffic between all Root Ports) and Endpoint: infinite FC units (Note 2).	Root Complex (not supporting peer-to-peer traffic between all Root Ports) and Endpoint: infinite FC units (Note 2).
	In Flit Mode, if [Merged] is enabled (see § Section 3.4.1), PD credits are used for Completions in the shared pool.	In Flit Mode, if [Merged] is enabled (see § Section 3.4.1), PD credits are used for Completions in the shared pool.	In Flit Mode, if [Merged] is enabled (see § Section 3.4.1), PD credits are used for Completions in the shared pool.

Notes:

1. Infinite header credits is an encoding that is interpreted as infinite by the Transmitter, which will, therefore, never throttle. In Flit Mode the [Infinite.3] encoding is used (see § Table 3-3). In Non-Flit Mode the [Infinite.1] or [Infinite.2] encodings are used (see § Table 3-2).
2. Infinite data credits is an encoding that is interpreted as infinite by the Transmitter, which will, therefore, never throttle. In Flit Mode the [Infinite.3] encoding is used (see § Table 3-3). In Non-Flit Mode the [Infinite.1] or [Infinite.2] encodings are used (see § Table 3-2).
3. **Rx_NP_MPS_Limit** is the maximum size Non-Posted TLP Payload accepted by the Receiver (in DW). Larger of:
 - Payload size supported by any implemented earmarked TLP Type values.
 - Receiver that supports DMWr routing capability or DMWr Completer capability:
 - If DMWr Request Routing Supported is 1: 128 bytes (8 credit units)
 - If DMWr Request Routing Supported is 0 and DMWr Completer Supported is 1 and DMWr Lengths Supported is 00b: 64 bytes (4 credit units)
 - If DMWr Request Routing Supported is 0 and DMWr Completer Supported is 1 and DMWr Lengths Supported is not 00b: 128 bytes (8 credit units)
 - Receiver that supports AtomicOp routing capability or any AtomicOp Completer capability:
 - If AtomicOp Routing Supported is 1: 32 bytes (2 credit units)
 - If AtomicOp Routing Supported is 0 and 128-bit CAS Completer Supported is 0: 16 bytes (1 credit unit)
 - If AtomicOp Routing Supported is 0 and 128-bit CAS Completer Supported is 1: 32 bytes (2 credit units)
 - 16 bytes (1 credit unit)

- A Root Complex that supports no peer-to-peer traffic between Root Ports must advertise infinite Completion credits on every Root Port.
- A Root Complex that supports peer-to-peer traffic between some or all of its Root Ports may optionally advertise non-infinite Completion credits on those Root Ports. In this case, the Root Complex must ensure that deadlocks are avoided and forward progress is maintained for completions directed towards the Root Complex. Note that temporary stalls of completion traffic (due to a temporary lack of credit) are possible since Non-Posted Requests forwarded by the RC may not have explicitly allocated completion buffer space.

- A Receiver that does not support Scaled Flow Control must never cumulatively issue more than 2047 outstanding unused credits to the Transmitter for data or 127 for header. A Receiver that supports Scaled Flow Control must never cumulatively issue more outstanding unused data or header credits to the Transmitter than the Max Credits values shown in § Table 3-4.
 - Components may optionally check for violations of this rule. If a component implementing this check determines a violation of this rule, the violation is a Flow Control Protocol Error (FCPE).
 - If checked, this is a reported error associated with the Receiving Port (see § Section 6.2)
- If [Infinite.1], [Infinite.2], or [Infinite.3] credit advertisement has been made during initialization, no Flow Control updates are required following initialization.
 - If UpdateFC DLLPs or Optimized_Update_FCs are sent, the credit value fields must be Clear and must be ignored by the Receiver. The Receiver may optionally check for non-zero update values (in violation of this rule). If a component implementing this check determines a violation of this rule, the violation is a Flow Control Protocol Error (FCPE)
 - If checked, this is a reported error associated with the Receiving Port (see § Section 6.2)
- If Scaled Flow Control is activated, the HdrScale and DataScale fields in the UpdateFC DLLPs must match the values advertised during initialization (see § Section 3.4.2) with the following exceptions.
 - In Flit Mode, when more than one VC is supported, it is permitted to advertise [Zero] shared credits during initialization. For VCs that initialized with [Zero] shared credits, the HdrScale and DataScale fields in shared credit UpdateFC DLLPs must match the non-[Zero] HdrScale and DataScale values used by other VCs. If [Zero] shared credits were advertised on all VCs, the HdrScale and DataScale fields in the corresponding shared credit UpdateFC DLLPs are undefined.
 - In Flit Mode, it is permitted to advertise [Merged] shared completion credits during initialization. In this situation, the HdrScale and DataScale fields in shared completion credit UpdateFC DLLPs must match the values advertised for shared posted credits during initialization. When [Merged] shared completion credits are advertised, at least one VC must advertise non-[Zero] shared posted completion credits.
 - The Receiver may optionally check for violations of this rule. If a Receiver implementing this check determines a violation of this rule, the violation is a Flow Control Protocol Error (FCPE).
 - If checked, this is a reported error associated with the Receiving Port (see § Section 6.2).
- A received TLP using a VC that is not enabled is a Malformed TLP.
 - VC0 is always enabled.
 - For VCs 1-7, a VC is considered enabled when the corresponding VC Enable/SVC Enable bit has been Set and FC negotiation for that VC has exited the FC_INIT1 state to the FC_INIT2 state (see § Section 3.4).
 - This is a reported error associated with the Receiving Port (see § Section 6.2).
- TLP transmission using any VC 0-7 is not permitted until initialization for that VC has completed by exiting FC_INIT2 state.

For VCs 1-7, software must use the corresponding VC Negotiation Pending/SVC Negotiation Pending bit in the corresponding VC Resource Status Register to ensure that a VC is not used until negotiation has completed by exiting the FC_INIT2 state in both components on a Link.

In Flit Mode, if software disables VC 1-7:

- Dedicated Credit counts are cleared.
- Shared credit counts are not affected. Shared credits are returned as usual when the associated TLPs are consumed by the Receiver and are available for use by the remaining VCs.
- Behavior is undefined if software subsequently re-enables VC1-7.

The **[Field Size]** parameter used in the following sections is described in § Table 2-51 (see § Section 3.4.2 for details of Scaled Flow Control).

Table 2-51 [Field Size] Values §

Scaled Flow Control Supported	HdrScale or DataScale	[Field Size] for PH, NPH, CplH	[Field Size] for PD, NPD, CplD
No	x	8	12
Yes	00b	8	12
Yes	01b	8	12
Yes	10b	10	14
Yes	11b	12	16

In Flit Mode, the following rules apply to [Merged] FC credits:

- Receivers are permitted to support [Merged] FC; Transmitters must support [Merged] FC.
- When [Merged] FC is enabled:
 - Shared Completion Header credits are [Merged] with shared Posted Header credits.
 - During FC initialization, shared Posted Header credits must be used to indicate the total [Merged] shared Header credit pool.
 - FC updates must indicate either shared Posted Header credits or shared Completion Header credits according to the type of credit freed by the Receiver.
 - Shared Completion Data credits are [Merged] with shared Posted Data credits.
 - During FC initialization, shared Posted Data credits must be used to indicate the total [Merged] shared Data credit pool.
 - FC updates must indicate either shared Posted Data credits or shared Completion Data credits according to the type of credit freed by the Receiver.
- Dedicated Header credits must not be [Merged].
- Dedicated Data credits must not be [Merged].
- Merging behavior for each link direction is independent. The Receivers on each end of a given Link choose whether or not to merge independently
- Use of [Merged] shared credits must be consistent across VCs. If one VC uses [Merged] shared credits, all VCs must also use [Merged] shared credits.
- Use of [Merged] shared credits must be consistent between Hdr and Data. If Hdr uses [Merged] shared credits, Data must also use [Merged] shared credits.
- When [Merged] FC is enabled, it must be ensured that a Requester's rate of Completion processing be matched to that Requester's rate of issuing the corresponding Requests as measured within a sliding window of not more than 100 μ s.

In Flit Mode, the Receiver must return credits indicating the VC of the buffer(s) freed. If [Merged] FC is enabled, the Receiver must return credits indicating the FC Type of the buffer(s) freed.

When more than one VC advertises shared credits with scale factor 01b, 10b, or 11b, that scale factor must be able to express all allocated shared credits, regardless of VC. For example, if VC0 and VC1 each advertise 120 header credits (i.e., a total of 240 credits), they must do so using a scale factor other than 1 (01b) since that scale factor is limited to 127 outstanding credits.

In Flit Mode, shared credits for Header and Data are managed in credit blocks, where a credit block consists of 4 credits of the appropriate type. Credit blocks are not affected by the scale factor. Credit blocks do not apply to dedicated credits.

Rules for FC accounting with credit blocks:

- In each VC, per FC Type, shared credits must be reserved by the Transmitter and released by the Receiver in units of credit blocks.
- When a single TLP does not fully consume all the credits in a credit block, the remaining credits in the credit block must be allocated for consumption only by TLP(s) in the same VC and of the same FC Type.
 - Credit block allocation must distinguish between Posted and Completion FC Types, regardless of whether [Merged] credits are used for FC accounting.
 - Once a credit block is allocated, it must be held open until fully consumed by TLP(s) in the same VC and of the same FC Type, or until the associated VC is disabled and/or a DL_Down condition is entered.
 - Once a credit block is allocated, it must be applied only for TLP(s) in the same VC and of the same FC Type, even if TLP(s) in other VCs and/or of other FC Types are Transmitted/Received.
- Receivers must advertise credits in units of whole credit blocks.

If Shared Credits are infinite for a given FC/VC, Shared and Dedicated Credits in all VCs for that FC must be infinite. For Example, if VC0 and VC1 are enabled and Shared Completion Header credits are infinite in VC0:

- Dedicated Completion Header credits in VC0 must be infinite.
- Dedicated Completion Header credits in VC1 must be infinite.
- Shared Completion Header credits in VC1 must be infinite.

IMPLEMENTATION NOTE: MOTIVATION FOR SHARED CREDIT BLOCKS §

Shared FC enables the reduced cost implementation of multiple Virtual Channels by allowing common sets of resources to be shared. However, cost and complexity are increased, relative to the use of only dedicated credits, by the need to track TLPs as they are stored and removed from these shared structures. Because TLPs in different VCs are unordered with respect to each other, and often have different traffic behaviors, shared resource tracking typically requires linked-list structures for tracking TLPs in each VC, and TLP storage quickly becomes fragmented.

If, as is typically the case, the TLP storage is block-oriented, then the additional constraints of efficient block management motivate that TLPs in a given VC can be packed together. However, it is very complex to manage sub-block level out-of-order removal of TLPs in different VCs and the subsequent re-allocation of that freed space. So, by requiring both the Transmitter and Receiver to explicitly recognize credit blocks, the Receiver's buffer management logic is considerably simplified, while maintaining the efficient use of Receiver resources.

§ Figure 2-98 illustrates how a series of received TLPs would be placed into the Receiver's buffers.

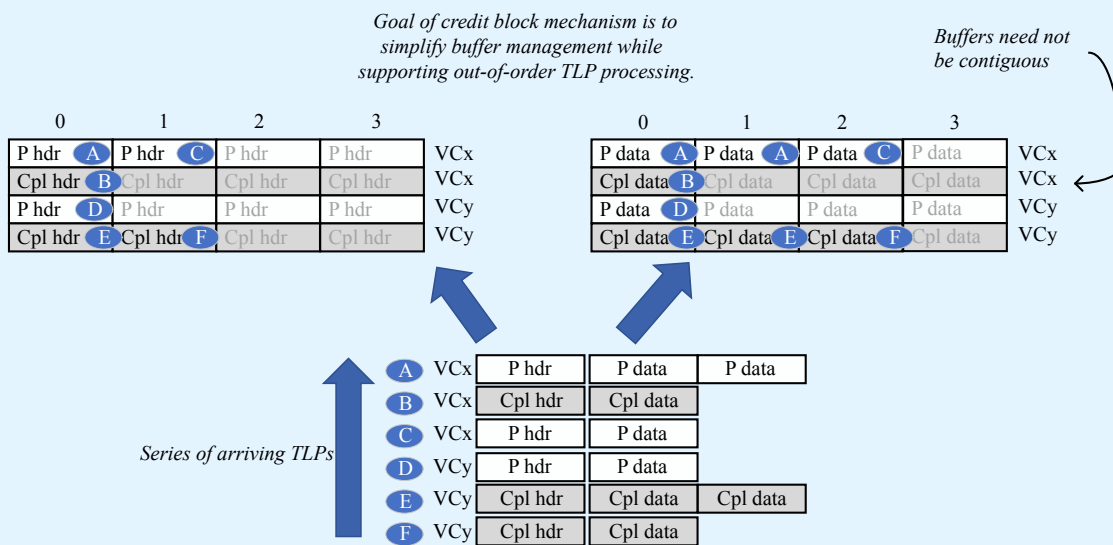


Figure 2-98 Credit Block Example §

2.6.1.1 FC Information Tracked by Transmitter §

- For each type of information tracked, there are two quantities (Non-Flit Mode) or six quantities (Flit Mode) tracked for Flow Control TLP Transmission gating:
 - CREDITS_CONSUMED** (per VC, all modes)
 - In Non-Flit Mode, CREDITS_CONSUMED is updated for all TLPs.
 - In Flit Mode, CREDITS_CONSUMED is updated for TLPs transmitted using dedicated credits.

- Count of the total number of FC units consumed by TLP Transmissions made since Flow Control initialization, modulo $2^{\text{[Field Size]}}$ (where [Field Size] is defined in § Table 2-51).
- Set to all 0's at interface initialization
- Set to 0 when VC Enable for VC[i] is Cleared.
- Updated for each TLP the Transaction Layer allows to pass the Flow Control gate for Transmission as shown:

$$\text{CREDITS_CONSUMED} := (\text{CREDITS_CONSUMED} + \text{Increment}) \bmod 2^{\text{[Field Size]}}$$

Equation 2-1 CREDITS_CONSUMED §

(Where *Increment* is the size in FC credits of the corresponding part of the TLP passed through the gate, and [Field Size] is defined in § Table 2-51)

- **SHARED_CREDITS_CONSUMED** (per VC, Flit Mode only)
 - In Non-Flit Mode, SHARED_CREDITS_CONSUMED is not used.
 - In Flit Mode, SHARED_CREDITS_CONSUMED is updated for TLPs transmitted using shared credits.
 - Count of the total number of FC units consumed by TLP Transmissions made since Flow Control initialization, modulo $2^{\text{[Field Size]}}$ (where [Field Size] is defined in § Table 2-51).
 - Set to all 0's at interface initialization
 - Updated for each TLP the Transaction Layer allows to pass the Flow Control gate for Transmission using shared credits as shown:

$$\text{SHARED_CREDITS_CONSUMED} := (\text{SHARED_CREDITS_CONSUMED} + \text{Increment}) \bmod 2^{\text{[Field Size]}}$$

Equation 2-2 SHARED_CREDITS_CONSUMED §

(Where *Increment* is the size in FC credits of the corresponding part of the TLP passed through the gate, and [Field Size] is defined in § Table 2-51)

- SHARED_CREDITS_CONSUMED is 0 for VCs that are not implemented or have never been enabled.
- SHARED_CREDITS_CONSUMED is preserved when VC Enable for VC 1-7 is Cleared.
- SHARED_CREDITS_CONSUMED is maintained independently for Posted and Completion credits even when [Merged] was selected by the Receiver.
- **SUM_SHARED_CREDITS_CONSUMED** (per Port, Flit Mode only)

$$\text{SUM_SHARED_CREDITS_CONSUMED} = \left(\sum_{i=0}^{i=7} \text{SHARED_CREDITS_CONSUMED}[i] \right) \bmod 2^{[\text{Field Size}]}$$

Equation 2-3 SUM_SHARED_CREDITS_CONSUMED §

- **SHARED_CREDITS_CONSUMED_CURRENTLY** (per VC, Flit Mode only, abbreviated as **SCCC** below)
 - In Non-Flit Mode, SHARED_CREDITS_CONSUMED_CURRENTLY is not used.
 - In Flit Mode, SHARED_CREDITS_CONSUMED_CURRENTLY is updated for TLPs transmitted using shared credits and for UpdateFC DLLPs that return shared credits.
 - Set to all 0's at interface initialization
 - Updated for each TLP the Transaction Layer allows to pass the Flow Control gate for Transmission using shared credits as shown:

$$\text{SCCC}[i] := (\text{SCCC}[i] + \text{Increment}) \bmod 2^{[\text{Field Size}+1]}$$

Equation 2-4 TLP SHARED_CREDITS_CONSUMED_CURRENTLY §

(Where *Increment* is the size in FC credits of the corresponding part of the TLP passed through the gate, and [Field Size] is defined in § Table 2-51)

- Updated for each UpdateFC releasing shared credit from VC[i]:

$$\text{SCCC}[i] := (\text{SCCC}[i] - (\text{UpdateFC value} - \text{SHARED_CREDIT_LIMIT}[i]) \bmod 2^{[\text{Field Size}]}) \bmod 2^{[\text{Field Size}+1]}$$

Equation 2-5 FC SHARED_CREDITS_CONSUMED_CURRENTLY §

- SHARED_CREDITS_CONSUMED_CURRENTLY is preserved when VC Enable for VC 1-7 is Cleared.
- SHARED_CREDITS_CONSUMED_CURRENTLY is maintained independently for Posted and Completion credits even when [Merged] was selected by the Receiver.
- **TOTAL_SHARED_CREDITS_AVAILABLE** (per Port, Flit Mode only)
 - In Non-Flit Mode, TOTAL_SHARED_CREDITS_AVAILABLE is not used.
 - In Flit Mode, TOTAL_SHARED_CREDITS_AVAILABLE contains the sum of the shared credits granted for all VCs during flow control initialization.
 - For [Merged], this initial value for all shared Completion credits is 0.
 - For [Zero], this initial value for that VC is 0.
 - TOTAL_SHARED_CREDITS_AVAILABLE is not affected when VC Enable for VC 1-7 is Cleared.
- **CREDIT_LIMIT** (per VC, all modes)
 - In Non-Flit Mode, CREDIT_LIMIT reflects all credit flow control updates.

- In Flit Mode, CREDIT_LIMIT reflects dedicated credit flow control updates.
- CREDIT_LIMIT contains the most recent number of FC units legally advertised by the Receiver. This quantity represents the total number of FC credits made available by the Receiver since Flow Control initialization, modulo $2^{[\text{Field Size}]}$ (where [Field Size] is defined in § Table 2-51).
- Undefined at interface initialization
- Set to the value indicated during Flow Control initialization
 - For “infinite” credits, this value is 0
- For each FC update received,
 - In Non-Flit Mode, if CREDIT_LIMIT is not equal to the update value, set CREDIT_LIMIT to the update value.
 - In Flit Mode, if Dedicated Flow Control is indicated (see § Figure 3-11) and CREDIT_LIMIT is not equal to the update value, set CREDIT_LIMIT to the update value.
- **SHARED_CREDIT_LIMIT** (per VC, Flit Mode only)
 - In Non-Flit Mode, SHARED_CREDIT_LIMIT is not used.
 - In Flit Mode, SHARED_CREDIT_LIMIT reflects shared credit flow control updates.
 - SHARED_CREDIT_LIMIT contains the most recent number of FC units legally advertised by the Receiver. This quantity represents the total number of shared FC credits made available by the Receiver since Flow Control initialization, modulo $2^{[\text{Field Size}]}$ (where [Field Size] is defined in § Table 2-51).
 - Undefined at interface initialization
 - Set to the value indicated during initial Flow Control initialization.
 - For [Merged], this initial value for all shared Completion credits is 0.
 - For [Zero], this initial value is 0.
 - SHARED_CREDIT_LIMIT is preserved when VC Enable for VC 1-7 is Cleared and also preserved during subsequent Flow Control initialization.
 - For each FC update received,
 - In Flit Mode, if Shared Flow Control is indicated (see § Figure 3-11) and if SHARED_CREDIT_LIMIT is not equal to the update value, set SHARED_CREDIT_LIMIT to the update value
- **SUM_SHARED_CREDIT_LIMIT** (per Port, Flit Mode only)
 - In Non-Flit Mode, SUM_SHARED_CREDIT_LIMIT is not used.
 - In Flit Mode, SUM_SHARED_CREDIT_LIMIT is defined by § Equation 2-6.

$$\text{SUM_SHARED_CREDIT_LIMIT} = \left(\sum_{i=0}^{i=7} \text{SHARED_CREDIT_LIMIT}[i] \right) \bmod 2^{[\text{Field Size}]}$$

Equation 2-6 SUM_SHARED_CREDIT_LIMIT §

This equation is not affected by [Merged]. When [Merged] is requested by the Receiver, Posted and Completion credits have distinct versions of SUM_SHARED_CREDIT_LIMIT.

- If a Transmitter detects that a TLP it is preparing to transmit is malformed, the Transmitter *MUST@FLIT* discard the TLP and handle the condition as an Uncorrectable Internal Error.
- If a Transmitter detects that a TLP it is preparing to transmit appears to be properly formed but with bad ECRC, the Transmitter *MUST@FLIT* transmit the TLP and update its internal Flow Control credits accordingly.
- The Transmitter gating function must determine if sufficient credits have been advertised to permit the transmission of a given TLP. If the Transmitter does not have enough credits to transmit the TLP, it must block the transmission of the TLP, possibly stalling other TLPs that are using the same Virtual Channel. The Transmitter must follow the ordering and deadlock avoidance rules specified in § Section 2.4 , which require that certain types of TLPs must bypass other specific types of TLPs when the latter are blocked. Note that TLPs using different Virtual Channels have no ordering relationship and must not block each other.
- In Flit Mode, the **shared transmitter gating function** test is performed as follows:
 - Credits must be allocated to specific VCs per the credit block rules defined in § Section 2.6.1 .
 - For each required type of credit, the number of credits required is calculated as:

SHARED_CUMULATIVE_CREDITS_REQUIRED = (SUM_SHARED_CREDITS_CONSUMED + credit units required for pending TLP) mod $2^{\text{[Field Size]}}$

Equation 2-7 SHARED_CUMULATIVE_CREDITS_REQUIRED §

This equation is not affected by [Merged]. When [Merged] is requested by the Receiver, Posted and Completion credits have distinct versions of SHARED_CUMULATIVE_CREDITS_REQUIRED.

- The transmitter is permitted to transmit a TLP if any of the following are true:
 - SHARED_CREDIT_LIMIT was “infinite” during Flow Control initialization.
 - For Non-Posted credits and for Posted and Completion credits when [Merged] was not requested by the Receiver, Shared Flow Control Usage Limit Enable is Clear and, for each type of information in the TLP, § Equation 2-8 is satisfied (using unsigned arithmetic):

$$\frac{(\text{SUM_SHARED_CREDIT_LIMIT} - \text{SHARED_CUMULATIVE_CREDITS_REQUIRED})}{\text{mod } 2^{\text{[Field Size]}}} < (2^{\text{[Field Size]}})/2$$

Equation 2-8 Shared Transmitter Gate non-[Merged] §

- For Posted and Completion credits when [Merged] was requested by the Receiver, Shared Flow Control Usage Limit Enable is Clear and, for each type of information in the TLP, § Equation 2-9 is satisfied (using unsigned arithmetic):

$$\begin{aligned} & (\text{SUM_SHARED_CREDIT_LIMIT_POSTED} \\ & + \text{SUM_SHARED_CREDIT_LIMIT_COMPLETION} \\ & - \text{SHARED_CUMULATIVE_CREDITS_REQUIRED_POSTED} \\ & - \text{SHARED_CUMULATIVE_CREDITS_REQUIRED_COMPLETION}) \\ & \text{mod } 2^{\text{[Field Size]}} < (2^{\text{[Field Size]}})/2 \end{aligned}$$

Equation 2-9 Shared Transmitter Gate [Merged] §

- For Non-Posted credits and for Posted and Completion credits when [Merged] was not requested by the Receiver, Shared Flow Control Usage Limit Enable is Set and, for each type of information in the TLP, § Equation 2-8 and § Equation 2-10 are both satisfied (using unsigned arithmetic)

$$\begin{aligned} & \text{SCCC} + \text{credit units required for pending TLP} \\ & \leq \text{TOTAL_SHARED_CREDITS_AVAILABLE} \times \text{Shared Flow Control Usage Limit} \times 0.125 \end{aligned}$$

Equation 2-10 Shared Transmitter Usage Limit Gate non-[Merged] §

- For Posted and Completion credits when [Merged] was requested by the Receiver, Shared Flow Control Usage Limit Enable is Set and, for each type of information in the TLP, § Equation 2-9 and § Equation 2-11 are both satisfied (using unsigned arithmetic)

$$\begin{aligned} & \text{SCCC} + \text{credit units required for pending TLP} \\ & \leq (\text{TOTAL_SHARED_CREDITS_AVAILABLE_POSTED} + \\ & \text{TOTAL_SHARED_CREDITS_AVAILABLE_COMPLETION}) \times \text{Shared Flow Control Usage Limit} \times \\ & 0.125 \end{aligned}$$

Equation 2-11 Shared Transmitter Usage Limit Gate [Merged] §

- If the above test does not permit a TLP to be transmitted, continue with the Transmitter gating function test below.
- Shared Flow Control is independent of the VC Arbitration mechanism described in § Section 6.3.3.2 .
- The **Transmitter gating function** test is performed as follows:
 - In Non-Flit Mode, this test applies to all TLPs.
 - In Flit Mode, this test applies to TLPs using dedicated credits. The shared transmitter gating function is used for TLPs using shared credits.
 - For each required type of credit, the number of credits required is calculated as:

$$\begin{aligned} & \text{CUMULATIVE_CREDITS_REQUIRED} = (\text{CREDITS_CONSUMED} + \text{credit units required for pending TLP}) \\ & \text{mod } 2^{\text{[Field Size]}} \end{aligned}$$

Equation 2-12 CUMULATIVE_CREDITS_REQUIRED §

- Unless CREDIT_LIMIT was specified as “infinite” during Flow Control initialization, the Transmitter is permitted to Transmit a TLP if, for each type of information in the TLP, the following equation is satisfied (using unsigned arithmetic):

$$(\text{CREDIT_LIMIT} - \text{CUMULATIVE_CREDITS_REQUIRED}) \bmod 2^{\text{[Field Size]}} \leq 2^{\text{[Field Size]}}/2$$

Equation 2-13 Transmitter Gate §

- If CREDIT_LIMIT was specified as “infinite” during Flow Control initialization, then the gating function is unconditionally satisfied for that type of credit.
- In Flit Mode, the TLP is transmitted with a Flit Mode Local TLP Prefix with the TLP Uses Dedicated Credits bit Set. This indicates that the flit is consuming dedicated credits.
- Note that some types of Transactions require more than one type of credit. (For example, Memory Write requests require PH and PD credits.)
- When accounting for credit use and return, information from different TLPs must not be mixed within one credit.
- When some TLP is blocked from Transmission by a lack of FC Credit, Transmitters must follow the ordering rules specified in § Section 2.4 when determining what types of TLPs must be permitted to bypass the stalled TLP.
- The return of FC credits for a Transaction must not be interpreted to mean that the Transaction has completed or achieved system visibility.
 - Flow Control credit return is used for receive buffer management only, and agents must not make any judgment about the Completion status or system visibility of a Transaction based on the return or lack of return of Flow Control information.
- In Non-Flit Mode, when a Transmitter sends a nullified TLP, the Transmitter does not modify CREDITS_CONSUMED for that TLP (see § Section 3.6.2.1).
- In Flit Mode, for all TLPs Transmitted, including nullified TLPs, Transmitters must modify CREDITS_CONSUMED (when TLP uses dedicated credits) or both SHARED_CREDITS_CONSUMED and SHARED_CREDITS_CONSUMED_CURRENTLY (when TLP uses shared credits).

2.6.1.2 FC Information Tracked by Receiver §

- For each type of information tracked, the following quantities are tracked for Flow Control TLP Receiver accounting. In Flit Mode, shared and dedicated credit versions of these are tracked independently.
 - **CREDITS_ALLOCATED**
 - Count of the total number of credits granted to the Transmitter since initialization, modulo $2^{\text{[Field Size]}}$ (where [Field Size] is defined in § Table 2-51)
 - Initially set according to the buffer size and allocation policies of the Receiver
 - If [Zero] or [Merged] were advertised by this Receiver, the corresponding CREDITS_ALLOCATED is set to 0
 - This value is included in the InitFC1, InitFC2, and UpdateFC DLLPs and in the Optimized_Update_FC (see § Section 3.5)
 - Incremented as the Receiver Transaction Layer makes additional receive buffer space available by processing Received TLPs. Optionally permitted to be incremented for dedicated credits or for shared credits when a single VC is using the shared pool, when the

Receiver Transaction Layer make additional buffer space available through other mechanisms (e.g., increasing the pool size).
Updated as shown:

$$\text{CREDITS_ALLOCATED} := (\text{CREDITS_ALLOCATED} + \text{Increment}) \bmod 2^{[\text{Field Size}]}$$

Equation 2-14 CREDITS_ALLOCATED §

(Where *Increment* corresponds to the credits made available, and [Field Size] is defined in § Table 2-51)

- For shared credits, CREDITS_ALLOCATED is preserved when VC Enable for VC 1-7 is Cleared and also preserved during subsequent Flow Control initialization.

◦ **CREDITS_RECEIVED**

- Mandatory for shared credits in Flit Mode.
- Otherwise, implemented when the optional error check described below is implemented.
- Count of the total number of FC units consumed by valid TLPs Received since Flow Control initialization, modulo $2^{[\text{Field Size}]}$ (where [Field Size] is defined in § Table 2-51)
- Set to all 0's at interface initialization
- Updated as shown:

$$\text{CREDITS_RECEIVED} := (\text{CREDITS_RECEIVED} + \text{Increment}) \bmod 2^{[\text{Field Size}]}$$

Equation 2-15 CREDITS_RECEIVED §

(Where *Increment* is the size in FC units of the corresponding part of the received TLP, and [Field Size] is defined in § Table 2-51)

for each Received TLP, provided that TLP:

- passes the Data Link Layer integrity checks
- is not malformed or (optionally) is malformed and is not ambiguous with respect to which buffer to release and is mapped to an initialized Virtual Channel
- does not consume more credits than have been allocated (see following rule)

For a TLP with an ECRC Check Failed error, but which otherwise is unambiguous with respect to which buffer to release, CREDITS_RECEIVED *MUST@FLIT* be updated.

- For shared credits, CREDITS_RECEIVED is preserved when VC Enable for VC 1-7 is Cleared and also preserved during subsequent Flow Control initialization.

• In Flit Mode, the Receiver accounting is modified as follows:

- If the TLP contained a Flit Mode Local TLP Prefix with the TLP Uses Dedicated Credits bit Set, update the dedicated CREDITS_ALLOCATED and dedicated CREDITS_RECEIVED for the associated VC.
- Otherwise, update the shared CREDITS_ALLOCATED and shared CREDITS_RECEIVED for the associated VC.
 - This accounting is not affected by [Merged]. Tracking is independent for Posted and Completion credits.

- Receivers are permitted to optimize their credit return mechanism to return shared and dedicated credits in a different order. For example, if shared TLP *S* and dedicated TLP *D* have the same credit type and VC and are received and processed in the order *S* followed by *D*, it is permitted to return *dedicated* credits when processing *S* as long as the corresponding *shared* credits are returned later when processing *D*.
- In Non-Flit Mode, if a Receiver implements the CREDITS_RECEIVED counter, then when a nullified TLP is received, the Receiver does not modify CREDITS_RECEIVED for that TLP (see § Section 3.6.2.1).
- In Flit Mode, Receivers that implement the CREDITS_RECEIVED counter modify CREDITS_RECEIVED even for nullified TLPs.
- A Receiver may optionally check for Receiver Overflow errors (TLPs exceeding CREDITS_ALLOCATED):
 - For Non-Flit Mode and for dedicated credits in Flit Mode, this is accomplished by checking § Equation 2-16 using unsigned arithmetic:

$$(\text{CREDITS_ALLOCATED} - \text{CREDITS_RECEIVED}) \bmod 2^{\text{[Field Size]}} \geq \frac{2^{\text{[Field Size]}}}{2}$$

Equation 2-16 Receiver Overflow Error Check Non-Flit / Dedicated §

- For shared Non-Posted and for shared Posted and Completion when [Merged] was not advertised by the Receiver, this is accomplished by checking § Equation 2-17 using unsigned arithmetic:

$$\left(\sum_{i=0}^{i=7} (\text{CREDITS_ALLOCATED}[i] - \text{CREDITS_RECEIVED}[i]) \right) \bmod 2^{\text{[Field Size]}} \geq \frac{2^{\text{[Field Size]}}}{2}$$

Equation 2-17 Receiver Overflow Error Check Non-Posted / Not [Merged] §

- For shared Posted and Completion when [Merged] was advertised by the Receiver, this is accomplished, by checking § Equation 2-18 using unsigned arithmetic:

$$temp1[i] \equiv CREDITS_ALLOCATED_{POSTED}[i] + CREDITS_ALLOCATED_{COMPLETION}[i]$$

$$temp2[i] \equiv CREDITS_RECEIVED_{POSTED}[i] + CREDITS_RECEIVED_{COMPLETION}[i]$$

$$\left(\sum_{i=0}^{i=7} (temp1[i] - temp2[i]) \right) \bmod 2^{[Field\ Size]} \geq \frac{2^{[Field\ Size]}}{2}$$

Equation 2-18 Receiver Overflow Error Check [Merged] §

If the check is implemented and this equation evaluates as true, the Receiver must:

- discard the TLP(s) without modifying the CREDITS_RECEIVED
- de-allocate any resources that it had allocated for the TLP(s)

If checked, this is a reported error associated with the Receiving Port (see § Section 6.2).

Note: Following a Receiver Overflow error, Receiver behavior is undefined, but it is encouraged that the Receiver continues to operate, processing Flow Control updates and accepting any TLPs that do not exceed allocated credits.

- For non-infinite NPH, NPD, PH, and CplH types, a Flow Control Update must be scheduled for Transmission each time the following events occur: In Non-Flit Mode, a Flow Control Update is an UpdateFC FCP. In Flit Mode, a Flow Control Update is either an UpdateFC FCP (for NPH, NPD, PH, and CplH, both Shared and Dedicated) or an Optimized_Update_FC (for Shared NPH and PH):
 - a. when scaled flow control is not activated and the number of available FC credits of a particular type is zero and one or more units of that type are made available by TLPs processed,
 - b. when scaled flow control is not activated, the NPD credit drops below 2, the Receiver supports either the AtomicOp routing capability or the 128-bit CAS Completer capability, and one or more NPD credits are made available by TLPs processed,
 - c. when scaled flow control is activated and the number of available FC credits of a particular type is zero or is below the scaled threshold and one or more units of that type are made available by TLPs processed so that the number of available credits is equal to or greater than the scaled threshold:
 - For Non-Flit Mode and for dedicated credits in Flit Mode, this threshold is 1 for HdrScale or DataScale of 01b, 4 for HdrScale or DataScale of 10b, and 16 for HdrScale or DataScale of 11b.

- For shared credits in Flit Mode, this threshold is 4 for HdrScale or DataScale of 01b, 4 for HdrScale or DataScale of 10b, and 16 for HdrScale or DataScale of 11b.
- d. when scaled flow control is activated in Non-Flit Mode and for dedicated credits in Flit Mode, the DataScale used for NPD is 01b, the NPD credit drops below 2, the Receiver supports either the AtomicOp routing capability or the 128-bit CAS Completer capability, and one or more NPD credits are made available by TLPs processed.
- e. For shared Non-Posted Data credits in Flit Mode, when the DataScale used for NPD is 01b, the NPD credit drops below 4, and 4 or more NPD credits are made available by TLPs processed.
- For non-infinite PD and CplD types, when the number of available credits is less than the number needed for the Rx_MPS_Limit, a Flow Control Update must be scheduled for Transmission each time one or more units of that type are made available by TLPs processed. In Non-Flit Mode, a Flow Control Update is an UpdateFC FCP. In Flit Mode, a Flow Control Update is either an UpdateFC FCP or an Optimized_Update_FC (for PD type).
 - For a Multi-Function Device where different Functions have different Rx_MPS_Limit values, the largest Rx_MPS_Limit value across all Functions must be used.

When multiple TLPs have been received, and some of the TLP(s) received consumed dedicated credits while other TLP(s) consumed shared credits, the Receiver must return all consumed dedicated credits prior to returning shared credits consumed by TLPs received after the TLP(s) received using dedicated credits. The Receiver is permitted to return the dedicated credits consumed prior to returning shared credits consumed by TLPs received after the TLP(s) using dedicated credits.

IMPLEMENTATION NOTE: RECEIVER HANDLING OF CREDIT RETURN FOR DEDICATED AND SHARED CREDITS §

The purpose of having some amount of Dedicated Credit per VC is to ensure that one VC cannot completely block another VC by consuming all available Shared Credit. To maintain this property it is necessary for the Receiver to ensure that Dedicated Credit is returned in a timely way - such that dedicated credits are returned at or before when the associated TLP(s) is(/are) consumed. In some implementations, buffers may be shared between Dedicated and Shared credits, and the distinction between the two types of Credits lies in how the buffer space is accounted for. In such implementations, it may be desirable to change the accounting for buffer space consumed using shared credits so that dedicated credits can be returned earlier to the Transmitter. To illustrate how this could work, consider the following example - TLPs A, B, C, and D are Received and consumed in that order. C uses dedicated credits while A, B, and D use shared credits.

- If $\text{size}_A \geq \text{size}_C$, the Receiver can return the dedicated credits for C when A is consumed.
- If $\text{size}_B \geq \text{size}_C$, the Receiver can return the dedicated credits for C when B is consumed.
- If $\text{size}_A + \text{size}_B \geq \text{size}_C$, the Receiver can return the dedicated credits for C when B is consumed.
- The Receiver is not permitted to delay the return of the dedicated credits for C to follow the time when D is consumed.

Other rules related to Flow Control:

- UpdateFC FCPs and Optimized_Update_FCs are permitted to be scheduled for Transmission more frequently than is required

- When the Link is in the L0 or L0s Link state, UpdateFC FCPs or Optimized_Update_FCs for each enabled type of non-infinite FC credit must be scheduled for transmission at least once every 30 μ s (-0%/+50%), except in Non-Flit Mode when the Extended Synch bit is Set, in which case the limit is 120 μ s (-0%/+50%).
 - This rule is optional when [Zero] dedicated FC credits are required as shown in § Table 3-3.
 - A timeout mechanism *MUST@FLIT* be implemented. If implemented, such a mechanism must:
 - be active only when the Link is in the L0 or L0s Link state
 - use a timer with a limit of 200 μ s (-0%/+50%), where the timer is reset by the receipt of any Init, UpdateFC FCP, or Optimized_Update_FC. Alternately, the timer may be reset by the receipt of any DLLP (see § Section 3.5)
 - upon timer expiration, instruct the Physical Layer to retrain the Link (via the LTSSM Recovery state, § Section 4.2.7.4)
 - in Non-Flit Mode, if an Infinite Credit advertisement has been made during initialization for all three FC types, this timeout mechanism must be disabled for that VC
 - in Flit Mode, if an Infinite Credit advertisement has been made during initialization for all six FC types, this timeout mechanism must be disabled for that VC

IMPLEMENTATION NOTE:

USE OF “INFINITE” FC ADVERTISEMENT §

For a given implementation it is possible that not all of the queue types need to be physically implemented in hardware for all Virtual Channels. For example, in a Device that does not support Flit Mode and whose Functions have no AtomicOp Completer, AtomicOp Routing capability, DMWr Completer, or DMWr Routing capability, there is no need to implement a Non-Posted Data queue for Virtual Channels other than VC0, since Non-Posted Requests with data are only allowed on Virtual Channel 0 for such Devices. For unimplemented queues, the Receiver can eliminate the need to present the appearance of tracking Flow Control credits by advertising infinite Flow Control credits during initialization.

IMPLEMENTATION NOTE:

NON-FLIT MODE FLOW CONTROL UPDATE LATENCY §

For components subject to receiving streams of TLPs, it is desirable to implement receive buffers larger than the minimum size required to prevent Transmitter throttling due to lack of available credits. Likewise, it is desirable to transmit UpdateFC FCPs such that the time required to send, receive, and process the UpdateFC prevents Transmitter throttling. Recommended maximum values for UpdateFC transmission latency during normal operation are shown in § Table 2-52, § Table 2-53, and § Table 2-54. Note that the values given in these tables do not account for any delays caused by the Receiver or Transmitter being in L0s, in Recovery, or for any delays caused by Retimers (see § Section 4.3.9) . For improved performance and/or power-saving, it may be desirable to use a Flow Control update policy that is more sophisticated than a simple timer. Any such policy is implementation specific, and beyond the scope of this document.

The values in the Tables are measured starting from when the Receiver Transaction Layer makes additional receive buffer space available by processing a received TLP, to when the first Symbol of the corresponding UpdateFC DLLP is transmitted.

For a Multi-Function Device where different Functions have different Rx_MPS_Limit values, it is strongly recommended that the smallest Rx_MPS_Limit value across all Functions be used.

*Table 2-52 Maximum UpdateFC Transmission Latency Guidelines
for 2.5 GT/s (Symbol Times) §*

		Link Operating Width						
		x1	x2	x4	x8	x12	x16	x32
Rx_MPS_Limit (bytes)	128	237	128	73	67	58	48	33
	256	416	217	118	107	90	72	45
	512	559	289	154	86	109	86	52
	1024	1071	545	282	150	194	150	84
	2048	2095	1057	538	278	365	278	148
	4096	4143	2081	1050	534	706	534	276

*Table 2-53 Maximum UpdateFC Transmission Latency Guidelines
for 5.0 GT/s (Symbol Times) §*

		Link Operating Width						
		x1	x2	x4	x8	x12	x16	x32
Rx_MPS_Limit (bytes)	128	288	179	124	118	109	99	84
	256	467	268	169	158	141	123	96
	512	610	340	205	137	160	137	103
	1024	1122	596	333	201	245	201	135
	2048	2146	1108	589	329	416	329	199

		Link Operating Width						
		x1	x2	x4	x8	x12	x16	x32
	4096	4194	2132	1101	585	757	585	327

Table 2-54 Maximum UpdateFC Transmission Latency Guidelines for 8.0 GT/s and Higher Data Rates (Symbol Times) §

		Link Operating Width						
		x1	x2	x4	x8	x12	x16	x32
Rx_MPS_Limit (bytes)	128	333	224	169	163	154	144	129
	256	512	313	214	203	186	168	141
	512	655	385	250	182	205	182	148
	1024	1167	641	378	246	290	246	180
	2048	2191	1153	634	374	461	374	244
	4096	4239	2177	1146	630	802	630	372

2.7 End-to-End Data Integrity §

Data integrity across a Link is provided by the Data Link Layer for NFM and by the Physical Layer Logical Block for FM. As TLPs are routed through intermediate components (i.e., Switches) a TLP may become corrupted, and the Link data integrity mechanisms will not detect such corruption. To ensure end-to-end data integrity detection in systems that require high data reliability, a Transaction Layer end-to-end 32-bit CRC (ECRC) can be applied to a TLP. The ECRC covers all bits that do not change as the TLP traverses the path (invariant fields). The ECRC is generated by the Transaction Layer in the source component, and checked (if supported) by the ultimate PCI Express Receiver, and optionally by intermediate Receivers. A Switch that supports ECRC checking must check ECRC on TLPs targeting the Switch itself. Such a Switch can optionally check ECRC on TLPs that it forwards. On TLPs that the Switch forwards, the Switch must preserve the error detecting properties of the ECRC, regardless of whether the Switch checks the ECRC, or if the ECRC check fails.⁵⁶

In some cases, the data in a TLP payload is known to be corrupt at the time the TLP is generated, or may become corrupted while passing through an intermediate component, such as a Switch. In these cases, error forwarding, also known as data poisoning, can be used to indicate the corruption to the device consuming the data. In FM, there are two different mechanisms for data poisoning, in support of distinct use models.

2.7.1 ECRC Rules §

The capability to generate and check ECRC is reported to software, and the ability to do so is enabled by software (see § Section 7.8.4.7).

- If a device Function is enabled to generate ECRC, it must calculate and apply ECRC for all TLPs originated by the Function

56. An exception is a Multicast TLP that an Egress Port is modifying due to the MC_Overlay mechanism. See § Section 6.14.5.

- For non-IDE TLPs that do not require FM/NFM translation, Switches must pass TLPs with ECRC unchanged from the Ingress Port to the Egress Port⁵⁷
 - For non-IDE TLPs that require FM/NFM translation, Switches must apply ECRC to the translated TLP and must ensure that the error detection capability of ECRC is maintained between the Ingress and Egress Ports; how this is done is outside the scope of this specification.
 - These rules do not apply for IDE TLPs, for which ECRC is not supported and for which FM/NFM translation is not possible.
- If a device supports ECRC generation/checking, at least one of its Functions must support Advanced Error Reporting (AER) (see § Section 6.2)
- If a device Function is enabled to check ECRC, it must do so for all TLPs with ECRC where the device is the ultimate PCI Express Receiver
 - Note that it is still possible for the Function to receive TLPs without ECRC, and these are processed normally - this is not an error

Note that a Switch may optionally perform ECRC checking on TLPs passing through the Switch. ECRC Errors detected by the Switch are reported as described in § Table 6-5, but do not alter the TLPs' passage through the Switch.⁵⁸

A 32-bit ECRC is calculated for the TLP (End-End TLP Prefixes/OHC, header, and data payload), but not, in FM, including any Trailer, using the following algorithm and appended to the end of the TLP (see § Figure 2-3):

- The ECRC value is calculated using the following algorithm (see § Figure 2-99)
- The polynomial used has coefficients expressed as 04C1 1DB7h
- The seed value (initial value for ECRC storage registers) is FFFF FFFFh
- All header fields, all End-End TLP Prefixes/OHC (if present), and the entire data payload (if present) are included in the ECRC calculation.
- Local TLP Prefixes (if present) are not included in the ECRC calculation.
- All Variant bits must be treated as Set for ECRC calculations.
- In Non-Flit Mode, the following bits are Variant:
 - TLP Header symbol 0, bit 0. This is bit 0 of the Type field⁵⁹. This bit in an End-End TLP Prefix is invariant.
 - TLP Header symbol 2, bit 6. This is either the EP bit or Reserved.
- In Flit Mode, the following bits are Variant:
 - TLP Header symbol 0, bit 0. This is bit 0 of the Type field⁶⁰.
 - TLP Header symbol 6, bit 7. This is either the EP bit or Reserved.
- All other fields are Invariant
- ECRC calculation starts with bit 0 of byte 0 and proceeds from bit 0 to bit 7 of each byte of the TLP
- The result of the ECRC calculation is complemented, and the complemented result bits are mapped into the 32-bit TLP Digest field (NFM), or Trailer (FM), as shown in § Table 2-55.

Table 2-55 Mapping of Bits into ECRC Field §

ECRC Result Bit	Corresponding Bit Position in the 32-bit TLP ECRC Field
0	7

57. An exception is a Multicast TLP that an Egress Port is modifying due to the MC_Overlay mechanism. See § Section 6.14.5.

58. An exception is a Multicast TLP that an Egress Port is modifying due to the MC_Overlay mechanism. See § Section 6.14.5.

59. Bit 0 of the Type field changes when a Configuration Request is changed from Type 1 to Type 0.

60. Bit 0 of the Type field changes when a Configuration Request is changed from Type 1 to Type 0.

ECRC Result Bit	Corresponding Bit Position in the 32-bit TLP ECRC Field
1	6
2	5
3	4
4	3
5	2
6	1
7	0
8	15
9	14
10	13
11	12
12	11
13	10
14	9
15	8
16	23
17	22
18	21
19	20
20	19
21	18
22	17
23	16
24	31
25	30
26	29
27	28
28	27
29	26
30	25
31	24

- In NFM, the 32-bit ECRC value is placed in the TLP Digest field at the end of the TLP (see § Figure 2-3). In FM, the 32-bit ECRC value is placed in the TLP Trailer.
- For TLPs including a TLP Digest field used for an ECRC value, Receivers that support end-to-end data integrity checking check the ECRC value in the TLP Digest field by:
 - applying the same algorithm used for ECRC calculation (above) to the received TLP, not including the 32-bit TLP Digest field of the received TLP, and then:
 - comparing the calculated result with the value in the TLP Digest field of the received TLP.
- Receivers that support end-to-end data integrity checks report violations as an ECRC Error. This reported error is associated with the Receiving Port (see § Section 6.2).

Beyond the stated error reporting semantics contained elsewhere in this specification, how ultimate PCI Express Receivers make use of the end-to-end data integrity check provided through the ECRC is beyond the scope of this document. Intermediate Receivers are still required to forward TLPs whose ECRC checks fail. A PCI Express-to-PCI/PCI-X Bridge is classified as an ultimate PCI Express Receiver with regard to ECRC checking.

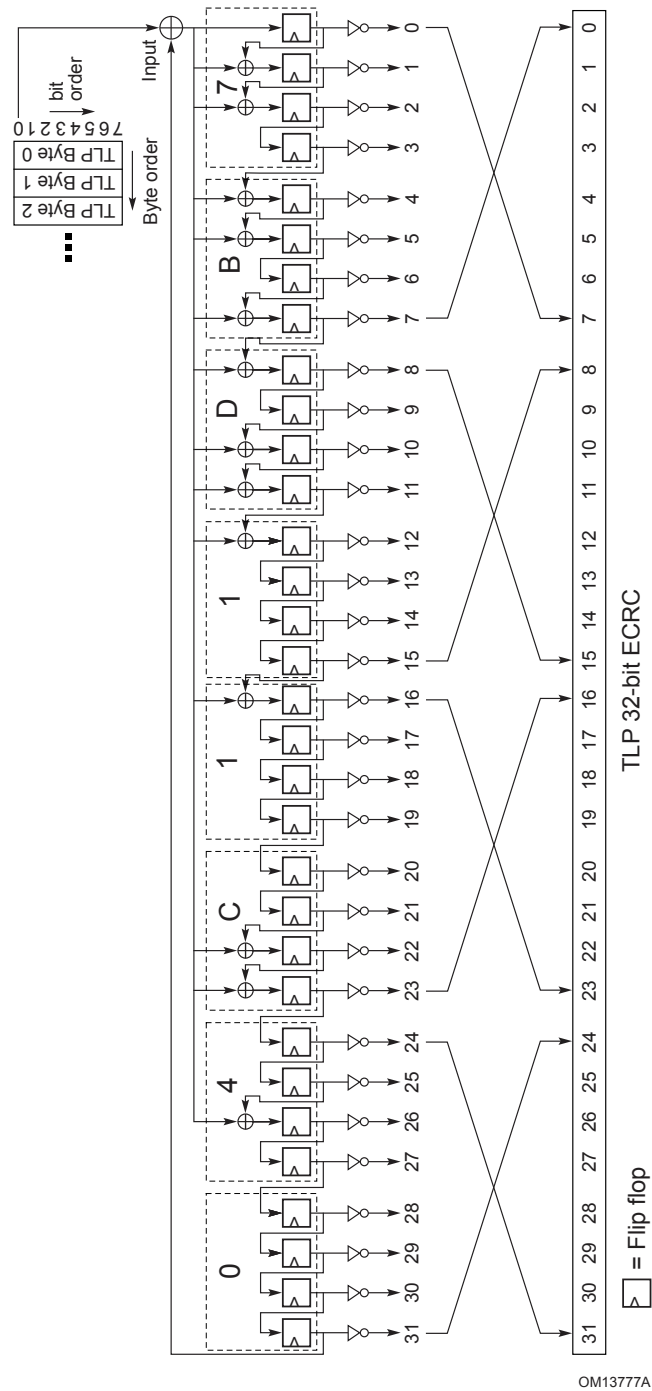


Figure 2-99 Calculation of 32-bit ECRC for TLP End to End Data Integrity Protection §

IMPLEMENTATION NOTE: PROTECTION OF TD BIT INSIDE SWITCHES (NFM) §

It is of utmost importance that Switches insure and maintain the integrity of the TD bit in TLPs that they receive and forward (i.e., by applying a special internal protection mechanism), since corruption of the TD bit will cause the ultimate target device to misinterpret the presence or absence of the TLP Digest field.

Similarly, it is strongly recommended that Switches provide internal protection to other Variant bits in TLPs that they receive and forward, as the end-to-end integrity of Variant bits is not sustained by the ECRC.

IMPLEMENTATION NOTE: DATA LINK LAYER DOES NOT HAVE INTERNAL TLP VISIBILITY (NFM) §

Since the Data Link Layer does not process the TLP header (it determines the start and end of the TLP based on indications from the Physical Layer), it is not aware of the existence of the TLP Digest field, and simply passes it to the Transaction Layer as a part of the TLP.

2.7.2 Error Forwarding (Data Poisoning) §

Error Forwarding (also known as data poisoning), is indicated by Setting the EP bit, or additionally, in FM, through the use of Physical Layer Logical Block mechanisms. In FM, either or both mechanisms are permitted to be applied to a TLP with a data payload, and the requirements defined in this specification for Receiver handling of poisoned TLPs are the same regardless of the poisoning mechanism applied. It is permitted for Receivers to additionally implement differentiated handling based on the type of poisoning mechanism applied, but such handling is outside the scope of this specification.

The rules for the use of the EP bit are specified in § Section 2.7.2.1 . The rules for the use of Physical Layer Logical Block mechanisms for data poisoning are specified in § Section 4.2.3.4 . Here are some examples of cases where Error Forwarding might be used:

- Example #1: A read from parity or ECC-protected memory encounters an uncorrectable error (EP bit)
- Example #2: An error detected at the source of a write directed towards system memory (EP bit)
- Example #3: Data integrity error on an internal data buffer or cache within a routing element (EP bit or Physical Layer Logical Block)

Considerations for the use of Error Forwarding

- Error Forwarding is only used for Read Completion Data, AtomicOp Completion Data, AtomicOp Request Data, or Write Data, never for the cases when the error is in the “header” (request phase, address/command, etc.). Requests/Completions with header errors cannot be forwarded in general since true destination cannot be positively known and, therefore, forwarding may cause direct or side effects such as data corruption, system failures, etc.
- Error Forwarding is used for controlled propagation of errors through the system, system diagnostics, etc.

- Note that Error forwarding does not cause Data Link Layer Retry - Poisoned TLPs will be retried only if there are transmission errors on the Link as determined by the TLP error detection mechanisms in the Data Link Layer.
 - The Poisoned TLP may ultimately cause the originator of the request to re-issue it (at the Transaction Layer or above) in the case of read operation or to take some other action. Such use of Error Forwarding information is beyond the scope of this specification.

2.7.2.1 Rules For Use of Data Poisoning §

- Support for TLP poisoning in a Transmitter is optional.
 - In FM, a Transmitter is permitted to support only the EP bit mechanism, only the Physical Layer Logical Block mechanism, or both, or neither.
- Data poisoning applies only to the data payload⁶¹ within a Write Request (Posted or Non-Posted), a Message with Data, an AtomicOp Request, a Read Completion, or an AtomicOp Completion.
- Poisoning of a TLP with a data payload in the Transaction Layer is indicated by a Set EP bit.
- When a routing element is translating a TLP from NFM to FM, if the EP bit is Set in the NFM TLP, then the EP bit must be Set in the FM TLP.
- When a routing element is translating a TLP from FM to NFM, if either poisoning mechanism has been applied to the FM TLP, then the EP bit must be Set in the NFM TLP.
- Transmitters are only permitted to poison TLPs that include a data payload. In FM, the EP bit is Reserved for TLPs that do not include a data payload. In NFM, the behavior of the Receiver is not specified if poisoning is indicated for any TLP that does not include a data payload.
- For IDE TLPs:
 - Only the original Transmitting Port is permitted to poison a TLP and must do so using the EP bit mechanism.
 - It is not permitted to use Physical Layer Logical Block mechanisms to poison a TLP; if data corruption is detected in an IDE TLP after the time the MAC has been generated, the IDE TLP must be forwarded without consideration of the detected corruption.
- If a Transmitter supports data poisoning, TLPs that are known at the Transaction Layer of the Transmitter to include a bad data payload must use the EP bit mechanism.
- For a routing element that supports data poisoning, if a non-IDE TLP is Received as poisoned using Physical Layer Logical Block mechanisms, that TLP must be transmitted at the Egress Port marked as poisoned using the EP bit mechanism.
- If a Downstream Port supports Poisoned TLP Egress Blocking, the Poisoned TLP Egress Blocking Enable bit is Set, and a poisoned TLP targets going out the Egress Port, the Port must handle the TLP as a Poisoned TLP Egress Blocked error unless there is a higher precedence error. See § Section 6.2.3.2.3 , § Section 6.2.5 , and § Section 7.9.14.3 . Further:
 - The Port must not transmit the TLP.
 - If DPC is not triggered and the TLP is a Non-Posted Request received on a non-UIO VC, the Port must return a Completion with Unsupported Request Completion Status. See § Section 6.2.3.2.4.1 .
 - If DPC is triggered the Port must behave as described in § Section 2.9.3 .
- For ultimate Completers:
 - The following Requests with poisoned data payload must not modify the value of the target location:
 - Configuration Write Request

61. A data payload includes TLPs with no bytes enabled (e.g., zero-length writes).

- Any of the following that target a control register or control structure in the Completer: I/O Write Request, Memory Write Request, or non-Vendor-Defined Message with data
- AtomicOp Request
- DMWr Request (see § Section 6.32)

Unless there is a higher precedence error, a Completer must handle these Requests as a Poisoned TLP Received error⁶², and the Completer must also return a Completion with a Completion Status of Unsupported Request (UR) if the Request is Non-Posted or UIO (see § Section 6.2.3.2.3 , § Section 6.2.3.2.4 , and § Section 6.2.5). Regardless of the severity of the reported error, the reported error must be handled as an uncorrectable error, not an Advisory Non-Fatal Error.

A Switch must route these Requests the same way it would route the same Request if it were not poisoned, unless a Request targets a location in the Switch itself, in which case the Switch is the Completer for that Request and must follow the above rules.

For some applications it may be desirable for the Completer to use poisoned data in Write Requests that do not target control registers or control structures - such use is not forbidden. Similarly, it may be desirable for the Requester to use data marked poisoned in Completions - such use is also not forbidden. The appropriate use of poisoned information is application specific, and is not discussed in this document.

This document does not define any mechanism for determining which part or parts of the data payload of a Poisoned TLP are actually corrupt and which, if any, are not corrupt.

2.8 Completion Timeout Mechanism §

In any split transaction protocol, there is a risk associated with the failure of a Requester to receive an expected Completion. To allow Requesters to attempt recovery from this situation in a standard manner, the Completion Timeout mechanism is defined. This mechanism is intended to be activated only when there is no reasonable expectation that the Completion will be returned, and should never occur under normal operating conditions. Note that the values specified here do not reflect expected service latencies, and must not be used to estimate typical response times.

PCI Express device Functions that issue Requests requiring Completions must implement the Completion Timeout mechanism. An exception is made for Configuration Requests (see below). The Completion Timeout mechanism is activated for each Request that requires one or more Completions when the Request is transmitted. Since Switches do not autonomously initiate Requests that need Completions, the requirement for Completion Timeout support is limited only to Root Complexes, PCI Express-PCI Bridges, and Endpoints.

The Completion Timeout mechanism may be disabled by configuration software by means of the Completion Timeout Disable mechanism (see § Section 7.5.3.15 and § Section 7.5.3.16).

The Completion Timeout limit is set in the Completion Timeout Value field of the Device Control 2 register. A Completion Timeout is a reported error associated with the Requester Function (see § Section 6.2). If the Completion Timeout programming mechanism is not supported, the Function *MUST@FLIT* implement a timeout value in the range 40 ms to 50 ms; when *Flit Mode Supported* is Clear, the Function must implement a timeout value in the range 50 µs to 50 ms, and it is strongly recommended that the value be at least 10 ms.

A Request for which there are multiple Completions must be considered completed only when all Completions have been received by the Requester.

For a Memory Read Request, if some, but not all, requested data is returned before the Completion Timeout timer expires, the Requester is permitted to keep or to discard the data that was returned prior to timer expiration.

62. Due to ambiguous language in earlier versions of this specification, a component is permitted to handle this error as an Unsupported Request, but this is strongly discouraged.

Completion Timeout expiration for a UIO Request does not necessarily indicate that the Request, or portions of the Request, succeeded or failed.

For a series of UIO Requests using the same Transaction ID, the Completion Timeout mechanism must be restarted for each UIO Request issued.

Completion Timeouts for Configuration Requests have special requirements for the support of PCI Express to PCI/PCI-X Bridges. PCI Express to PCI/PCI-X Bridges, by default, are not enabled to return Request Retry Status (RRS) for Configuration Requests to a PCI/PCI-X device behind the Bridge. This may result in lengthy completion delays that must be comprehended by the Completion Timeout value in the Root Complex. System software may enable PCI Express to PCI/PCI-X Bridges to return RRS for Configuration Requests by setting the Bridge Configuration Retry Enable bit in the Device Control register, subject to the restrictions noted in the [\[PCIe-to-PCI-PCI-X-Bridge\]](#).

IMPLEMENTATION NOTE:

COMPLETION TIMEOUT PREFIX/HEADER LOG CAPABLE §

The prefix/header of the Request TLP associated with a Completion Timeout may optionally be recorded by Requesters that implement the [AER Extended Capability](#). Support for recording of the prefix/header is indicated by the value of the Completion Timeout Prefix/Header Log Capable bit in the Advanced Error Capabilities and Control register.

A Completion Timeout may be the result of improper configuration, system failure, or async removal (see § Section 6.7.6). In order for host software to distinguish a Completion Timeout error after which continued normal operation is not possible (e.g., after one caused by improper configuration or a system failure) from one where continued normal operation is possible (e.g., after an async removal), it is strongly encouraged that Requesters log the Request TLP prefix/header associated with the Completion Timeout.

2.9 Link Status Dependencies §

2.9.1 Transaction Layer Behavior in DL_Down Status §

[DL_Down](#) status indicates that there is no connection with another component on the Link, or that the connection with the other component has been lost and is not recoverable by the Physical or Data Link Layers. This section specifies the Transaction Layer's behavior if DPC has not been triggered and the Data Link Layer reports [DL_Down](#) status to the Transaction Layer, indicating that the Link is non-operational. § Section 2.9.3 specifies the behavior if DPC has been triggered.

- For a Port with [DL_Down](#) status, the Transaction Layer is not required to accept received TLPs from the Data Link Layer, provided that these TLPs have not been acknowledged by the Data Link Layer. Such TLPs do not modify receive Flow Control credits.

For a Downstream Port, [DL_Down](#) status is handled by:

- Initializing back to their default state any buffers or internal states associated with outstanding requests transmitted Downstream
 - Port configuration registers must not be affected, except as required to update status associated with the transition to [DL_Down](#).

- For Non-Posted Requests and UIO Requests, forming completions for any Requests submitted by the device core for Transmission, returning Unsupported Request Completion Status, then discarding the Requests
 - This is a reported error associated with the Function for the (virtual) Bridge associated with the Port (see § Section 6.2). For Root Ports, the reporting of this error is optional.

- Non-Posted Requests and UIO Requests already being processed by the Transaction Layer, for which it may not be practical to return Completions, are discarded.

Note: This is equivalent to the case where the Request had been Transmitted but not yet Completed before the Link status became DL_Down.

- These cases are handled by the Requester using the Completion Timeout mechanism.

Note: The point at which a Non-Posted Request or UIO Request becomes “uncompletable” is implementation specific.

- The Port must terminate any PME_Turn_Off handshake Requests targeting the Port in such a way that the Port is considered to have acknowledged the PME_Turn_Off request (see the Implementation Note in § Section 5.3.3.2.1).
 - The Port must handle Vendor-Defined Message Requests as described in § Section 2.2.8.6 (e.g., silently discard Vendor-Defined Type 1 Messages Requests that it is not designed to receive) since the DL_Down prevents the Request from reaching its targeted Function.
 - For all other Posted Requests, discarding the Requests
 - This is a reported error associated with the Function for the (virtual) Bridge associated with the Port (see § Section 6.2), and must be reported as an Unsupported Request. For Root Ports, the reporting of this error is optional.
 - For a Posted Request already being processed by the Transaction Layer, the Port is permitted not to report the error.
- Note: This is equivalent to the case where the Request had been Transmitted before the Link status became DL_Down

Note: The point at which a Posted Request becomes “unreportable” is implementation specific.

- Discarding all Completions submitted by the device core for Transmission

For an Upstream Port, DL_Down status is handled as a reset by:

- Returning all PCI Express-specific registers, state machines and externally observable state to the specified default or initial conditions (except for registers defined as sticky - see § Section 7.4)
- Discarding all TLPs being processed
- For Switch and Bridge propagating hot reset to all associated Downstream Ports. In Switches that support Link speeds greater than 5.0 GT/s, the Upstream Port must direct the LTSSM of each Downstream Port to the Hot Reset state, but not hold the LTSSMs in that state. This permits each Downstream Port to begin Link training immediately after its hot reset completes. This behavior is recommended for all Switches.

2.9.2 Transaction Layer Behavior in DL_Up Status §

DL_Up status indicates that a connection has been established with another component on the associated Link. This section specifies the Transaction Layer's behavior when the Data Link Layer reports entry to the DL_Up status to the Transaction Layer, indicating that the Link is operational. The Transaction Layer of a Port with DL_Up status must accept received TLPs that conform to the other rules of this specification.

For a Downstream Port on a Root Complex or a Switch:

- When transitioning from a non-DL_Up status to a DL_Up status and the Auto Slot Power Limit Disable bit is Clear in the Slot Control Register, the Port must initiate the transmission of a Set_Slot_Power_Limit Message to the other component on the Link to convey the value programmed in the Slot Power Limit Scale and Slot Power Limit Value fields of the Slot Capabilities Register. This Transmission is optional if the Slot Capabilities Register has not yet been initialized.

2.9.3 Transaction Layer Behavior During Downstream Port Containment §

During Downstream Port Containment (DPC), the LTSSM associated with the Downstream Port is directed to the Disabled state. Once it reaches the Disabled state, it remains there as long as the DPC Trigger Status bit in the DPC Status Register is Set. See § Section 6.2.11 for requirements on how long software must leave the Downstream Port in DPC. This section specifies the Transaction Layer's behavior once DPC has been triggered, and as long as the Downstream Port remains in DPC.

- Once DPC has been triggered, no additional (Upstream) TLPs are accepted from the Data Link Layer.
- If the condition that triggered DPC was associated with an Upstream TLP, any subsequent Upstream TLPs that were already accepted from the Data Link Layer must be discarded silently.

The Downstream Port handles (Downstream) TLPs submitted by the device core in the following manner.

- If the condition that triggered DPC was associated with a Downstream TLP, any prior Downstream TLPs are permitted to be dropped silently or transmitted before the Link goes down. Otherwise, the following rules apply.
 - For each Non-Posted Request or UIO Request, the Port must return a Completion and discard the Request silently. The Completer ID field must contain the value associated with the Downstream Port.
 - If the DPC Completion Control bit is Set in the DPC Control Register, then Completions are generated with Unsupported Request (UR) Completion Status.
 - If the DPC Completion Control bit is Clear, Completions are generated with Completer Abort (CA) Completion Status.
 - The Port must terminate any PME_Turn_Off handshake Requests targeting the Port in such a way that the Port is considered to have acknowledged the PME_Turn_Off Request (see the Implementation Note in § Section 5.3.3.2.1).
 - The Port must handle Vendor-Defined Message Requests as described in § Section 2.2.8.6 . (e.g., silently discard Vendor Defined_Type 1 Message Requests that it is not designed to receive) since the DL_Down prevents the Request from reaching its targeted Function.
 - For all other Posted Requests and Completions, the Port must silently discard the TLP.

For any outstanding Non-Posted Requests or UIO Requests where DPC being triggered prevents their associated Completions from being returned, the following apply:

- For Root Ports that support RP Extensions for DPC, the Root Port may track certain Non-Posted Requests and UIO Requests and, when DPC is triggered, synthesize a Completion for each tracked Request. This helps avoid Completion Timeouts that would otherwise occur as a side-effect of DPC being triggered. Each synthesized Completion must have a UR or CA Completion Status as determined by the DPC Completion Control bit. The set of Non-Posted Requests and UIO Requests that get tracked is implementation specific, but it is strongly recommended that all Non-Posted Requests and UIO Requests that are generated by host processor instructions (e.g., “read”, “write”, “load”, “store”, or one that corresponds to an AtomicOp) be tracked. Other

candidates for tracking include peer-to-peer Requests coming from other Root Ports and Requests coming from RCiEPs.

- Otherwise, the associated Requesters may encounter Completion Timeouts. The software solution stack should comprehend and account for this possibility.